HOLCF-extra

alex

October 28, 2026

Contents

0.1 Step functions 14

1 Lemmas concerning completion, complete sets and closed sets **24**
1.1 Strict sums 35
1.2 Cartesian products 39
1.3 Smash products 40
1.4 Joins: auxiliary lemmas 53

2 Distributive bounded-complete cpos **54**

3 Instances **55**

4 Continuity of meet, and distributivity from compacts **56**

5 Filter sums **60**
5.1 The meet form, in bounded-complete pcpos 61
5.2 Instances 62

6 Separated sums **63**
6.1 Atoms of the domain constructions 65
6.2 Instances 66

7 Distributivity of smash products **67**

8 dI-domains **70**
8.1 Transfer along isomorphisms 71
8.2 Instances 71

9 Bounded-complete domains with a top **74**

10 Algebraic lattices are bifinite **77**

11 Order structure of domain-package types **80**
11.1 Transfer along isomorphisms 80
11.2 Map functionals 83

12 Order predicates as deflation locales **83**
12.1 Bounded completeness 84
12.2 Separation 90
12.3 Separated sums 93
12.4 Distributivity 97

13 Deriving order-class instances of domain-package types **101**
13.1 Fixed-point induction on a map definition 101
13.2 Registries 101
13.3 Methods 102
13.4 Future work: a derivation command 104

14 MUB-closure (in an arbitrary partial order) **105**

15 Fixed sets of deflations are MUB-closed **106**

16 Finiteness of MUB-closures in bifinite domains **107**

17 The deflation determined by a MUB-closed set of compacts **111**

18 The lattice of finite deflations **115**

19 Algebraic deflations are determined by their compact fixes **119**

20 Lower powerdomain elements are generated by their maximal members **130**

21 Binds and maps through members **131**

22 Rows with a top adjoined **133**

23 The bag powerdomain **144**
23.1 Splitting plans 144
23.2 The basis 148
23.3 Basis constructors 149
23.4 Basis preorder 151
23.5 Basic plans 154
23.6 Labelled families 155
23.7 Type definition 160
23.8 Unit, plus and omega 161
23.9 Embedding-projection pair with the convex powerdomain . . 163
23.10Mapping 165

theory *Top* **imports** *HOLCF* **begin**

datatype *'a::type top* = *Of* (*top-down*: *'a*) | *Top*

instantiation *top* :: (*below*) *below*
begin
definition *below-top-def*:
(⊑) *a b* ≡ *case b of Top* ⇒ *True* |
  Of b ⇒ (*case a of Top* ⇒ *False* | *Of a* ⇒ *a* ⊑ *b*)

instance ..

theorem *below-Top*[*simp*]: *x* ⊑ *Top* **by** (*simp add*: *below-top-def*)

theorem *Top-below*[*simp*]: *Top* ⊑ *x* ⟷ *x* = *Top* **by** (*cases x*) (*simp-all add*: *below-top-def*)

theorem *below-Of*: *Of x* ⊑ *Of y* ⟷ *x* ⊑ *y* **by** (*simp add*: *below-top-def*)

end

instance *top* :: (*po*) *po*
  by (*intro-classes*)
   (*auto simp add*: *below-top-def*
     intro: *below-trans below-antisym*
     split: *top.splits*)

lemma *not-Top*: **assumes** *x* ≠ *Top* **obtains** *y* **where** *x* = *Of y*
using *assms* **by** (*cases x*) *simp-all*

lemma *ch2ch-top*:
  assumes *chain S*
    Top ∉ *range S*
  obtains *T* **where** *chain T S* = (λ*i*. *Of* (*T i*))
proof −
  from ‹*Top* ∉ *range S*›[*THEN notE*, *OF range-eqI*] **have** *S i* ≠ *Top* **for** *i* **by** *metis*
  then obtain *T* **where** *T*: *S* = (λ*i*. *Of* (*T i*)) **using** *not-Top* **by** *meson*
  then have *chain T* **using** ‹*chain S*›[*THEN chainE*, *unfolded below-top-def*]
    by (*auto intro*: *chainI*)
  from *this T* **show** *?thesis* **by** (*rule that*)
qed

lemma *top-chlub-Of*: **assumes** *chain* (*T*::*nat*⇒*'a*::*cpo*)
  shows *range* (λ*i*. *Of* (*T i*)) <<| *Of* (⨆ *i*. *T i*)
proof (*intro is-lubI is-ubI*)
  show *x* ∈ *range* (λ*i*. *Of* (*T i*)) ⟹ *x* ⊑ *Of* (⨆ *i*. *T i*)
    for *x* **by** (*auto simp add*: *is-ub-thelub*[*OF assms*] *below-Of*)
  show *range* (λ*i*. *Of* (*T i*)) <| *u* ⟹ *Of* (⨆ *i*. *T i*) ⊑ *u* **for** *u*

```
    apply (induction u)
     apply (fastforce intro: lub-below[OF assms] dest: is-ubD simp add: below-Of)
    apply simp
    .
qed

lemma top-lub-Top:
Top ∈ X ⟹ X <<| Top
  by (intro is-lubI is-ubI) (auto dest: is-ubD)

lemma lub-Top-iff: range (S::nat⇒'a::cpo top) <<| Top ⟷ Top ∈ range S if
chain S
proof (cases Top ∈ range S)
  case True
  then show ?thesis by (simp add: top-lub-Top)
next
  case False
  from ‹chain S› this obtain T where chain T S = (λi. Of (T i))
    by (rule ch2ch-top)
  then have range S <<| Of (⨆ i. T i) by (blast intro: top-chlub-Of)
  with ‹Top ∉ range S› show ?thesis by (blast dest: is-lub-unique)
qed

instantiation top :: (cpo) cpo
begin

instance
proof
  fix S :: nat ⇒ 'a top assume chain S
  show ∃ x. range S <<| x
  proof (cases Top ∈ range S)
    case True
    then show ?thesis by (blast intro: top-lub-Top)
  next
    case False
   from ‹chain S› this show ?thesis by (blast elim: ch2ch-top intro: top-chlub-Of)
  qed
qed

theorem compact-Top: compact (Top :: 'a top)
  unfolding compact-def
proof (rule admI)
  fix Y :: nat ⇒ 'a top assume chain Y
  assume ∀ i. ¬(Top ⊑ Y i)
  then have ⋀i. Y i ≠ Top by simp
  then have Top ∉ range Y by (metis rangeE)
  from ‹chain Y› this have (⨆ i. Y i) ≠ Top
    using cpo-lubI lub-Top-iff by metis
  then show ¬(Top ⊑ (⨆ i. Y i)) by simp
```

qed

end

instantiation *top* :: (*pcpo*) *pcpo*
begin

lemma *of-bot-below*: *Of* $\bot \sqsubseteq y$ **by** (*cases y*) (*simp-all add*: *below-Of*)

instance by (*intro-classes*, *intro exI*[*of - Of* $\bot$] *allI of-bot-below*)

theorem *top-bot*: $(\bot::'a\ top) = Of\ \bot$
by (*rule bottomI*[*symmetric*]) (*rule of-bot-below*)

end

definition *down* $\equiv (\Lambda\ x.\ Of\ x)$

theorem *cont-Of*[*simp*]: *cont Of*
by (*rule contI*) (*rule top-chlub-Of*)

theorem *down-beta*: $down{\cdot}x = Of\ x$
unfolding *down-def* **by** *simp*
theorem *top-cases*[*case-names down Top*, *cases type*: *top*]:
$(\bigwedge p.\ x = down{\cdot}p \Longrightarrow P) \Longrightarrow (x = Top \Longrightarrow P) \Longrightarrow P$
by (*cases x*) (*simp-all add*: *down-beta*)

theorem *down-inject*: $down{\cdot}x = down{\cdot}y \Longrightarrow x = y$
by (*simp add*: *down-beta*)

lemma *top-chain-cases* [*consumes 1*]:
fixes *Y* :: *nat* $\Rightarrow$ $'a$::*cpo top*
assumes *chain Y*
obtains (*down*) *X* **where** *chain X Y* $= (\lambda i.\ down{\cdot}(X\ i))$ $(\bigsqcup i.\ Y\ i) = down{\cdot}(\bigsqcup i.\ X\ i)$
| (*Top*) *finite-chain Y* $\exists i.\ Y\ i = Top$ $(\bigsqcup i.\ Y\ i) = Top$
proof (*cases Top* $\in$ *range Y*)
assume *Top* $\in$ *range Y*
then have *3*: $(\bigsqcup i.\ Y\ i) = Top$ **using** *lub-Top-iff*[*OF assms*] *lub-eqI* **by** *blast*
with *assms* ‹*Top* $\in$ *range Y*› **have** *1*: *finite-chain Y*
using *finite-chain-def maxinch-is-thelub* **by** *fastforce*
from ‹*Top* $\in$ *range Y*›[*THEN rangeE*] **have** *2*: $\exists i.\ Y\ i = Top$ **by** *metis*
from *1 2 3* **show** *thesis* **by** (*rule that*(*2*))
next
assume *Top* $\notin$ *range Y*
from *ch2ch-top*[*OF assms this*] **obtain** *X* **where** *1*: *chain X* **and** *2*: $Y = (\lambda i.\ Of\ (X\ i))$ **.**
then have *range Y* $<<|$ *Of* $(\bigsqcup i.\ X\ i)$ **using** *top-chlub-Of* **by** *blast*

```
  then have 3: (⨆ i. Y i) = Of (⨆ i. X i) by (rule lub-eqI)
  from 1 2 3 show thesis by (rule that(1)[unfolded down-beta])
qed

definition cmap-top ≡ (Λ f x. map-top (λx. f·x) x)

lemma cmap-top-cont1[cont2cont]: cont (map-top f) if cont f
proof (rule contI)
  fix Y :: nat ⇒ 'a top assume chain Y
  then show range (λi. map-top f (Y i)) <<| map-top f (⨆ i. Y i)
  proof (cases rule: top-chain-cases)
    case (down X)
    from that ‹chain X› have fX: chain (λi. f (X i)) by (rule ch2ch-cont)
    from ‹Y = (λi. down·(X i))› have (λi. map-top f (Y i)) = (λi. Of (f (X i)))
      by (simp add: down-beta)
   then have range (λi. map-top f (Y i)) <<| Of (⨆ i. f (X i)) using top-chlub-Of[OF
fX]
      by presburger
   also have (⨆ i. f (X i)) = f (⨆ i. X i) using that ‹chain X› by (rule cont2con-
tlubE[symmetric])
    also from ‹(⨆ i. Y i) = down·(⨆ i. X i)› have Of ... = map-top f (⨆ i. Y i)
      by (simp add: down-beta)
    finally show ?thesis .
  next
    case Top
   from ‹finite-chain Y› obtain i where max-in-chain i Y by (rule finite-chainE)
    with ‹(⨆ i. Y i) = Top› have Y i = Top using maxinch-is-thelub[OF ‹chain
Y›]
      by simp
    then have map-top f (Y i) = Top by simp
    then have Top ∈ range (λi. map-top f (Y i)) using range-eqI by metis
    then have ... <<| Top by (rule top-lub-Top)
    also have Top = map-top f (⨆ i. Y i) using ‹(⨆ i. Y i) = Top› by simp
    finally show ?thesis .
  qed
qed

lemma cmap-top-cont2[cont2cont]: cont (λf. map-top (λx. f·x) x)
  by (cases x) (auto simp add: down-beta intro: cont-compose)

lemma cmap-top-beta: cmap-top·f·x = map-top (λx. f·x) x
  unfolding cmap-top-def
  by (simp add: cmap-top-cont1 cmap-top-cont2 cont2cont-LAM)

lemma cmap-top-down[simp]: cmap-top·f·(down·x) = down·(f·x) by (simp add:
cmap-top-beta down-beta)
lemma cmap-top-Top[simp]: cmap-top·f·Top = Top by (simp add: cmap-top-beta)
lemma cmap-top-ID[simp]: cmap-top·ID = ID
proof (rule cfun-eqI)
```

```
  show cmap-top·ID·x = ID·x for x
    by (cases x) simp-all
qed

lemma down-strict: down·⊥ = ⊥ by (simp add: down-beta top-bot)

lemma down-below-of: down·x ⊑ down·y ⟹ x ⊑ y by (simp add: down-beta
below-Of)
lemma down-below-iff[iff]: down·x ⊑ down·y ⟷ x ⊑ y by (rule iffI) (auto intro:
monofun-cfun-arg down-below-of)

lemma down-ne-Top[simp]: down·x ≠ Top Top ≠ down·x by (simp-all add: down-beta)

lemma approx-chain-top:
  assumes approx-chain a
  shows approx-chain (λi. cmap-top·(a i))
proof −
  interpret approx-chain a by fact
  show ?thesis
  proof
    show chain (λi. cmap-top·(a i)) by (simp add: ch2ch-Rep-cfunR)
    show (⨆ i. cmap-top·(a i)) = ID by (simp add: contlub-cfun-arg[symmetric])
    show cmap-top·(a i)·(cmap-top·(a i)·x) = cmap-top·(a i)·x for i x
      by (cases x) (simp-all add: approx-idem)
    show cmap-top·(a i)·x ⊑ x for i x by (cases x) (simp-all add: approx-below)
    show finite {x. cmap-top·(a i)·x = x} for i
    proof −
      have {x. cmap-top·(a i)·x = x} = insert Top {down·x | x. (a i)·x = x}
        (is ?l = ?r)
      proof (rule set-eqI)
        fix x show x ∈ ?l ⟷ x ∈ ?r
          by (cases x) (auto dest: down-inject)
      qed
      also have finite ...
        using finite-deflation.finite-fixes[OF finite-deflation-approx]
        by simp
      finally show ?thesis.
    qed
  qed
qed

instance top :: (bifinite) bifinite
  by intro-classes (blast intro: bifinite[THEN exE, THEN approx-chain-top])

definition Iftop where Iftop t f x ≡ case x of Top ⇒ t | Of x ⇒ f·x

lemma Iftop-down: Iftop t f (down·x) = f·x by (simp add: Iftop-def down-beta)

lemma Iftop-top: Iftop t f Top = t by (simp add: Iftop-def)
```

lemma *cont-Iftop*: *cont* (λx. *Iftop t f x*) **if** *t-is-top*: $\bigwedge x$. $x \sqsubseteq t$
proof (*rule contI*)
  fix *Y* :: *nat* $\Rightarrow$ *'b top* **assume** *chain Y*
  then show *range* (λi. *Iftop t f* (*Y i*)) <<|
        Iftop t f ($\bigsqcup i$. *Y i*)
  proof (*cases Y rule*: *top-chain-cases*)
    case (*down X*)
    from ‹*Y* = (λi. *down*·(*X i*))› **have** ∗:(λi. *Iftop t f* (*Y i*)) = (λi. *f*·(*X i*))
      by (*simp add*: *Iftop-down*)
    from ‹($\bigsqcup i$. *Y i*) = *down*·($\bigsqcup i$. *X i*)› **have** ∗∗: *Iftop t f* ($\bigsqcup i$. *Y i*) = *f*·($\bigsqcup i$. *X i*)
      by (*simp add*: *Iftop-down*)
    from ‹*chain X*› **show** *?thesis* **unfolding** ∗ ∗∗
      by (*intro contE*) *auto*
  next
    case *Top*
   from ‹*finite-chain Y*› **obtain** *i* **where** *max-in-chain i Y* **by** (*rule finite-chainE*)
    with ‹($\bigsqcup i$. *Y i*) = *Top*› **have** *Top* = *Y i* **using** *maxinch-is-thelub*[*OF* ‹*chain Y*›] **by** *simp*
    then have *t* = *Iftop t f* (*Y i*) **by** (*simp add*: *Iftop-top*)
    then have *t* ∈ *range* (λi. *Iftop t f* (*Y i*)) **by** (*rule range-eqI*)
    then have *range* (λi. *Iftop t f* (*Y i*)) <<| *t*
      unfolding *is-lub-def is-ub-def* **using** *t-is-top* **by** *blast*
    moreover have *t* = *Iftop t f* ($\bigsqcup i$. *Y i*) **by** (*simp add*: ‹($\bigsqcup i$. *Y i*) = *Top*› *Iftop-top*)
    ultimately show *?thesis* **by** *presburger*
  qed
qed

definition *up-top-to-top-up* :: *'a*::*cpo u top* → *'a top u*
  where *up-top-to-top-up* = (Λ *x*. *Iftop* (*up*·*Top*) (Λ (*up*·*x*). *up*·(*down*·*x*)) *x*)

lemma *utttu-beta*: *up-top-to-top-up*·*x* = *Iftop* (*up*·*Top*) (Λ (*up*·*x*). *up*·(*down*·*x*)) *x*
  unfolding *up-top-to-top-up-def*
proof (*rule beta-cfun*, *rule cont-Iftop*)
  show *x* ⊑ *up*·*Top* **for** *x* **by** (*cases x*) *simp-all*
qed

definition *top-up-to-up-top* :: *'a*::*cpo top u* → *'a u top*
  where *top-up-to-up-top* = (Λ (*up*·*x*). *Iftop Top* (*down oo up*) *x*)

lemma *tutut-beta-minor*: (Λ *x*. *Iftop Top* (*down oo up*) *x*)·*x* = *Iftop Top* (*down oo up*) *x*
  by (*rule beta-cfun*, *rule cont-Iftop*, *rule below-Top*)

theorem *utttu-cases*:
  up-top-to-top-up·*Top* = *up*·*Top*

up-top-to-top-up·(*down*·⊥) = ⊥
up-top-to-top-up·(*down*·(*up*·*x*)) = *up*·(*down*·*x*)
by (*simp-all add*: *utttu-beta Iftop-top Iftop-down*)

theorem *tutut-cases*:
top-up-to-up-top·(*up*·*Top*) = *Top*
top-up-to-up-top·(⊥) = *down*·⊥
top-up-to-up-top·(*up*·(*down*·*x*)) = *down*·(*up*·*x*)
unfolding *top-up-to-up-top-def*
by (*simp-all add*: *tutut-beta-minor Iftop-top Iftop-down down-strict*)

lemma *top-up-iso1*: *up-top-to-top-up*·(*top-up-to-up-top*·*x*) = *x*
proof −
consider *x* = *up*·*Top* | *y* **where** *x* = *up*·(*down*·*y*) | *x* = ⊥
by (*metis top-cases upE*)
then show *?thesis* **by** *cases* (*simp-all add*: *utttu-cases tutut-cases*)
qed

lemma *top-up-iso2*: *top-up-to-up-top*·(*up-top-to-top-up*·*x*) = *x*
proof −
consider *x* = *Top* | *y* **where** *x* = *down*·(*up*·*y*) | *x* = *down*·⊥
by (*metis top-cases upE*)
then show *?thesis* **by** *cases* (*simp-all add*: *utttu-cases tutut-cases*)
qed

instance *top* :: (*profinite*) *profinite*
proof
from *profinite* **obtain** *a* :: *nat* ⇒ *'a u* → *'a u* **where** *approx-chain a* **..**
note *this* = *approx-chain-top*[*OF this*]
from *approx-chain-iso*[*OF this top-up-iso1 top-up-iso2*]
show ∃ (*a* :: *nat* ⇒ *'a top u* → *'a top u*). *approx-chain a*
by *fast*
qed

hide-const (**open**) *Of*

end
theory *FiniteCF* **imports** *HOLCF* **begin**

default-sort *type*

lemma (**in** *po*) *finite-has-maximal*:
fixes *A* :: *'a set*
shows ⟦*finite A*; *A* ≠ {}⟧ ⟹ ∃ *x*∈*A*. ∀ *y*∈*A*. *x* ⊑ *y* ⟶ *x* = *y*
proof (*induct rule*: *finite-ne-induct*)
case (*singleton x*)
show *?case* **by** *simp*
next
case (*insert a A*)

```
  from ‹∃ x∈A. ∀ y∈A. x ⊑ y ⟶ x = y›
  obtain x where x: x ∈ A
          and x-eq: ⋀y. ⟦y ∈ A; x ⊑ y⟧ ⟹ x = y by fast
  show ?case
  proof (intro bexI ballI impI)
    fix y
    assume y ∈ insert a A and (if x ⊑ a then a else x) ⊑ y
    thus (if x ⊑ a then a else x) = y
      by (auto split: if-splits intro: below-antisym dest: below-trans x-eq)
  next
    show (if x ⊑ a then a else x) ∈ insert a A
      by (simp add: x)
  qed
qed

lemma (in po) smyth-ub-mono:
  assumes ∀ x ∈ A. ∃ y ∈ B. x ⊑ y
  B <| b
shows A <| b
  using assms by (blast intro: is-ubI below-trans dest: is-ubD)

lemma (in po) lub-mono-general:
  assumes A <<| a B <<| b and smyth-ord: ∀ x ∈ A. ∃ y ∈ B. x ⊑ y
  shows a ⊑ b
proof −
  from ‹B <<| b› have B <| b by (rule is-lubD1)
  with smyth-ord have A <| b by (rule smyth-ub-mono)
  with ‹A <<| a› show a ⊑ b by (rule is-lubD2)
qed

lemma (in po) ch2ch-lub-gen:
  assumes lub: ⋀i. J i <<| Y i
    and smyth-ord: ⋀i. ∀ x ∈ J i. ∃ y ∈ J (Suc i). x ⊑ y
  shows chain Y
  by (rule chainI) (rule lub-mono-general[OF lub lub smyth-ord])


theorem ch2ch-lub:
  assumes chain: ⋀j. j ∈ J ⟹ chain (λi. Y i j)
    and lub: ⋀i. Y i ‘ J <<| lub (Y i ‘ J)
  shows chain (λi. lub (Y i ‘ J))
proof −
  have Y (Suc i) ‘ J <| x ⟹ Y i ‘ J <| x for i x
  proof (rule ub-imageI, drule ub-imageD)
    fix j assume j ∈ J
    then have chain (λi. Y i j) by (rule chain)
    then have Y i j ⊑ Y (Suc i) j by (rule chainE)
    also assume Y (Suc i) j ⊑ x
    finally show Y i j ⊑ x .
```

```
  qed
  from lub[THEN is-lubD2, OF this, OF lub[THEN is-lubD1]] show ?thesis
    by (rule chainI)
qed

theorem chain-lub-family:
  fixes Y :: nat ⇒ 'a::type ⇒ 'b::cpo
  assumes chain: ⋀j. j ∈ J ⟹ chain (λi. Y i j)
    and lub: ⋀i. Y i ‘ J <<| lub (Y i ‘ J)
  defines L ≡ (⨆ i. lub (Y i ‘ J))
  shows (λj. ⨆ i. Y i j) ‘ J <<| L
proof (rule is-lubI)
  have chain (λi. lub (Y i ‘ J)) using chain lub by (rule ch2ch-lub)
  then have lub-L: range (λi. lub (Y i ‘ J)) <<| L
    unfolding L-def by (rule cpo-lubI)
  show (λj. ⨆ i. Y i j) ‘ J <| L
  proof (rule ub-imageI)
    fix j assume j: j ∈ J
    {
      fix i
      have Y i j ⊑ lub (Y i ‘ J)
        using lub[of i, THEN is-lubD1, THEN ub-imageD, OF j] .
      also have ... ⊑ L
        using lub-L[THEN is-lubD1, THEN ub-imageD] by blast
      finally have Y i j ⊑ L .
    }
    from chain[OF j] this show (⨆ i. Y i j) ⊑ L
      by (rule lub-below)
  qed
  show (λj. ⨆ i. Y i j) ‘ J <| u ⟹ L ⊑ u for u
  proof −
    assume ub: (λj. ⨆ i. Y i j) ‘ J <| u
    {
      fix j assume j: j ∈ J
      from chain[OF j] have Y i j ⊑ (⨆ i. Y i j) for i by (rule is-ub-thelub)
      also assume (⨆ i. Y i j) ⊑ u
      finally have Y i j ⊑ u for i .
    }
    from this ub[THEN ub-imageD] have Y i j ⊑ u if j ∈ J for i j using that by
blast
    from lub[THEN is-lubD2, OF ub-imageI, OF this] have lub (Y i ‘ J) ⊑ u for
i by simp
    from lub-L[THEN is-lubD2, OF ub-imageI, OF this] show L ⊑ u.
  qed
qed

corollary chain-lub-family-converse:
  fixes Y :: nat ⇒ 'a::type ⇒ 'b::cpo
  assumes chain: ⋀j. j ∈ J ⟹ chain (λi. Y i j)
```

and *lub*: $\bigwedge i.\ Y\ i\ `\ J <<|\ lub\ (Y\ i\ `\ J)$
shows *range* $(\lambda i.\ lub\ (Y\ i\ `\ J)) <<|\ (\bigsqcup j{\in}J.\ (\bigsqcup i.\ Y\ i\ j))$
proof −
have *chain* $(\lambda i.\ lub\ (Y\ i\ `\ J))$ **using** *chain lub* **by** (*rule ch2ch-lub*)
then have ∗: *range* $(\lambda i.\ lub\ (Y\ i\ `\ J)) <<|\ (\bigsqcup i.\ lub\ (Y\ i\ `\ J))$
by (*rule cpo-lubI*)
from *chain lub* **have** $((\lambda j.\ \bigsqcup i.\ Y\ i\ j)\ `\ J) <<|\ (\bigsqcup i.\ lub\ (Y\ i\ `\ J))$
by (*rule chain-lub-family*)
then have *lub* $((\lambda j.\ \bigsqcup i.\ Y\ i\ j)\ `\ J) = (\bigsqcup i.\ lub\ (Y\ i\ `\ J))$
by (*rule lub-eqI*)
with ∗ **show** *?thesis* **by** *simp*
qed

lemma (**in** *approx-chain*) *approx-ch2ch-lub*:
assumes $\bigwedge i.\ (\lambda x.\ approx\ i{\cdot}x)\ `\ S <<|\ (\bigsqcup x \in S.\ approx\ i{\cdot}x)$
shows *chain* $(\lambda i.\ (\bigsqcup\ x \in S.\ approx\ i{\cdot}x))$
by (*rule ch2ch-lub*) (*simp-all add*: *assms*)

lemma (**in** *approx-chain*) *approx-lub-family*:
assumes $\bigwedge i.\ (\lambda x.\ approx\ i{\cdot}x)\ `\ S <<|\ (\bigsqcup x \in S.\ approx\ i{\cdot}x)$
shows $S <<|\ (\bigsqcup i.\ \bigsqcup x \in S.\ approx\ i{\cdot}x)$
proof −
have $S = (\lambda j.\ \bigsqcup i.\ approx\ i{\cdot}j)\ `\ S$
proof −
have $(\bigsqcup i.\ approx\ i{\cdot}j) = j$ **for** j
using *lub-approx chain-approx*[*THEN contlub-cfun-fun*] **by** *simp*
then show *?thesis* **by** *simp*
qed
also have ... $<<|\ (\bigsqcup i.\ \bigsqcup x \in S.\ approx\ i{\cdot}x)$
by (*intro chain-lub-family assms*) *simp*
finally show *?thesis* **.**
qed

end
theory *Definitions* **imports** *HOLCF* **begin**

lemma (**in** *pcpo*) *lub-empty*[*simp*]: $\{\} <<|\ \bot$ **by** (*simp add*: *is-lub-def*)

context *po* **begin**

definition *below-strict* (**infix** ‹⊏› *50*) **where** $a \sqsubset b = (a \sqsubseteq b \wedge \neg(b \sqsubseteq a))$

Stated as a fact rather than a *sublocale*: a sublocale into a locale with derived constants breaks later subclasses whose assumptions need a finer sort (Isabelle class bug).

lemma *below-order*: *class.order below below-strict*
by *unfold-locales* (*fact below-strict-def below-refl below-trans below-antisym*)+

theorem *lub-below-iff'*: $A <<| a \Longrightarrow a \sqsubseteq x \longleftrightarrow (\forall y \in A.\ y \sqsubseteq x)$
using *is-lub-below-iff is-ub-def* **by** *blast*

theorem *lub-below-iff-b*: $f\ `\ A <<| a \Longrightarrow a \sqsubseteq x \longleftrightarrow (\forall y \in A.\ f\ y \sqsubseteq x)$
using *lub-below-iff'* **by** *blast*

definition *join* (**infixl** ‹⊔› *65*) **where**
$a \sqcup b \equiv lub\ \{a,\ b\}$

lemma *join-sym*: $a \sqcup b = b \sqcup a$
by (*simp add*: *join-def insert-commute*)

lemma *join-idem*[*simp*]: $a \sqcup a = a$
by (*simp add*: *join-def*)

lemma *join-absorb*: $a \sqsubseteq b \Longrightarrow a \sqcup b = b$
by (*simp add*: *join-def lub-bin*)

lemma *join-absorb2*: $a \sqsubseteq b \Longrightarrow b \sqcup a = b$
by (*simp add*: *join-sym join-absorb*)

lemma *bin-join-ex-symm*: $\{a,\ b\} <<| z \longleftrightarrow \{b,\ a\} <<| z$
unfolding *insert-commute* **..**

lemma *binLubI*:
assumes $a \sqsubseteq w\ b \sqsubseteq w\ \bigwedge z.\ a \sqsubseteq z \Longrightarrow b \sqsubseteq z \Longrightarrow w \sqsubseteq z$
shows $\{a,\ b\} <<| w$
unfolding *is-lub-def is-ub-def* **using** *assms* **by** *blast*

definition *is-glb A a* $\equiv \forall b.\ (\forall x \in A.\ b \sqsubseteq x) \longleftrightarrow b \sqsubseteq a$

definition *glb A* $\equiv$ *THE a. is-glb A a*

lemma *is-glbI*:
assumes $\bigwedge x.\ x \in A \Longrightarrow a \sqsubseteq x$
and $\bigwedge b.\ \forall x \in A.\ b \sqsubseteq x \Longrightarrow b \sqsubseteq a$
shows *is-glb A a*
unfolding *is-glb-def*
using *assms*
by (*auto simp add*: *is-glb-def dest*: *below-trans*)

lemma *is-glb-unique*: *is-glb A a* $\Longrightarrow$ *is-glb A b* $\Longrightarrow a = b$
unfolding *is-glb-def* **by** (*blast intro*: *below-antisym*)

lemma *is-glb-glb*: *is-glb A a* $\Longrightarrow$ *is-glb A* (*glb A*)
unfolding *glb-def* **by** (*rule theI*[*OF - is-glb-unique*])

lemma *glb-eqI*: *is-glb A a* $\Longrightarrow$ *glb A* $= a$

using *is-glb-unique*[*OF is-glb-glb*] **.**

lemma *is-glb-below*: *is-glb S a* $\Longrightarrow$ $x \in S \Longrightarrow a \sqsubseteq x$
unfolding *is-glb-def* **by** *auto*

lemma *is-glb-greatest*: *is-glb S a* $\Longrightarrow$ $(\forall x \in S.\ b \sqsubseteq x) \Longrightarrow b \sqsubseteq a$
unfolding *is-glb-def* **by** *auto*

definition *meet* :: $'a \Rightarrow 'a \Rightarrow 'a$ (**infixl** ‹⊓› *70*)
where $a \sqcap b = glb\ \{a, b\}$

lemma *meet-idem*: $a \sqcap a = a$
unfolding *meet-def*
by (*rule glb-eqI*) (*simp add*: *is-glb-def*)

lemma *meet-sym*: $a \sqcap b = b \sqcap a$
by (*simp add*: *meet-def insert-commute*)

lemma *meet-absorb*: $a \sqsubseteq b \Longrightarrow a \sqcap b = a$
unfolding *meet-def*
by (*rule glb-eqI*) (*auto simp add*: *is-glb-def dest*: *below-trans*)

lemma *meet-absorb2*: $a \sqsubseteq b \Longrightarrow b \sqcap a = a$
by (*simp add*: *meet-sym meet-absorb*)

end

Binder syntax for *glb*, mirroring HOLCF's $\bigsqcup$ for *lub*. HOL's $\bigsqcap$ for *Inf* lives in the bundle *lattice-syntax*, which *Main* leaves switched off.

syntax (*ASCII*)
-BGlb :: [*pttrn*, *'a set*, *'b*] $\Rightarrow$ *'b* (‹(‹*indent=3 notation=‹binder GLB›› GLB* -:-./ -)› [*0*,*0*, *10*] *10*)

syntax
-BGlb :: [*pttrn*, *'a set*, *'b*] $\Rightarrow$ *'b* (‹(‹*indent=3 notation=‹binder* $\bigsqcap$*››*$\bigsqcap$ -∈-./ -)› [*0*,*0*, *10*] *10*)

syntax-consts
-BGlb $\rightleftharpoons$ *glb*

translations
GLB x:*A*. *t* $\rightleftharpoons$ *CONST glb* ((λ*x*. *t*) ‘ *A*)

context *po*
begin

abbreviation *Glb* (**binder** ‹$\bigsqcap$› *10*)
where $\bigsqcap n.\ t\ n \equiv glb\ (range\ t)$

end

A constant defined in a class context exists twice: at locale level (*po.meet* ($\sqsubseteq$)) and at class level (($\sqcap$)), the latter introduced by a kernel definition stored as *po-class.meet-dict*: ($\sqcap$) $\equiv$ *po.meet* ($\sqsubseteq$). Collected here (turned around) for use as *rewrites* when interpreting our class locales at concrete types, whose obligations and facts otherwise come out in locale vocabulary.

named-theorems *class-locale-eqs* ‹*locale−level vs class−level copies of class−context constants*›

lemmas *po-locale-eqs* [*class-locale-eqs*] =
po-class.is-ub-dict[*symmetric*] *po-class.is-lub-dict*[*symmetric*] *po-class.lub-dict*[*symmetric*]
po-class.is-glb-dict[*symmetric*] *po-class.glb-dict*[*symmetric*]
po-class.meet-dict[*symmetric*] *po-class.join-dict*[*symmetric*]

lemma (**in** *pcpo*) *join-bottom*[*simp*]: $\bot \sqcup a = a$ $a \sqcup \bot = a$
by (*simp-all add*: *join-absorb join-absorb2*)

lemma *obtain-compact-chain*:
fixes x :: *'a*::*profinite*
obtains X **where** *chain* X $\forall i.$ *compact* $(X\ i)$ $x = (\bigsqcup i.\ X\ i)$
proof −
from *compact-basis.obtain-principal-chain*
obtain B' **where** B': $\forall i.\ B'\ i \sqsubseteq B'\ (Suc\ i)$ $up{\cdot}x = (\bigsqcup i.$ *Rep-compact-basis* $(B'\ i))$
.
define X' **where** $X'\ i =$ *Rep-compact-basis* $(B'\ i)$ **for** i
have *chain* X' **unfolding** X'-*def* **using** $B'(1)$ **by** (*rule compact-basis.ch2ch-principal*)
have *upx*: $up{\cdot}x = (\bigsqcup i.\ X'\ i)$ **unfolding** X'-*def* **by** *fact*
have *compact* $(X'\ i)$ **for** i **unfolding** X'-*def* **by** *simp*
obtain X n **where** *upX*: $\forall i.\ up{\cdot}(X\ i) = X'\ (i + n)$
and *chain* X **and** x: $x = (\bigsqcup i.\ X\ i)$
using ‹*chain* X'›
proof (*cases rule*: *up-chain-cases*)
case *1*
then have $(\bigsqcup i.\ X'\ i) = \bot$ **by** *simp*
with *upx* **show** *?thesis* **by** *simp*
next
case (*2* X n)
with *upx* **show** *?thesis* **using** *that* **by** *simp*
qed
from *upX* ‹$\bigwedge i.$ *compact* $(X'\ i)$› *compact-upD* **have** $\forall i.$ *compact* $(X\ i)$
by *metis*
from ‹*chain* X› *this* x **show** *?thesis* **by** (*rule that*)
qed

lemma *compact-induct'*:
fixes x :: *'a*::*profinite*

assumes *adm*: *adm P*
assumes *compact*: $\bigwedge xx.$ *compact xx* $\Longrightarrow$ *xx* $\sqsubseteq$ *x* $\Longrightarrow$ *P xx*
shows *P x*
proof −
obtain *X* **where** *X*: *chain X* $\forall\, i.$ *compact* (*X i*) *x* = ($\bigsqcup i.$ *X i*)
by (*rule obtain-compact-chain*)
from *X*(*1*, *3*) **have** $\bigwedge i.$ *X i* $\sqsubseteq$ *x* **using** *is-ub-thelub* **by** *blast*
from *X*(*2*) *this* **have** $\bigwedge i.$ *P* (*X i*) **using** *compact* **by** *blast*
with *adm*[*THEN admD*, *OF X*(*1*)] *X*(*3*) **show** *P x* **by** *presburger*
qed

0.1 Step functions

The step function *stepfn c y* is *y* above *c* and ⊥ elsewhere; it is continuous when *c* is compact. It lies below *f* exactly when *y* $\sqsubseteq$ *f*·*c*.

definition *stepfn c y* ≡ (Λ *x*. *if c* $\sqsubseteq$ *x then y else* ⊥)

lemma *monofun-stepfn*: *monofun* (λ*x*::′*a*::*cpo*. *if c* $\sqsubseteq$ *x then y else* ⊥)
by (*rule monofunI*) (*auto dest*: *below-trans*)

lemma *cont-stepfn*: *cont* (λ*x*::′*a*::*cpo*. *if c* $\sqsubseteq$ *x then y else* ⊥) (**is** *cont ?f*) **if** *compact c*
proof (*rule contI2*)
show *monofun ?f* **by** (*rule monofun-stepfn*)
fix *Y* :: *nat* ⇒ ′*a* **assume** *chain Y* **and** *chain-F*: *chain* (λ*i*. *?f* (*Y i*))
show *?f* ($\bigsqcup i.$ *Y i*) $\sqsubseteq$ ($\bigsqcup i.$ *?f* (*Y i*))
proof (*cases c* $\sqsubseteq$ ($\bigsqcup i.$ *Y i*))
case *True*
with ‹*compact c*› ‹*chain Y*› **have** ∃ *i*. *c* $\sqsubseteq$ *Y i* **by** (*rule compactD2*)
then obtain *i* **where** *i*: *c* $\sqsubseteq$ *Y i* **..**
from *i* ‹*chain Y*›[*THEN chain-mono*] **have** *Y-mono*: *i* ≤ *j* $\Longrightarrow$ *c* $\sqsubseteq$ *Y j* **for** *j*
by (*rule below-trans*)
from *i* **have** *1*: *?f* (*Y i*) = *y* **by** *simp*
with *Y-mono* **have** *i* ≤ *j* $\Longrightarrow$ *?f* (*Y i*) = *?f* (*Y j*) **for** *j* **by** *simp*
then have *max*: *max-in-chain i* (λ*i*. *?f* (*Y i*)) **by** (*rule max-in-chainI*)
from *maxinch-is-thelub*[*OF chain-F*] *this 1* **have** *1*: ($\bigsqcup i.$ *?f* (*Y i*)) = *y*
by *presburger*
from *True* **have** *2*: *?f* ($\bigsqcup i.$ *Y i*) = *y* **by** *simp*
show *?thesis* **unfolding** *1 2* **by** (*rule below-refl*)
next
case *False*
then have *1*: *?f* ($\bigsqcup i.$ *Y i*) = ⊥ **by** *simp*
show *?thesis* **unfolding** *1* **by** (*rule minimal*)
qed
qed

lemma *stepfn-beta*: *stepfn c y*·*x* = (*if c* $\sqsubseteq$ *x then y else* ⊥) **if** *compact c*
unfolding *stepfn-def*
by (*rule beta-cfun*) (*rule cont-stepfn*[*OF that*])

lemma *stepfn-below-iff*: **assumes** *compact c* **shows** *stepfn c y* $\sqsubseteq$ *f* $\longleftrightarrow$ *y* $\sqsubseteq$ *f*·*c*
unfolding *cfun-below-iff stepfn-beta*[*OF assms*]
apply (*simp only*: *minimal split*: *if-splits*)
by *simp* (*blast intro*: *monofun-cfun-arg*[*of c - f*] *below-trans*[*of y f*·*c*])

end
theory *Directed* **imports** *HOLCF FiniteCF HOL.Zorn* **begin**

default-sort *type*

context *po* **begin**
definition *directed* :: *'a set* ⇒ *bool* **where** *directed S* ≡ ∀ *x* ∈ *S*. ∀ *y* ∈ *S*. ∃ *z* ∈ *S*. *x* $\sqsubseteq$ *z* ∧ *y* $\sqsubseteq$ *z*

lemma *directedI*[*intro*]:
assumes ⋀*x y*. *x* ∈ *S* ⟹ *y* ∈ *S* ⟹ ∃ *z* ∈ *S*. *x* $\sqsubseteq$ *z* ∧ *y* $\sqsubseteq$ *z*
shows *directed S*
using *assms* **by** (*simp add*: *directed-def*)

lemma *directedE*[*elim*]:
assumes *directed S x* ∈ *S y* ∈ *S*
obtains *z* **where** *z* ∈ *S x* $\sqsubseteq$ *z y* $\sqsubseteq$ *z*
using *assms* **by** (*auto simp add*: *directed-def*)

lemma *directed-finite-has-lub*:
assumes *directed*: *directed S*
and *finite-ne*: *finite S S* ≠ {}
shows ∃ *x* ∈ *S*. *S* <<| *x*
proof −
from *finite-ne*
obtain *w* **where** *w*: *w* ∈ *S*
and *w-max*: ⋀*x*. *x* ∈ *S* ⟹ *w* $\sqsubseteq$ *x* ⟹ *w* = *x*
by (*blast dest*: *finite-has-maximal*)
{
fix *x* **assume** *x*: *x* ∈ *S*
define *T* **where** *T* ≡ {*y* ∈ *S*. *x* $\sqsubseteq$ *y*}
from *finite-ne x*
have *finite T T* ≠ {} **unfolding** *T-def*
by *auto*
then obtain *wx* **where** *wx* ∈ *T*
and *wx-max'*: ⋀*y*. *y* ∈ *T* ⟹ *wx* $\sqsubseteq$ *y* ⟹ *wx* = *y*
by (*blast dest*: *finite-has-maximal*)
from ‹*wx* ∈ *T*› *T-def* **have** *x* $\sqsubseteq$ *wx wx* ∈ *S* **by** *auto*
have *wx-max*: *wx* = *y* **if** *y* ∈ *S wx* $\sqsubseteq$ *y* **for** *y*
proof −
from *that below-trans*[*OF* ‹*x* $\sqsubseteq$ *wx*›]
have *y* ∈ *T* **by** (*simp add*: *T-def*)
with *that* **show** *?thesis* **by** (*simp add*: *wx-max'*)

```
    qed
    have wx = w proof −
      from directed ‹wx ∈ S› ‹w ∈ S› obtain w′
        where w′ ∈ S wx ⊑ w′ w ⊑ w′
        by auto
      with w-max wx-max show ?thesis by blast
    qed
    with ‹x ⊑ wx› have x ⊑ w by simp
  }
  then have S <| w by (rule is-ubI)
  from this ‹w ∈ S› have S <<| w by (rule is-lub-maximal)
  with ‹w ∈ S› show ?thesis by rule
qed

corollary directed-finite-lub:
  directed S ⟹ finite S ⟹ S ≠ {} ⟹ S <<| lub S
  by (blast dest: directed-finite-has-lub intro: is-lub-lub)


corollary (in pcpo) directed-finite-lub-pcpo:
  fixes S :: 'a set
  assumes directed S finite S
  shows S <<| lub S
proof (cases S = {})
  case True
  then have S <<| ⊥ by (simp add: is-lub-def is-ub-def)
  then show ?thesis by (rule is-lub-lub)
next
  case False
  with assms show ?thesis by (rule directed-finite-lub)
qed

corollary finite-directed-lub-mem:
lub S ∈ S if directed S finite S S ≠ {}
proof −
  from that obtain w where w ∈ S S <<| w
    using directed-finite-has-lub by blast
  then show ?thesis by (subst lub-eqI)
qed

lemma directed-mono-image:
  fixes f :: 'a ⇒ 'b::po
  assumes ⋀x y. x ⊑ y ⟹ f x ⊑ f y
  directed S
shows directed (f ‘ S)
  using assms(2)
  unfolding directed-def po-class.directed-def
  by (blast intro: assms(1))
```

```
lemma chain-directed: directed (range C) if chain C
proof (rule, clarsimp)
  fix i j
  from chain-mono[OF that]
  show ∃ k. C i ⊑ C k ∧ C j ⊑ C k
    by (cases rule: le-cases[of i j]) blast+
qed

end

class dcpo = po +
  assumes dcpo: ⋀S. directed S ⟹ ∃ x. S <<| x
begin
theorem directed-lub: directed S ⟹ S <<| lub S
  using dcpo is-lub-lub by auto
end

subclass (in dcpo) cpo
proof
  fix S :: nat ⇒ 'a
  assume chain S
  from chain-directed[OF this] show ∃ x. range S <<| x
    using directed-lub by blast
qed

subclass (in dcpo) pcpo
proof
  have directed {} by auto
  then obtain x where {} <<| x using dcpo by blast
  then have x ⊑ y for y using is-lubD2 is-ubI by simp
  then show ∃ x. ∀ y. x ⊑ y by auto
qed

instance bifinite ⊆ dcpo
proof
  fix S :: 'a set
  assume directed S
  obtain a :: nat ⇒ 'a → 'a where approx-chain a
    using bifinite ..
  then interpret approx-chain a .
  define T where T i = (λx. a i·x) ‘ S for i
  {
    fix i
    have directed (T i)
      using monofun-cfun-arg ‹directed S›
      unfolding T-def
      by (rule directed-mono-image)
    moreover have finite (T i)
      by (simp add: T-def finite-deflation.finite-image)
```

```
    ultimately have T i <<| lub (T i)
      by (rule directed-finite-lub-pcpo)
  }
  from approx-lub-family[OF this[unfolded T-def]]
  show ∃x. S <<| x ..
qed

primrec chainify :: 'a::po set ⇒ (nat ⇒ 'a) ⇒ (nat ⇒ 'a)
  where chainify A f 0 = f 0
  | chainify A f (Suc n) = (SOME x. x ∈ A ∧ chainify A f n ⊑ x ∧ f (Suc n) ⊑
x)

lemma chainifyPropI:
  assumes directed S
    f (Suc i) ∈ S chainify S f i ∈ S
  shows chainify S f (Suc i) ∈ S
    ∧ chainify S f i ⊑ chainify S f (Suc i)
    ∧ f (Suc i) ⊑ chainify S f (Suc i)
proof −
  from assms obtain x where x ∈ S ∧ chainify S f i ⊑ x ∧ f (Suc i) ⊑ x
    by auto
  from someI[of λx. x ∈ S ∧ chainify S f i ⊑ x ∧ f (Suc i) ⊑ x, OF this]
  show ?thesis by simp
qed

lemma chainify-directed-mem:
  assumes directed S ⋀i. f i ∈ S
  shows chainify S f i ∈ S
proof (induct i)
  case 0
  then show ?case by (simp add: assms(2))
next
  case (Suc i)
  from chainifyPropI[OF assms this] show ?case by blast
qed

lemma chainify-directed-chain:
  assumes directed S ⋀i. f i ∈ S
  shows chain (chainify S f)
proof (rule chainI)
  fix i
  have chainify S f i ∈ S
    using assms by (rule chainify-directed-mem)
  from chainifyPropI[OF assms this]
  show chainify S f i ⊑ chainify S f (Suc i) by simp
qed

lemma chainify-directed-below:
  assumes directed S ⋀i. f i ∈ S
```

```
  shows f i ⊑ chainify S f i
proof (cases i)
  case 0
  then show ?thesis by simp
next
  case (Suc i)
  have chainify S f i ∈ S
    using assms by (rule chainify-directed-mem)
  from chainifyPropI[OF assms this]
  show ?thesis unfolding Suc by simp
qed

lemma lub-subset-mono:
  assumes A <<| a B <<| b A ⊆ B
  shows a ⊑ b
  using assms(3) by (intro lub-mono-general[OF assms(1,2)]) blast

lemma obtain-directed-chain:
  fixes S :: 'a::bifinite set
  assumes directed S S ≠ {}
  obtains C where chain C ∀ i. C i ∈ S (⨆ i. C i) = lub S
proof −
  obtain a :: nat ⇒ 'a → 'a where approx-chain a
    using bifinite ..
  then interpret approx-chain a .
  define T where T i = (λx. a i·x) ‘ S for i
  have directed (T i) for i
    using monofun-cfun-arg ‹directed S›
    unfolding T-def
    by (rule directed-mono-image)
  moreover have finite (T i) for i
    by (simp add: T-def finite-deflation.finite-image)
  ultimately have T-lub: T i <<| lub (T i) for i
    by (rule directed-finite-lub-pcpo)
  then have SleT: S <<| (⨆ i. lub (T i))
    unfolding T-def
    by (rule approx-lub-family)
  from T-lub have chainT: chain (λi. lub (T i))
    unfolding T-def
    by (rule approx-ch2ch-lub)
  have T i ≠ {} for i by (simp add: ‹S ≠ {}› T-def)
  with ‹⋀i. directed (T i)› ‹⋀i. finite (T i)›
  have lub (T i) ∈ T i for i
    by (rule finite-directed-lub-mem)
  then have ∃ x ∈ S. a i·x = lub (T i) for i
    unfolding T-def using imageE by metis
  with approx-below have ∃ x ∈ S. lub (T i) ⊑ x for i
    by metis
  then obtain f where f: f i ∈ S and f-above: lub (T i) ⊑ f i for i
```

```
    by metis
  define C where C i ≡ chainify S f i for i
  from ‹directed S› f have
    Cmem: ⋀i. C i ∈ S
    and chainC: chain C
    and aboveC: ⋀i. f i ⊑ C i
    unfolding C-def
    by (auto intro:
        chainify-directed-mem chainify-directed-chain
        chainify-directed-below)
  from f-above aboveC have lub (T i) ⊑ C i for i
    by (rule below-trans)
  with chainT chainC have TleC: (⨆ i. lub (T i)) ⊑ (⨆ i. C i)
    by (rule lub-mono)
  from SleT have S <<| lub S by (rule is-lub-lub)
  with cpo-lubI[OF chainC] have CleS: (⨆ i. C i) ⊑ lub S
    by (blast intro: Cmem lub-subset-mono)
  from SleT have lub S = (⨆ i. lub (T i)) by (rule lub-eqI)
  with CleS TleC below-antisym have (⨆ i. C i) = lub S by auto
  from chainC Cmem this show thesis using that by presburger
qed
```

In a bifinite (hence directed-complete) domain a continuous function preserves *directed* lubs, not just lubs of chains: the image of a nonempty directed set has the image of the lub as its least upper bound. The directed set is replaced by a cofinal chain via $[\![directed\ ?S;\ ?S \neq \{\};\ \bigwedge C.\ [\![chain\ C;\ \forall i.\ C\ i \in ?S;\ (\bigsqcup i.\ C\ i) = lub\ ?S]\!] \Longrightarrow ?thesis]\!] \Longrightarrow ?thesis$, on which ordinary continuity applies.

Combined with distributivity this shows that binary meet preserves *all* existing lubs finite ones by the distributive law *meet-join-distrib*, directed ones by this lemma.

```
lemma cont-directed-is-lub:
  fixes S :: 'a::bifinite set
  assumes cont: cont f and dir: directed S and ne: S ≠ {}
  shows f ‘ S <<| f (lub S)
proof −
  have mono: monofun f using cont by (rule cont2mono)
  from dir ne obtain C where chainC: chain C and Cmem: ∀ i. C i ∈ S
    and Club: (⨆ i. C i) = lub S
    by (rule obtain-directed-chain)
  have fC: f (lub S) = (⨆ i. f (C i))
    using Club cont2contlubE[OF cont chainC] by simp
  show f ‘ S <<| f (lub S)
  proof (rule is-lubI)
    show f ‘ S <| f (lub S)
    proof (rule is-ubI)
      fix z assume z ∈ f ‘ S
      then obtain s where s: s ∈ S and z: z = f s by blast
```

```
    from dir have S <<| lub S by (rule directed-lub)
    with s have s ⊑ lub S by (auto dest: is-lubD1 is-ubD)
    then have f s ⊑ f (lub S) by (rule monofunE[OF mono])
    with z show z ⊑ f (lub S) by simp
   qed
  next
   fix u assume ub: f ‘ S <| u
   have chfC: chain (λi. f (C i)) using mono chainC by (rule ch2ch-monofun)
   have (⨆ i. f (C i)) ⊑ u
   proof (rule lub-below[OF chfC])
    fix i
    from Cmem have f (C i) ∈ f ‘ S by blast
    with ub show f (C i) ⊑ u by (rule is-ubD)
   qed
   with fC show f (lub S) ⊑ u by simp
  qed
qed

corollary cont-directed-lub:
  fixes S :: 'a::bifinite set
  assumes cont f and directed S and S ≠ {}
  shows lub (f ‘ S) = f (lub S)
  using cont-directed-is-lub[OF assms] by (rule lub-eqI)

lemma directed-admD:
  fixes D :: 'a::bifinite set
  assumes adm P and D: directed D D ≠ {}
    and forall: ∀ x ∈ D. P x
  shows P (lub D)
proof −
  from obtain-directed-chain[OF D]
  obtain C where chain C and mem: ∀ i. C i ∈ D and lub: (⨆ i. C i) = lub D .
  from forall mem have ∀ i. P (C i) by fast
  with admD[OF ‹adm P› ‹chain C›] lub
  show P (lub D) by simp
qed

definition Chain C ≡ ∀ a ∈ C. ∀ b ∈ C. a ⊑ b ∨ b ⊑ a

lemma ChainE: assumes Chain C a ∈ C b ∈ C
  obtains a ⊑ b | b ⊑ a
  using assms unfolding Chain-def by auto

lemma Chain-directedI: Chain C ⟹ directed C
  unfolding Chain-def directed-def by metis

lemma Chain-mono-image: monofun f ⟹ Chain C ⟹ Chain (f ‘ C)
  unfolding Chain-def
  by (auto dest: monofunE)
```

lemma *chain-ChainI*: *Chain* (*range C*) **if** *chain C*
unfolding *Chain-def*
proof (*intro strip*)
fix *a b* **assume** *a* ∈ *range C b* ∈ *range C*
then obtain *i j* **where** *ab*: *a* = *C i b* = *C j* **by** *blast*
from *chain-mono*[*OF that*] **have** *C i* ⊑ *C j* ∨ *C j* ⊑ *C i*
by (*cases rule*: *le-cases*[*of i j*]) *auto*
with *ab* **show** *a* ⊑ *b* ∨ *b* ⊑ *a* **by** *simp*
qed

lemma *po-Zorn-lemma*:
fixes *A* :: *'a*::*po set*
assumes ⋀*C*. *Chain C* ⟹ *C* ⊆ *A* ⟹ ∃ *u* ∈ *A*. *C* <| *u*
shows ∃ *m* ∈ *A*. ∀ *a* ∈ *A*. *m* ⊑ *a* ⟶ *a* = *m*
proof (*rule predicate-Zorn*)
show *partial-order-on A* (*relation-of below A*)
by (*rule partial-order-on-relation-ofI*)
(*auto intro*: *below-trans below-antisym*)
next
fix *C* **assume** *C* ∈ *Zorn.Chains* (*relation-of below A*)
then have *Chain C C* ⊆ *A*
by (*auto simp add*: *Zorn.Chains-def Chain-def relation-of-def*)
then have ∃ *u* ∈ *A*. *C* <| *u* **by** (*rule assms*)
then show ∃ *u* ∈ *A*. ∀ *a* ∈ *C*. *a* ⊑ *u* **by** (*simp add*: *is-ub-def*)
qed

theorem *obtain-total-above*:
fixes *S* :: *'a*::*po set* **and** *x* :: *'a*
assumes *x*: *x* ∈ *S* **and** *directed*: ⋀*T*. *T* ⊆ *S* ⟹ *directed T* ⟹ ∃ *a* ∈ *S*. *T* <|
a
obtains *y* **where** *y* ∈ *S x* ⊑ *y* ∀ *z* ∈ *S*. *y* ⊑ *z* ⟶ *y* = *z*
proof −
define *S-above* **where** *S-above* = {*y* ∈ *S*. *x* ⊑ *y*}
have *S-above-iff*: *y* ∈ *S-above* ⟷ *x* ⊑ *y* ∧ *y* ∈ *S* **for** *y*
by (*auto simp add*: *S-above-def*)
have ∃ *m* ∈ *S-above*. ∀ *y* ∈ *S-above*. *m* ⊑ *y* ⟶ *y* = *m*
proof (*rule po-Zorn-lemma*)
fix *C* **assume** *Chain C C* ⊆ *S-above*
then have *C* ⊆ *S directed C*
by (*auto intro*: *Chain-directedI simp*: *S-above-def*)
then obtain *a* **where** *a* ∈ *S C* <| *a*
by (*blast dest*: *directed*)
show ∃ *u* ∈ *S-above*. *C* <| *u*
proof (*cases C* = {})
case *True*
then show *?thesis*
by (*intro bexI*[**where** *x*=*x*]) (*auto simp add*: *x S-above-def*)
next

```
      case False
      then obtain y where y ∈ C by blast
      with ‹C <| a› have y ⊑ a by (rule is-ubD)
      from ‹y ∈ C› ‹C ⊆ S-above› S-above-def
      have x ⊑ y by auto
      from below-trans[OF this ‹y ⊑ a›] ‹a ∈ S›
      have a ∈ S-above by (simp add: S-above-def)
      from this ‹C <| a› show ?thesis by blast
    qed
  qed
  then obtain m where m ∈ S-above and m-max: ∀ y. y ∈ S-above ⟶ m ⊑ y
⟶ m = y
    by blast
  from ‹m ∈ S-above› have m ∈ S x ⊑ m by (simp-all add: S-above-iff)
  have m = y if y ∈ S m ⊑ y for y
  proof −
    from ‹x ⊑ m› ‹m ⊑ y› have x ⊑ y by (rule below-trans)
    with ‹y ∈ S› have y ∈ S-above by (simp add: S-above-iff)
    with m-max ‹m ⊑ y› show m = y by blast
  qed
  from ‹m ∈ S› ‹x ⊑ m› this
  show thesis
    using that[of m]
    by blast
qed

lemma directed-join-of-finite-subsets:
  fixes A :: 'a::dcpo set
  assumes lubs: ⋀B. B ∈ Fpow(A) ⟹ B <<| lub B
  shows directed-lubs: directed (lub ‘ Fpow(A))
    and lub-lub: A <<| lub (lub ‘ Fpow(A))
proof −
  show directed (lub ‘ Fpow(A))
  proof (intro directedI, safe)
    fix x y assume xy: x ∈ Fpow(A) y ∈ Fpow(A)
    then have un: x ∪ y ∈ Fpow(A) by (auto simp add: Fpow-def)
    with xy lubs have lub x ⊑ lub (x ∪ y) ∧ lub y ⊑ lub (x ∪ y)
      using lub-subset-mono by fast
    with un show ∃ z ∈ lub ‘ Fpow(A). lub x ⊑ z ∧ lub y ⊑ z by fast
  qed
  then have pa-lub: lub ‘ Fpow(A) <<| lub (lub ‘ Fpow(A)) by (rule directed-lub)
  show A <<| lub (lub ‘ Fpow(A))
  proof (intro is-lubI is-ubI)
    fix x assume x ∈ A
    then have {x} ∈ Fpow(A) by (simp add: Fpow-def)
    then have x ∈ lub ‘ Fpow(A) by force
    with pa-lub[THEN is-lubD1, THEN is-ubD] show x ⊑ lub (lub ‘ Fpow(A)) .
  next
    fix u assume A <| u
```

then have $B \in Fpow(A) \Longrightarrow B <| u$ **for** B **by** (*auto simp add*: *Fpow-def is-ub-def*)
with *lubs*[*THEN is-lubD2*] **have** $B \in Fpow(A) \Longrightarrow lub\ B \sqsubseteq u$ **for** B **by** *simp*
then have $lub\ `\ Fpow(A) <| u$ **by** (*rule ub-imageI*)
with *pa-lub*[*THEN is-lubD2*] **show** $lub\ (lub\ `\ Fpow(A)) \sqsubseteq u$ **.**
qed
qed

1 Lemmas concerning completion, complete sets and closed sets

context fixes $A :: 'a::bifinite\ set$
begin

definition $chain\text{-}complete \equiv \forall C.\ chain\ C \wedge range\ C \subseteq A \longrightarrow Lub\ C \in A$

definition $dcomplete \equiv \forall B.\ B \subseteq A \wedge directed\ B \wedge B \neq \{\} \longrightarrow lub\ B \in A$

lemma *chain-complete-iff-dcomplete*:
chain-complete $\longleftrightarrow$ *dcomplete*
unfolding *chain-complete-def dcomplete-def*
apply (*rule iffI*)
using *obtain-directed-chain*
apply (*metis image-subsetI subsetD*)
using *chain-directed* **by** *blast*

end

A compact element below the lub of a directed set is below a member; the approximants of the lub are the union of the approximants.

lemma *compact-directedD*:
fixes $x :: 'a::bifinite$
assumes *D*: *directed D* $D \neq \{\}$ **and** *x*: *compact x*
shows $x \sqsubseteq lub\ D \longleftrightarrow (\exists y \in D.\ x \sqsubseteq y)$
proof (*safe*)
assume $x \sqsubseteq lub\ D$
from *obtain-directed-chain*[*OF D*]
obtain *C* **where** *chain C* $\forall i.\ C\ i \in D$ $(\bigsqcup i.\ C\ i) = lub\ D$ **.**
with $x\ \langle x \sqsubseteq lub\ D \rangle$ **have** $\exists i.\ x \sqsubseteq C\ i$
by (*auto intro*: *compactD2*[*of x C*])
with $\langle \forall i.\ C\ i \in D \rangle$ **show** $\exists y \in D.\ x \sqsubseteq y$ **by** *fast*
next
fix *y* **assume** $y \in D$
with *directed-lub*[*OF D*(*1*)] *is-lubD1 is-ubD* **have** $y \sqsubseteq lub\ D$ **by** *blast*
moreover assume $x \sqsubseteq y$
ultimately show $x \sqsubseteq lub\ D$ **by** (*rule below-trans*[*rotated*])
qed

lemma *directed-approximants*:
fixes $D :: ('a::bifinite)\ set$
assumes *directed* D $D \neq \{\}$
shows *approximants* (*lub* D) $= (\bigcup x \in D.\ approximants\ x)$
unfolding *approximants-def*
using *compact-directedD*[*OF assms*] **by** *auto*

end
theory *Bounded-Complete* **imports** *HOLCF Top Definitions FiniteCF Directed*
begin

default-sort *po*

class *bounded-complete* = *pcpo* +
assumes *ex-bound-ne-ex-lub*: $A \neq \{\} \Longrightarrow A <| b \Longrightarrow \exists a.\ A <<| a$
begin

lemma *ex-bound-ex-lub*: $A <| b \Longrightarrow \exists a.\ A <<| a$
by (*cases* $A = \{\}$) (*auto intro*: *ex-bound-ne-ex-lub lub-empty*)

lemma *ex-bound-lub*: $A <| b \Longrightarrow A <<| lub\ A$
using *ex-bound-ex-lub*[*THEN exE*] *is-lub-lub* **by** *blast*

lemma *join-is-lub*: $x \sqsubseteq z \Longrightarrow y \sqsubseteq z \Longrightarrow \{x, y\} <<| x \sqcup y$
using *ex-bound-lub*[*of* $\{x, y\}$ z] **by** (*simp add*: *join-def*)

lemma *join-belowI*: $x \sqsubseteq z \Longrightarrow y \sqsubseteq z \Longrightarrow x \sqcup y \sqsubseteq z$
using *join-is-lub*[*THEN lub-below-iff'*]
by *simp*

lemma *lub-belowI*: $A \neq \{\} \Longrightarrow A <| b \Longrightarrow lub\ A \sqsubseteq b$
using *ex-bound-lub*[*THEN is-lubD2*]
by *simp*

lemma *below-lubI*: $A <| b \Longrightarrow x \in A \Longrightarrow x \sqsubseteq lub\ A$
using *ex-bound-lub*[*THEN is-lubD1*, *THEN is-ubD*] **.**

lemma *is-glb*: **fixes** $S :: 'a\ set$ **assumes** $S \neq \{\}$
obtains s **where** *is-glb* S s
proof −
from *assms* **obtain** y **where** $y \in S$ **by** *blast*
define B **where** $B \equiv \{x.\ \forall y \in S.\ x \sqsubseteq y\}$
have $\bot \in B$ **unfolding** *B-def* **by** *simp*
then have *1*: $B \neq \{\}$ **by** *auto*
from ‹$y \in S$› **have** *2*: $B <| y$ **by** (*simp add*: *B-def is-ub-def*)
from *1 2* **obtain** s **where** s: $B <<| s$ **using** *ex-bound-ex-lub* **by** *auto*
have *sinB*: $s \sqsubseteq y$ **if** $y \in S$ **for** y
proof −

```
    from that have B <| y by (simp add: B-def is-ub-def)
    with s show ?thesis by (rule is-lubD2)
  qed
  have x ⊑ s ⟷ (∀ y ∈ S. x ⊑ y) for x
  proof
    show x ⊑ s ⟹ (∀ y ∈ S. x ⊑ y)
      using below-trans[of x s, OF - sinB] by blast
    show (∀ y ∈ S. x ⊑ y) ⟹ x ⊑ s
      using s[THEN is-lubD1, THEN is-ubD, unfolded B-def]
      by simp
  qed
  with ‹S ≠ {}› have is-glb S s unfolding is-glb-def by simp
  then show ?thesis by (rule that)
qed

lemma glb: fixes S :: 'a set
  assumes S ≠ {}
  shows is-glb S (glb S)
  using is-glb[OF assms] by (blast intro: is-glb-glb)

lemma glb-below: x ∈ S ⟹ glb S ⊑ x
  by (auto intro: glb is-glb-below)

lemma glb-greatest: S ≠ {} ⟹ (∀ x ∈ S. b ⊑ x) ⟹ b ⊑ glb S
  by (auto intro: glb is-glb-greatest)

lemma glb-below-iff: S ≠ {} ⟹ (b ⊑ glb S) = (∀ x ∈ S. b ⊑ x)
  by (drule glb) (simp add: is-glb-def)

lemma glb-eq-lub:
  assumes S ≠ {}
  shows glb S = lub {x. ∀ y ∈ S. x ⊑ y} (is - = lub ?B)
proof (rule glb-eqI, rule is-glbI)
  obtain y where y ∈ S using assms by blast
  then have ?B <| y by (simp add: is-ub-def)
  then have B: ?B <<| lub ?B by (rule ex-bound-lub)
  show lub ?B ⊑ x if x ∈ S for x
    by (rule is-lubD2[OF B]) (simp add: is-ub-def that)
  show b ⊑ lub ?B if ∀ x ∈ S. b ⊑ x for b
    by (rule is-ubD[OF is-lubD1[OF B]]) (simp add: that)
qed

lemma meet-below1: a ⊓ b ⊑ a and meet-below2: a ⊓ b ⊑ b
  unfolding meet-def
  by (fast intro: glb-below)+

lemma meet-below-iff: c ⊑ a ⊓ b ⟷ c ⊑ a ∧ c ⊑ b
  unfolding meet-def
  by (auto simp add: glb-below-iff)
```

lemma *meet-greatest*: $c \sqsubseteq a \Longrightarrow c \sqsubseteq b \Longrightarrow c \sqsubseteq a \sqcap b$
by (*auto simp add*: *meet-below-iff*)

lemma *meet-sym*: $a \sqcap b = b \sqcap a$
unfolding *meet-def*
by (*simp only*: *insert-commute*)

definition *joinable* (**infix** $\Uparrow$ *50*)
where $a \Uparrow b \equiv \exists z.\ a \sqsubseteq z \wedge b \sqsubseteq z$

lemma *joinable-lub*: $a \Uparrow b \Longrightarrow \{a, b\} <<| a \sqcup b$
unfolding *joinable-def*
by (*fast intro*: *join-is-lub*)

lemma *joinableI*: $a \sqsubseteq c \Longrightarrow b \sqsubseteq c \Longrightarrow a \Uparrow b$
unfolding *joinable-def*
by *fast*

lemma *joinableD*: $a \Uparrow b \Longrightarrow a \sqsubseteq a \sqcup b\ a \Uparrow b \Longrightarrow b \sqsubseteq a \sqcup b$
using *joinable-lub*
by (*auto dest*: *is-lubD1 is-ubD*)

lemma *joinable-mono1*: $a \Uparrow b \Longrightarrow c \sqsubseteq a \Longrightarrow c \Uparrow b$
apply (*frule joinableD*(*1*), *drule joinableD*(*2*))
apply (*rule joinableI*[*of c a* $\sqcup$ *b b*])
apply (*rule below-trans*[*of c a*])
.

lemma *joinable-refl*[*simp*]: $a \Uparrow a$
unfolding *joinable-def join-def* **by** *fast*

lemma *joinable-sym*: $a \Uparrow b \longleftrightarrow b \Uparrow a$
unfolding *joinable-def*
by *fast*

lemma *joinable-mono2*: $a \Uparrow b \Longrightarrow c \sqsubseteq b \Longrightarrow a \Uparrow c$
using *joinable-mono1 joinable-sym* **by** *metis*

A fact rather than a *sublocale*, for the same reason as *below-order*.

lemma *semilattice-meet*: *class.semilattice-inf meet below below-strict*
by (*rule class.semilattice-inf.intro*[*OF below-order*])
(*unfold-locales*, (*fact meet-below1 meet-below2 meet-greatest*)+)

lemma *join-mono1*: $c \sqcup b \sqsubseteq a \sqcup b$ **if** $a \Uparrow b\ c \sqsubseteq a$
proof −
from *that* **have** $c \sqsubseteq a \sqcup b$ **by** (*blast intro*: *below-trans joinableD*(*1*))
also from *that* **have** $b \sqsubseteq a \sqcup b$ **by** (*blast intro*: *joinableD*(*2*))
finally(*join-belowI*) **show** *?thesis***.**

qed

lemma *join-mono2*: $a \sqcup c \sqsubseteq a \sqcup b$ **if** $a \Uparrow b\ c \sqsubseteq b$
by (*metis join-mono1 join-sym joinable-sym that*)

lemma *join-absorbI*: $a \sqsubseteq b \Longrightarrow a \sqcup b = b$
by (*meson below-antisym below-refl join-belowI joinableD*(*2*) *joinableI*)

lemma *joinable-if-below1*[*simp*]: $a \sqsubseteq b \Longrightarrow a \Uparrow b$
by (*auto intro*: *joinableI*)

lemma *joinable-if-below2*[*simp*]: $a \sqsubseteq b \Longrightarrow b \Uparrow a$
by (*auto intro*: *joinableI*)

corollary *join-absorb-meet*[*simp*]: $(a \sqcap b) \sqcup b = b$
by (*auto intro*: *join-absorbI meet-below2*)

corollary *join-absorb-meet2*[*simp*]: $a \sqcup (a \sqcap b) = a$
using *join-absorb-meet join-sym meet-sym* **by** *metis*

corollary *join-absorb-meet3*[*simp*]: $(b \sqcap a) \sqcup b = b$
by (*simp add*: *meet-sym*)

corollary *join-absorb-meet4*[*simp*]: $a \sqcup (b \sqcap a) = a$
by (*simp add*: *meet-sym*)

lemma *meet-absorbI*: $a \sqsubseteq b \Longrightarrow a \sqcap b = a$
by (*rule semilattice-inf.inf-absorb1*[*OF semilattice-meet*])

corollary *meet-absorb-join*[*simp*]: $a \Uparrow b \Longrightarrow (a \sqcup b) \sqcap a = a$
using *meet-absorbI*[*OF joinableD*(*1*)] **by** (*simp add*: *meet-sym*)

lemma *join-assoc*: **assumes** $a \Uparrow b\ b \Uparrow c\ a \sqcup b \Uparrow c\ a \Uparrow b \sqcup c$
shows $(a \sqcup b) \sqcup c = a \sqcup (b \sqcup c)$
by (*intro below-antisym join-belowI joinableD assms*)
(*metis join-belowI joinableD assms below-trans*)+

lemma *joinable-simps*[*simp*]:
assumes $a \Uparrow b$
shows
$a \Uparrow a \sqcup b\ a \sqcup b \Uparrow a$
$b \Uparrow a \sqcup b\ a \sqcup b \Uparrow b$
$a \Uparrow b \sqcap d\ a \sqcap c \Uparrow b$
$a \sqcap c \Uparrow b \sqcap d$
proof −
from *assms* **show** $a \sqcap c \Uparrow b$ **by** (*auto intro*: *joinable-mono1 meet-below1*)
then show $a \sqcap c \Uparrow b \sqcap d$ **by** (*auto intro*: *joinable-mono2 meet-below1*)
from *assms* **show** $a \Uparrow b \sqcap d$ **by** (*auto intro*: *joinable-mono2 meet-below1*)
qed (*auto intro*: *joinable-mono1 joinable-mono2 joinableD*[*OF assms*] *joinableI*)

end

lemmas *bounded-complete-locale-eqs* [*class-locale-eqs*] =
bounded-complete-class.joinable-dict[*symmetric*]

lemma *bounded-complete-finite-joinI*:
assumes $\bigwedge a\ b\ c{::}'a{::}dcpo.\ [\![\ a \sqsubseteq c;\ b \sqsubseteq c\ ]\!] \Longrightarrow \exists z.\ \{a, b\} <<| z$
shows *OFCLASS*('*a*, *bounded-complete-class*)
proof
from *assms* **have** *joinI*: $a \sqsubseteq c \Longrightarrow b \sqsubseteq c \Longrightarrow \{a, b\} <<| a \sqcup b$ **for** $a\ b\ c :: 'a$
unfolding *join-def* **using** *is-lub-lub* **by** *blast*
have $B <| z \Longrightarrow \exists b.\ B <<| b$ **if** *finite B*
for *z* **and** $B :: 'a\ set$
using *that*(*1*)
proof (*induction B rule*: *finite-induct*)
case *empty*
then show *?case* **by** (*blast intro*: *lub-empty*)
next
case (*insert x F*)
from *insert.prems* **have** *prev*: $F <| z$ **and** *x*: $x \sqsubseteq z$ **by** *simp-all*
from *insert.IH*[*OF prev*] **obtain** *w* **where** *w*: $F <<| w$ **..**
from *w prev* **have** $w \sqsubseteq z$ **by** (*rule is-lubD2*)
with *x* **have** *pairlub*: $\{x, w\} <<| x \sqcup w$ **by** (*rule joinI*)
from *is-lubD1*[*OF this*] **have** *belows*: $x \sqsubseteq x \sqcup w\ w \sqsubseteq x \sqcup w$
by *simp-all*
have (*insert x F*) $<<| x \sqcup w$
proof (*rule is-lubI*)
show *insert x F* $<| x \sqcup w$ **unfolding** *is-ub-insert*
using *is-lubD1 pairlub w is-ub-upward belows* **by** *blast*
next
fix *u* **assume** *insert x F* $<| u$
then have $F <| u$ **and** $x \sqsubseteq u$ **by** *simp-all*
with *is-lubD2*[*OF w*] **have** $\{x, w\} <| u$ **by** *fastforce*
with *is-lubD2*[*OF pairlub*] **show** $x \sqcup w \sqsubseteq u$ **.**
qed
then show *?case* **..**
qed
then have *finite-lub*: *finite* $B \Longrightarrow B <| z \Longrightarrow B <<| lub\ B$ **for** *z* **and** $B :: 'a\ set$
using *is-lub-lub* **by** *auto*
fix *A* **and** $b :: 'a$ **assume** $A \neq \{\}\ A <| b$
from ‹$A <| b$› **have** $B \in Fpow(A) \Longrightarrow B <| b$ **for** *B*
unfolding *Fpow-def is-ub-def* **by** *fast*
moreover have $B \in Fpow(A) \Longrightarrow$ *finite B* **for** *B* **by** (*simp add*: *Fpow-def*)
ultimately have $B \in Fpow(A) \Longrightarrow B <<| lub\ B$ **for** *B*
using *finite-lub* **by** *fast*
with *directed-join-of-finite-subsets*(*2*)
have $A <<| lub\ (lub\ `\ Fpow(A))$ **by** *blast*
then show $\exists a.\ A <<| a$ **..**

qed

lemma (**in** *cpo*) *bin-join-chain1*:
assumes *cX*: *chain X*
and *Z*: $\bigwedge i.\ \{X\ i,\ y\} <<|\ Z\ i$
shows *chain Z* **and** $\{\bigsqcup i.\ X\ i,\ y\} <<|\ (\bigsqcup i.\ Z\ i)$
proof −
from *Z* **show** *cZ*: *chain Z*
by (*rule ch2ch-lub-gen*) (*blast intro*: *chainE*[*OF cX*])
show $\{\bigsqcup i.\ X\ i,\ y\} <<|\ (\bigsqcup i.\ Z\ i)$
proof (*rule is-lubI*)
from *Z* **have** $X\ i \sqsubseteq Z\ i$ **for** *i* **using** *is-lubD1*[*THEN is-ubD*] **by** *blast*
with *cX cZ* **have** *1*: $(\bigsqcup i.\ X\ i) \sqsubseteq (\bigsqcup i.\ Z\ i)$ **by** (*rule lub-mono*)
from *Z* **have** $y \sqsubseteq Z\ i$ **for** *i* **using** *is-lubD1*[*THEN is-ubD*] **by** *blast*
with *cZ* **have** *2*: $y \sqsubseteq (\bigsqcup i.\ Z\ i)$ **by** (*rule below-lub*)
from *1 2* **show** $\{\bigsqcup i.\ X\ i,\ y\} <|\ (\bigsqcup i.\ Z\ i)$ **by** (*intro is-ubI*) *auto*
next
fix *u* **assume** *u*: $\{\bigsqcup i.\ X\ i,\ y\} <|\ u$
from *is-ubD*[*OF u*] **have** *1*: $(\bigsqcup i.\ X\ i) \sqsubseteq u$ **and** *2*: $y \sqsubseteq u$ **by** *auto*
from *cX 1* **have** *1*: $X\ i \sqsubseteq u$ **for** *i* **using** *lub-below-iff* **by** *blast*
from *1 2* **have** $\{X\ i,\ y\} <|\ u$ **for** *i* **by** (*intro is-ubI*) *auto*
with *Z* **have** $Z\ i \sqsubseteq u$ **for** *i* **by** (*rule is-lubD2*)
with *cZ* **show** $(\bigsqcup i.\ Z\ i) \sqsubseteq u$ **by** (*rule lub-below*)
qed
qed

lemma (**in** *cpo*) *ex-join-adm*: *adm* $(\lambda x.\ \exists z.\ \{x,\ y\} <<|\ z)$
proof (*rule admI*)
fix *X* :: $nat \Rightarrow 'a$
assume *cX*: *chain X*
assume $\forall i.\ \exists z.\ \{X\ i,\ y\} <<|\ z$
then obtain *Z* **where** *Z*: $\{X\ i,\ y\} <<|\ Z\ i$ **for** *i* **by** *metis*
with *cX bin-join-chain1*(*2*) **show** $\exists z.\ \{\bigsqcup i.\ X\ i,\ y\} <<|\ z$ **by** *blast*
qed

lemma (**in** *cpo*) *ex-join-adm2*: *adm* $(\lambda y.\ \exists z.\ \{x,\ y\} <<|\ z)$
unfolding *bin-join-ex-symm* **by** (*fact ex-join-adm*)

lemma *compact-eq-approx2*:
fixes *a b* :: *'a*::*bifinite*
assumes *compact a compact b*
obtains *d* **where** *finite-deflation d* $d{\cdot}a = a$ $d{\cdot}b = b$
proof −
from *bifinite* **obtain** *approx* :: $nat \Rightarrow 'a \to 'a$ **where** *approx-chain approx* **..**
then interpret *approx-chain approx* **.**
note *approx-mono* = *chain-mono*[*OF chain-approx*,
THEN deflation.belowD[*OF deflation-approx*]]
from *assms compact-eq-approx* **obtain** *i j* **where** *approx i*$\cdot a = a$ *approx j*$\cdot b = b$
by *presburger*

```
  with approx-mono have approx (max i j)·a = a approx (max i j)·b = b
    by (cases rule: le-cases[of i j], simp-all)+
  with finite-deflation-approx show ?thesis by (rule that)
qed

lemma compact-lubI:
  fixes a b :: 'a::bifinite
  assumes compacts: compact a compact b
  assumes compactRule: ⋀c. compact c ⟹ a ⊑ c ⟹ b ⊑ c ⟹ a ⊑ F a b ∧ b
⊑ F a b ∧ F a b ⊑ c
 and c: a ⊑ c b ⊑ c
shows {a, b} <<| F a b
proof (rule is-lubI)
  from compacts obtain d where finite-deflation d and fixing: d·a = a d·b = b
    by (rule compact-eq-approx2)
  then interpret finite-deflation d by presburger
  from c have a ⊑ d·c b ⊑ d·c
    by (subst fixing[symmetric], fast intro: monofun-cfun-arg)+
  moreover have compact (d·c) by (rule compact)
  ultimately have a ⊑ F a b b ⊑ F a b using compactRule by auto
  then show {a, b} <| F a b by simp
next
  fix u
  show {a, b} <| u ⟹ F a b ⊑ u
  proof (induction u rule: compact-induct', simp add: compacts)
    fix uu assume compact uu {a, b} <| uu
    with compactRule show F a b ⊑ uu by auto
  qed
qed
```

Bounded completeness through any chain of maps with lub *ID* (e.g. the *take* functions of a domain, or an approximation chain): it suffices that pairs of approximants have lubs under any common bound.

```
lemma chain-bounded-lub:
  fixes t :: nat ⇒ 'a::cpo → 'a
  assumes t: chain t (⨆i. t i) = ID
    and lubs: ⋀n a b c. t n·a ⊑ c ⟹ t n·b ⊑ c ⟹ ∃z. {t n·a, t n·b} <<| z
    and (a::'a) ⊑ c b ⊑ c
  shows ∃z. {a, b} <<| z
proof −
  have reach: (⨆n. t n·x) = x for x
    by (simp add: contlub-cfun-fun[OF t(1), symmetric] t(2))
  have chx: chain (λn. t n·x) for x by (rule ch2ch-Rep-cfunL[OF t(1)])
  have below: t n·x ⊑ x for n x
    using is-ub-thelub[OF chx, of n x] by (simp add: reach)
  define L where L n = lub {t n·a, t n·b} for n
  have L: {t n·a, t n·b} <<| L n for n
  proof −
    have t n·a ⊑ c t n·b ⊑ c
```

```
    using below ‹a ⊑ c› ‹b ⊑ c› by (blast intro: below-trans)+
   from lubs[OF this] obtain z where {t n·a, t n·b} <<| z by (elim exE)
   then show ?thesis unfolding L-def by (rule is-lub-lub)
  qed
  have chL: chain L
  proof (rule chainI)
   fix n
   have t n·a ⊑ L (Suc n) t n·b ⊑ L (Suc n)
    using chainE[OF chx] is-lubD1[OF L, THEN is-ubD]
    by (blast intro: below-trans)+
   then show L n ⊑ L (Suc n) by (blast intro: is-lubD2[OF L] is-ubI)
  qed
  have {a, b} <<| (⨆ n. L n)
  proof (rule is-lubI)
   have (⨆ n. t n·a) ⊑ (⨆ n. L n) (⨆ n. t n·b) ⊑ (⨆ n. L n)
    using is-lubD1[OF L, THEN is-ubD] by (blast intro: lub-mono chx chL)+
   then show {a, b} <| (⨆ n. L n) by (simp add: reach is-ub-def)
  next
   fix u assume {a, b} <| u
   then have a ⊑ u b ⊑ u by (auto dest: is-ubD)
   then have L n ⊑ u for n
    using below by (blast intro: is-lubD2[OF L] is-ubI below-trans)
   then show (⨆ n. L n) ⊑ u by (rule lub-below[OF chL])
  qed
  then show ?thesis ..
qed

lemma bounded-complete-chainI:
  fixes t :: nat ⇒ 'a::dcpo → 'a
  assumes chain t (⨆ i. t i) = ID
    and ⋀n a b c. t n·a ⊑ c ⟹ t n·b ⊑ c ⟹ ∃ z. {t n·a, t n·b} <<| z
  shows OFCLASS('a, bounded-complete-class)
  by (rule bounded-complete-finite-joinI) (rule chain-bounded-lub[OF assms])
```

The compact version is the approximation-chain instance.

```
lemma bounded-complete-finite-compact-joinI:
  assumes ⋀a b c::'a::bifinite. ⟦ compact a; compact b; a ⊑ c; b ⊑ c ⟧ ⟹ ∃ z. {a,
b} <<| z
  shows OFCLASS('a, bounded-complete-class)
proof (rule bounded-complete-finite-joinI)
  fix a b c :: 'a assume a ⊑ c b ⊑ c
  from bifinite obtain approx :: nat ⇒ 'a → 'a where approx-chain approx ..
  then interpret approx-chain approx .
  show ∃ z. {a, b} <<| z
   by (rule chain-bounded-lub[OF chain-approx lub-approx - ‹a ⊑ c› ‹b ⊑ c›], rule
assms) simp-all
qed

lemma bounded-complete-finite-compact-joinI':
```

assumes
$\bigwedge a\ b\ c{::}'a{::}bifinite.\ [\![\ compact\ a;\ compact\ b;\ compact\ c;\ a \sqsubseteq c;\ b \sqsubseteq c]\!] \Longrightarrow a \sqsubseteq F\ a\ b$
$\bigwedge a\ b\ c{::}'a{::}bifinite.\ [\![\ compact\ a;\ compact\ b;\ compact\ c;\ a \sqsubseteq c;\ b \sqsubseteq c]\!] \Longrightarrow b \sqsubseteq F\ a\ b$
$\bigwedge a\ b\ c{::}'a{::}bifinite.\ [\![\ compact\ a;\ compact\ b;\ compact\ c;\ a \sqsubseteq c;\ b \sqsubseteq c]\!] \Longrightarrow F\ a\ b \sqsubseteq c$
shows *OFCLASS*('a, *bounded-complete-class*)
proof (*rule bounded-complete-finite-compact-joinI*)
fix *a b c* :: 'a
assume *compact a compact b a* $\sqsubseteq$ *c b* $\sqsubseteq$ *c*
with *assms* **have** {*a*, *b*} <<| *F a b*
by (*intro compact-lubI*[*of a b F c*]) *presburger*+
then show $\exists z.$ {*a*, *b*} <<| *z* **..**
qed

lemma *down-below*: *fup*·*ID*·*x* $\sqsubseteq$ *y* $\longleftrightarrow$ *x* $\sqsubseteq$ *up*·*y*
by (*cases x*) *simp-all*

corollary *down-below-ub*: ($\lambda x.$ *fup*·*ID*·*x*) ‘ *X* <| *y* $\longleftrightarrow$ *X* <| *up*·*y*
unfolding *is-ub-def*
using *down-below* **by** *blast*

instance *u* :: (*bounded-complete*) *bounded-complete*
proof
fix *A* :: 'a *u set* **assume** *A* $\neq$ {}
fix *b* **assume** *A* <| *b*
show $\exists a.$ *A* <<| *a*
proof (*cases A* <| $\bot$)
case *True*
from *is-ubD*[*OF this*] **have** *A* = {} $\vee$ *A* = {$\bot$} **by** *auto*
then have *A* <<| $\bot$ **by** *auto*
then show *?thesis* **..**
next
case *False*
let *?down* = ($\lambda x.$ *fup*·*ID*·*x*)
from ‹*A* <| *b*› **have** *1*: *?down* ‘ *A* <| *?down b*
by (*simp add*: *is-ub-def monofun-cfun-arg*)
define *a* **where** *a* = *lub* (*?down* ‘ *A*)
thm *ex-bound-lub*
from *1* **have** *a*: *?down* ‘ *A* <<| *a* **unfolding** *a-def* **by** (*rule ex-bound-lub*)
have *A* <<| *up*·*a*
proof (*rule is-lubI*)
show *A* <| *up*·*a* **using** *a*[*THEN is-lubD1*] *down-below-ub* **by** *blast*
fix *u* **assume** *A* <| *u*
with ‹¬(*A* <| $\bot$)› **obtain** *u'* **where** ∗: *u* = *up*·*u'* *A* <| *up*·*u'* **by** (*cases u*) *simp-all*
then have *?down* ‘ *A* <| *u'* **by** (*simp add*: *down-below-ub*)
with *a* **have** *a* $\sqsubseteq$ *u'* **by** (*rule is-lubD2*)

```
    with ∗ show up·a ⊑ u by simp
   qed
   then show ?thesis ..
  qed
qed

lemma meet-bottom [simp]:
  fixes x :: 'a::bounded-complete
  shows x ⊓ ⊥ = ⊥ ⊥ ⊓ x = ⊥
  by (simp-all add: meet-sym meet-absorbI)

lemma up-joinable-iff [simp]:
  fixes x y :: 'a::bounded-complete
  shows up·x ⇑ up·y ⟷ x ⇑ y
proof
  assume up·x ⇑ up·y
  then obtain z where up·x ⊑ z up·y ⊑ z unfolding joinable-def by blast
  then show x ⇑ y by (cases z) (auto intro: joinableI)
next
  assume x ⇑ y
  then obtain z where x ⊑ z y ⊑ z unfolding joinable-def by blast
  then show up·x ⇑ up·y by (auto intro: joinableI[of - up·z])
qed

lemma up-join:
  fixes x y :: 'a::bounded-complete
  assumes x ⇑ y shows up·x ⊔ up·y = up·(x ⊔ y)
proof (rule below-antisym)
  have xy: up·x ⇑ up·y using assms by simp
  show up·x ⊔ up·y ⊑ up·(x ⊔ y)
    by (rule join-belowI) (simp-all add: joinableD[OF assms])
  have up·x ⊑ up·x ⊔ up·y up·y ⊑ up·x ⊔ up·y using joinableD[OF xy] by simp-all
  then show up·(x ⊔ y) ⊑ up·x ⊔ up·y
    by (cases up·x ⊔ up·y) (simp-all add: join-belowI)
qed

lemma up-meet [simp]:
  fixes x y :: 'a::bounded-complete
  shows up·x ⊓ up·y = up·(x ⊓ y)
proof (rule below-antisym)
  show up·(x ⊓ y) ⊑ up·x ⊓ up·y
    by (rule meet-greatest) (simp-all add: meet-below1 meet-below2)
  have up·x ⊓ up·y ⊑ up·x up·x ⊓ up·y ⊑ up·y by (rule meet-below1, rule meet-below2)
  then show up·x ⊓ up·y ⊑ up·(x ⊓ y)
    by (cases up·x ⊓ up·y) (simp-all add: meet-greatest)
qed
```

1.1 Strict sums

A bounded set of a strict sum lies on one side, so its lub is computed there: projecting onto the side with *sscase* transports bounds back and forth.

lemma *ssum-lub-sinl*:
fixes $A :: ('a{::}bounded\text{-}complete \oplus 'b{::}bounded\text{-}complete)\ set$
assumes $A <| sinl{\cdot}b$ **shows** $\exists\, a.\ A <<| a$
proof −
define d **where** $d = (sscase{\cdot}ID{\cdot}\bot :: 'a \oplus 'b \to 'a)$
define l **where** $l = lub\ ((\lambda x.\ d{\cdot}x)\ `\ A)$
have *sinl-d-below*: $sinl{\cdot}(d{\cdot}u) \sqsubseteq u$ **for** u **by** (*cases u*) (*simp-all add*: *d-def*)
have *sinl-d*: $x = sinl{\cdot}(d{\cdot}x)$ **if** $x \sqsubseteq sinl{\cdot}b$ **for** x
using *that* **by** (*cases x*) (*simp-all add*: *d-def*)
have $d{\cdot}x \sqsubseteq b$ **if** $x \in A$ **for** x
using *monofun-cfun-arg*[*OF is-ubD*[*OF assms that*], *of d*] **by** (*cases b* = ⊥) (*simp-all add*: *d-def*)
then have $(\lambda x.\ d{\cdot}x)\ `\ A <|\ b$ **by** (*auto intro*: *is-ubI*)
then have *l*: $(\lambda x.\ d{\cdot}x)\ `\ A <<|\ l$ **unfolding** *l-def* **by** (*rule ex-bound-lub*)
have $A <<| sinl{\cdot}l$
proof (*rule is-lubI*)
show $A <|\ sinl{\cdot}l$
proof (*rule is-ubI*)
fix x **assume** *x*: $x \in A$
have $d{\cdot}x \sqsubseteq l$ **using** x **by** (*blast intro*: *is-ubD*[*OF is-lubD1*[*OF l*]])
then have $sinl{\cdot}(d{\cdot}x) \sqsubseteq sinl{\cdot}l$ **by** (*rule monofun-cfun-arg*)
then show $x \sqsubseteq sinl{\cdot}l$
by (*rule back-subst*) (*rule sinl-d*[*OF is-ubD*[*OF assms x*], *symmetric*])
qed
next
fix u **assume** $A <|\ u$
then have $(\lambda x.\ d{\cdot}x)\ `\ A <|\ d{\cdot}u$ **by** (*auto intro*!: *is-ubI monofun-cfun-arg dest*: *is-ubD*)
with l **have** $l \sqsubseteq d{\cdot}u$ **by** (*rule is-lubD2*)
then have $sinl{\cdot}l \sqsubseteq sinl{\cdot}(d{\cdot}u)$ **by** (*rule monofun-cfun-arg*)
then show $sinl{\cdot}l \sqsubseteq u$ **using** *sinl-d-below* **by** (*rule below-trans*)
qed
then show *?thesis* **..**
qed

lemma *ssum-lub-sinr*:
fixes $A :: ('a{::}bounded\text{-}complete \oplus 'b{::}bounded\text{-}complete)\ set$
assumes $A <| sinr{\cdot}b$ **shows** $\exists\, a.\ A <<| a$
proof −
define d **where** $d = (sscase{\cdot}\bot{\cdot}ID :: 'a \oplus 'b \to 'b)$
define l **where** $l = lub\ ((\lambda x.\ d{\cdot}x)\ `\ A)$
have *sinr-d-below*: $sinr{\cdot}(d{\cdot}u) \sqsubseteq u$ **for** u **by** (*cases u*) (*simp-all add*: *d-def*)
have *sinr-d*: $x = sinr{\cdot}(d{\cdot}x)$ **if** $x \sqsubseteq sinr{\cdot}b$ **for** x
using *that* **by** (*cases x*) (*simp-all add*: *d-def*)
have $d{\cdot}x \sqsubseteq b$ **if** $x \in A$ **for** x

```
    using monofun-cfun-arg[OF is-ubD[OF assms that], of d] by (cases b = ⊥)
(simp-all add: d-def)
  then have (λx. d·x) ‘ A <| b by (auto intro: is-ubI)
  then have l: (λx. d·x) ‘ A <<| l unfolding l-def by (rule ex-bound-lub)
  have A <<| sinr·l
  proof (rule is-lubI)
    show A <| sinr·l
    proof (rule is-ubI)
      fix x assume x: x ∈ A
      have d·x ⊑ l using x by (blast intro: is-ubD[OF is-lubD1[OF l]])
      then have sinr·(d·x) ⊑ sinr·l by (rule monofun-cfun-arg)
      then show x ⊑ sinr·l
        by (rule back-subst) (rule sinr-d[OF is-ubD[OF assms x], symmetric])
    qed
  next
    fix u assume A <| u
    then have (λx. d·x) ‘ A <| d·u by (auto intro!: is-ubI monofun-cfun-arg dest:
is-ubD)
    with l have l ⊑ d·u by (rule is-lubD2)
    then have sinr·l ⊑ sinr·(d·u) by (rule monofun-cfun-arg)
    then show sinr·l ⊑ u using sinr-d-below by (rule below-trans)
  qed
  then show ?thesis ..
qed

instance ssum :: (bounded-complete, bounded-complete) bounded-complete
proof
  fix A :: ('a ⊕ 'b) set and b assume b: A <| b
  show ∃ a. A <<| a
  proof (cases b)
    case bottom with b show ?thesis using ssum-lub-sinl[of A ⊥] by simp
  next
    case (sinl x) with b show ?thesis by (simp add: ssum-lub-sinl)
  next
    case (sinr y) with b show ?thesis by (simp add: ssum-lub-sinr)
  qed
qed

context
  fixes x x' :: 'a::bounded-complete and y y' :: 'b::bounded-complete
begin

lemma sinl-meet [simp]: sinl·x ⊓ sinl·x' = (sinl·(x ⊓ x') :: 'a ⊕ 'b)
proof (rule below-antisym)
  let ?m = sinl·x ⊓ sinl·x' :: 'a ⊕ 'b
  show sinl·(x ⊓ x') ⊑ ?m
    by (rule meet-greatest) (simp-all add: meet-below1 meet-below2)
  have ?m ⊑ sinl·x ?m ⊑ sinl·x' by (rule meet-below1, rule meet-below2)
  then show ?m ⊑ sinl·(x ⊓ x')
```

by (*cases ?m*) (*simp-all add*: *meet-greatest*)
qed

lemma *sinr-meet* [*simp*]: *sinr·y* ⊓ *sinr·y′* = (*sinr·*(*y* ⊓ *y′*) :: *′a* ⊕ *′b*)
proof (*rule below-antisym*)
let *?m* = *sinr·y* ⊓ *sinr·y′* :: *′a* ⊕ *′b*
show *sinr·*(*y* ⊓ *y′*) ⊑ *?m*
by (*rule meet-greatest*) (*simp-all add*: *meet-below1 meet-below2*)
have *?m* ⊑ *sinr·y* *?m* ⊑ *sinr·y′* **by** (*rule meet-below1*, *rule meet-below2*)
then show *?m* ⊑ *sinr·*(*y* ⊓ *y′*)
by (*cases ?m*) (*simp-all add*: *meet-greatest*)
qed

lemma *sinl-sinr-meet* [*simp*]: *sinl·x* ⊓ *sinr·y* = (⊥ :: *′a* ⊕ *′b*) *sinr·y* ⊓ *sinl·x* = (⊥ :: *′a* ⊕ *′b*)
proof −
have *sinl·x* ⊓ *sinr·y* ⊑ *sinl·x* *sinl·x* ⊓ *sinr·y* ⊑ (*sinr·y* :: *′a* ⊕ *′b*)
by (*rule meet-below1*, *rule meet-below2*)
then show *sinl·x* ⊓ *sinr·y* = (⊥ :: *′a* ⊕ *′b*)
by (*cases sinl·x* ⊓ *sinr·y* :: *′a* ⊕ *′b*) *simp-all*
then show *sinr·y* ⊓ *sinl·x* = (⊥ :: *′a* ⊕ *′b*) **by** (*simp add*: *meet-sym*)
qed

lemma *sinl-joinable-iff* [*simp*]: (*sinl·x* :: *′a* ⊕ *′b*) ⇑ *sinl·x′* ⟷ *x* ⇑ *x′*
proof
assume (*sinl·x* :: *′a* ⊕ *′b*) ⇑ *sinl·x′*
then obtain *z* :: *′a* ⊕ *′b* **where** *sinl·x* ⊑ *z* *sinl·x′* ⊑ *z* **unfolding** *joinable-def*
by *blast*
then show *x* ⇑ *x′* **by** (*cases z*) (*auto intro*: *joinableI*)
next
assume *x* ⇑ *x′*
then obtain *z* **where** *x* ⊑ *z* *x′* ⊑ *z* **unfolding** *joinable-def* **by** *blast*
then show (*sinl·x* :: *′a* ⊕ *′b*) ⇑ *sinl·x′* **by** (*auto intro*: *joinableI*[*of* - *sinl·z*])
qed

lemma *sinr-joinable-iff* [*simp*]: (*sinr·y* :: *′a* ⊕ *′b*) ⇑ *sinr·y′* ⟷ *y* ⇑ *y′*
proof
assume (*sinr·y* :: *′a* ⊕ *′b*) ⇑ *sinr·y′*
then obtain *z* :: *′a* ⊕ *′b* **where** *sinr·y* ⊑ *z* *sinr·y′* ⊑ *z* **unfolding** *joinable-def*
by *blast*
then show *y* ⇑ *y′* **by** (*cases z*) (*auto intro*: *joinableI*)
next
assume *y* ⇑ *y′*
then obtain *z* **where** *y* ⊑ *z* *y′* ⊑ *z* **unfolding** *joinable-def* **by** *blast*
then show (*sinr·y* :: *′a* ⊕ *′b*) ⇑ *sinr·y′* **by** (*auto intro*: *joinableI*[*of* - *sinr·z*])
qed

lemma *sinl-sinr-joinable-iff* [*simp*]:
(*sinl·x* :: *′a* ⊕ *′b*) ⇑ *sinr·y* ⟷ *x* = ⊥ ∨ *y* = ⊥ (*sinr·y* :: *′a* ⊕ *′b*) ⇑ *sinl·x* ⟷

$x = \bot \vee y = \bot$
proof −
have $(sinl{\cdot}x :: 'a \oplus 'b) \Uparrow sinr{\cdot}y \longleftrightarrow x = \bot \vee y = \bot$
proof
assume $(sinl{\cdot}x :: 'a \oplus 'b) \Uparrow sinr{\cdot}y$
then obtain $z :: 'a \oplus 'b$ **where** $sinl{\cdot}x \sqsubseteq z$ $sinr{\cdot}y \sqsubseteq z$ **unfolding** *joinable-def*
by *blast*
then show $x = \bot \vee y = \bot$ **by** (*cases z*) *simp-all*
qed *auto*
then show $(sinl{\cdot}x :: 'a \oplus 'b) \Uparrow sinr{\cdot}y \longleftrightarrow x = \bot \vee y = \bot$
$(sinr{\cdot}y :: 'a \oplus 'b) \Uparrow sinl{\cdot}x \longleftrightarrow x = \bot \vee y = \bot$ **by** (*simp-all add*: *joinable-sym*)
qed

lemma *sinl-join*: $x \Uparrow x' \Longrightarrow sinl{\cdot}x \sqcup sinl{\cdot}x' = (sinl{\cdot}(x \sqcup x') :: 'a \oplus 'b)$
proof (*rule below-antisym*)
assume *xx*: $x \Uparrow x'$
let *?j* $= sinl{\cdot}x \sqcup sinl{\cdot}x' :: 'a \oplus 'b$
have *j*: $(sinl{\cdot}x :: 'a \oplus 'b) \Uparrow sinl{\cdot}x'$ **using** *xx* **by** *simp*
show $?j \sqsubseteq sinl{\cdot}(x \sqcup x')$
by (*rule join-belowI*) (*simp-all add*: *joinableD*[*OF xx*])
have $sinl{\cdot}x \sqsubseteq ?j$ $sinl{\cdot}x' \sqsubseteq ?j$ **using** *joinableD*[*OF j*] **by** *simp-all*
then show $sinl{\cdot}(x \sqcup x') \sqsubseteq ?j$
by (*cases ?j*) (*simp-all add*: *join-belowI*)
qed

lemma *sinr-join*: $y \Uparrow y' \Longrightarrow sinr{\cdot}y \sqcup sinr{\cdot}y' = (sinr{\cdot}(y \sqcup y') :: 'a \oplus 'b)$
proof (*rule below-antisym*)
assume *yy*: $y \Uparrow y'$
let *?j* $= sinr{\cdot}y \sqcup sinr{\cdot}y' :: 'a \oplus 'b$
have *j*: $(sinr{\cdot}y :: 'a \oplus 'b) \Uparrow sinr{\cdot}y'$ **using** *yy* **by** *simp*
show $?j \sqsubseteq sinr{\cdot}(y \sqcup y')$
by (*rule join-belowI*) (*simp-all add*: *joinableD*[*OF yy*])
have $sinr{\cdot}y \sqsubseteq ?j$ $sinr{\cdot}y' \sqsubseteq ?j$ **using** *joinableD*[*OF j*] **by** *simp-all*
then show $sinr{\cdot}(y \sqcup y') \sqsubseteq ?j$
by (*cases ?j*) (*simp-all add*: *join-belowI*)
qed

end

1.2 Cartesian products

Everything is componentwise.

lemma *prod-ub-iff*: $A <| b \longleftrightarrow fst \mathbin{`} A <| fst\ b \wedge snd \mathbin{`} A <| snd\ b$
by (*auto simp*: *is-ub-def below-prod-def*)

instance *prod* :: (*bounded-complete*, *bounded-complete*) *bounded-complete*
proof
fix $A :: ('a \times 'b)$ *set* **and** *b* **assume** $A <| b$
then have $fst \mathbin{`} A <<| lub\ (fst \mathbin{`} A)$ $snd \mathbin{`} A <<| lub\ (snd \mathbin{`} A)$

by (*auto simp*: *prod-ub-iff intro*: *ex-bound-lub*)
then have $A <<| (lub\ (fst\ `\ A),\ lub\ (snd\ `\ A))$
by (*auto simp*: *is-lub-def prod-ub-iff below-prod-def*)
then show $\exists\, a.\ A <<|\ a$ **..**
qed

lemma *prod-joinable-iff*: $x \Uparrow y \longleftrightarrow fst\ x \Uparrow fst\ y \wedge snd\ x \Uparrow snd\ y$
unfolding *joinable-def below-prod-def* **by** (*metis fst-conv snd-conv*)

context
fixes $x\ y :: 'a{::}bounded\text{-}complete \times 'b{::}bounded\text{-}complete$
begin

lemma *prod-meet*: $x \sqcap y = (fst\ x \sqcap fst\ y,\ snd\ x \sqcap snd\ y)$
proof (*rule below-antisym*)
show $x \sqcap y \sqsubseteq (fst\ x \sqcap fst\ y,\ snd\ x \sqcap snd\ y)$
using *meet-below1*[*of x y*] *meet-below2*[*of x y*] **by** (*simp add*: *below-prod-def meet-greatest*)
show $(fst\ x \sqcap fst\ y,\ snd\ x \sqcap snd\ y) \sqsubseteq x \sqcap y$
by (*rule meet-greatest*) (*simp-all add*: *below-prod-def meet-below1 meet-below2*)
qed

lemma *prod-join*: $x \Uparrow y \Longrightarrow x \sqcup y = (fst\ x \sqcup fst\ y,\ snd\ x \sqcup snd\ y)$
proof (*rule below-antisym*)
assume *xy*: $x \Uparrow y$
then have *c*: $fst\ x \Uparrow fst\ y\ snd\ x \Uparrow snd\ y$ **by** (*simp-all add*: *prod-joinable-iff*)
show $x \sqcup y \sqsubseteq (fst\ x \sqcup fst\ y,\ snd\ x \sqcup snd\ y)$
by (*rule join-belowI*) (*simp-all add*: *below-prod-def joinableD*[*OF c*(*1*)] *joinableD*[*OF c*(*2*)])
show $(fst\ x \sqcup fst\ y,\ snd\ x \sqcup snd\ y) \sqsubseteq x \sqcup y$
using *joinableD*[*OF xy*] **by** (*simp add*: *below-prod-def join-belowI*)
qed

end

1.3 Smash products

Componentwise, through *below-sprod*; the only care needed is that the pair of component lubs does not collapse, which holds as soon as one element is nonbottom.

instance *sprod* :: (*bounded-complete*, *bounded-complete*) *bounded-complete*
proof
fix $A :: ('a \otimes 'b)\ set$ **and** *b* **assume** *ne*: $A \neq \{\}$ **and** *ub*: $A <|\ b$
show $\exists\, a.\ A <<|\ a$
proof (*cases* $A = \{\bot\}$)
case *True* **then show** *?thesis* **by** (*auto intro*: *is-lubI is-ubI*)
next
case *False*
with *ne* **obtain** *x0* **where** *x0*: $x0 \in A\ x0 \neq \bot$ **by** *blast*

```
    have (λx. sfst·x) ‘ A <| sfst·b (λx. ssnd·x) ‘ A <| ssnd·b
      using ub by (auto intro!: is-ubI monofun-cfun-arg dest: is-ubD)
    then obtain L1 L2 where L1: (λx. sfst·x) ‘ A <<| L1 and L2: (λx. ssnd·x)
‘ A <<| L2
      by (blast dest: ex-bound-ex-lub)
    have le: sfst·x ⊑ L1 ssnd·x ⊑ L2 if x ∈ A for x
      using that is-lubD1[OF L1] is-lubD1[OF L2] by (auto dest: is-ubD)
    have nb: L1 ≠ ⊥ L2 ≠ ⊥
      using le[OF x0(1)] x0(2) by (auto dest: bottomI)
    have A <<| (:L1, L2:)
    proof (rule is-lubI)
      show A <| (:L1, L2:)
        using le nb by (auto intro!: is-ubI simp: below-sprod)
    next
      fix u assume u: A <| u
      have L1 ⊑ sfst·u L2 ⊑ ssnd·u
        using u by (auto intro!: is-lubD2[OF L1] is-lubD2[OF L2] is-ubI mono-
fun-cfun-arg
          dest: is-ubD)
      then show (:L1, L2:) ⊑ u using nb by (simp add: below-sprod)
    qed
    then show ?thesis ..
  qed
qed

lemma sfst-spair-below: sfst·(:x, y:) ⊑ x and ssnd-spair-below: ssnd·(:x, y:) ⊑ y
  by (cases y = ⊥; simp) (cases x = ⊥; simp)

context
  fixes x x′ :: 'a::bounded-complete and y y′ :: 'b::bounded-complete
begin
```

Meets are componentwise without side conditions: a collapsing component makes the whole pair $\bot$, by strictness of *spair*.

```
lemma spair-meet: (:x, y:) ⊓ (:x′, y′:) = (:x ⊓ x′, y ⊓ y′:)
proof (rule below-antisym)
  let ?m = (:x, y:) ⊓ (:x′, y′:)
  have sfst·?m ⊑ x ⊓ x′ ssnd·?m ⊑ y ⊓ y′
   by (intro meet-greatest below-trans[OF monofun-cfun-arg[OF meet-below1] sfst-spair-below]
        below-trans[OF monofun-cfun-arg[OF meet-below2] sfst-spair-below]
        below-trans[OF monofun-cfun-arg[OF meet-below1] ssnd-spair-below]
        below-trans[OF monofun-cfun-arg[OF meet-below2] ssnd-spair-below])+
  then have (:sfst·?m, ssnd·?m:) ⊑ (:x ⊓ x′, y ⊓ y′:) by (intro monofun-cfun)
simp-all
  then show ?m ⊑ (:x ⊓ x′, y ⊓ y′:) by (simp only: spair-sfst-ssnd)
  show (:x ⊓ x′, y ⊓ y′:) ⊑ ?m
   by (intro meet-greatest monofun-cfun monofun-cfun-arg meet-below1 meet-below2)
qed
```

context
  assumes *nb*: $x \neq \bot\ y \neq \bot\ x' \neq \bot\ y' \neq \bot$
begin

lemma *spair-joinable-iff*: $(:x,\ y:) \Uparrow (:x',\ y':) \longleftrightarrow x \Uparrow x' \wedge y \Uparrow y'$
proof
  assume $(:x,\ y:) \Uparrow (:x',\ y':)$
  then obtain u **where** $(:x,\ y:) \sqsubseteq u\ (:x',\ y':) \sqsubseteq u$ **unfolding** *joinable-def* **by** *blast*
  then show $x \Uparrow x' \wedge y \Uparrow y'$ **using** *nb* **by** (*auto simp*: *below-sprod intro*: *joinableI*)
next
  assume $x \Uparrow x' \wedge y \Uparrow y'$
  then have $(:x,\ y:) \sqsubseteq (:x \sqcup x',\ y \sqcup y':)\ (:x',\ y':) \sqsubseteq (:x \sqcup x',\ y \sqcup y':)$
    by (*auto intro*!: *monofun-cfun monofun-cfun-arg joinableD*)
  then show $(:x,\ y:) \Uparrow (:x',\ y':)$ **by** (*rule joinableI*)
qed

lemma *spair-join*:
  assumes $x \Uparrow x'\ y \Uparrow y'$ **shows** $(:x,\ y:) \sqcup (:x',\ y':) = (:x \sqcup x',\ y \sqcup y':)$
proof (*rule below-antisym*)
  have *j*: $(:x,\ y:) \Uparrow (:x',\ y':)$ **using** *assms* **by** (*simp add*: *spair-joinable-iff*)
  show $(:x,\ y:) \sqcup (:x',\ y':) \sqsubseteq (:x \sqcup x',\ y \sqcup y':)$
    using *assms* **by** (*intro join-belowI monofun-cfun monofun-cfun-arg joinableD*)
  have $x \sqsubseteq$ *sfst*·$((:x,\ y:) \sqcup (:x',\ y':))\ x' \sqsubseteq$ *sfst*·$((:x,\ y:) \sqcup (:x',\ y':))$
    $y \sqsubseteq$ *ssnd*·$((:x,\ y:) \sqcup (:x',\ y':))\ y' \sqsubseteq$ *ssnd*·$((:x,\ y:) \sqcup (:x',\ y':))$
    using *monofun-cfun-arg*[*OF joinableD*(*1*)[*OF j*], *of sfst*] *monofun-cfun-arg*[*OF joinableD*(*2*)[*OF j*], *of sfst*]
     monofun-cfun-arg[*OF joinableD*(*1*)[*OF j*], *of ssnd*] *monofun-cfun-arg*[*OF joinableD*(*2*)[*OF j*], *of ssnd*]
    using *nb* **by** *simp-all*
  then have $(:x \sqcup x',\ y \sqcup y':) \sqsubseteq (:$*sfst*·$((:x,\ y:) \sqcup (:x',\ y':))$, *ssnd*·$((:x,\ y:) \sqcup (:x',\ y':)):)$
    by (*intro monofun-cfun monofun-cfun-arg join-belowI*)
  then show $(:x \sqcup x',\ y \sqcup y':) \sqsubseteq (:x,\ y:) \sqcup (:x',\ y':)$ **by** (*simp only*: *spair-sfst-ssnd*)
qed

end

end

subclass (**in** *flat*) *bounded-complete*
proof
  fix $A :: 'a\ set$ **and** $b :: 'a$
  assume $A \neq \{\}$
  assume $A <| b$
  then have $\forall x \in A.\ x = \bot \vee x = b$ **using** *is-ubD*[*THEN ax-flat*] **by** *blast*
  with ‹$A \neq \{\}$› **consider** $A = \{b\} \mid A = \{\bot\} \mid A = \{\bot,\ b\}$
    by *fast*
  then show $\exists z.\ A <<| z$
    by (*cases*) (*auto intro*: *is-lubI is-ubI dest*: *is-ubD*)

qed

instantiation *fun* :: (*type*, *bounded-complete*) *bounded-complete*
begin

lemma *fun-ub-iff*: $A <| b \longleftrightarrow (\forall p.\ (\lambda f.\ f\ p)\ `\ A <|\ b\ p)$
by (*auto simp add*: *is-ub-def fun-below-iff*)

lemma *fun-lub-iff*: $A <<| b \longleftrightarrow (\forall p.\ (\lambda f.\ f\ p)\ `\ A <<|\ b\ p)$
(**is** *?f* $\longleftrightarrow$ $(\forall p.\ ?p\ p)$)
proof
show $(\forall p.\ ?p\ p) \Longrightarrow ?f$
by (*auto simp add*: *is-lub-def is-ub-def fun-ub-iff fun-below-iff*)
show $(\forall p.\ ?p\ p)$ **if** *?f*
proof (*intro strip is-lubI*)
from *that* **have** *b-ub*: $A <|\ b$ **by** (*rule is-lubD1*)
fix *p*
show $(\lambda f.\ f\ p)\ `\ A <|\ b\ p$ **using** *b-ub fun-ub-iff* **by** *fast*
fix *u*
assume $(\lambda f.\ f\ p)\ `\ A <|\ u$
with *b-ub* **have** $A <|\ b(p := u)$
unfolding *fun-ub-iff* **by** *simp*
with *that* **have** $b \sqsubseteq b(p := u)$
by (*rule is-lubD2*)
from *fun-belowD*[*OF this*, *of p*] **show** $b\ p \sqsubseteq u$ **by** *simp*
qed
qed

lemma *fun-bounded-complete*:
fixes $A :: ('a \Rightarrow 'b)\ set$
and *b*
assumes $A <|\ b$
shows $A <<| (\lambda p.\ (\bigsqcup f \in A.\ f\ p))$
proof −
from *assms* **have** $(\lambda f.\ f\ p)\ `\ A <|\ b\ p$ **for** *p*
by (*simp add*: *fun-ub-iff*)
then have $(\lambda f.\ f\ p)\ `\ A <<| (\bigsqcup f \in A.\ f\ p)$ **for** *p*
by (*rule ex-bound-lub*)
then show $A <<| (\lambda p.\ (\bigsqcup f \in A.\ f\ p))$
by (*simp add*: *fun-lub-iff*)
qed

instance proof qed (*blast intro*: *fun-bounded-complete*)

end

lemma *fun-lub*:
fixes $A :: ('a{::}type \Rightarrow 'b{::}bounded\text{-}complete)\ set$

```
  and b
 assumes A <| b
 shows lub A = (λp. (⨆f ∈ A. f p))
 using lub-eqI[OF fun-bounded-complete, OF assms] .

lemma fun-join:
 fixes f g :: 'a::type ⇒ 'b::bounded-complete
 assumes f ⇑ g
 shows f ⊔ g = (λx. f x ⊔ g x)
proof −
 from assms obtain z where {f, g} <| z
  by (auto simp add: joinable-def is-ub-def)
 from fun-lub[OF this]
 show ?thesis unfolding join-def by simp
qed

lemma fun-meet:
 fixes f g :: 'a::type ⇒ 'b::bounded-complete
 shows f ⊓ g = (λx. f x ⊓ g x)
proof (rule below-antisym)
 show f ⊓ g ⊑ (λx. f x ⊓ g x)
 proof (rule fun-belowI)
  fix x
  have (f ⊓ g) x ⊑ f x (f ⊓ g) x ⊑ g x
   by (auto intro: fun-belowD meet-below1 meet-below2)
  thus (f ⊓ g) x ⊑ (λx. f x ⊓ g x) x by (simp add: meet-greatest)
 qed
next
 show (λx. f x ⊓ g x) ⊑ f ⊓ g
  by (rule meet-greatest; rule fun-belowI) (simp-all add: meet-below1 meet-below2)
qed


instantiation cfun :: (cpo, bounded-complete) bounded-complete
begin

lemma cfun-ub-iff: A <| b ⟷ (∀ p. (λf. f·p) ‘ A <| b·p)
 by (auto simp add: is-ub-def cfun-below-iff)

lemma cfun-lubI: (⋀p. (λf. f·p) ‘ A <<| b·p) ⟹ (A <<| b)
by (simp add: is-lub-def cfun-ub-iff cfun-below-iff)

context
 fixes  A :: ('a → 'b) set
  and b
 assumes Ab: A <| b
begin

private lemma pointwise-lubs: (λf. f·p) ‘ A <<| (⨆ f ∈ A. f·p) for p
```

proof −
from *Ab* **have** $(\lambda f.\ f{\cdot}p)$ ‘ $A <|\ b{\cdot}p$ **for** *p*
by (*auto dest*: *is-ubD intro*!: *is-ubI monofun-cfun-fun*)
then show *?thesis*
by (*rule ex-bound-lub*)
qed

lemma *cfun-bound-cont*[*simp*]: $(\Lambda\ p.\ (\bigsqcup f \in A.\ f{\cdot}p)){\cdot}p = (\bigsqcup\ f \in A.\ f{\cdot}p)$ **for** *p*
proof (*rule beta-cfun*, *rule contI*)
fix $Y :: nat \Rightarrow 'a$ **assume** *chain Y*
then have *range* $(\lambda i.\ \bigsqcup f{\in}A.\ f{\cdot}(Y\ i)) <<|\ (\bigsqcup f{\in}A.\ (\bigsqcup i.\ f{\cdot}(Y\ i)))$
by (*auto intro*: *chain-lub-family-converse pointwise-lubs*)
then show *range* $(\lambda i.\ \bigsqcup f{\in}A.\ f{\cdot}(Y\ i)) <<|\ (\bigsqcup f{\in}A.\ f{\cdot}(\bigsqcup i.\ Y\ i))$
by (*simp add*: ‹*chain Y*› *contlub-cfun-arg*)
qed

lemma *cfun-bounded-complete*: $A <<|\ (\Lambda\ p.\ (\bigsqcup f \in A.\ f{\cdot}p))$
using *pointwise-lubs* **by** (*simp add*: *is-lub-def is-ub-def cfun-below-iff*)
end

instance proof qed (*blast intro*: *cfun-bounded-complete*)

end

lemma *cfun-lub*:
fixes $A :: ('a{::}cpo \to 'b{::}bounded\text{-}complete)\ set$
and *b*
assumes $A <|\ b$
shows $(lub\ A){\cdot}p = (\bigsqcup f \in A.\ f{\cdot}p)$
using *cfun-bounded-complete*[*OF assms*, *THEN lub-eqI*]
cfun-bound-cont[*OF assms*] **by** *simp*

lemma *cfun-join*:
fixes $f\ g :: 'a{::}cpo \to 'b{::}bounded\text{-}complete$
assumes $f \Uparrow g$
shows $(f \sqcup g){\cdot}p = f{\cdot}p \sqcup g{\cdot}p$
proof −
from *assms* **obtain** *z* **where** $\{f, g\} <|\ z$
by (*auto simp add*: *joinable-def is-ub-def*)
from *cfun-lub*[*OF this*]
show *?thesis* **unfolding** *join-def* **by** *simp*
qed

instantiation *sfun* :: (*pcpo*, *bounded-complete*) *bounded-complete*
begin

context
fixes *A* **and** $b :: 'a \to! 'b$
assumes *Ab*: $A <|\ b$

begin

private lemma *sfun-cfun-lub*: $(\lambda f.\ \mathit{sfun\text{-}rep}{\cdot}f)\ `\ A <<|\ (\bigsqcup f \in A.\ \mathit{sfun\text{-}rep}{\cdot}f)$
proof −
  from *Ab* **have** $(\lambda f.\ \mathit{sfun\text{-}rep}{\cdot}f)\ `\ A <|\ \mathit{sfun\text{-}rep}{\cdot}b$
    by (*auto dest*: *is-ubD intro*!: *is-ubI monofun-cfun-arg*)
  then show *?thesis* **by** (*rule ex-bound-lub*)
qed

lemma *sfun-lub-strict*: $\mathit{strictify}{\cdot}(\bigsqcup f \in A.\ \mathit{sfun\text{-}rep}{\cdot}f) = (\bigsqcup f \in A.\ \mathit{sfun\text{-}rep}{\cdot}f)$
(**is** $\mathit{strictify}{\cdot}?\mathit{lub} = ?\mathit{lub}$)
proof (*rule below-antisym*)
  show $\mathit{strictify}{\cdot}?\mathit{lub} \sqsubseteq ?\mathit{lub}$ **by** (*rule cfun-belowI*) (*simp add*: *strictify-conv-if*)
  show $?\mathit{lub} \sqsubseteq \mathit{strictify}{\cdot}?\mathit{lub}$ **using** *sfun-cfun-lub*
    apply (*rule is-lubD2*, *intro is-ubI cfun-belowI*)
    by (*auto simp add*: *strictify-conv-if*
      intro!: *sfun-cfun-lub*[*THEN is-lubD1*, *THEN is-ubD*, *THEN monofun-cfun-fun*])
qed

lemma *sfun-bounded-complete*:
  $A <<|\ \mathit{sfun\text{-}abs}{\cdot}(\bigsqcup f \in A.\ \mathit{sfun\text{-}rep}{\cdot}f)$ (**is** $?A <<|\ \mathit{sfun\text{-}abs}{\cdot}?\mathit{lub}$)
proof −
  from *sfun-cfun-lub* **have** $(\lambda f.\ \mathit{sfun\text{-}rep}{\cdot}f)\ `\ A <<|\ \mathit{sfun\text{-}rep}{\cdot}(\mathit{sfun\text{-}abs}{\cdot}?\mathit{lub})$
    by (*simp add*: *sfun-lub-strict*)
  then show *?thesis*
    by (*simp add*: *is-lub-def is-ub-def sfun-below-iff*)
qed

end

instance proof qed (*blast intro*: *sfun-bounded-complete*)

end

default-sort *bifinite*

lemma (**in** *bounded-complete*) *lattice*:
  assumes $\bigwedge a\ b.\ a \Uparrow b$
  shows *class.lattice meet below below-strict join*
 by (*intro class.lattice.intro semilattice-meet class.semilattice-sup.intro below-order*)
    (*unfold-locales*, *auto intro*: *joinableD*[*OF assms*] *join-belowI*)

lemma (**in** *bounded-complete*) *bdd-above-iff*:
$\mathit{preorder.bdd\text{-}above}\ (\sqsubseteq)\ A \longleftrightarrow (\exists\, a.\ A <|\ a)$
 unfolding *preorder.bdd-above-def*[*OF class.order.axioms*(*1*)[*OF below-order*]] *is-ub-def*
  by *fast*

lemma (**in** *bounded-complete*) *cclattice*:
  assumes $\bigwedge a\ b.\ a \Uparrow b$

shows *class.conditionally-complete-lattice glb lub meet below below-strict join*
by (*intro class.conditionally-complete-lattice.intro lattice*[*OF assms*])
(*unfold-locales*, *auto intro*: *joinableD*[*OF assms*] *join-belowI glb-below glb-greatest lub-belowI*[*unfolded is-ub-def*] *below-lubI simp add*: *bdd-above-iff*)

instantiation *lower-pd* :: (*bifinite*) *bounded-complete*
begin
instance proof (*intro bounded-complete-finite-compact-joinI'*)
fix *a b c* :: *'a lower-pd*
show $a \sqsubseteq a \cup\flat b$ $b \sqsubseteq a \cup\flat b$ **by** (*fact lower-plus-below1 lower-plus-below2*)+
assume $a \sqsubseteq c$ $b \sqsubseteq c$
then show $a \cup\flat b \sqsubseteq c$ **unfolding** *lower-plus-below-iff* **by** *presburger*
qed

lemma *joinable*[*simp*]: $(a :: 'a\ lower\text{-}pd) \Uparrow b$ **using** *lower-plus-below1 lower-plus-below2*
by (*rule joinableI*)

lemma *lower-join-eq*: $(a :: 'a\ lower\text{-}pd) \sqcup b = a \cup\flat b$
proof (*rule below-antisym*)
from *lower-plus-below1 lower-plus-below2* **show** $a \sqcup b \sqsubseteq a \cup\flat b$ **by** (*rule join-belowI*)
from *joinableD*[*OF joinable*] **show** $a \cup\flat b \sqsubseteq a \sqcup b$
by (*rule lower-plus-least*)
qed

global-interpretation *lower-pd-lattice*: *conditionally-complete-lattice glb* :: (*'a lower-pd*) *set* ⇒ (*'a lower-pd*) *lub meet below below-strict join*
using *cclattice*[*OF joinable*] **.**

end

definition *min-ub* :: *'a*::*po* ⇒ *'a* ⇒ *'a* ⇒ *bool*
where $\textit{min-ub}\ a\ b\ z \equiv a \sqsubseteq z \wedge b \sqsubseteq z \wedge (\forall w.\ a \sqsubseteq w \longrightarrow b \sqsubseteq w \longrightarrow w \sqsubseteq z \longrightarrow w = z)$

lemma *min-ubI*[*intro*]:
assumes $a \sqsubseteq z$ $b \sqsubseteq z$ $\bigwedge w.\ a \sqsubseteq w \Longrightarrow b \sqsubseteq w \Longrightarrow w \sqsubseteq z \Longrightarrow w = z$
shows *min-ub a b z*
unfolding *min-ub-def* **using** *assms* **by** *blast*

lemma *min-ub-of-compact*: **fixes** *a b* :: *'a*::*bifinite* **assumes** *compact a compact b*
shows *min-ub-below*: $a \sqsubseteq z \Longrightarrow b \sqsubseteq z \Longrightarrow \exists w.\ \textit{min-ub}\ a\ b\ w \wedge w \sqsubseteq z$
and *min-ub-compact*: $\bigwedge w.\ \textit{min-ub}\ a\ b\ w \Longrightarrow \textit{compact}\ w$
and *min-ub-finite*: *finite* $\{w.\ \textit{min-ub}\ a\ b\ w\}$
proof −
from *bifinite* **obtain** *approx* :: *nat* ⇒ *'a* → *'a* **where** *approx-chain approx* **..**
then interpret *approx-chain approx* **.**
have *idem-mono*: $i \leq j \Longrightarrow \textit{approx}\ i{\cdot}x = x \Longrightarrow \textit{approx}\ j{\cdot}x = x$ **for** *i j x*
proof −

assume $i \leq j$ **then have** *approx* $i{\cdot}x \sqsubseteq$ *approx* $j{\cdot}x$
by (*rule chain-approx*[*THEN chain-mono*, *THEN monofun-cfun-fun*])
also assume *approx* $i{\cdot}x = x$
also have *approx* $j{\cdot}x \sqsubseteq x$ **by** (*rule approx-below*)
finally show *?thesis* **..**
qed

from *compact-eq-approx assms*(*1*,*2*) **obtain** i j **where** f: *approx* $i{\cdot}a = a$ *approx* $j{\cdot}b = b$
by *blast*
define k **where** $k = max\ i\ j$
with f **have** *idem*: *approx* $k{\cdot}a = a$ *approx* $k{\cdot}b = b$
by (*auto intro*: *idem-mono max.cobounded1 max.cobounded2*)
define D **where** $D \equiv range\ (\lambda x.\ approx\ k{\cdot}x)$
have *finite* D **unfolding** *D-def* **by** (*rule finite-range-approx*)

{
fix $z :: 'a$
define z' **where** $z' =$ *approx* $k{\cdot}z$
assume $a \sqsubseteq z$ $b \sqsubseteq z$
with *idem* **have** *deflated*: $a \sqsubseteq z'$ $b \sqsubseteq z'$
unfolding *z'-def*
by (*metis monofun-cfun-arg*)+
define B **where** $B = \{x \in D.\ a \sqsubseteq x \wedge b \sqsubseteq x \wedge x \sqsubseteq z'\}$
from ‹*finite D*› **have** *fin*: *finite* B **unfolding** *B-def* **by** *force*
have $z' \in D$ **unfolding** *z'-def D-def* **by** *simp*
then have $z' \in B$ **unfolding** *B-def* **using** *deflated* **by** *force*
then have *ne*: $B \neq \{\}$ **by** *blast*
from *fin ne* **have** $\exists\, mu \in B.\ (\forall\, x \in B.\ x \sqsubseteq mu \longrightarrow x = mu)$
proof (*induction B rule*: *finite-ne-induct*)
case (*singleton x*)
then show *?case* **by** *simp*
next
case (*insert nu F*)
show *?case*
by (*metis insert.IH below-trans insert-iff po-eq-conv*)
qed
then obtain *mu* **where** $mu \in B$ **and** *muR*: $\bigwedge x.\ x \in B \Longrightarrow x \sqsubseteq mu \Longrightarrow x = mu$
by *blast*
have *mu-idem*: *approx* $k{\cdot}mu = mu$
using ‹$mu \in B$›[*unfolded B-def D-def*]
approx-idem **by** *blast*
have *mu-lower*: $mu \sqsubseteq z'$ **using** ‹$mu \in B$›[*unfolded B-def*] **by** *blast*
have *min-ub a b mu*
proof
from ‹$mu \in B$›[*unfolded B-def*] **show** $a \sqsubseteq mu$ $b \sqsubseteq mu$ **by** *simp-all*
fix w **assume** $a \sqsubseteq w$ $b \sqsubseteq w$ $w \sqsubseteq mu$
with *idem mu-idem* **have** $a \sqsubseteq$ *approx* $k{\cdot}w \wedge b \sqsubseteq$ *approx* $k{\cdot}w$

```
      and under: approx k·w ⊑ mu
      by (metis monofun-cfun-arg)+
    with mu-lower have approx k·w ∈ B unfolding B-def D-def
      using below-trans by blast
    from this under have approx k·w = mu by (rule muR)
    then have mu ⊑ w using approx-below by blast
    with ‹w ⊑ mu› show w = mu by (rule below-antisym)
  qed
  moreover from mu-lower[unfolded z'-def] approx-below have mu ⊑ z
    by (rule below-trans)
  ultimately show ∃ w. min-ub a b w ∧ w ⊑ z by blast
 }
 {
  have w-idem: approx k·w = w if min-ub a b w for w
  proof −
    from that have a ⊑ w ∧ b ⊑ w by (simp add: min-ub-def)
    with idem have a ⊑ approx k·w ∧ b ⊑ approx k·w
      by (metis monofun-cfun-arg)
    with that approx-below show ?thesis by (simp add: min-ub-def)
  qed
  then have {w. min-ub a b w} ⊆ D unfolding D-def
    by (simp add: Collect-mono-iff deflation.range-eq-fixes
        deflation-approx)
  from this ‹finite D› show finite {w. min-ub a b w} by (rule finite-subset)
   from w-idem show min-ub a b w ⟹ compact w for w by (metis com-
pact-approx)
 }
qed



context begin
private alias Cb = Rep-compact-basis
definition pd-sup-set :: 'a pd-basis ⇒ 'a pd-basis ⇒ 'a compact-basis set
  where pd-sup-set A B ≡
⋃ a ∈ Rep-pd-basis A. ⋃ b ∈ Rep-pd-basis B. {w. min-ub (Cb a) (Cb b) (Cb w)}

lemma finite-pd-sup-set: finite (pd-sup-set A B)
proof −
 {
  fix a b :: 'a compact-basis
  have c: compact (Cb a) compact (Cb b) by simp-all
  note min-ub-finite = min-ub-finite[OF c]
  note min-ub-compact = min-ub-compact[OF c]
  have finite {w. min-ub (Cb a) (Cb b) w}
    by (auto intro: min-ub-finite)
  moreover have {w. min-ub (Cb a) (Cb b) (Cb w)} = Abs-compact-basis ‘ ...
    using Abs-compact-basis-inverse'[OF min-ub-compact, symmetric]
    by (auto intro: Rep-compact-basis-inverse[symmetric])
```

ultimately have *finite* {*w*. *min-ub* (*Rep-compact-basis a*) (*Rep-compact-basis b*) (*Rep-compact-basis w*)}
using *finite-imageI* **by** *simp*
}
with *finite-Rep-pd-basis*
show *?thesis* **unfolding** *pd-sup-set-def* **by** *blast*
qed

definition *pd-sup A B* ≡ *Abs-pd-basis* (*pd-sup-set A B*)

lemma *pd-sup-rep*: *pd-sup-set A B* ≠ {} ⟹ *Rep-pd-basis* (*pd-sup A B*) = *pd-sup-set A B*
unfolding *pd-sup-def*
using *finite-pd-sup-set*
Abs-pd-basis-inverse[*unfolded pd-basis-def mem-Collect-eq*]
by *blast*

lemma *min-ub-Cb-def*:
fixes *a b z* :: 'a::*bifinite compact-basis*
shows *min-ub* (*Cb a*) (*Cb b*) (*Cb z*) ⟷ *a* ⊑ *z* ∧ *b* ⊑ *z* ∧ (∀ *w*. *a* ⊑ *w* ⟶ *b* ⊑ *w* ⟶ *w* ⊑ *z* ⟶ *w* = *z*)
proof −
let *?noncompact* = ∀ *w*. *Cb a* ⊑ *w* ⟶ *Cb b* ⊑ *w* ⟶ *w* ⊑ *Cb z* ⟶ *w* = *Cb z*
let *?compact* = ∀ *w*. *a* ⊑ *w* ⟶ *b* ⊑ *w* ⟶ *w* ⊑ *z* ⟶ *w* = *z*
have *?noncompact* ⟷ *?compact*
term *type-definition*
proof
show *?noncompact* ⟹ *?compact* **unfolding** *compact-le-def*
using *type-definition.Rep-inject*[*OF type-definition-compact-basis*]
by *blast*
show *?noncompact* **if** *?compact*
proof (*intro allI*)
show *Cb a* ⊑ *w* ⟶ *Cb b* ⊑ *w* ⟶ *w* ⊑ *Cb z* ⟶ *w* = *Cb z* (**is** *?P w*) **for** *w*
proof (*induction w rule*: *compact-basis.principal-induct*)
from *Rep-compact-basis'*[*unfolded compact-def*]
show *adm ?P* **by** *simp*
from *that* **show** *?P* (*Cb w*) **for** *w* **by** *auto*
qed
qed
qed
then show *?thesis* **unfolding** *min-ub-def* **by** *simp*
qed

lemma *min-ub-below'*:
fixes *a z* :: 'a::*bifinite compact-basis*
shows *a* ⊑ *z* ⟹ *b* ⊑ *z* ⟹ ∃ *w*. *min-ub* (*Cb a*) (*Cb b*) (*Cb w*) ∧ *w* ⊑ *z*
proof −
assume *a* ⊑ *z b* ⊑ *z*

from *this*[*unfolded compact-le-def*] **obtain** *w* **where** *1*: *min-ub* (*Cb a*) (*Cb b*) *w* **and** *2*: $w \sqsubseteq Cb\ z$
using *min-ub-below Rep-compact-basis'* **by** *blast*
from *1* **have** *compact w* **using** *Rep-compact-basis' min-ub-compact* **by** *blast*
then have $w = Cb\ (Abs\text{-}compact\text{-}basis\ w)$ **by** *simp*
with *1 2 compact-le-def* **show** *?thesis* **by** *metis*
qed

lemma *pd-sup-ex*: **assumes** $A \leq\sharp\ Z\ B \leq\sharp\ Z$
shows *pd-sup-ne*: *pd-sup-set A B* $\neq \{\}$
and *pd-sup-le*: *pd-sup A B* $\leq\sharp\ Z$
proof −
{
fix *z* **assume** *z*: $z \in$ *Rep-pd-basis Z*
from *z assms*(*1*)[*unfolded upper-le-def*] **obtain** *a* **where** *a*: $a \in$ *Rep-pd-basis A* $a \sqsubseteq z$
by *blast*
from *z assms*(*2*)[*unfolded upper-le-def*] **obtain** *b* **where** *b*: $b \in$ *Rep-pd-basis B* $b \sqsubseteq z$
by *blast*
from *a*(*2*) *b*(*2*) **obtain** *w* **where** *w*: *min-ub* (*Cb a*) (*Cb b*) (*Cb w*) $w \sqsubseteq z$ **using** *min-ub-below'*
by *blast*
from *a*(*1*) *b*(*1*) *w*(*1*) **have** $w \in$ *pd-sup-set A B* **unfolding** *pd-sup-set-def* **by** *blast*
with *w*(*2*) **have** $\exists\, w \in$ *pd-sup-set A B*. $w \sqsubseteq z$ **by** *blast*
}
note *sup-finder* = *this*
from *Rep-pd-basis-nonempty* **obtain** *z* **where** $z \in$ *Rep-pd-basis Z* **by** *blast*
from *sup-finder*[*OF this*] **show** *pd-sup-set A B* $\neq \{\}$ **by** *blast*
then have *Rep-pd-basis* (*pd-sup A B*) = *pd-sup-set A B* **by** (*rule pd-sup-rep*)
with *sup-finder* **show** *pd-sup A B* $\leq\sharp\ Z$ **unfolding** *upper-le-def* **by** *blast*
qed

lemma *pd-sup-ge*:
assumes *pd-sup-set A B* $\neq \{\}$
shows $A \leq\sharp$ *pd-sup A B B* $\leq\sharp$ *pd-sup A B*
unfolding *upper-le-def pd-sup-rep*[*OF assms*]
pd-sup-set-def min-ub-Cb-def
by *blast*+

end

instantiation *upper-pd* :: (*bifinite*) *bounded-complete*
begin

instance
proof (*rule bounded-complete-finite-compact-joinI'*)

fix $x\ y\ z :: 'a\ upper\text{-}pd$
assume z: $x \sqsubseteq z\ y \sqsubseteq z$
define $unp :: 'a\ upper\text{-}pd \Rightarrow 'a\ pd\text{-}basis$
where $unp\ x = (SOME\ xb.\ x = upper\text{-}principal\ xb)$ **for** x
have $unp\text{-}p$: $compact\ x \Longrightarrow x = upper\text{-}principal\ (unp\ x)$ **for** x
unfolding $unp\text{-}def$
by $(drule\ upper\text{-}pd.compact\text{-}imp\text{-}principal)\ (rule\ someI\text{-}ex)$
define xb **where** $xb = unp\ x$
define yb **where** $yb = unp\ y$
define zb **where** $zb = unp\ z$
assume $compact\ x\ compact\ y\ compact\ z$
with $upper\text{-}pd.compact\text{-}imp\text{-}principal$
have $princ$: $x = upper\text{-}principal\ xb\ y = upper\text{-}principal\ yb$
$z = upper\text{-}principal\ zb$ **unfolding** $xb\text{-}def\ yb\text{-}def\ zb\text{-}def$ **by** $(fast\ intro: unp\text{-}p)+$
with z **have** $xb \leq\sharp\ zb\ yb \leq\sharp\ zb$ **by** $simp\text{-}all$
from $pd\text{-}sup\text{-}ex[OF\ this]$
have $xb \leq\sharp\ pd\text{-}sup\ xb\ yb\ yb \leq\sharp\ pd\text{-}sup\ xb\ yb\ pd\text{-}sup\ xb\ yb \leq\sharp\ zb$
by $(auto\ intro: pd\text{-}sup\text{-}ge)$
with $princ$ **show**
$x \sqsubseteq upper\text{-}principal\ (pd\text{-}sup\ xb\ yb)$
$y \sqsubseteq upper\text{-}principal\ (pd\text{-}sup\ xb\ yb)$
$upper\text{-}principal\ (pd\text{-}sup\ xb\ yb) \sqsubseteq z$ **by** $simp\text{-}all$
qed

lemma $upper\text{-}meet\text{-}eq$: $x \sqcap y = x \cup\sharp\ y$
proof $(rule\ below\text{-}antisym)$
show $x \cup\sharp\ y \sqsubseteq x \sqcap y$ **using** $meet\text{-}below\text{-}iff\ upper\text{-}plus\text{-}below1\ upper\text{-}plus\text{-}below2$
by $fast$
show $x \sqcap y \sqsubseteq x \cup\sharp\ y$ **using** $upper\text{-}below\text{-}plus\text{-}iff\ meet\text{-}below1\ meet\text{-}below2$ **by** $fast$
qed

end

instantiation $top :: (bounded\text{-}complete)\ bounded\text{-}complete$
begin

lemma $top\text{-}is\text{-}ub\text{-}transfer$: $(\lambda x.\ down{\cdot}x)\ `\ A <|\ down{\cdot}w \longleftrightarrow A <|\ w$
unfolding $is\text{-}ub\text{-}def$ **by** $simp$

lemma $top\text{-}is\text{-}lub\text{-}transfer$: $A <<|\ w \Longrightarrow (\lambda x.\ down{\cdot}x)\ `\ A <<|\ down{\cdot}w$
proof $(rule\ is\text{-}lubI)$
show $A <<|\ w \Longrightarrow (\lambda x.\ down{\cdot}x)\ `\ A <|\ down{\cdot}w$ **by** $(auto\ dest: is\text{-}lubD1\ simp\ add: top\text{-}is\text{-}ub\text{-}transfer)$
show $A <<|\ w \Longrightarrow (\lambda x.\ down{\cdot}x)\ `\ A <|\ u \Longrightarrow down{\cdot}w \sqsubseteq u$ **for** u
by $(cases\ u)\ (auto\ dest: is\text{-}lubD2\ simp\ add: top\text{-}is\text{-}ub\text{-}transfer)$
qed

lemma $is\text{-}ub\text{-}Top$: $A <|\ Top$

by (*auto intro*: *is-ubI*)

instance
proof
fix A :: $'a$ *top set* **and** b
show $\exists\, a.\ A <<| a$
proof (*cases Top* $\in A$)
case *True*
then have $A <<|$ *Top* **by** (*rule top-lub-Top*)
then show *?thesis* **..**
next
case *False*
then have *1*: $\forall\, x \in A.\ \exists\, y.\ x = down{\cdot}y$ **by** (*metis top-cases*)
define A' **where** $A' = \{y.\ \exists\, x \in A.\ x = down{\cdot}y\}$
with *1* **have** *2*: $A = (\lambda x.\ down{\cdot}x)$ ‘ A' **by** *fast*
show *?thesis* **proof** (*cases* $\exists\, w.\ A' <<| w$)
case *True*
then show *?thesis* **unfolding** *2* **by** (*fast intro*: *top-is-lub-transfer*)
next
case *False*
with *ex-bound-ex-lub* **have** *3*: $\nexists\, w.\ A' <| w$ **by** *auto*
then have *4*: $A <| w \Longrightarrow w = Top$ **for** w
unfolding *2* **using** *top-is-ub-transfer* **by** (*cases w*) *auto*
then have $A <<|$ *Top* **by** (*auto intro*: *is-lubI simp add*: *is-ub-Top*)
then show *?thesis* **..**
qed
qed
qed

lemmas *top-is-lub* = *ex-bound-ex-lub*[*OF is-ub-Top*]
lemmas *top-lub* = *ex-bound-lub*[*OF is-ub-Top*]

lemmas *top-join-is-lub* = *join-is-lub*[*OF below-Top below-Top*]

lemma [*simp*]: (*a* :: $'a$ *top*) $\Uparrow$ *b* **by** (*auto intro*: *joinableI below-Top*)

end

1.4 Joins: auxiliary lemmas

lemma *bottom-joinable*: $\bot \Uparrow$ (*y*::$'a$::*bounded-complete*)
by (*auto simp add*: *joinable-def*)

lemma *cfun-arg-joinable*:
fixes *a b* :: $'a$::*bounded-complete* **and** *f* :: $'a \to 'b$::*bounded-complete*
assumes $a \Uparrow b$ **shows** $f{\cdot}a \Uparrow f{\cdot}b$
using *assms* **unfolding** *joinable-def* **by** (*blast intro*: *monofun-cfun-arg*)

Deflations lie below *ID*, so any two are joinable.

lemma *deflation-joinable*:
assumes *deflation p* **and** *deflation q* **shows** $p \Uparrow q$
by (*rule joinableI*[*OF deflation.below-ID*[*OF assms*(*1*)] *deflation.below-ID*[*OF assms*(*2*)]])

lemma *compact-join*:
fixes $x\ y :: 'a$::*bounded-complete*
assumes *x*: *compact x* **and** *y*: *compact y* **and** *xy*: $x \Uparrow y$
shows *compact* $(x \sqcup y)$
proof (*rule compactI2*)
fix $Y :: nat \Rightarrow 'a$ **assume** *Y*: *chain Y* **and** *le*: $x \sqcup y \sqsubseteq (\bigsqcup i.\ Y\ i)$
from *le joinableD*[*OF xy*] **have** $x \sqsubseteq (\bigsqcup i.\ Y\ i)$ $y \sqsubseteq (\bigsqcup i.\ Y\ i)$
by (*blast intro*: *below-trans*)+
then obtain *i j* **where** $x \sqsubseteq Y\ i$ $y \sqsubseteq Y\ j$
using *compactD2*[*OF x Y*] *compactD2*[*OF y Y*] **by** *blast*
then have $x \sqsubseteq Y\ (max\ i\ j)$ $y \sqsubseteq Y\ (max\ i\ j)$
using *chain-mono*[*OF Y*] **by** (*blast intro*: *below-trans max.cobounded1 max.cobounded2*)+
then show $\exists i.\ x \sqcup y \sqsubseteq Y\ i$ **by** (*blast intro*: *join-belowI*)
qed

lemma *insert-join-is-lub*:
fixes $S :: 'a$::*bounded-complete set*
assumes *Sl*: $S <<|\ l$ **and** *j*: $z \Uparrow l$
shows *insert z S* $<<|\ z \sqcup l$
proof (*rule is-lubI*)
show *insert z S* $<|\ z \sqcup l$
proof (*rule is-ubI*)
fix *w* **assume** *w*: $w \in$ *insert z S*
show $w \sqsubseteq z \sqcup l$
proof (*cases* $w = z$)
case *True* **then show** *?thesis* **by** (*simp add*: *joinableD*(*1*)[*OF j*])
next
case *False* **with** *w* **have** $w \in S$ **by** *simp*
with *is-lubD1*[*OF Sl*] **have** $w \sqsubseteq l$ **by** (*rule is-ubD*)
also have $l \sqsubseteq z \sqcup l$ **by** (*rule joinableD*(*2*)[*OF j*])
finally show *?thesis* **.**
qed
qed
next
fix *u* **assume** *insert z S* $<|\ u$
then have *zu*: $z \sqsubseteq u$ **and** *Su*: $S <|\ u$ **by** (*auto simp*: *is-ub-insert*)
from *Sl Su* **have** $l \sqsubseteq u$ **by** (*rule is-lubD2*)
with *zu* **show** $z \sqcup l \sqsubseteq u$ **by** (*rule join-belowI*)
qed

end
theory *Distributive* **imports** *Bounded-Complete* **begin**

2 Distributive bounded-complete cpos

A bounded-complete cpo is *distributive* when binary meet distributes over joins of joinable pairs.

class *distributive-cpo* = *bounded-complete* +
  assumes *meet-join-distrib*: $a \Uparrow b \Longrightarrow c \sqcap (a \sqcup b) = (c \sqcap a) \sqcup (c \sqcap b)$
begin

lemma *meet-join-distrib2*: $a \Uparrow b \Longrightarrow (a \sqcup b) \sqcap c = (a \sqcap c) \sqcup (b \sqcap c)$
  by (*simp add*: *meet-join-distrib meet-sym*)

lemma *join-meet-distrib*: $a \sqcup (b \sqcap c) = (a \sqcup b) \sqcap (a \sqcup c)$ **if** $a \Uparrow b$ $a \Uparrow c$
proof −
  have *d1*: $(a \sqcup b) \sqcap c = (a \sqcap c) \sqcup (b \sqcap c)$
    by (*auto intro*: *meet-join-distrib2* ‹$a \Uparrow b$›)
  have $a \sqcup (b \sqcap c) = (a \sqcup (a \sqcap c)) \sqcup (b \sqcap c)$
    by (*simp add*: *join-absorb-meet2*)
  also have ... $= a \sqcup ((a \sqcap c) \sqcup (b \sqcap c))$
    by (*rule join-assoc*, *simp-all add*: *that d1*[*symmetric*])
  also have ... $= a \sqcup ((a \sqcup b) \sqcap c)$
    by (*simp add*: *d1*)
  also have ... $= ((a \sqcup b) \sqcap a) \sqcup ((a \sqcup b) \sqcap c)$
    by (*simp add*: *that*)
  also have ... $= (a \sqcup b) \sqcap (a \sqcup c)$
    by (*simp add*: *that meet-join-distrib meet-sym*)
  finally show *?thesis* **.**
qed

end

class *scott-domain* = *domain* + *distributive-cpo*

3 Instances

Two joinable elements of a flat domain are comparable, so distributivity is a case split.

instance *flat* ⊆ *distributive-cpo*
proof
  fix *a b c* :: $'a$ **assume** *ab*: $a \Uparrow b$
  have *mb*: $x \sqcap \bot = \bot$ **for** x :: $'a$ **by** (*simp add*: *meet-sym meet-absorbI*)
  from *joinableD*[*OF ab*] **have** $a = \bot \vee b = \bot \vee a = b$
    by (*metis ax-flat*)
  then show $c \sqcap (a \sqcup b) = c \sqcap a \sqcup (c \sqcap b)$
    by (*elim disjE*) (*simp-all add*: *mb join-bottom*)
qed

instance *fun* :: (*type*, *distributive-cpo*) *distributive-cpo*

```
proof
  fix a b c :: 'a ⇒ 'b
  assume ab: a ⇑ b
  have ptw: a x ⇑ b x for x
  proof −
    from ab obtain z where a ⊑ z b ⊑ z unfolding joinable-def by blast
    thus a x ⇑ b x by (meson fun-belowD joinableI)
  qed
  have cab: (c ⊓ a) ⇑ (c ⊓ b) by (rule joinableI[OF meet-below1 meet-below1])
  have c ⊓ (a ⊔ b) = (λx. c x ⊓ (a x ⊔ b x))
    using ab by (simp add: fun-meet fun-join)
  also have ... = (λx. (c x ⊓ a x) ⊔ (c x ⊓ b x))
    using ptw by (simp add: meet-join-distrib)
  also have ... = (c ⊓ a) ⊔ (c ⊓ b)
    using cab by (simp add: fun-meet fun-join)
  finally show c ⊓ (a ⊔ b) = (c ⊓ a) ⊔ (c ⊓ b) .
qed

instance u :: (distributive-cpo) distributive-cpo
proof
  fix a b c :: 'a u assume ab: a ⇑ b
  show c ⊓ (a ⊔ b) = c ⊓ a ⊔ (c ⊓ b)
  proof (cases a; cases b; cases c)
    fix a' b' c' assume ∗: a = up·a' b = up·b' c = up·c'
    with ab have a'b': a' ⇑ b' by simp
    have c' ⊓ a' ⇑ c' ⊓ b' by (rule joinableI[OF meet-below1 meet-below1])
    with ∗ a'b' show ?thesis by (simp add: up-join meet-join-distrib)
  qed simp-all
qed

instance ssum :: (distributive-cpo, distributive-cpo) distributive-cpo
proof
  fix a b c :: 'a ⊕ 'b assume ab: a ⇑ b
  have ml: x ⊓ x' ⇑ x ⊓ x'' for x x' x'' :: 'a by (rule joinableI[OF meet-below1
meet-below1])
  have mr: y ⊓ y' ⇑ y ⊓ y'' for y y' y'' :: 'b by (rule joinableI[OF meet-below1
meet-below1])
  show c ⊓ (a ⊔ b) = c ⊓ a ⊔ (c ⊓ b)
    using ab by (cases a; cases b; cases c)
      (simp-all add: sinl-join sinr-join meet-join-distrib ml mr)
qed

instance prod :: (distributive-cpo, distributive-cpo) distributive-cpo
proof
  fix a b c :: 'a × 'b assume ab: a ⇑ b
  then have c: fst a ⇑ fst b snd a ⇑ snd b by (simp-all add: prod-joinable-iff)
  have c ⊓ a ⇑ c ⊓ b by (rule joinableI[OF meet-below1 meet-below1])
  with ab c show c ⊓ (a ⊔ b) = c ⊓ a ⊔ (c ⊓ b)
    by (simp add: prod-meet prod-join meet-join-distrib)
```

qed

4 Continuity of meet, and distributivity from compacts

In a bifinite bounded-complete domain binary meet is continuous: this is the *approximants* observation a compact below $(\bigsqcup i.\ X\ i) \sqcap y$ already lies below some $X\ i \sqcap y$.

lemma *approximants-meet*:
  approximants $((x::'a::\{$*bifinite*, *bounded-complete*$\}) \sqcap y) =$ *approximants* $x \cap$ *approximants* y
  unfolding *approximants-def* **by** (*auto simp add*: *meet-below-iff*)

lemma *cont-meet-arg*: *cont* $(\lambda x::'a::\{$*bifinite*, *bounded-complete*$\}.\ x \sqcap y)$
proof −
  have *mono*: *monofun* $(\lambda x::'a.\ x \sqcap y)$
  proof (*rule monofunI*)
    fix $a\ b :: 'a$ **assume** $a \sqsubseteq b$
    have $a \sqcap y \sqsubseteq b$ **using** *meet-below1*[*of a y*] ‹$a \sqsubseteq b$› **by** (*rule below-trans*)
    moreover have $a \sqcap y \sqsubseteq y$ **by** (*rule meet-below2*)
    ultimately show $a \sqcap y \sqsubseteq b \sqcap y$ **by** (*rule meet-greatest*)
  qed
  show *?thesis*
  proof (*rule contI2*[*OF mono*])
    fix $X :: nat \Rightarrow 'a$ **assume** X: *chain X*
    assume *chXy*: *chain* $(\lambda i.\ X\ i \sqcap y)$
    show $(\bigsqcup i.\ X\ i) \sqcap y \sqsubseteq (\bigsqcup i.\ X\ i \sqcap y)$
      by (*rule compact-basis.belowI*) (*simp add*: *compact-basis.rep-lub X chXy approximants-meet*)
  qed
qed

lemma *cont-meet-arg2*: *cont* $(\lambda y::'a::\{$*bifinite*, *bounded-complete*$\}.\ x \sqcap y)$
  using *cont-meet-arg*[*of x*] **by** (*simp add*: *meet-sym*)

lemma *cont2cont-meet* [*simp*, *cont2cont*]:
  fixes $f\ g :: 'b::cpo \Rightarrow 'a::\{$*bifinite*, *bounded-complete*$\}$
  assumes *contf*: *cont f* **and** *contg*: *cont g* **shows** *cont* $(\lambda x.\ f\ x \sqcap g\ x)$
  by (*rule cont-apply* [*OF contg cont-meet-arg2 cont-compose* [*OF cont-meet-arg contf*]])

lemma *meet-lub-chains*:
  fixes $X\ Y :: nat \Rightarrow 'a::\{$*bifinite*, *bounded-complete*$\}$
  assumes *chain X chain Y* **shows** $(\bigsqcup i.\ X\ i) \sqcap (\bigsqcup i.\ Y\ i) = (\bigsqcup i.\ X\ i \sqcap Y\ i)$
proof −
  have *c*: *cont* $(\lambda p::'a \times 'a.\ fst\ p \sqcap snd\ p)$ **by** *simp*
  have *ch*: *chain* $(\lambda i.\ (X\ i,\ Y\ i))$ **using** *assms* **by** (*simp add*: *chain-def*)

have $(\bigsqcup i.\ (X\ i,\ Y\ i)) = ((\bigsqcup i.\ X\ i), (\bigsqcup i.\ Y\ i))$ **using** *assms* **by** (*rule lub-Pair*)
with *cont2contlubE*[*OF c ch*] **show** *?thesis* **by** *simp*
qed

Joins of pointwise joinable chains: the lubs are joinable, and joining commutes with lubs.

lemma *lub-join-chains*:
fixes $X\ Y :: nat \Rightarrow 'a$::*bounded-complete*
assumes X: *chain* X **and** Y: *chain* Y **and** j: $\bigwedge i.\ X\ i \Uparrow Y\ i$
shows *chain* $(\lambda i.\ X\ i \sqcup Y\ i)$ $(\bigsqcup i.\ X\ i) \Uparrow (\bigsqcup i.\ Y\ i)$
$(\bigsqcup i.\ X\ i \sqcup Y\ i) = (\bigsqcup i.\ X\ i) \sqcup (\bigsqcup i.\ Y\ i)$
proof −
have *XY*: $X\ i \sqsubseteq (\bigsqcup i.\ X\ i)$ $Y\ i \sqsubseteq (\bigsqcup i.\ Y\ i)$ **for** i **using** $X\ Y$ **by** (*simp-all add*: *is-ub-thelub*)
show *ch*: *chain* $(\lambda i.\ X\ i \sqcup Y\ i)$
proof (*rule chainI*)
fix i
have $X\ i \sqsubseteq X\ (Suc\ i) \sqcup Y\ (Suc\ i)$ $Y\ i \sqsubseteq X\ (Suc\ i) \sqcup Y\ (Suc\ i)$
using *chainE*[*OF X*, *of i*] *chainE*[*OF Y*, *of i*] *joinableD*[*OF j*[*of Suc i*]]
by (*blast intro*: *below-trans*)+
then show $X\ i \sqcup Y\ i \sqsubseteq X\ (Suc\ i) \sqcup Y\ (Suc\ i)$ **by** (*rule join-belowI*)
qed
have $(\bigsqcup i.\ X\ i) \sqsubseteq (\bigsqcup i.\ X\ i \sqcup Y\ i)$ $(\bigsqcup i.\ Y\ i) \sqsubseteq (\bigsqcup i.\ X\ i \sqcup Y\ i)$
using *joinableD*[*OF j*] **by** (*blast intro*: *lub-mono X Y ch*)+
then show *jl*: $(\bigsqcup i.\ X\ i) \Uparrow (\bigsqcup i.\ Y\ i)$ **by** (*rule joinableI*)
show $(\bigsqcup i.\ X\ i \sqcup Y\ i) = (\bigsqcup i.\ X\ i) \sqcup (\bigsqcup i.\ Y\ i)$
proof (*rule below-antisym*)
show $(\bigsqcup i.\ X\ i \sqcup Y\ i) \sqsubseteq (\bigsqcup i.\ X\ i) \sqcup (\bigsqcup i.\ Y\ i)$
proof (*rule lub-below*[*OF ch*])
fix i
show $X\ i \sqcup Y\ i \sqsubseteq (\bigsqcup i.\ X\ i) \sqcup (\bigsqcup i.\ Y\ i)$
using *XY joinableD*[*OF jl*] **by** (*blast intro*: *join-belowI below-trans*)
qed
show $(\bigsqcup i.\ X\ i) \sqcup (\bigsqcup i.\ Y\ i) \sqsubseteq (\bigsqcup i.\ X\ i \sqcup Y\ i)$
using ‹$(\bigsqcup i.\ X\ i) \sqsubseteq$ -› ‹$(\bigsqcup i.\ Y\ i) \sqsubseteq$ -› **by** (*rule join-belowI*)
qed
qed

Distributivity through any chain of maps with lub *ID* (e.g. the *take* functions of a domain, or an approximation chain): it suffices to meet with approximants $t\ n{\cdot}c$, against arbitrary joinable a, b; the limit is taken in c only, by continuity of meet.

lemma *chain-distrib*:
fixes $t :: nat \Rightarrow 'a$::{*bifinite*, *bounded-complete*} $\to 'a$ **and** $a\ b\ c :: 'a$
assumes t: *chain* t $(\bigsqcup i.\ t\ i) = ID$
and *law*: $\bigwedge n\ a\ b.\ a \Uparrow b \Longrightarrow t\ n{\cdot}c \sqcap (a \sqcup b) = t\ n{\cdot}c \sqcap a \sqcup t\ n{\cdot}c \sqcap b$
and $a \Uparrow b$
shows $c \sqcap (a \sqcup b) = c \sqcap a \sqcup c \sqcap b$
proof −

have *ch*: *chain* ($\lambda n.\ t\ n{\cdot}c$) **by** (*rule ch2ch-Rep-cfunL*[*OF t*(*1*)])
have *reach*: ($\bigsqcup n.\ t\ n{\cdot}c$) = *c* **by** (*simp add*: *contlub-cfun-fun*[*OF t*(*1*), *symmetric*] *t*(*2*))
have *lim*: $c \sqcap y = (\bigsqcup n.\ t\ n{\cdot}c \sqcap y)$ **and** *chy*: *chain* ($\lambda n.\ t\ n{\cdot}c \sqcap y$) **for** *y*
using *cont2contlubE*[*OF cont-meet-arg ch*, *of y*] *ch2ch-cont*[*OF cont-meet-arg ch*, *of y*]
by (*simp-all add*: *reach*)
have *j*: $t\ n{\cdot}c \sqcap a \Uparrow t\ n{\cdot}c \sqcap b$ **for** *n* **by** (*rule joinableI*[*OF meet-below1 meet-below1*])
have $c \sqcap (a \sqcup b) = (\bigsqcup n.\ t\ n{\cdot}c \sqcap (a \sqcup b))$ **by** (*rule lim*)
also have $\ldots = (\bigsqcup n.\ t\ n{\cdot}c \sqcap a \sqcup t\ n{\cdot}c \sqcap b)$ **by** (*simp only*: *law*[*OF* ‹$a \Uparrow b$›])
also have $\ldots = (\bigsqcup n.\ t\ n{\cdot}c \sqcap a) \sqcup (\bigsqcup n.\ t\ n{\cdot}c \sqcap b)$ **by** (*rule lub-join-chains*(*3*)[*OF chy chy j*])
also have $\ldots = c \sqcap a \sqcup c \sqcap b$ **by** (*simp only*: *lim*[*symmetric*])
finally show *?thesis* **.**
qed

lemma *distributive-cpo-chainI*:
fixes *t* :: *nat* ⇒ *'a*::{*bifinite*, *bounded-complete*} → *'a*
assumes *t*: *chain t* ($\bigsqcup i.\ t\ i$) = *ID*
and *law*: $\bigwedge n\ a\ b\ c.\ a \Uparrow b \Longrightarrow t\ n{\cdot}c \sqcap (a \sqcup b) = t\ n{\cdot}c \sqcap a \sqcup t\ n{\cdot}c \sqcap b$
shows *OFCLASS*(*'a*, *distributive-cpo-class*)
by *intro-classes* (*rule chain-distrib*[*OF t law*])

For a fixed *c*, the law against compact joinable pairs extends to all joinable pairs (approximating *a* and *b*; meet is continuous in its second argument).

lemma *compact-meet-distrib*:
fixes *a b c* :: *'a*::{*bifinite*, *bounded-complete*}
assumes *law*: $\bigwedge a\ b.$ *compact a* ⟹ *compact b* ⟹ $a \Uparrow b \Longrightarrow c \sqcap (a \sqcup b) = c \sqcap a \sqcup c \sqcap b$
and $a \Uparrow b$
shows $c \sqcap (a \sqcup b) = c \sqcap a \sqcup c \sqcap b$
proof −
from *bifinite* **obtain** *approx* :: *nat* ⇒ *'a* → *'a* **where** *approx-chain approx* **..**
then interpret *approx-chain approx* **.**
have *reach*: ($\bigsqcup i.$ *approx i*·*x*) = *x* **for** *x*
by (*simp add*: *contlub-cfun-fun*[*OF chain-approx*, *symmetric*])
have *ch*: *chain* ($\lambda i.$ *approx i*·*x*) **for** *x* **by** (*rule ch2ch-Rep-cfunL*[*OF chain-approx*])
have *lim*: $c \sqcap (\bigsqcup i.\ X\ i) = (\bigsqcup i.\ c \sqcap X\ i)$ **and** *chX*: *chain* ($\lambda i.\ c \sqcap X\ i$) **if** *chain X* **for** *X*
using *cont2contlubE*[*OF cont-meet-arg2 that*] *ch2ch-cont*[*OF cont-meet-arg2 that*] **by** *simp-all*
define *A* **where** *A* = ($\lambda i.$ *approx i*·*a*)
define *B* **where** *B* = ($\lambda i.$ *approx i*·*b*)
have *chs*: *chain A chain B* **unfolding** *A-def B-def* **by** (*rule ch*)+
have *AB*: *A i* ⇑ *B i* **for** *i*
unfolding *A-def B-def* **using** ‹$a \Uparrow b$› **by** (*rule cfun-arg-joinable*)
have *j*: $c \sqcap A\ i \Uparrow c \sqcap B\ i$ **for** *i* **by** (*rule joinableI*[*OF meet-below1 meet-below1*])
have $c \sqcap (a \sqcup b) = c \sqcap ((\bigsqcup i.\ A\ i) \sqcup (\bigsqcup i.\ B\ i))$ **by** (*simp only*: *A-def B-def reach*)

also have ... = $c \sqcap (\bigsqcup i.\ A\ i \sqcup B\ i)$ **by** (*simp only*: *lub-join-chains*(*3*)[*OF chs AB*])
also have ... = $(\bigsqcup i.\ c \sqcap (A\ i \sqcup B\ i))$ **by** (*rule lim*[*OF lub-join-chains*(*1*)[*OF chs AB*]])
also have ... = $(\bigsqcup i.\ c \sqcap A\ i \sqcup c \sqcap B\ i)$
using *AB* **by** (*simp add*: *law A-def B-def*)
also have ... = $(\bigsqcup i.\ c \sqcap A\ i) \sqcup (\bigsqcup i.\ c \sqcap B\ i)$
by (*rule lub-join-chains*(*3*)[*OF chX*[*OF chs*(*1*)] *chX*[*OF chs*(*2*)] *j*])
also have ... = $c \sqcap a \sqcup c \sqcap b$
by (*simp only*: *lim*[*OF chs*(*1*), *symmetric*] *lim*[*OF chs*(*2*), *symmetric*]) (*simp only*: *A-def B-def reach*)
finally show *?thesis* **.**
qed

So distributivity of a bifinite bounded-complete domain need only be checked on compacts: the approximation-chain instance of *chain-distrib*.

lemma *distributive-cpo-compactI*:
assumes $\bigwedge a\ b\ c{::}'a{::}\{$*bifinite*, *bounded-complete*$\}$. *compact* $a \Longrightarrow$ *compact* $b \Longrightarrow$ *compact* $c \Longrightarrow$
$a \Uparrow b \Longrightarrow c \sqcap (a \sqcup b) = c \sqcap a \sqcup (c \sqcap b)$
shows *OFCLASS*($'a$, *distributive-cpo-class*)
proof *intro-classes*
fix $a\ b\ c :: 'a$ **assume** $a \Uparrow b$
from *bifinite* **obtain** *approx* :: *nat* $\Rightarrow 'a \to 'a$ **where** *approx-chain approx* **..**
then interpret *approx-chain approx* **.**
show $c \sqcap (a \sqcup b) = c \sqcap a \sqcup c \sqcap b$
by (*rule chain-distrib*[*OF chain-approx lub-approx* - ‹$a \Uparrow b$›], *rule compact-meet-distrib*, *rule assms*) *simp-all*
qed

end
theory *Filter-Sum* **imports** *Distributive* **begin**

5 Filter sums

A pcpo is a *filter sum* when any two nonbottom elements with a common upper bound also have a common nonbottom lower bound. In a bounded-complete pcpo this says that joinable nonbottom elements have a nonbottom meet (*filter-sum-meet* below). Equivalently, every connected component of the punctured space $D - \{\bot\}$ is a filter, i.e. $D \cong (\Sigma_i\ F_i)_\bot$. This is what keeps the smash product distributive. (When every component has a least element, D is a *separated sum*, the lifted disjoint sum of pointed domains.)

class *filter-sum* = *pcpo* +
assumes *filter-sum*: $x \neq \bot \Longrightarrow y \neq \bot \Longrightarrow x \sqsubseteq u \Longrightarrow y \sqsubseteq u \Longrightarrow \exists l.\ l \neq \bot \wedge l \sqsubseteq x \wedge l \sqsubseteq y$

Bounded-above nonbottom elements have nonbottom lower bounds with any

third element together: if z has one with y, it has one with x. No lattice operations needed.

lemma *filter-sum-lower-cong*:
fixes $x\ y\ z :: 'a$::*filter-sum*
assumes x: $x \neq \bot$ **and** xu: $x \sqsubseteq u$ **and** yu: $y \sqsubseteq u$
and w: $w \neq \bot\ w \sqsubseteq z\ w \sqsubseteq y$
shows $\exists v.\ v \neq \bot \wedge v \sqsubseteq z \wedge v \sqsubseteq x$
proof −
have $w \sqsubseteq u$ **using** ‹$w \sqsubseteq y$› *yu* **by** (*rule below-trans*)
from *filter-sum*[*OF* ‹$w \neq \bot$› *x this xu*] **obtain** v **where** $v \neq \bot\ v \sqsubseteq w\ v \sqsubseteq x$ **by** *fast*
with ‹$w \sqsubseteq z$› **show** *?thesis* **by** (*fast dest*: *below-trans*)
qed

Separation through any chain of maps with lub *ID* (e.g. the *take* functions of a domain): it suffices to separate approximants $t\ n{\cdot}x$ from arbitrary y. A nonbottom x has a nonbottom approximant, and a witness below it lies below x.

lemma *filter-sum-chainI*:
fixes $t :: nat \Rightarrow 'a$::*pcpo* $\to 'a$
assumes t: *chain* t $(\bigsqcup i.\ t\ i) = ID$
and *sep*: $\bigwedge n\ x\ y\ u.\ t\ n{\cdot}x \neq \bot \Longrightarrow y \neq \bot \Longrightarrow t\ n{\cdot}x \sqsubseteq u \Longrightarrow y \sqsubseteq u \Longrightarrow$
$\exists l.\ l \neq \bot \wedge l \sqsubseteq t\ n{\cdot}x \wedge l \sqsubseteq y$
shows *OFCLASS*('*a*, *filter-sum-class*)
proof
fix $x\ y\ u :: 'a$ **assume** $x \neq \bot\ y \neq \bot\ x \sqsubseteq u\ y \sqsubseteq u$
have *chx*: *chain* $(\lambda n.\ t\ n{\cdot}x)$ **by** (*rule ch2ch-Rep-cfunL*[*OF t*(*1*)])
have *reach*: $(\bigsqcup n.\ t\ n{\cdot}x) = x$ **by** (*simp add*: *contlub-cfun-fun*[*OF t*(*1*), *symmetric*] *t*(*2*))
have *below*: $t\ n{\cdot}x \sqsubseteq x$ **for** n **using** *is-ub-thelub*[*OF chx*, *of n*] **by** (*simp add*: *reach*)
have $\exists n.\ t\ n{\cdot}x \neq \bot$ **using** ‹$x \neq \bot$› *lub-eq-bottom-iff*[*OF chx*] **by** (*simp add*: *reach*)
then obtain n :: *nat* **where** *n*: $t\ n{\cdot}x \neq \bot$ **by** (*elim exE*)
have $t\ n{\cdot}x \sqsubseteq u$ **using** *below* ‹$x \sqsubseteq u$› **by** (*rule below-trans*)
from *sep*[*OF n* ‹$y \neq \bot$› *this* ‹$y \sqsubseteq u$›] **obtain** $l :: 'a$ **where** *l*: $l \neq \bot\ l \sqsubseteq t\ n{\cdot}x\ l \sqsubseteq y$
by (*elim exE conjE*)
from *l*(*2*) *below* **have** $l \sqsubseteq x$ **by** (*rule below-trans*)
with *l*(*1*,*3*) **show** $\exists l.\ l \neq \bot \wedge l \sqsubseteq x \wedge l \sqsubseteq y$ **by** (*intro exI*[*of - l*]) *simp*
qed

5.1 The meet form, in bounded-complete pcpos

A nonbottom meet is the same as a nonbottom common lower bound; through this, the lattice-free statements above become statements about meets.

lemma *nonbot-meet-iff*:

fixes $x\ y :: 'a$::*bounded-complete*
shows $x \sqcap y \neq \bot \longleftrightarrow (\exists l.\ l \neq \bot \wedge l \sqsubseteq x \wedge l \sqsubseteq y)$
proof
assume $x \sqcap y \neq \bot$
then show $\exists l.\ l \neq \bot \wedge l \sqsubseteq x \wedge l \sqsubseteq y$
by (*intro exI*[*of* - $x \sqcap y$] *conjI meet-below1 meet-below2*)
next
assume $\exists l.\ l \neq \bot \wedge l \sqsubseteq x \wedge l \sqsubseteq y$
then obtain $l :: 'a$ **where** $l \neq \bot\ l \sqsubseteq x\ l \sqsubseteq y$ **by** (*elim exE conjE*)
then show $x \sqcap y \neq \bot$ **by** (*metis bottomI meet-greatest*)
qed

lemma *filter-sum-meet*:
fixes $x\ y :: 'a$::{*filter-sum*, *bounded-complete*}
assumes $x \Uparrow y\ x \sqcap y = \bot$ **shows** $x = \bot \vee y = \bot$
proof −
from ‹$x \Uparrow y$› **obtain** $u :: 'a$ **where** $x \sqsubseteq u\ y \sqsubseteq u$ **unfolding** *joinable-def* **by** (*elim exE conjE*)
with *filter-sum*[*of x y u*] ‹$x \sqcap y = \bot$› **show** *?thesis*
by (*auto simp*: *nonbot-meet-iff*[*symmetric*])
qed

Joinable nonbottom elements meet any third element to $\bot$ together.

lemma *filter-sum-meet-bottom-cong*:
fixes $x\ y\ z :: 'a$::{*filter-sum*, *bounded-complete*}
assumes $x \Uparrow y\ x \neq \bot\ y \neq \bot\ z \sqcap x = \bot$ **shows** $z \sqcap y = \bot$
proof (*rule ccontr*)
assume $z \sqcap y \neq \bot$
then obtain $w :: 'a$ **where** w: $w \neq \bot\ w \sqsubseteq z\ w \sqsubseteq y$
unfolding *nonbot-meet-iff* **by** (*elim exE conjE*)
from ‹$x \Uparrow y$› **obtain** $u :: 'a$ **where** u: $x \sqsubseteq u\ y \sqsubseteq u$ **unfolding** *joinable-def* **by** (*elim exE conjE*)
have $\exists v.\ v \neq \bot \wedge v \sqsubseteq z \wedge v \sqsubseteq x$
using *filter-sum-lower-cong*[*of x u y w z*] ‹$x \neq \bot$› $u\ w$ **by** *simp*
then have $z \sqcap x \neq \bot$ **by** (*simp add*: *nonbot-meet-iff*)
with ‹$z \sqcap x = \bot$› **show** *False* **by** *simp*
qed

5.2 Instances

lemma (**in** *flat*) *flat-joinableE*: **assumes** $a \Uparrow b$ **obtains** $a = \bot \mid b = \bot \mid a = b$
by (*metis ax-flat joinableD assms*)

instance *flat* ⊆ *filter-sum*
proof
fix $x\ y\ u :: 'a$ **assume** $x \neq \bot\ y \neq \bot\ x \sqsubseteq u\ y \sqsubseteq u$
then have $x = y$ **by** (*metis ax-flat*)
with ‹$x \neq \bot$› **show** $\exists l.\ l \neq \bot \wedge l \sqsubseteq x \wedge l \sqsubseteq y$ **by** (*intro exI*[*of* - x]) *simp*
qed

Lifting needs no assumption on the argument beyond pointedness: $up{\cdot}\bot$ lies below every nonbottom element.

instance $u :: (pcpo)$ *filter-sum*
proof
  fix $x\ y\ u :: 'a\ u$ **assume** $x \neq \bot\ y \neq \bot$
  then show $\exists\, l.\ l \neq \bot \wedge l \sqsubseteq x \wedge l \sqsubseteq y$
    by (*intro exI*[*of* - $up{\cdot}\bot$]) (*cases x*; *cases y*; *simp*)
qed

A common upper bound forces both elements into the same summand; separate there.

instance *ssum* :: ({*pcpo*, *filter-sum*}, {*pcpo*, *filter-sum*}) *filter-sum*
proof
  fix $x\ y\ u :: 'a \oplus 'b$ **assume** *nb*: $x \neq \bot\ y \neq \bot$ **and** *le*: $x \sqsubseteq u\ y \sqsubseteq u$
  have *L*: $\exists\, l.\ l \neq \bot \wedge l \sqsubseteq (sinl{\cdot}a :: 'a \oplus 'b) \wedge l \sqsubseteq sinl{\cdot}b$
    if $a \neq \bot\ b \neq \bot\ a \sqsubseteq c\ b \sqsubseteq c$ **for** $a\ b\ c :: 'a$
  proof −
    from *filter-sum*[*OF that*] **obtain** $l :: 'a$ **where** $l \neq \bot\ l \sqsubseteq a\ l \sqsubseteq b$
      by (*elim exE conjE*)
    then show *?thesis* **by** (*intro exI*[*of* - $sinl{\cdot}l$]) *simp*
  qed
  have *R*: $\exists\, l.\ l \neq \bot \wedge l \sqsubseteq (sinr{\cdot}a :: 'a \oplus 'b) \wedge l \sqsubseteq sinr{\cdot}b$
    if $a \neq \bot\ b \neq \bot\ a \sqsubseteq c\ b \sqsubseteq c$ **for** $a\ b\ c :: 'b$
  proof −
    from *filter-sum*[*OF that*] **obtain** $l :: 'b$ **where** $l \neq \bot\ l \sqsubseteq a\ l \sqsubseteq b$
      by (*elim exE conjE*)
    then show *?thesis* **by** (*intro exI*[*of* - $sinr{\cdot}l$]) *simp*
  qed
  show $\exists\, l.\ l \neq \bot \wedge l \sqsubseteq x \wedge l \sqsubseteq y$
  proof (*cases x*)
    case *bottom* **with** *nb* **show** *?thesis* **by** *simp*
  next
    case (*sinl a*)
   from *le*(*1*) *sinl* **obtain** *c* **where** *u*: $u = sinl{\cdot}c$ **and** *ac*: $a \sqsubseteq c$ **by** (*cases u*) *auto*
    from *le*(*2*) *nb*(*2*) *u* **obtain** *b* **where** *y*: $y = sinl{\cdot}b\ b \neq \bot$ **and** *bc*: $b \sqsubseteq c$ **by** (*cases y*) *auto*
    from *L*[*OF sinl*(*2*) *y*(*2*) *ac bc*] **show** *?thesis* **by** (*simp only*: *sinl*(*1*) *y*(*1*))
  next
    case (*sinr a*)
    from *le*(*1*) *sinr* **obtain** *c* **where** *u*: $u = sinr{\cdot}c$ **and** *ac*: $a \sqsubseteq c$ **by** (*cases u*) *auto*
    from *le*(*2*) *nb*(*2*) *u* **obtain** *b* **where** *y*: $y = sinr{\cdot}b\ b \neq \bot$ **and** *bc*: $b \sqsubseteq c$ **by** (*cases y*) *auto*
    from *R*[*OF sinr*(*2*) *y*(*2*) *ac bc*] **show** *?thesis* **by** (*simp only*: *sinr*(*1*) *y*(*1*))
  qed
qed

Componentwise: the pair of the two components' witnesses.

instance *sprod* :: ({*pcpo*, *filter-sum*}, {*pcpo*, *filter-sum*}) *filter-sum*

proof
  fix $x\ y\ u :: 'a \otimes 'b$ **assume** *nb*: $x \neq \bot\ y \neq \bot$ **and** *le*: $x \sqsubseteq u\ y \sqsubseteq u$
  have *S*: $\exists l.\ l \neq \bot \wedge l \sqsubseteq (:a,\ b:) \wedge l \sqsubseteq (:a',\ b':)$
    if $a \neq \bot\ b \neq \bot\ a' \neq \bot\ b' \neq \bot\ a \sqsubseteq c\ a' \sqsubseteq c\ b \sqsubseteq d\ b' \sqsubseteq d$
    for $a\ a'\ c :: 'a$ **and** $b\ b'\ d :: 'b$
  proof −
    from *filter-sum*[*OF* ‹$a \neq \bot$› ‹$a' \neq \bot$› ‹$a \sqsubseteq c$› ‹$a' \sqsubseteq c$›]
    obtain *la* :: $'a$ **where** $la \neq \bot\ la \sqsubseteq a\ la \sqsubseteq a'$ **by** (*elim exE conjE*)
    moreover from *filter-sum*[*OF* ‹$b \neq \bot$› ‹$b' \neq \bot$› ‹$b \sqsubseteq d$› ‹$b' \sqsubseteq d$›]
    obtain *lb* :: $'b$ **where** $lb \neq \bot\ lb \sqsubseteq b\ lb \sqsubseteq b'$ **by** (*elim exE conjE*)
    ultimately show *?thesis* **using** *that*
      by (*intro exI*[*of* - (:*la*, *lb*:)] *conjI*) (*simp-all only*: *spair-below spair-bottom-iff simp-thms*)
  qed
  obtain *x1 x2* **where** *x*: $x = (:x1,\ x2:)\ x1 \neq \bot\ x2 \neq \bot$ **using** *nb*(*1*) **by** (*cases x*) *auto*
  obtain *y1 y2* **where** *y*: $y = (:y1,\ y2:)\ y1 \neq \bot\ y2 \neq \bot$ **using** *nb*(*2*) **by** (*cases y*) *auto*
  obtain *u1 u2* **where** *u*: $u = (:u1,\ u2:)$ **using** *le*(*1*) *x* **by** (*cases u*) *auto*
  have $x1 \sqsubseteq u1\ x2 \sqsubseteq u2\ y1 \sqsubseteq u1\ y2 \sqsubseteq u2$
    using *le*(*1*)[*unfolded x*(*1*) *u*] *le*(*2*)[*unfolded y*(*1*) *u*]
    by (*simp-all only*: *spair-below x*(*2*,*3*) *y*(*2*,*3*) *simp-thms*)
  from *S*[*OF x*(*2*,*3*) *y*(*2*,*3*) *this*(*1*,*3*,*2*,*4*)] **show** $\exists l.\ l \neq \bot \wedge l \sqsubseteq x \wedge l \sqsubseteq y$
    by (*simp only*: *x*(*1*) *y*(*1*))
qed

6 Separated sums

An *atom* is a minimal nonbottom element. A pcpo is a *separated sum* when any two nonbottom elements with a common upper bound lie above a common atom: atomicity and separation in one statement. Equivalently every component of the punctured space $D - \{\bot\}$ has a least element, i.e. $D \cong (\Sigma_i\ C_i)_\bot$ for pointed C_i, the separated sum of the C_i; that least element is the component's unique atom.

definition *atom* :: $'a$::*pcpo* ⇒ *bool* **where**
  $atom\ l \longleftrightarrow l \neq \bot \wedge (\forall z.\ z \sqsubseteq l \longrightarrow z = \bot \vee z = l)$

lemma *atomI*: $l \neq \bot \Longrightarrow (\bigwedge z.\ z \sqsubseteq l \Longrightarrow z = \bot \vee z = l) \Longrightarrow atom\ l$
  by (*simp add*: *atom-def*)

lemma *atom-nonbottom*: $atom\ l \Longrightarrow l \neq \bot$
  by (*simp add*: *atom-def*)

lemma *atomD*: $atom\ l \Longrightarrow z \sqsubseteq l \Longrightarrow z = \bot \vee z = l$
  by (*simp add*: *atom-def*)

class *separated-sum* = *pcpo* +

assumes *separated-sum*: $x \neq \bot \Longrightarrow y \neq \bot \Longrightarrow x \sqsubseteq u \Longrightarrow y \sqsubseteq u \Longrightarrow \exists\, l.\ atom\ l \wedge l \sqsubseteq x \wedge l \sqsubseteq y$

instance *separated-sum* $\subseteq$ *filter-sum*
proof
fix *x y u* :: *'a* **assume** $x \neq \bot\ y \neq \bot\ x \sqsubseteq u\ y \sqsubseteq u$
from *separated-sum*[*OF this*] **show** $\exists\, l.\ l \neq \bot \wedge l \sqsubseteq x \wedge l \sqsubseteq y$
by (*blast dest*: *atom-nonbottom*)
qed

lemma *atom-below*:
fixes *x* :: *'a*::*separated-sum*
assumes $x \neq \bot$ **obtains** *l* **where** *atom l l* $\sqsubseteq x$
using *separated-sum*[*OF assms assms below-refl below-refl*] **by** *blast*

Two atoms with a common upper bound are equal.

lemma *atom-unique*:
fixes *a b* :: *'a*::*separated-sum*
assumes *a*: *atom a* **and** *b*: *atom b* **and** $a \sqsubseteq u\ b \sqsubseteq u$ **shows** $a = b$
proof −
from *separated-sum*[*OF atom-nonbottom*[*OF a*] *atom-nonbottom*[*OF b*] *assms*(*3*,*4*)]
obtain *l* **where** *l*: *atom l l* $\sqsubseteq a\ l \sqsubseteq b$ **by** *blast*
have $l = a$ **using** *atomD*[*OF a l*(*2*)] *atom-nonbottom*[*OF l*(*1*)] **by** *blast*
moreover have $l = b$ **using** *atomD*[*OF b l*(*3*)] *atom-nonbottom*[*OF l*(*1*)] **by** *blast*
ultimately show *?thesis* **by** *simp*
qed

In an algebraic domain atoms are compact: some compact approximant is nonbottom, and lies below the atom.

lemma *atom-compact*:
fixes *a* :: *'a*::*bifinite* **assumes** *a*: *atom a* **shows** *compact a*
proof −
obtain *C* **where** *C*: *chain C* $\forall\, i.$ *compact* (*C i*) $a = (\bigsqcup i.\ C\ i)$ **by** (*rule obtain-compact-chain*)
have $\exists\, i.\ C\ i \neq \bot$ **using** *C*(*1*,*3*) *atom-nonbottom*[*OF a*] *lub-eq-bottom-iff* **by** *blast*
then obtain *i* **where** *i*: $C\ i \neq \bot$ **by** *blast*
have $C\ i \sqsubseteq a$ **using** *is-ub-thelub*[*OF C*(*1*)] *C*(*3*) **by** *simp*
with *atomD*[*OF a*] *i* **have** $C\ i = a$ **by** *blast*
with *C*(*2*) **show** *?thesis* **by** *metis*
qed

6.1 Atoms of the domain constructions

lemma *atom-up-bottom*: *atom* (*up*·(⊥::*'a*::*pcpo*))
proof (*rule atomI*)
fix *z* :: *'a u* **assume** $z \sqsubseteq up{\cdot}\bot$
then show $z = \bot \vee z = up{\cdot}\bot$ **by** (*cases z*) *simp-all*

qed *simp*

lemma *atom-sinl-iff*: *atom* (*sinl*·*x* :: *'a*::*pcpo* ⊕ *'b*::*pcpo*) ⟷ *atom x*
proof
  assume *a*: *atom* (*sinl*·*x* :: *'a* ⊕ *'b*)
  show *atom x*
  proof (*rule atomI*)
    show *x* ≠ ⊥ **using** *atom-nonbottom*[*OF a*] **by** *auto*
    fix *z* **assume** *z* ⊑ *x*
    then have (*sinl*·*z* :: *'a* ⊕ *'b*) ⊑ *sinl*·*x* **by** *simp*
    from *atomD*[*OF a this*] **show** *z* = ⊥ ∨ *z* = *x* **by** *simp*
  qed
next
  assume *a*: *atom x*
  show *atom* (*sinl*·*x* :: *'a* ⊕ *'b*)
  proof (*rule atomI*)
    show (*sinl*·*x* :: *'a* ⊕ *'b*) ≠ ⊥ **using** *atom-nonbottom*[*OF a*] **by** *simp*
    fix *z* :: *'a* ⊕ *'b* **assume** *z*: *z* ⊑ *sinl*·*x*
    show *z* = ⊥ ∨ *z* = *sinl*·*x*
    proof (*cases z*)
      case (*sinl z'*) **with** *z atomD*[*OF a*, *of z'*] **show** *?thesis* **by** *auto*
    qed (*use z* **in** *auto*)
  qed
qed

lemma *atom-sinr-iff*: *atom* (*sinr*·*x* :: *'a*::*pcpo* ⊕ *'b*::*pcpo*) ⟷ *atom x*
proof
  assume *a*: *atom* (*sinr*·*x* :: *'a* ⊕ *'b*)
  show *atom x*
  proof (*rule atomI*)
    show *x* ≠ ⊥ **using** *atom-nonbottom*[*OF a*] **by** *auto*
    fix *z* **assume** *z* ⊑ *x*
    then have (*sinr*·*z* :: *'a* ⊕ *'b*) ⊑ *sinr*·*x* **by** *simp*
    from *atomD*[*OF a this*] **show** *z* = ⊥ ∨ *z* = *x* **by** *simp*
  qed
next
  assume *a*: *atom x*
  show *atom* (*sinr*·*x* :: *'a* ⊕ *'b*)
  proof (*rule atomI*)
    show (*sinr*·*x* :: *'a* ⊕ *'b*) ≠ ⊥ **using** *atom-nonbottom*[*OF a*] **by** *simp*
    fix *z* :: *'a* ⊕ *'b* **assume** *z*: *z* ⊑ *sinr*·*x*
    show *z* = ⊥ ∨ *z* = *sinr*·*x*
    proof (*cases z*)
      case (*sinr z'*) **with** *z atomD*[*OF a*, *of z'*] **show** *?thesis* **by** *auto*
    qed (*use z* **in** *auto*)
  qed
qed

lemma *atom-spair*:

assumes *a*: *atom* (*a*::′*a*::*pcpo*) **and** *b*: *atom* (*b*::′*b*::*pcpo*) **shows** *atom* (:*a*, *b*:)
proof (*rule atomI*)
show (:*a*, *b*:) ≠ ⊥ **using** *atom-nonbottom*[*OF a*] *atom-nonbottom*[*OF b*] **by** *simp*
fix *z* **assume** *z*: *z* ⊑ (:*a*, *b*:)
show *z* = ⊥ ∨ *z* = (:*a*, *b*:)
proof (*cases z*)
case (*spair z1 z2*)
with *z atom-nonbottom*[*OF a*] *atom-nonbottom*[*OF b*] **have** *z1* ⊑ *a z2* ⊑ *b*
by (*simp-all add*: *spair-below*)
with *atomD*[*OF a*] *atomD*[*OF b*] *spair* **show** *?thesis* **by** *auto*
qed *simp*
qed

6.2 Instances

instance *flat* ⊆ *separated-sum*
proof
fix *x y u* :: ′*a* **assume** *x* ≠ ⊥ *y* ≠ ⊥ *x* ⊑ *u y* ⊑ *u*
then have *x* = *y* **by** (*metis ax-flat*)
moreover have *atom x* **by** (*rule atomI*[*OF* ‹*x* ≠ ⊥›]) (*metis ax-flat*)
ultimately show ∃ *l*. *atom l* ∧ *l* ⊑ *x* ∧ *l* ⊑ *y* **by** *blast*
qed

instance *u* :: (*pcpo*) *separated-sum*
proof
fix *x y u* :: ′*a u* **assume** *x* ≠ ⊥ *y* ≠ ⊥
then show ∃ *l*. *atom l* ∧ *l* ⊑ *x* ∧ *l* ⊑ *y*
by (*intro exI*[*of* - *up*·⊥]) (*cases x*; *cases y*; *simp add*: *atom-up-bottom*)
qed

instance *ssum* :: (*separated-sum*, *separated-sum*) *separated-sum*
proof
fix *x y u* :: ′*a* ⊕ ′*b* **assume** *nb*: *x* ≠ ⊥ *y* ≠ ⊥ **and** *le*: *x* ⊑ *u y* ⊑ *u*
show ∃ *l*. *atom l* ∧ *l* ⊑ *x* ∧ *l* ⊑ *y*
proof (*cases x*)
case *bottom* **with** *nb* **show** *?thesis* **by** *simp*
next
case (*sinl a*)
from *le*(*1*) *sinl* **obtain** *c* **where** *u*: *u* = *sinl*·*c* **and** *ac*: *a* ⊑ *c* **by** (*cases u*) *auto*
from *le*(*2*) *nb*(*2*) *u* **obtain** *b* **where** *y*: *y* = *sinl*·*b b* ≠ ⊥ **and** *bc*: *b* ⊑ *c* **by** (*cases y*) *auto*
from *separated-sum*[*OF sinl*(*2*) *y*(*2*) *ac bc*] **obtain** *l* **where** *atom l l* ⊑ *a l* ⊑ *b* **by** *blast*
with *sinl y* **show** *?thesis* **by** (*intro exI*[*of* - *sinl*·*l*]) (*simp add*: *atom-sinl-iff*)
next
case (*sinr a*)
from *le*(*1*) *sinr* **obtain** *c* **where** *u*: *u* = *sinr*·*c* **and** *ac*: *a* ⊑ *c* **by** (*cases u*) *auto*
from *le*(*2*) *nb*(*2*) *u* **obtain** *b* **where** *y*: *y* = *sinr*·*b b* ≠ ⊥ **and** *bc*: *b* ⊑ *c* **by**

```
(cases y) auto
    from separated-sum[OF sinr(2) y(2) ac bc] obtain l where atom l l ⊑ a l ⊑
b by blast
    with sinr y show ?thesis by (intro exI[of - sinr·l]) (simp add: atom-sinr-iff)
  qed
qed

instance sprod :: (separated-sum, separated-sum) separated-sum
proof
  fix x y u :: 'a ⊗ 'b assume nb: x ≠ ⊥ y ≠ ⊥ and le: x ⊑ u y ⊑ u
  obtain x1 x2 where x: x = (:x1, x2:) x1 ≠ ⊥ x2 ≠ ⊥ using nb(1) by (cases
x) auto
  obtain y1 y2 where y: y = (:y1, y2:) y1 ≠ ⊥ y2 ≠ ⊥ using nb(2) by (cases
y) auto
  obtain u1 u2 where u: u = (:u1, u2:) using le(1) x by (cases u) auto
  have x1 ⊑ u1 x2 ⊑ u2 y1 ⊑ u1 y2 ⊑ u2
    using le(1)[unfolded x(1) u] le(2)[unfolded y(1) u]
    by (simp-all only: spair-below x(2,3) y(2,3) simp-thms)
  then obtain l1 l2 where atom l1 l1 ⊑ x1 l1 ⊑ y1 atom l2 l2 ⊑ x2 l2 ⊑ y2
    using separated-sum[OF x(2) y(2)] separated-sum[OF x(3) y(3)] by meson
  then show ∃ l. atom l ∧ l ⊑ x ∧ l ⊑ y
    using x y by (intro exI[of - (:l1, l2:)])
      (simp add: atom-spair spair-below atom-nonbottom)
qed
```

7 Distributivity of smash products

Separation of both components keeps ⊗ distributive: joinable nonbottom pairs meet a third pair to ⊥ together, so either both sides collapse or everything is componentwise.

```
instance sprod :: ({filter-sum, distributive-cpo}, {filter-sum, distributive-cpo})
  distributive-cpo
proof
  fix a b c :: 'a ⊗ 'b assume a ⇑ b
  show c ⊓ (a ⊔ b) = c ⊓ a ⊔ c ⊓ b
  proof (cases a = ⊥ ∨ b = ⊥ ∨ c = ⊥)
    case True then show ?thesis by (elim disjE) simp-all
  next
    case False
    then obtain a1 a2 b1 b2 c1 c2 where
      ab: a = (:a1, a2:) b = (:b1, b2:) c = (:c1, c2:) and
      nb: a1 ≠ ⊥ a2 ≠ ⊥ b1 ≠ ⊥ b2 ≠ ⊥ c1 ≠ ⊥ c2 ≠ ⊥
      by (cases a; cases b; cases c) auto
   from ‹a ⇑ b› have j: a1 ⇑ b1 a2 ⇑ b2 by (simp-all add: ab nb spair-joinable-iff)
    have cong: c1 ⊓ a1 = ⊥ ⟷ c1 ⊓ b1 = ⊥ c2 ⊓ a2 = ⊥ ⟷ c2 ⊓ b2 = ⊥
    using filter-sum-meet-bottom-cong[OF j(1) nb(1,3)] filter-sum-meet-bottom-cong[OF
j(2) nb(2,4)]
      filter-sum-meet-bottom-cong[OF joinable-sym[THEN iffD1, OF j(1)] nb(3,1)]
```

```
      filter-sum-meet-bottom-cong[OF joinable-sym[THEN iffD1, OF j(2)] nb(4,2)]
      by blast+
    have mj: x ⊓ y ⇑ x ⊓ z for x y z :: 'c::bounded-complete
      by (rule joinableI[OF meet-below1 meet-below1])
    have lhs: c ⊓ (a ⊔ b) = (:c1 ⊓ a1 ⊔ c1 ⊓ b1, c2 ⊓ a2 ⊔ c2 ⊓ b2:)
     by (simp only: ab spair-join[OF nb(1,2,3,4) j] spair-meet meet-join-distrib[OF
j(1)]
        meet-join-distrib[OF j(2)])
    have rhs: c ⊓ a ⊔ c ⊓ b = (:c1 ⊓ a1, c2 ⊓ a2:) ⊔ (:c1 ⊓ b1, c2 ⊓ b2:)
      by (simp only: ab spair-meet)
    show ?thesis
    proof (cases c1 ⊓ a1 = ⊥ ∨ c2 ⊓ a2 = ⊥)
      case True
      with cong show ?thesis unfolding lhs rhs by auto
    next
      case False
      with cong have c1 ⊓ a1 ≠ ⊥ c2 ⊓ a2 ≠ ⊥ c1 ⊓ b1 ≠ ⊥ c2 ⊓ b2 ≠ ⊥ by
simp-all
      then show ?thesis unfolding lhs rhs by (simp add: spair-join mj)
    qed
  qed
qed
```

If one component is flat, the other needs no separation: joinable nonbottom elements of a flat type are equal, so that component contributes no proper joins. These cannot be instances (they would conflict with the arity above), so they are stated as the class law.

```
lemma sprod-flat-left-distrib:
  fixes a b c :: 'a::flat ⊗ 'b::distributive-cpo
  assumes a ⇑ b shows c ⊓ (a ⊔ b) = c ⊓ a ⊔ c ⊓ b
proof (cases a = ⊥ ∨ b = ⊥ ∨ c = ⊥)
  case True then show ?thesis by (elim disjE) simp-all
next
  case False
  then obtain a1 a2 b1 b2 c1 c2 where
    ab: a = (:a1, a2:) b = (:b1, b2:) c = (:c1, c2:) and
    nb: a1 ≠ ⊥ a2 ≠ ⊥ b1 ≠ ⊥ b2 ≠ ⊥ c1 ≠ ⊥ c2 ≠ ⊥
    by (cases a; cases b; cases c) auto
  from ‹a ⇑ b› have j: a1 ⇑ b1 a2 ⇑ b2 by (simp-all add: ab nb spair-joinable-iff)
  from j(1) nb(1,3) have a1 = b1 by (blast elim: flat-joinableE)
  with ab(2) have b: b = (:a1, b2:) by simp
  have mj: x ⊓ y ⇑ x ⊓ z for x y z :: 'c::bounded-complete
    by (rule joinableI[OF meet-below1 meet-below1])
  have lhs: c ⊓ (a ⊔ b) = (:c1 ⊓ a1, c2 ⊓ a2 ⊔ c2 ⊓ b2:)
   by (simp only: ab(1,3) b spair-join[OF nb(1,2,1,4) joinable-refl j(2)] spair-meet
join-idem
      meet-join-distrib[OF j(2)])
  have rhs: c ⊓ a ⊔ c ⊓ b = (:c1 ⊓ a1, c2 ⊓ a2:) ⊔ (:c1 ⊓ a1, c2 ⊓ b2:)
    by (simp only: ab(1,3) b spair-meet)
```

show *?thesis*
proof (*cases c1* ⊓ *a1* = ⊥ ∨ *c2* ⊓ *a2* = ⊥ ∨ *c2* ⊓ *b2* = ⊥)
case *True* **then show** *?thesis* **unfolding** *lhs rhs* **by** (*elim disjE*) *simp-all*
next **case** *False* **then show** *?thesis* **unfolding** *lhs rhs* **by** (*simp add*: *spair-join mj*)
qed
qed

global-interpretation *sprod-flat-left*: *distributive-cpo*
below :: ('*a*::*flat* ⊗ '*b*::*distributive-cpo*) ⇒ - ⇒ *bool*
rewrites *po.is-ub* (⊑) = ((<|) :: ('*a* ⊗ '*b*) *set* ⇒ -)
and *po.is-lub* (⊑) = ((<<|) :: ('*a* ⊗ '*b*) *set* ⇒ -)
and *po.lub* (⊑) = (*lub* :: ('*a* ⊗ '*b*) *set* ⇒ -)
and *po.is-glb* (⊑) = (*is-glb* :: ('*a* ⊗ '*b*) *set* ⇒ -)
and *po.glb* (⊑) = (*glb* :: ('*a* ⊗ '*b*) *set* ⇒ -)
and *po.meet* (⊑) = ((⊓) :: '*a* ⊗ '*b* ⇒ -)
and *po.join* (⊑) = ((⊔) :: '*a* ⊗ '*b* ⇒ -)
and *bounded-complete.joinable* (⊑) = ((⇑) :: '*a* ⊗ '*b* ⇒ -)
by *unfold-locales* (*simp-all only*: *class-locale-eqs sprod-flat-left-distrib*)

lemma *sprod-flat-right-distrib*:
fixes *a b c* :: '*a*::*distributive-cpo* ⊗ '*b*::*flat*
assumes *a* ⇑ *b* **shows** *c* ⊓ (*a* ⊔ *b*) = *c* ⊓ *a* ⊔ *c* ⊓ *b*
proof (*cases a* = ⊥ ∨ *b* = ⊥ ∨ *c* = ⊥)
case *True* **then show** *?thesis* **by** (*elim disjE*) *simp-all*
next
case *False*
then obtain *a1 a2 b1 b2 c1 c2* **where**
ab: *a* = (:*a1*, *a2*:) *b* = (:*b1*, *b2*:) *c* = (:*c1*, *c2*:) **and**
nb: *a1* ≠ ⊥ *a2* ≠ ⊥ *b1* ≠ ⊥ *b2* ≠ ⊥ *c1* ≠ ⊥ *c2* ≠ ⊥
by (*cases a*; *cases b*; *cases c*) *auto*
from ‹*a* ⇑ *b*› **have** *j*: *a1* ⇑ *b1 a2* ⇑ *b2* **by** (*simp-all add*: *ab nb spair-joinable-iff*)
from *j*(*2*) *nb*(*2*,*4*) **have** *a2* = *b2* **by** (*blast elim*: *flat-joinableE*)
with *ab*(*2*) **have** *b*: *b* = (:*b1*, *a2*:) **by** *simp*
have *mj*: *x* ⊓ *y* ⇑ *x* ⊓ *z* **for** *x y z* :: '*c*::*bounded-complete*
by (*rule joinableI*[*OF meet-below1 meet-below1*])
have *lhs*: *c* ⊓ (*a* ⊔ *b*) = (:*c1* ⊓ *a1* ⊔ *c1* ⊓ *b1*, *c2* ⊓ *a2*:)
by (*simp only*: *ab*(*1*,*3*) *b spair-join*[*OF nb*(*1*,*2*,*3*,*2*) *j*(*1*) *joinable-refl*] *spair-meet join-idem*
meet-join-distrib[*OF j*(*1*)])
have *rhs*: *c* ⊓ *a* ⊔ *c* ⊓ *b* = (:*c1* ⊓ *a1*, *c2* ⊓ *a2*:) ⊔ (:*c1* ⊓ *b1*, *c2* ⊓ *a2*:)
by (*simp only*: *ab*(*1*,*3*) *b spair-meet*)
show *?thesis*
proof (*cases c2* ⊓ *a2* = ⊥ ∨ *c1* ⊓ *a1* = ⊥ ∨ *c1* ⊓ *b1* = ⊥)
case *True* **then show** *?thesis* **unfolding** *lhs rhs* **by** (*elim disjE*) *simp-all*
next
case *False* **then show** *?thesis* **unfolding** *lhs rhs* **by** (*simp add*: *spair-join mj*)
qed
qed

global-interpretation *sprod-flat-right*: *distributive-cpo*
below :: (*'a*::*distributive-cpo* ⊗ *'b*::*flat*) ⇒ - ⇒ *bool*
rewrites *po.is-ub* (⊑) = ((<|) :: (*'a* ⊗ *'b*) *set* ⇒ -)
and *po.is-lub* (⊑) = ((<<|) :: (*'a* ⊗ *'b*) *set* ⇒ -)
and *po.lub* (⊑) = (*lub* :: (*'a* ⊗ *'b*) *set* ⇒ -)
and *po.is-glb* (⊑) = (*is-glb* :: (*'a* ⊗ *'b*) *set* ⇒ -)
and *po.glb* (⊑) = (*glb* :: (*'a* ⊗ *'b*) *set* ⇒ -)
and *po.meet* (⊑) = ((⊓) :: *'a* ⊗ *'b* ⇒ -)
and *po.join* (⊑) = ((⊔) :: *'a* ⊗ *'b* ⇒ -)
and *bounded-complete.joinable* (⊑) = ((⇑) :: *'a* ⊗ *'b* ⇒ -)
by *unfold-locales* (*simp-all only*: *class-locale-eqs sprod-flat-right-distrib*)

end
theory *DI* **imports** *Filter-Sum* **begin**

8 dI-domains

A *dI-domain* (Berry) is a distributive bifinite domain with property I: every compact element has finitely many elements below it. Everything below a compact element is then compact itself.

class *dI* = *bifinite* + *distributive-cpo* +
assumes *finite-below-compact*: *compact k* ⟹ *finite* {*x*. *x* ⊑ *k*}

lemma *compact-below-compact*:
fixes *x* :: *'a*::*dI*
assumes *k*: *compact k* **and** *xk*: *x* ⊑ *k*
shows *compact x*
proof −
obtain *a* :: *nat* ⇒ *'a* → *'a* **where** *a*: *approx-chain a* **using** *bifinite* **by** *blast*
interpret *approx-chain a* **by** (*fact a*)
let *?Y* = λ*i*. *a i*·*x*
have *ch*: *chain ?Y* **by** (*rule ch2ch-Rep-cfunL*[*OF chain-approx*])
have *range ?Y* ⊆ {*y*. *y* ⊑ *k*} **using** *approx-below xk* **by** (*blast intro*: *below-trans*)
then have *finite* (*range ?Y*) **by** (*rule finite-subset*) (*rule finite-below-compact*[*OF k*])
then obtain *i* **where** *max-in-chain i ?Y*
using *finite-range-imp-finch*[*OF ch*] **unfolding** *finite-chain-def* **by** *blast*
then have *l*: (⨆ *j*. *?Y j*) = *?Y i* **using** *maxinch-is-thelub*[*OF ch*] **by** *blast*
have (⨆ *j*. *?Y j*) = *x* **by** (*simp add*: *contlub-cfun-fun*[*OF chain-approx*, *symmetric*])
with *l* **have** *x* = *a i*·*x* **by** *simp*
then show *?thesis* **by** (*metis compact-approx*)
qed

8.1 Transfer along isomorphisms

Domain types are dI-domains when their representation is: the derivation machinery for map functions does not apply, since property I is not preserved by recursion.

lemma *dI-iso*:
  fixes *abs* :: *'r*::*dI* → *'t*::{*bifinite*, *distributive-cpo*} **and** *rep*
  assumes *iso abs rep* **shows** *OFCLASS*(*'t*, *dI-class*)
proof (*intro-classes*)
  fix *k* :: *'t* **assume** *compact k*
  then have *c*: *compact* (*rep*·*k*) **by** (*rule iso.compact-rep*[*OF assms*])
  have {*x*. *x* ⊑ *k*} ⊆ (λ*y*. *abs*·*y*) ‘ {*y*. *y* ⊑ *rep*·*k*}
  proof
    fix *x* **assume** *x* ∈ {*x*. *x* ⊑ *k*}
    then have *rep*·*x* ⊑ *rep*·*k* **by** (*simp add*: *monofun-cfun-arg*)
    moreover have *x* = *abs*·(*rep*·*x*) **by** (*simp add*: *iso.rep-iso*[*OF assms*])
    ultimately show *x* ∈ (λ*y*. *abs*·*y*) ‘ {*y*. *y* ⊑ *rep*·*k*} **by** *blast*
  qed
  then show *finite* {*x*. *x* ⊑ *k*}
    by (*rule finite-subset*) (*rule finite-imageI*[*OF finite-below-compact*[*OF c*]])
qed

8.2 Instances

Flat domains have at most two elements below anything.

instance *lift* :: (*countable*) *dI*
proof
  fix *k* :: *'a lift*
  show *finite* {*x*. *x* ⊑ *k*}
    by (*rule finite-subset*[*of* - {⊥, *k*}]) (*auto dest*: *flat-eq*)
qed

Lifting adds one element below everything.

instance *u* :: (*dI*) *dI*
proof
  fix *k* :: *'a u* **assume** *k*: *compact k*
  show *finite* {*x*. *x* ⊑ *k*}
  proof (*cases k rule*: *upE*)
    case *bottom*
    then show *?thesis* **by** *simp*
  next
    case (*up a*)
    with *k* **have** *a*: *compact a* **by** *simp*
    have {*x*. *x* ⊑ *k*} ⊆ *insert* ⊥ ((λ*z*. *up*·*z*) ‘ {*z*. *z* ⊑ *a*})
    proof
      fix *x* **assume** *x* ∈ {*x*. *x* ⊑ *k*}
      with *up* **show** *x* ∈ *insert* ⊥ ((λ*z*. *up*·*z*) ‘ {*z*. *z* ⊑ *a*}) **by** (*cases x rule*: *upE*)
auto

```
    qed
    then show ?thesis
      by (rule finite-subset) (simp add: finite-below-compact[OF a])
  qed
qed
```

Below an injection lie the injections of what lies below its argument.

```
instance ssum :: (dI, dI) dI
proof
  fix k :: 'a ⊕ 'b assume k: compact k
  show finite {x. x ⊑ k}
  proof (cases k rule: ssumE)
    case bottom
    then show ?thesis by simp
  next
    case (sinl a)
    with k have a: compact a by simp
    have {x. x ⊑ k} ⊆ (λz. sinl·z) ‘ {z. z ⊑ a}
    proof
      fix x assume x ∈ {x. x ⊑ k}
      with sinl show x ∈ (λz. sinl·z) ‘ {z. z ⊑ a}
        by (cases x rule: ssumE) (auto intro: image-eqI[where x = ⊥])
    qed
    then show ?thesis by (rule finite-subset) (simp add: finite-below-compact[OF
a])
  next
    case (sinr b)
    with k have b: compact b by simp
    have {x. x ⊑ k} ⊆ (λz. sinr·z) ‘ {z. z ⊑ b}
    proof
      fix x assume x ∈ {x. x ⊑ k}
      with sinr show x ∈ (λz. sinr·z) ‘ {z. z ⊑ b}
        by (cases x rule: ssumE) (auto intro: image-eqI[where x = ⊥])
    qed
    then show ?thesis by (rule finite-subset) (simp add: finite-below-compact[OF
b])
  qed
qed
```

Below a pair lie the pairs of what lies below its components.

```
instance prod :: (dI, dI) dI
proof
  fix k :: 'a × 'b assume k: compact k
  obtain a b where ab: k = (a, b) by (cases k)
  with k have a: compact a and b: compact b by simp-all
  have {x. x ⊑ k} ⊆ {z. z ⊑ a} × {z. z ⊑ b}
    using ab by (auto simp: below-prod-def)
  then show finite {x. x ⊑ k}
   by (rule finite-subset) (simp add: finite-below-compact[OF a] finite-below-compact[OF
```

b])
qed

Strict products are distributive over filter-sum factors (see *Filter-Sum*).

instance *sprod* :: ({*filter-sum*, *dI*}, {*filter-sum*, *dI*}) *dI*
proof
  fix *k* :: *'a* ⊗ *'b* **assume** *k*: *compact k*
  show *finite* {*x*. *x* ⊑ *k*}
  proof (*cases k rule*: *sprodE*)
    case *bottom*
    then show *?thesis* **by** *simp*
  next
    case (*spair a b*)
   with *k* **have** *a*: *compact a* **and** *b*: *compact b* **by** (*simp-all add*: *compact-spair-iff*)
    have {*x*. *x* ⊑ *k*} ⊆ (λ(*y*, *z*). (:*y*, *z*:)) ` ({*y*. *y* ⊑ *a*} × {*z*. *z* ⊑ *b*})
    proof
      fix *x* **assume** *x*: *x* ∈ {*x*. *x* ⊑ *k*}
      show *x* ∈ (λ(*y*, *z*). (:*y*, *z*:)) ` ({*y*. *y* ⊑ *a*} × {*z*. *z* ⊑ *b*})
      proof (*cases x rule*: *sprodE*)
        case *bottom*
        then show *?thesis*
            by (*intro image-eqI*[**where** *x* = (⊥ :: *'a*, ⊥ :: *'b*)]) (*simp-all add*: *mem-Times-iff*)
      next
        case (*spair c d*)
        with *x* ‹*k* = (:*a*, *b*:)› **have** *c* ⊑ *a* *d* ⊑ *b* **by** (*simp-all add*: *spair-below*)
        with *spair* **show** *?thesis* **by** (*intro image-eqI*[**where** *x* = (*c*, *d*)]) *simp-all*
      qed
    qed
    then show *?thesis*
    by (*rule finite-subset*) (*simp add*: *finite-below-compact*[*OF a*] *finite-below-compact*[*OF b*])
  qed
qed

end
theory *Lattice-Complete* **imports** *Distributive* **begin**

9 Bounded-complete domains with a top

A bounded-complete pcpo with a top element is a complete lattice under its information order: every set is bounded, hence has a lub. The lattice laws are stated as facts, with the join and meet of *bounded-complete* as the lattice operations and no joinability side conditions. They are deliberately not interpreted globally: an interpretation copies HOL's classical rules into the claset, restated at sort *lattice-complete*, and since *blast* ignores sorts during search it then tries them at every type (the copy of *order-refl* [*iff*]

made an unrelated *pcpo*-level blast search for over 1400 s). Interpret them in a *context* instead and re-export the lemmas needed.

class *lattice-complete* = *bounded-complete* +
  assumes *ex-top*: $\exists\, t.\ \forall\, x.\ x \sqsubseteq t$

definition *lattice-top* :: *'a*::*lattice-complete* **where**
  lattice-top = *lub UNIV*

lemma *below-lattice-top* [*simp*]: $x \sqsubseteq$ (*lattice-top* :: *'a*::*lattice-complete*)
proof −
  obtain *t* :: *'a* **where** $\forall\, x.\ x \sqsubseteq t$ **using** *ex-top* **by** *blast*
  then have *UNIV* <| *t* **by** (*simp add*: *is-ub-def*)
  then have *UNIV* <<| *lub* (*UNIV* :: *'a set*) **by** (*rule ex-bound-lub*)
  then show *?thesis* **unfolding** *lattice-top-def* **by** (*blast dest*: *is-lubD1 is-ubD*)
qed

lemma *is-ub-lattice-top* [*simp*]: *S* <| (*lattice-top* :: *'a*::*lattice-complete*)
  by (*simp add*: *is-ub-def*)

lemma *lattice-is-lub*: *S* <<| *lub* (*S* :: *'a*::*lattice-complete set*)
  by (*rule ex-bound-lub*[*OF is-ub-lattice-top*])

lemma *joinable-lattice* [*simp*]: (*a* :: *'a*::*lattice-complete*) ⇑ *b*
  by (*rule joinableI*[*OF below-lattice-top below-lattice-top*])

lemma *lattice-lub-upper*: $x \in S \Longrightarrow x \sqsubseteq$ *lub* (*S* :: *'a*::*lattice-complete set*)
  by (*rule is-ubD*[*OF is-lubD1*[*OF lattice-is-lub*]])

lemma *lattice-lub-least*: *S* <| *b* ⟹ *lub* (*S* :: *'a*::*lattice-complete set*) ⊑ *b*
  by (*rule is-lubD2*[*OF lattice-is-lub*])

Every set has a glb, the lub of its lower bounds (also for {}, unlike *?S* ≠ {} ⟹ *glb ?S* = *lub* {*x*. ∀ *y*∈*?S*. *x* ⊑ *y*}).

lemma *lattice-is-glb*: *is-glb S* (*lub* {*x*. ∀ *y*∈*S*. *x* ⊑ *y*} :: *'a*::*lattice-complete*)
  by (*rule is-glbI*) (*auto intro*: *lattice-lub-least lattice-lub-upper simp*: *is-ub-def*)

lemma *lattice-glb-eq-lub*: *glb S* = *lub* {*x*. ∀ *y*∈*S*. *x* ⊑ (*y* :: *'a*::*lattice-complete*)}
  by (*rule glb-eqI*[*OF lattice-is-glb*])

lemma *lattice-glb-greatest*: (⋀*y*. *y* ∈ *S* ⟹ *b* ⊑ *y*) ⟹ *b* ⊑ *glb* (*S* :: *'a*::*lattice-complete set*)
  unfolding *lattice-glb-eq-lub* **by** (*rule lattice-lub-upper*) *simp*

theorem *complete-lattice-below*:
  class.complete-lattice glb lub (⊓) (⊑) (⊏) (⊔) ⊥ (*lattice-top* :: *'a*::*lattice-complete*)
proof (*rule class.complete-lattice.intro*[*OF class.lattice.intro*[*OF semilattice-meet*]])
  show *class.semilattice-sup* (⊔) (⊑) ((⊏) :: *'a* ⇒ -)
    by (*rule class.semilattice-sup.intro*[*OF below-order*])
      (*unfold-locales*, *simp-all add*: *joinableD join-belowI*)

show *class.complete-lattice-axioms glb lub* (⊑) ⊥ (*lattice-top* :: *'a*)
proof
show *lub* {} = (⊥ :: *'a*) **by** (*rule lub-eqI*) *simp*
qed (*simp-all add*: *glb-below lattice-glb-greatest lattice-lub-upper lattice-lub-least is-ub-def*,
simp add: *lattice-glb-eq-lub lattice-top-def*)
qed

theorem *distrib-lattice-below*:
class.distrib-lattice (⊓) (⊑) (⊏) ((⊔) :: *'a*::{*lattice-complete*, *distributive-cpo*} ⇒ -)
by (*rule class.distrib-lattice.intro*[*OF class.complete-lattice.axioms*(*1*)[*OF complete-lattice-below*]])
(*unfold-locales*, *simp add*: *join-meet-distrib*)

The lattice laws needed elsewhere, re-exported from local interpretations under *lattice-* names (the interpretations themselves stay local, see above).

context
begin

interpretation *lc*: *complete-lattice glb* :: *'a*::*lattice-complete set* ⇒ *'a lub* (⊓) (⊑) (⊏) (⊔) ⊥ *lattice-top*
by (*fact complete-lattice-below*)

lemmas *lattice-join-assoc* = *lc.sup-assoc*
and *lattice-join-commute* = *lc.sup-commute*
and *lattice-join-left-commute* = *lc.sup-left-commute*
and *lattice-join-ge1* = *lc.sup-ge1*
and *lattice-join-ge2* = *lc.sup-ge2*
and *lattice-join-least* = *lc.sup-least*
and *lattice-join-mono* = *lc.sup-mono*
and *lattice-join-absorb1* = *lc.sup-absorb1*
and *lattice-join-absorb2* = *lc.sup-absorb2*
and *lattice-join-bot-left* = *lc.sup-bot-left*
and *lattice-join-bot-right* = *lc.sup-bot-right*
and *lattice-meet-assoc* = *lc.inf-assoc*
and *lattice-meet-commute* = *lc.inf-commute*
and *lattice-meet-left-commute* = *lc.inf-left-commute*
and *lattice-meet-le1* = *lc.inf-le1*
and *lattice-meet-le2* = *lc.inf-le2*
and *lattice-meet-greatest* = *lc.inf-greatest*
and *lattice-meet-mono* = *lc.inf-mono*
and *lattice-meet-absorb1* = *lc.inf-absorb1*
and *lattice-meet-absorb2* = *lc.inf-absorb2*
and *lattice-meet-idem* = *lc.inf-idem*
and *lattice-meet-left-idem* = *lc.inf-left-idem*
and *lattice-meet-bot-left* = *lc.inf-bot-left*
and *lattice-meet-bot-right* = *lc.inf-bot-right*
and *lattice-meet-top-left* = *lc.inf-top-left*

and *lattice-meet-join-absorb* = *lc.inf-sup-absorb*
and *lattice-join-meet-absorb* = *lc.sup-inf-absorb*
and *lattice-lub-bot-conv* = *lc.Sup-bot-conv*
and *lattice-join-ac* = *lc.sup-aci*
and *lattice-meet-ac* = *lc.inf-aci*

end

context
begin

interpretation *lcd*: *distrib-lattice* (⊓) (⊑) (⊏) (⊔) :: 'a::{*lattice-complete*, *distributive-cpo*} ⇒ -
by (*fact distrib-lattice-below*)

lemmas *lattice-meet-join-distrib1* = *lcd.inf-sup-distrib1*
and *lattice-meet-join-distrib2* = *lcd.inf-sup-distrib2*
and *lattice-join-meet-distrib1* = *lcd.sup-inf-distrib1*
and *lattice-join-meet-distrib2* = *lcd.sup-inf-distrib2*

end

Adjoining a top to a bounded-complete domain makes it a complete lattice.

instance *top* :: (*bounded-complete*) *lattice-complete*
by *intro-classes* (*rule exI*[*of* - *Top*], *simp*)

instance *fun* :: (*type*, *lattice-complete*) *lattice-complete*
by *intro-classes* (*rule exI*[*of* - λ-. *lattice-top*], *simp add*: *fun-below-iff*)

instance *prod* :: (*lattice-complete*, *lattice-complete*) *lattice-complete*
by *intro-classes* (*rule exI*[*of* - (*lattice-top*, *lattice-top*)], *simp add*: *below-prod-def*)

lemma *lattice-top-prod*: *lattice-top* = (*lattice-top* :: 'a::*lattice-complete*, *lattice-top* :: 'b::*lattice-complete*)
by (*rule below-antisym*) (*simp add*: *below-prod-def*, *rule below-lattice-top*)

lemma *lattice-join-prod*: *x* ⊔ *y* = (*fst x* ⊔ *fst y*, *snd x* ⊔ *snd y*)
for *x y* :: 'a::*lattice-complete* × 'b::*lattice-complete*
by (*rule prod-join*[*OF joinable-lattice*])

lemma *lattice-top-fun*: *lattice-top* = (λ-. *lattice-top* :: 'b::*lattice-complete*)
by (*rule below-antisym*) (*simp add*: *fun-below-iff*, *rule below-lattice-top*)

lemma *fold-join-ge-init*: *b* ⊑ *fold* (⊔) *xs* (*b*::'a::*lattice-complete*)
by (*induct xs arbitrary*: *b*) (*simp-all*, *metis lattice-join-ge2 below-trans*)

lemma *fold-join-upper*: *x* ∈ *set xs* ⟹ *x* ⊑ *fold* (⊔) *xs* (*b*::'a::*lattice-complete*)
proof (*induct xs arbitrary*: *b*)
case (*Cons y xs*)

show *?case*
proof (*cases* $x = y$)
case *True*
have $x \sqsubseteq x \sqcup b$ **by** (*rule lattice-join-ge1*)
also have $\ldots \sqsubseteq$ *fold* ($\sqcup$) *xs* ($x \sqcup b$) **by** (*rule fold-join-ge-init*)
finally show *?thesis* **using** *True* **by** *simp*
next
case *False*
with *Cons* **show** *?thesis* **by** *simp*
qed
qed *simp*

end
theory *Algebraic-Lattice* **imports** *Lattice-Complete* **begin**

10 Algebraic lattices are bifinite

If every element of a complete lattice is the lub of the compacts below it, and the compacts are enumerated by c, then truncating to the lub of the first n compacts below the argument is an approximation chain.

definition *approx-alg* :: (*nat* $\Rightarrow$ *'a*::*lattice-complete*) $\Rightarrow$ *nat* $\Rightarrow$ *'a* $\rightarrow$ *'a* **where**
approx-alg c n = (Λ *x*. *lub* $\{c\ i \mid i.\ i \leq n \wedge c\ i \sqsubseteq x\}$)

lemma *lattice-lub-mono*: $S \subseteq T \Longrightarrow$ *lub* (*S* :: *'a*::*lattice-complete set*) $\sqsubseteq$ *lub T*
by (*rule lattice-lub-least*) (*auto simp*: *is-ub-def intro*: *lattice-lub-upper*)

lemma *common-chain-index*:
fixes *Y* :: *nat* $\Rightarrow$ *'a*::*cpo* **and** *c* :: *nat* $\Rightarrow$ *'a* **and** *n* :: *nat*
assumes *Y*: *chain Y* **and** *c*: $\bigwedge i.$ *compact* (*c i*)
shows $\exists J.\ \forall i{\leq}n.\ c\ i \sqsubseteq (\bigsqcup j.\ Y\ j) \longrightarrow c\ i \sqsubseteq Y\ J$
proof (*induct n*)
case *0*
obtain *j* **where** *c 0* $\sqsubseteq (\bigsqcup j.\ Y\ j) \longrightarrow c\ 0 \sqsubseteq Y\ j$ **using** *compactD2*[*OF c Y*] **by** *blast*
then show *?case* **by** *blast*
next
case (*Suc n*)
then obtain *J* **where** *J*: $\forall i{\leq}n.\ c\ i \sqsubseteq (\bigsqcup j.\ Y\ j) \longrightarrow c\ i \sqsubseteq Y\ J$ **by** *blast*
obtain *j* **where** *j*: *c* (*Suc n*) $\sqsubseteq (\bigsqcup j.\ Y\ j) \longrightarrow c$ (*Suc n*) $\sqsubseteq Y\ j$ **using** *compactD2*[*OF c Y*] **by** *blast*
have *JJ*: *Y J* $\sqsubseteq$ *Y* (*max J j*) **and** *jJ*: *Y j* $\sqsubseteq$ *Y* (*max J j*) **by** (*simp-all add*: *chain-mono*[*OF Y*])
show *?case*
proof (*intro exI*[*of - max J j*] *allI impI*)
fix *i* **assume** *i*: $i \leq$ *Suc n c i* $\sqsubseteq (\bigsqcup j.\ Y\ j)$
show *c i* $\sqsubseteq$ *Y* (*max J j*)
proof (*cases* $i \leq n$)
case *True* **with** *J i JJ* **show** *?thesis* **by** (*blast intro*: *below-trans*)

```
    next
      case False with i have i = Suc n by simp
      with j i jJ show ?thesis by (blast intro: below-trans)
    qed
  qed
qed

lemma cont-approx-alg:
  fixes c :: nat ⇒ 'a::lattice-complete
  assumes c: ⋀i. compact (c i)
  shows cont (λx. lub {c i |i. i ≤ n ∧ c i ⊑ x}) (is cont ?f)
proof (rule contI2)
  show monofun ?f by (rule monofunI, rule lattice-lub-mono) (auto intro: be-
low-trans)
  fix Y :: nat ⇒ 'a assume Y: chain Y and fY: chain (λi. ?f (Y i))
  obtain J where J: ∀ i≤n. c i ⊑ (⨆j. Y j) ⟶ c i ⊑ Y J
    using common-chain-index[where c = c and n = n, OF Y c] by blast
  have ?f (⨆j. Y j) ⊑ ?f (Y J) by (rule lattice-lub-mono) (use J in auto)
  also have ... ⊑ (⨆i. ?f (Y i)) by (rule is-ub-thelub[OF fY])
  finally show ?f (⨆i. Y i) ⊑ (⨆i. ?f (Y i)) .
qed

lemma approx-alg-beta:
  (⋀i. compact (c i)) ⟹ approx-alg c n·x = lub {c i |i. i ≤ n ∧ c i ⊑ x}
  unfolding approx-alg-def by (simp add: cont-approx-alg)

lemma approx-chain-algebraic:
  fixes c :: nat ⇒ 'a::lattice-complete
  assumes comp: range c = {k. compact k} and alg: ⋀x :: 'a. x = lub {k. compact
k ∧ k ⊑ x}
  shows approx-chain (approx-alg c)
proof −
  have c: ⋀i. compact (c i) using comp by blast
  have ch: chain (approx-alg c)
    by (rule chainI, rule cfun-belowI) (simp add: approx-alg-beta[OF c], rule lat-
tice-lub-mono, auto)
  have below: approx-alg c n·x ⊑ x for n and x :: 'a
   by (simp add: approx-alg-beta[OF c], rule lattice-lub-least) (auto simp: is-ub-def)
  show ?thesis
  proof
    show chain (approx-alg c) by (fact ch)
    show (⨆i. approx-alg c i) = ID
    proof (rule cfun-eqI)
      fix x :: 'a
      have x ⊑ (⨆i. approx-alg c i·x)
      proof (subst alg[of x], rule lattice-lub-least, rule is-ubI)
        fix k assume k ∈ {k. compact k ∧ k ⊑ x}
        then obtain i where ki: k = c i and kx: k ⊑ x using comp by blast
        have k ⊑ approx-alg c i·x
```

```
      by (simp add: approx-alg-beta[OF c], rule lattice-lub-upper) (use ki kx in
blast)
    also have ... ⊑ (⨆ i. approx-alg c i·x) by (rule is-ub-thelub[OF ch2ch-Rep-cfunL[OF
ch]])
      finally show k ⊑ (⨆ i. approx-alg c i·x) .
    qed
   moreover have (⨆ i. approx-alg c i·x) ⊑ x by (rule lub-below[OF ch2ch-Rep-cfunL[OF
ch] below])
    ultimately show (⨆ i. approx-alg c i)·x = ID·x
      by (simp add: contlub-cfun-fun[OF ch] below-antisym)
  qed
  show approx-alg c i·(approx-alg c i·x) = approx-alg c i·x for i and x :: 'a
  proof −
    have {c j |j. j ≤ i ∧ c j ⊑ approx-alg c i·x} = {c j |j. j ≤ i ∧ c j ⊑ x}
      using below[of i x]
    by (auto intro: below-trans simp: approx-alg-beta[OF c] intro!: lattice-lub-upper)
    then show ?thesis by (simp only: approx-alg-beta[OF c])
  qed
  show approx-alg c i·x ⊑ x for i and x :: 'a by (rule below)
  show finite {x. approx-alg c i·x = x} for i
  proof −
    have {x. approx-alg c i·x = x} ⊆ lub ‘ Pow (c ‘ {..i})
    proof
      fix x assume x ∈ {x. approx-alg c i·x = x}
     then have x = lub {c j |j. j ≤ i ∧ c j ⊑ x} by (simp add: approx-alg-beta[OF
c])
      moreover have {c j |j. j ≤ i ∧ c j ⊑ x} ∈ Pow (c ‘ {..i}) by auto
      ultimately show x ∈ lub ‘ Pow (c ‘ {..i}) by blast
    qed
    then show ?thesis by (rule finite-subset) simp
  qed
 qed
qed

lemma bifinite-algebraic-lattice:
 assumes comp: ∃ c :: nat ⇒ 'a::lattice-complete. range c = {k. compact k}
   and alg: ⋀x :: 'a. x = lub {k. compact k ∧ k ⊑ x}
 shows OFCLASS('a, bifinite-class)
proof
 obtain c :: nat ⇒ 'a where range c = {k. compact k} using comp by blast
 then show ∃ a :: nat ⇒ 'a → 'a. approx-chain a by (blast intro: approx-chain-algebraic
alg)
qed

end
theory Domain-Order
 imports Filter-Sum
begin
```

11 Order structure of domain-package types

The domain package presents each domain as isomorphic, through *abs* and *rep* satisfying the laws of *iso*, to a combination of its constructor arguments. An isomorphism of pointed cpos is an order isomorphism, so it preserves bounds, lubs and meets.

For recursive domains the order classes are derived from the map functional: each class property is stated for the image of a deflation (a locale over *deflation*), pushed through *ssum-map*, *sprod-map*, *u-map* and the *abs*/*rep* isomorphism, and established for *T-map*·*v* by fixed-point induction; at $v = ID$ this gives the instance. This is the *minimal invariant* technique of Pitts (*Relational Properties of Domains*, Information and Computation 127, 1996, Definition 3.2 and Section 6): *ID* is the least fixed point of $e \mapsto abs\ oo\ F\ e\ oo\ rep$, so an admissible property holding at $\bot$ and preserved by that map holds at *ID*. Dependencies between properties are expressed as sort constraints on the component types rather than as hypotheses on the deflations: each class is derived for the type before the classes that need it.

11.1 Transfer along isomorphisms

lemma *iso-is-ub-iff*:
  fixes *abs* :: $'r$::*pcpo* $\to$ $'t$::*pcpo* **and** *rep*
  assumes *iso abs rep* **shows** $(\lambda x.\ rep{\cdot}x)$ ‘ $A <| l \longleftrightarrow A <| abs{\cdot}l$
proof −
  interpret *iso abs rep* **by** *fact*
  have $x \sqsubseteq abs{\cdot}l \longleftrightarrow rep{\cdot}x \sqsubseteq l$ **for** *x* **by** (*subst rep-below*[*symmetric*]) *simp*
  then show *?thesis* **unfolding** *is-ub-def* **by** *auto*
qed

lemma *iso-is-lub-iff*:
  fixes *abs* :: $'r$::*pcpo* $\to$ $'t$::*pcpo* **and** *rep*
  assumes *iso abs rep* **shows** $(\lambda x.\ rep{\cdot}x)$ ‘ $A <<| l \longleftrightarrow A <<| abs{\cdot}l$
proof −
  interpret *iso abs rep* **by** *fact*
  note *ub* = *iso-is-ub-iff*[*OF assms*]
  show *?thesis*
  proof
    assume *l*: $(\lambda x.\ rep{\cdot}x)$ ‘ $A <<| l$
    show $A <<| abs{\cdot}l$
    proof (*rule is-lubI*)
      show $A <| abs{\cdot}l$ **using** *is-lubD1*[*OF l*] *ub* **by** *blast*
      fix *u* **assume** $A <| u$
      then have $(\lambda x.\ rep{\cdot}x)$ ‘ $A <| rep{\cdot}u$ **using** *ub*[*of A rep*·*u*] **by** *simp*
      with *l* **have** $l \sqsubseteq rep{\cdot}u$ **by** (*rule is-lubD2*)
      then have $abs{\cdot}l \sqsubseteq abs{\cdot}(rep{\cdot}u)$ **by** (*rule monofun-cfun-arg*)
      then show $abs{\cdot}l \sqsubseteq u$ **by** *simp*
    qed

```
  next
    assume l: A <<| abs·l
    show (λx. rep·x) ‘ A <<| l
    proof (rule is-lubI)
      show (λx. rep·x) ‘ A <| l using is-lubD1[OF l] ub by blast
      fix v assume (λx. rep·x) ‘ A <| v
      then have A <| abs·v using ub by blast
      with l have abs·l ⊑ abs·v by (rule is-lubD2)
      then show l ⊑ v by (simp add: abs-below)
    qed
  qed
qed

lemma bounded-complete-iso:
  fixes abs :: 'r::bounded-complete → 't::pcpo and rep
  assumes iso abs rep shows OFCLASS('t, bounded-complete-class)
proof intro-classes
  interpret iso abs rep by fact
  fix A :: 't set and b assume ne: A ≠ {} and ub: A <| b
  from ub have ub': (λx. rep·x) ‘ A <| rep·b using iso-is-ub-iff[OF assms, of A
rep·b] by simp
  from ne have (λx. rep·x) ‘ A ≠ {} by simp
 then have ∃ l. (λx. rep·x) ‘ A <<| l using ub' by (rule bounded-complete-class.ex-bound-ne-ex-lub)
  then obtain l :: 'r where (λx. rep·x) ‘ A <<| l by (elim exE)
  then have A <<| abs·l by (rule iso-is-lub-iff[OF assms, THEN iffD1])
  then show ∃ a. A <<| a ..
qed

lemma iso-rep-meet:
  fixes abs :: 'r::bounded-complete → 't::bounded-complete and rep
  assumes iso abs rep shows rep·(x ⊓ y) = rep·x ⊓ rep·y
proof (rule below-antisym)
  interpret iso abs rep by fact
  show rep·(x ⊓ y) ⊑ rep·x ⊓ rep·y
    by (intro meet-greatest monofun-cfun-arg meet-below1 meet-below2)
  have abs·(rep·x ⊓ rep·y) ⊑ abs·(rep·x) abs·(rep·x ⊓ rep·y) ⊑ abs·(rep·y)
    by (intro monofun-cfun-arg meet-below1 meet-below2)+
  then have abs·(rep·x ⊓ rep·y) ⊑ x ⊓ y by (simp add: meet-greatest)
   then show rep·x ⊓ rep·y ⊑ rep·(x ⊓ y) by (simp add: iso-swap[symmetric]
abs-below[symmetric])
qed

lemma iso-rep-joinable-iff:
  fixes abs :: 'r::bounded-complete → 't::bounded-complete and rep
  assumes iso abs rep shows rep·x ⇑ rep·y ⟷ x ⇑ y
proof
  interpret iso abs rep by fact
  assume rep·x ⇑ rep·y
  then obtain z where zx: rep·x ⊑ z and zy: rep·y ⊑ z unfolding joinable-def
```

by *blast*
have $abs{\cdot}(rep{\cdot}x) \sqsubseteq abs{\cdot}z$ **using** *zx* **by** (*rule monofun-cfun-arg*)
moreover have $abs{\cdot}(rep{\cdot}y) \sqsubseteq abs{\cdot}z$ **using** *zy* **by** (*rule monofun-cfun-arg*)
ultimately show $x \Uparrow y$ **by** (*simp add*: *joinableI*)
next
interpret *iso abs rep* **by** *fact*
assume $x \Uparrow y$
then obtain *z* **where** $x \sqsubseteq z$ $y \sqsubseteq z$ **unfolding** *joinable-def* **by** *blast*
then have $rep{\cdot}x \sqsubseteq rep{\cdot}z$ $rep{\cdot}y \sqsubseteq rep{\cdot}z$ **by** (*simp-all add*: *monofun-cfun-arg*)
then show $rep{\cdot}x \Uparrow rep{\cdot}y$ **by** (*rule joinableI*)
qed

lemma *iso-rep-join*:
fixes *abs* :: *'r*::*bounded-complete* $\to$ *'t*::*bounded-complete* **and** *rep*
assumes *iso abs rep* $x \Uparrow y$ **shows** $rep{\cdot}(x \sqcup y) = rep{\cdot}x \sqcup rep{\cdot}y$
proof (*rule below-antisym*)
interpret *iso abs rep* **by** *fact*
have *j*: $rep{\cdot}x \Uparrow rep{\cdot}y$ **using** *iso-rep-joinable-iff*[*OF assms*(*1*)] *assms*(*2*) **by** *simp*
have $abs{\cdot}(rep{\cdot}x) \sqsubseteq abs{\cdot}(rep{\cdot}x \sqcup rep{\cdot}y)$ $abs{\cdot}(rep{\cdot}y) \sqsubseteq abs{\cdot}(rep{\cdot}x \sqcup rep{\cdot}y)$
by (*rule monofun-cfun-arg*[*OF joinableD*(*1*)[*OF j*]], *rule monofun-cfun-arg*[*OF joinableD*(*2*)[*OF j*]])
then have $x \sqcup y \sqsubseteq abs{\cdot}(rep{\cdot}x \sqcup rep{\cdot}y)$ **by** (*simp add*: *join-belowI*)
then show $rep{\cdot}(x \sqcup y) \sqsubseteq rep{\cdot}x \sqcup rep{\cdot}y$ **by** (*simp add*: *iso-swap*[*symmetric*] *rep-below*[*symmetric*])
have $rep{\cdot}x \sqsubseteq rep{\cdot}(x \sqcup y)$ $rep{\cdot}y \sqsubseteq rep{\cdot}(x \sqcup y)$
by (*rule monofun-cfun-arg*[*OF joinableD*(*1*)[*OF assms*(*2*)]],
rule monofun-cfun-arg[*OF joinableD*(*2*)[*OF assms*(*2*)]])
then show $rep{\cdot}x \sqcup rep{\cdot}y \sqsubseteq rep{\cdot}(x \sqcup y)$ **by** (*rule join-belowI*)
qed

lemma *distributive-cpo-iso*:
fixes *abs* :: *'r*::*distributive-cpo* $\to$ *'t*::*bounded-complete* **and** *rep*
assumes *iso abs rep* **shows** *OFCLASS*(*'t*, *distributive-cpo-class*)
proof *intro-classes*
interpret *iso abs rep* **by** *fact*
fix *a b c* :: *'t* **assume** *ab*: $a \Uparrow b$
have *cab*: $c \sqcap a \Uparrow c \sqcap b$ **by** (*rule joinableI*[*OF meet-below1 meet-below1*])
have $rep{\cdot}(c \sqcap (a \sqcup b)) = rep{\cdot}c \sqcap (rep{\cdot}a \sqcup rep{\cdot}b)$
by (*simp add*: *iso-rep-meet*[*OF assms*] *iso-rep-join*[*OF assms ab*])
also have $\ldots = (rep{\cdot}c \sqcap rep{\cdot}a) \sqcup (rep{\cdot}c \sqcap rep{\cdot}b)$
using *ab* **by** (*simp add*: *meet-join-distrib iso-rep-joinable-iff*[*OF assms*])
also have $\ldots = rep{\cdot}(c \sqcap a \sqcup (c \sqcap b))$
by (*simp add*: *iso-rep-meet*[*OF assms*] *iso-rep-join*[*OF assms cab*])
finally show $c \sqcap (a \sqcup b) = c \sqcap a \sqcup (c \sqcap b)$ **by** (*simp add*: *rep-eq*)
qed

11.2 Map functionals

The domain package defines *T-map* $\equiv \Lambda\ v.\ \mu\ f.\ abs\ oo\ F\ v\ f\ oo\ rep$, with F built from the rep-type combinators. A predicate preserved by F and holding at $\bot$ holds for the map by fixed-point induction, if admissible; at $v = ID$ (*T-map*$\cdot ID = ID$) this yields the instance. Only the preservation step needs proof search.

definition *preserves* :: $((\prime a{::}pcpo \to \prime a) \Rightarrow bool) \Rightarrow ((\prime a \to \prime a) \to (\prime a \to \prime a)) \Rightarrow bool$ **where**
  $preserves\ P\ F \longleftrightarrow (\forall f.\ P\ f \longrightarrow P\ (F{\cdot}f))$

lemma *preserves-LAM*: $cont\ G \Longrightarrow (\bigwedge f.\ P\ f \Longrightarrow P\ (G\ f)) \Longrightarrow preserves\ P\ (\Lambda\ f.\ G\ f)$
  by (*simp add*: *preserves-def*)

lemma *fix-preserves*: $adm\ P \Longrightarrow P\ \bot \Longrightarrow preserves\ P\ F \Longrightarrow P\ (fix{\cdot}F)$
  by (*erule fix-ind*) (*simp-all add*: *preserves-def*)

12 Order predicates as deflation locales

The class properties, relativised to the image of a deflation. Bundling *deflation* into every predicate keeps the connective rules uniform: the deflation part rides along through the HOLCF *deflation-∗-map* lemmas inside each connective rule, and admissibility gains *adm-deflation*. The image of a chain's lub is approximated by the images of its members, which gives admissibility of the order predicates.

locale *bounded-complete-defl* = *deflation d* **for** $d :: \prime a{::}bifinite \to \prime a$ +
  assumes *pair-lubs*: $d{\cdot}a \sqsubseteq c \Longrightarrow d{\cdot}b \sqsubseteq c \Longrightarrow \exists z.\ \{d{\cdot}a,\ d{\cdot}b\} <<|\ z$

locale *filter-sum-defl* = *deflation d* **for** $d :: \prime a{::}bifinite \to \prime a$ +
  assumes *filter-sum-image*:
    $d{\cdot}x \neq \bot \Longrightarrow y \neq \bot \Longrightarrow d{\cdot}x \sqsubseteq u \Longrightarrow y \sqsubseteq u \Longrightarrow \exists l.\ l \neq \bot \wedge l \sqsubseteq d{\cdot}x \wedge l \sqsubseteq y$

lemma (**in** *bounded-complete-defl*) *deflation*: *deflation d* **by** (*fact deflation-axioms*)

lemma (**in** *filter-sum-defl*) *deflation*: *deflation d* **by** (*fact deflation-axioms*)

12.1 Bounded completeness

lemma *bounded-complete-deflI*:
  $deflation\ d \Longrightarrow (\bigwedge a\ b\ c.\ d{\cdot}a \sqsubseteq c \Longrightarrow d{\cdot}b \sqsubseteq c \Longrightarrow \exists z.\ \{d{\cdot}a,\ d{\cdot}b\} <<|\ z) \Longrightarrow bounded\text{-}complete\text{-}defl\ d$
  by (*simp add*: *bounded-complete-defl-def bounded-complete-defl-axioms-def*)

lemma *bounded-complete-defl-bottom*: *bounded-complete-defl* $(\bot :: \prime a{::}bifinite \to \prime a)$
  by (*rule bounded-complete-deflI*[*OF deflation-bottom*]) (*auto intro*: *is-lub-singleton*)

lemma *bounded-complete-defl-ID*: *bounded-complete-defl* (*ID* :: *'a*::{*bifinite*, *bounded-complete*} → *'a*)
proof (*rule bounded-complete-deflI*[*OF deflation-ID*])
  fix *a b c* :: *'a* **assume** *ID*·*a* ⊑ *c ID*·*b* ⊑ *c*
  then have {*ID*·*a*, *ID*·*b*} <| *c* **by** (*simp add*: *is-ub-def*)
  then show ∃ *z*. {*ID*·*a*, *ID*·*b*} <<| *z* **by** (*rule ex-bound-ex-lub*)
qed

lemma *up-is-lub-pair*: {*x*, *y*} <<| *z* ⟹ {*up*·*x*, *up*·*y*} <<| *up*·*z*
proof (*rule is-lubI*)
  assume *z*: {*x*, *y*} <<| *z*
  then have *x* ⊑ *z y* ⊑ *z* **by** (*auto dest*: *is-lubD1 is-ubD*)
  then show {*up*·*x*, *up*·*y*} <| *up*·*z* **by** (*auto intro*!: *is-ubI*)
next
  fix *u* **assume** *z*: {*x*, *y*} <<| *z* **and** *u*: {*up*·*x*, *up*·*y*} <| *u*
  then have *up*·*x* ⊑ *u up*·*y* ⊑ *u* **by** (*auto dest*: *is-ubD*)
  then obtain *u'* **where** *u* = *up*·*u' x* ⊑ *u' y* ⊑ *u'* **by** (*cases u*) *auto*
  with *z* **show** *up*·*z* ⊑ *u* **by** (*auto intro*: *is-lubD2 is-ubI*)
qed

lemma *bounded-complete-defl-u-map*:
  assumes *f*: *bounded-complete-defl f* **shows** *bounded-complete-defl* (*u-map*·*f*)
proof (*rule bounded-complete-deflI*[*OF deflation-u-map*[*OF bounded-complete-defl.axioms*(*1*)[*OF f*]]])
  fix *a b c* **assume** *ac*: *u-map*·*f*·*a* ⊑ *c* **and** *bc*: *u-map*·*f*·*b* ⊑ *c*
  show ∃ *z*. {*u-map*·*f*·*a*, *u-map*·*f*·*b*} <<| *z*
  proof (*cases a* = ⊥ ∨ *b* = ⊥)
    case *True* **then show** *?thesis*
    proof
      assume *a* = ⊥ **then show** *?thesis*
        by (*intro exI*[*of* - *u-map*·*f*·*b*]) (*simp add*: *is-lub-bin*)
    next
      assume *b* = ⊥ **then show** *?thesis*
        by (*intro exI*[*of* - *u-map*·*f*·*a*]) (*simp add*: *is-lub-bin insert-commute*)
    qed
  next
    case *False*
    then obtain *x y* **where** *ab*: *a* = *up*·*x b* = *up*·*y* **by** (*cases a*; *cases b*) *auto*
    with *ac* **obtain** *c'* **where** *c*: *c* = *up*·*c' f*·*x* ⊑ *c'* **by** (*cases c*) *auto*
    with *ab bc* **have** *f*·*y* ⊑ *c'* **by** *simp*
    from *bounded-complete-defl.pair-lubs*[*OF f* ‹*f*·*x* ⊑ *c'*› *this*]
    obtain *z* **where** {*f*·*x*, *f*·*y*} <<| *z* **by** (*elim exE*)
    then have {*up*·(*f*·*x*), *up*·(*f*·*y*)} <<| *up*·*z* **by** (*rule up-is-lub-pair*)
    then show *?thesis* **by** (*auto simp*: *ab*)
  qed
qed

lemma *sinl-is-lub-pair*:

assumes *z*: $\{x, y\} <<| z$ **shows** $\{sinl{\cdot}x, sinl{\cdot}y\} <<| (sinl{\cdot}z :: 'a{::}pcpo \oplus 'b{::}pcpo)$
proof (*rule is-lubI*)
have $x \sqsubseteq z$ $y \sqsubseteq z$ **using** *z* **by** (*auto dest*: *is-lubD1 is-ubD*)
then show $\{sinl{\cdot}x, sinl{\cdot}y\} <| (sinl{\cdot}z :: 'a \oplus 'b)$ **by** (*auto intro*!: *is-ubI*)
next
fix $u :: 'a \oplus 'b$ **assume** $\{sinl{\cdot}x, sinl{\cdot}y\} <| u$
then have *u*: $sinl{\cdot}x \sqsubseteq u$ $sinl{\cdot}y \sqsubseteq u$ **by** (*auto dest*: *is-ubD*)
show $sinl{\cdot}z \sqsubseteq u$
proof (*cases u*)
case *bottom*
with *u* **have** $\{x, y\} <| \bot$ **by** (*auto intro*!: *is-ubI*)
with *z* **have** $z = \bot$ **by** (*blast dest*: *is-lubD2 intro*: *bottomI*)
then show *?thesis* **by** *simp*
next
case (*sinl w*)
with *u* **have** $\{x, y\} <| w$ **by** (*auto intro*!: *is-ubI*)
with *z sinl* **show** *?thesis* **by** (*simp add*: *is-lubD2*)
next
case (*sinr w*)
with *u* **have** $\{x, y\} <| \bot$ **by** (*auto intro*!: *is-ubI*)
with *z* **have** $z = \bot$ **by** (*blast dest*: *is-lubD2 intro*: *bottomI*)
then show *?thesis* **by** *simp*
qed
qed

lemma *sinr-is-lub-pair*:
assumes *z*: $\{x, y\} <<| z$ **shows** $\{sinr{\cdot}x, sinr{\cdot}y\} <<| (sinr{\cdot}z :: 'a{::}pcpo \oplus 'b{::}pcpo)$
proof (*rule is-lubI*)
have $x \sqsubseteq z$ $y \sqsubseteq z$ **using** *z* **by** (*auto dest*: *is-lubD1 is-ubD*)
then show $\{sinr{\cdot}x, sinr{\cdot}y\} <| (sinr{\cdot}z :: 'a \oplus 'b)$ **by** (*auto intro*!: *is-ubI*)
next
fix $u :: 'a \oplus 'b$ **assume** $\{sinr{\cdot}x, sinr{\cdot}y\} <| u$
then have *u*: $sinr{\cdot}x \sqsubseteq u$ $sinr{\cdot}y \sqsubseteq u$ **by** (*auto dest*: *is-ubD*)
show $sinr{\cdot}z \sqsubseteq u$
proof (*cases u*)
case *bottom*
with *u* **have** $\{x, y\} <| \bot$ **by** (*auto intro*!: *is-ubI*)
with *z* **have** $z = \bot$ **by** (*blast dest*: *is-lubD2 intro*: *bottomI*)
then show *?thesis* **by** *simp*
next
case (*sinl w*)
with *u* **have** $\{x, y\} <| \bot$ **by** (*auto intro*!: *is-ubI*)
with *z* **have** $z = \bot$ **by** (*blast dest*: *is-lubD2 intro*: *bottomI*)
then show *?thesis* **by** *simp*
next
case (*sinr w*)
with *u* **have** $\{x, y\} <| w$ **by** (*auto intro*!: *is-ubI*)
with *z sinr* **show** *?thesis* **by** (*simp add*: *is-lubD2*)

qed
qed

lemma *ex-lub-pair-bottom*: $x = \bot \vee y = \bot \Longrightarrow \exists\, z.\ \{x, y\} <<|\ (z::'a::pcpo)$
proof (*elim disjE*)
assume $x = \bot$ **then show** *?thesis* **by** (*intro exI*[*of* - *y*]) (*simp add*: *is-lub-bin*)
next
assume $y = \bot$ **then show** *?thesis* **by** (*intro exI*[*of* - *x*]) (*simp add*: *is-lub-bin insert-commute*)
qed

lemma *bounded-complete-defl-ssum-map*:
assumes *f*: *bounded-complete-defl f* **and** *g*: *bounded-complete-defl g*
shows *bounded-complete-defl* (*ssum-map*·*f*·*g*)
proof (*rule bounded-complete-deflI*)
show *deflation* (*ssum-map*·*f*·*g*)
by (*rule deflation-ssum-map*[*OF bounded-complete-defl.axioms*(*1*)[*OF f*] *bounded-complete-defl.axioms*(*1*)[*OF g*]])
fix *a b c*
assume *ac*: *ssum-map*·*f*·*g*·*a* $\sqsubseteq$ *c* **and** *bc*: *ssum-map*·*f*·*g*·*b* $\sqsubseteq$ *c*
show $\exists\, z.\ \{$*ssum-map*·*f*·*g*·*a*, *ssum-map*·*f*·*g*·*b*$\} <<|\ z$
proof (*cases ssum-map*·*f*·*g*·*a* $= \bot \vee$ *ssum-map*·*f*·*g*·*b* $= \bot$)
case *True* **then show** *?thesis* **by** (*rule ex-lub-pair-bottom*)
next
case *False*
then have *A*: *ssum-map*·*f*·*g*·*a* $\neq \bot$ **and** *B*: *ssum-map*·*f*·*g*·*b* $\neq \bot$ **by** *simp-all*
show *?thesis*
proof (*cases c*)
case *bottom* **with** *ac A* **show** *?thesis* **by** *simp*
next
case (*sinl w*)
obtain *x* **where** *a*: *a* = *sinl*·*x x* $\neq \bot$ *f*·*x* $\sqsubseteq$ *w*
using *ac A sinl* **by** (*cases a*) *auto*
obtain *y* **where** *b*: *b* = *sinl*·*y y* $\neq \bot$ *f*·*y* $\sqsubseteq$ *w*
using *bc B sinl* **by** (*cases b*) *auto*
from *bounded-complete-defl.pair-lubs*[*OF f* ‹*f*·*x* $\sqsubseteq$ *w*› ‹*f*·*y* $\sqsubseteq$ *w*›]
obtain *z* **where** $\{f{\cdot}x, f{\cdot}y\} <<|\ z$ **by** (*elim exE*)
then have $\{$*sinl*·(*f*·*x*), *sinl*·(*f*·*y*)$\} <<|$ (*sinl*·*z* :: - $\oplus$ -) **by** (*rule sinl-is-lub-pair*)
with *a b* **show** *?thesis* **by** *auto*
next
case (*sinr w*)
obtain *x* **where** *a*: *a* = *sinr*·*x x* $\neq \bot$ *g*·*x* $\sqsubseteq$ *w*
using *ac A sinr* **by** (*cases a*) *auto*
obtain *y* **where** *b*: *b* = *sinr*·*y y* $\neq \bot$ *g*·*y* $\sqsubseteq$ *w*
using *bc B sinr* **by** (*cases b*) *auto*
from *bounded-complete-defl.pair-lubs*[*OF g* ‹*g*·*x* $\sqsubseteq$ *w*› ‹*g*·*y* $\sqsubseteq$ *w*›]
obtain *z* **where** $\{g{\cdot}x, g{\cdot}y\} <<|\ z$ **by** (*elim exE*)
then have $\{$*sinr*·(*g*·*x*), *sinr*·(*g*·*y*)$\} <<|$ (*sinr*·*z* :: - $\oplus$ -) **by** (*rule sinr-is-lub-pair*)
with *a b* **show** *?thesis* **by** *auto*

```
    qed
  qed
qed

lemma spair-is-lub-pair:
  assumes z1: {x1, y1} <<| z1 and z2: {x2, y2} <<| z2
    and nb: x1 ≠ ⊥ x2 ≠ ⊥ y1 ≠ ⊥ y2 ≠ ⊥
  shows {(:x1, x2:), (:y1, y2:)} <<| (:z1, z2:)
proof −
  have ub: x1 ⊑ z1 y1 ⊑ z1 x2 ⊑ z2 y2 ⊑ z2
    using z1 z2 by (auto dest: is-lubD1 is-ubD)
  with nb have nbz: z1 ≠ ⊥ z2 ≠ ⊥ by auto
  show ?thesis
  proof (rule is-lubI)
  from ub nb nbz show {(:x1, x2:), (:y1, y2:)} <| (:z1, z2:)
    by (auto intro!: is-ubI simp: below-sprod)
next
  fix u assume {(:x1, x2:), (:y1, y2:)} <| u
  then have (:x1, x2:) ⊑ u (:y1, y2:) ⊑ u by (auto dest: is-ubD)
  with nb have {x1, y1} <| sfst·u {x2, y2} <| ssnd·u
    by (auto intro!: is-ubI simp: below-sprod)
  with z1 z2 have z1 ⊑ sfst·u z2 ⊑ ssnd·u by (auto dest: is-lubD2)
  with nbz show (:z1, z2:) ⊑ u by (simp add: below-sprod)
  qed
qed

lemma bounded-complete-defl-sprod-map:
  assumes f: bounded-complete-defl f and g: bounded-complete-defl g
  shows bounded-complete-defl (sprod-map·f·g)
proof (rule bounded-complete-deflI)
  show deflation (sprod-map·f·g)
    by (rule deflation-sprod-map[OF bounded-complete-defl.axioms(1)[OF f]
        bounded-complete-defl.axioms(1)[OF g]])
  fix a b c
  assume ac: sprod-map·f·g·a ⊑ c and bc: sprod-map·f·g·b ⊑ c
  show ∃ z. {sprod-map·f·g·a, sprod-map·f·g·b} <<| z
  proof (cases sprod-map·f·g·a = ⊥ ∨ sprod-map·f·g·b = ⊥)
    case True then show ?thesis by (rule ex-lub-pair-bottom)
  next
    case False
    then have A: sprod-map·f·g·a ≠ ⊥ and B: sprod-map·f·g·b ≠ ⊥ by simp-all
    obtain a1 a2 where a: a = (:a1, a2:) a1 ≠ ⊥ a2 ≠ ⊥ using A by (cases a)
auto
    obtain b1 b2 where b: b = (:b1, b2:) b1 ≠ ⊥ b2 ≠ ⊥ using B by (cases b)
auto
    from A a have fa: f·a1 ≠ ⊥ g·a2 ≠ ⊥ by simp-all
    from B b have fb: f·b1 ≠ ⊥ g·b2 ≠ ⊥ by simp-all
    from ac a fa have f·a1 ⊑ sfst·c g·a2 ⊑ ssnd·c by (simp-all add: below-sprod)
     moreover from bc b fb have f·b1 ⊑ sfst·c g·b2 ⊑ ssnd·c by (simp-all add:
```

below-sprod)
ultimately obtain *z1 z2* **where** $\{f{\cdot}a1,\ f{\cdot}b1\} <<|\ z1\ \{g{\cdot}a2,\ g{\cdot}b2\} <<|\ z2$
using *bounded-complete-defl.pair-lubs*[*OF f*] *bounded-complete-defl.pair-lubs*[*OF g*] **by** *meson*
then have $\{(:f{\cdot}a1,\ g{\cdot}a2:),\ (:f{\cdot}b1,\ g{\cdot}b2:)\} <<|\ (:z1,\ z2:)$
using *fa fb* **by** (*rule spair-is-lub-pair*)
with *a b* **show** *?thesis* **by** *auto*
qed
qed

lemma *bounded-complete-defl-iso*:
fixes *abs* :: *'r*::*bifinite* $\to$ *'t*::*bifinite* **and** *rep*
assumes *iso-abs*: $\bigwedge x.\ rep{\cdot}(abs{\cdot}x) = x$ **and** *iso-rep*: $\bigwedge y.\ abs{\cdot}(rep{\cdot}y) = y$
and *f*: *bounded-complete-defl f*
shows *bounded-complete-defl* (*abs oo f oo rep*)
proof (*rule bounded-complete-deflI*)
have *iso*: *iso abs rep* **by** (*rule iso.intro*[*OF iso-abs iso-rep*])
interpret *iso abs rep* **by** (*fact iso*)
show *deflation* (*abs oo f oo rep*)
by (*rule deflation-abs-rep*[*OF abs-iso rep-iso bounded-complete-defl.axioms*(*1*)[*OF f*]])
fix *a b c* **assume** $(abs\ oo\ f\ oo\ rep){\cdot}a \sqsubseteq c$ $(abs\ oo\ f\ oo\ rep){\cdot}b \sqsubseteq c$
then have $f{\cdot}(rep{\cdot}a) \sqsubseteq rep{\cdot}c\ f{\cdot}(rep{\cdot}b) \sqsubseteq rep{\cdot}c$ **by** (*simp-all add*: *abs-below*[*symmetric*])
from *bounded-complete-defl.pair-lubs*[*OF f this*]
obtain *z* :: *'r* **where** z: $\{f{\cdot}(rep{\cdot}a),\ f{\cdot}(rep{\cdot}b)\} <<|\ z$ **by** (*elim exE*)
have $(\lambda x.\ rep{\cdot}x)\ `\ \{abs{\cdot}(f{\cdot}(rep{\cdot}a)),\ abs{\cdot}(f{\cdot}(rep{\cdot}b))\} = \{f{\cdot}(rep{\cdot}a),\ f{\cdot}(rep{\cdot}b)\}$ **by** *simp*
with *z* **have** $\{abs{\cdot}(f{\cdot}(rep{\cdot}a)),\ abs{\cdot}(f{\cdot}(rep{\cdot}b))\} <<|\ abs{\cdot}z$
by (*simp only*: *iso-is-lub-iff*[*OF iso*, *symmetric*])
then show $\exists z.\ \{(abs\ oo\ f\ oo\ rep){\cdot}a,\ (abs\ oo\ f\ oo\ rep){\cdot}b\} <<|\ z$ **by** *auto*
qed

lemma *adm-bounded-complete-defl*: *adm* (*bounded-complete-defl* :: (*'a*::*bifinite* $\to$ *'a*) $\Rightarrow$ *bool*)
proof (*rule admI*)
fix *Y* :: *nat* $\Rightarrow$ *'a* $\to$ *'a* **assume** *Y*: *chain Y* **and** *P*: $\forall i.$ *bounded-complete-defl* (*Y i*)
have *chx*: *chain* $(\lambda n.\ Y\ n{\cdot}x)$ **for** *x* **by** (*rule ch2ch-Rep-cfunL*[*OF Y*])
have *reach*: $(\bigsqcup n.\ Y\ n{\cdot}x) = (\bigsqcup i.\ Y\ i){\cdot}x$ **for** *x* **by** (*rule contlub-cfun-fun*[*OF Y*, *symmetric*])
have *below*: $Y\ n{\cdot}x \sqsubseteq (\bigsqcup i.\ Y\ i){\cdot}x$ **for** *n x*
using *is-ub-thelub*[*OF chx*, *of n x*] **by** (*simp only*: *reach*)
show *bounded-complete-defl* $(\bigsqcup i.\ Y\ i)$
proof (*rule bounded-complete-deflI*)
show *deflation* $(\bigsqcup i.\ Y\ i)$
by (*rule admD*[*OF adm-deflation Y*]) (*rule bounded-complete-defl.axioms*(*1*)[*OF P*[*rule-format*]])
fix *a b c* **assume** *ac*: $(\bigsqcup i.\ Y\ i){\cdot}a \sqsubseteq c$ **and** *bc*: $(\bigsqcup i.\ Y\ i){\cdot}b \sqsubseteq c$
define *L* **where** $L\ n = lub\ \{Y\ n{\cdot}a,\ Y\ n{\cdot}b\}$ **for** *n*

```
    have L: {Y n·a, Y n·b} <<| L n for n
    proof −
      have Y n·a ⊑ c Y n·b ⊑ c
        using below-trans[OF below[of n a] ac] below-trans[OF below[of n b] bc] .
      from bounded-complete-defl.pair-lubs[OF P[rule-format] this]
      obtain z where {Y n·a, Y n·b} <<| z by (elim exE)
      then show ?thesis unfolding L-def by (rule is-lub-lub)
    qed
    have chL: chain L
    proof (rule chainI)
      fix n
      have Y n·a ⊑ L (Suc n) Y n·b ⊑ L (Suc n)
          using chainE[OF chx] is-lubD1[OF L, THEN is-ubD] by (blast intro:
below-trans)+
      then show L n ⊑ L (Suc n) by (blast intro: is-lubD2[OF L] is-ubI)
    qed
    have {(⨆ i. Y i)·a, (⨆ i. Y i)·b} <<| (⨆ n. L n)
    proof (rule is-lubI)
      have (⨆ n. Y n·a) ⊑ (⨆ n. L n) (⨆ n. Y n·b) ⊑ (⨆ n. L n)
        using is-lubD1[OF L, THEN is-ubD] by (blast intro: lub-mono chx chL)+
       then show {(⨆ i. Y i)·a, (⨆ i. Y i)·b} <| (⨆ n. L n) by (simp add: reach
is-ub-def)
    next
      fix u assume {(⨆ i. Y i)·a, (⨆ i. Y i)·b} <| u
      then have (⨆ i. Y i)·a ⊑ u (⨆ i. Y i)·b ⊑ u by (auto dest: is-ubD)
      then have L n ⊑ u for n
          using below[of n a] below[of n b] by (blast intro: is-lubD2[OF L] is-ubI
below-trans)
      then show (⨆ n. L n) ⊑ u by (rule lub-below[OF chL])
    qed
    then show ∃ z. {(⨆ i. Y i)·a, (⨆ i. Y i)·b} <<| z ..
  qed
qed
```

Antitone among deflations: a smaller deflation has a smaller image.

```
lemma bounded-complete-defl-below:
  assumes le: d ⊑ e and e: bounded-complete-defl e and d: deflation d
  shows bounded-complete-defl d
proof (rule bounded-complete-deflI[OF d])
  have de: deflation e by (rule bounded-complete-defl.axioms(1)[OF e])
  fix a b c assume d·a ⊑ c d·b ⊑ c
  moreover have e·(d·a) = d·a e·(d·b) = d·b by (rule deflation-below-comp2[OF
d de le])+
  ultimately show ∃ z. {d·a, d·b} <<| z
    using bounded-complete-defl.pair-lubs[OF e, of d·a c d·b] by simp
qed

lemma bounded-complete-class-deflI:
  assumes d: bounded-complete-defl (ID :: 'a::bifinite → 'a)
```

shows *OFCLASS*('*a*, *bounded-complete-class*)
proof (*rule bounded-complete-chainI*[*of* λ*n*. *ID*])
show *chain* (λ*n*. *ID* :: '*a* → '*a*) **by** *simp*
show (⨆ *n*. *ID* :: '*a* → '*a*) = *ID* **by** *simp*
fix *n* **and** *a b c* :: '*a*
assume *ID*·*a* ⊑ *c* *ID*·*b* ⊑ *c*
then show ∃ *z*. {*ID*·*a*, *ID*·*b*} <<| *z* **by** (*rule bounded-complete-defl.pair-lubs*[*OF d*])
qed

12.2 Separation

Separation is stated one-sidedly: a nonbottom image element shares a nonbottom lower bound with every nonbottom element it is joinable with, the second element being arbitrary. At *ID* this is *filter-sum*. The one-sided form is admissible without conditions on the images (a witness found at a finite stage stays a witness in the limit), and it is the shape that carries over to separated sums, where the witness is a global atom.

lemma *filter-sum-deflI*:
assumes *deflation d*
and ⋀*x y u*. *d*·*x* ≠ ⊥ ⟹ *y* ≠ ⊥ ⟹ *d*·*x* ⊑ *u* ⟹ *y* ⊑ *u* ⟹ ∃ *l*. *l* ≠ ⊥ ∧ *l* ⊑ *d*·*x* ∧ *l* ⊑ *y*
shows *filter-sum-defl d*
using *assms* **by** (*simp add*: *filter-sum-defl-def filter-sum-defl-axioms-def*)

lemma *filter-sum-defl-bottom*: *filter-sum-defl* (⊥ :: '*a*::*bifinite* → '*a*)
by (*rule filter-sum-deflI*[*OF deflation-bottom*]) *simp*

lemma *filter-sum-defl-ID*: *filter-sum-defl* (*ID* :: '*a*::{*bifinite*, *filter-sum*} → '*a*)
by (*rule filter-sum-deflI*[*OF deflation-ID*]) (*simp add*: *filter-sum*)

lemma *filter-sum-class-deflI*:
assumes *d*: *filter-sum-defl* (*ID* :: '*a*::*bifinite* → '*a*)
shows *OFCLASS*('*a*, *filter-sum-class*)
proof
fix *x y u* :: '*a* **assume** *x* ≠ ⊥ *y* ≠ ⊥ *x* ⊑ *u* *y* ⊑ *u*
then show ∃ *l*. *l* ≠ ⊥ ∧ *l* ⊑ *x* ∧ *l* ⊑ *y*
using *filter-sum-defl.filter-sum-image*[*OF d*, *of x y u*] **by** *simp*
qed

Lifting separates unconditionally: only the deflation property of f is needed.

lemma *filter-sum-defl-u-map*:
assumes *f*: *deflation* (*f* :: '*a*::*bifinite* → '*a*) **shows** *filter-sum-defl* (*u-map*·*f*)
proof (*rule filter-sum-deflI*[*OF deflation-u-map*[*OF f*]])
fix *x y u* :: '$a_\perp$ **assume** *x*: *u-map*·*f*·*x* ≠ ⊥ **and** *y*: *y* ≠ ⊥
from *x* **obtain** *x'* **where** *x* = *up*·*x'* **by** (*cases x*) *auto*
moreover from *y* **obtain** *y'* **where** *y* = *up*·*y'* **by** (*cases y*) *auto*

ultimately show $\exists\, l.\ l \neq \bot \wedge l \sqsubseteq u\text{-}map{\cdot}f{\cdot}x \wedge l \sqsubseteq y$ **by** (*intro exI*[*of* - *up*·⊥])
simp
qed

lemma *filter-sum-defl-ssum-map*:
assumes *f*: *filter-sum-defl f* **and** *g*: *filter-sum-defl g*
shows *filter-sum-defl* (*ssum-map*·*f*·*g*)
proof (*rule filter-sum-deflI*)
show *deflation* (*ssum-map*·*f*·*g*)
using *f g* **by** (*intro deflation-ssum-map*) (*simp-all add*: *filter-sum-defl-def*)
fix *x y u* **assume** *x*: *ssum-map*·*f*·*g*·*x* ≠ ⊥ **and** *y*: *y* ≠ ⊥
and *xu*: *ssum-map*·*f*·*g*·*x* ⊑ *u* **and** *yu*: *y* ⊑ *u*
show ∃ *l*. *l* ≠ ⊥ ∧ *l* ⊑ *ssum-map*·*f*·*g*·*x* ∧ *l* ⊑ *y*
proof (*cases x*)
case *bottom* **with** *x* **show** *?thesis* **by** *simp*
next
case (*sinl x'*)
with *x* **have** *fx*: *f*·*x'* ≠ ⊥ **and** *img*: *ssum-map*·*f*·*g*·*x* = *sinl*·(*f*·*x'*) **by** *simp-all*
obtain *u'* **where** *u*: *u* = *sinl*·*u'* *f*·*x'* ⊑ *u'*
using *xu fx img* **by** (*cases u*) *auto*
obtain *y'* **where** *yy*: *y* = *sinl*·*y'* *y'* ≠ ⊥ *y'* ⊑ *u'*
using *yu y u* **by** (*cases y*) *auto*
obtain *l'* **where** *l'*: *l'* ≠ ⊥ *l'* ⊑ *f*·*x'* *l'* ⊑ *y'*
using *filter-sum-defl.filter-sum-image*[*OF f fx yy*(*2*) *u*(*2*) *yy*(*3*)] **by** *auto*
show *?thesis* **using** *l' img yy*(*1*) **by** (*intro exI*[*of* - *sinl*·*l'*]) *simp*
next
case (*sinr x'*)
with *x* **have** *gx*: *g*·*x'* ≠ ⊥ **and** *img*: *ssum-map*·*f*·*g*·*x* = *sinr*·(*g*·*x'*) **by** *simp-all*
obtain *u'* **where** *u*: *u* = *sinr*·*u'* *g*·*x'* ⊑ *u'*
using *xu gx img* **by** (*cases u*) *auto*
obtain *y'* **where** *yy*: *y* = *sinr*·*y'* *y'* ≠ ⊥ *y'* ⊑ *u'*
using *yu y u* **by** (*cases y*) *auto*
obtain *l'* **where** *l'*: *l'* ≠ ⊥ *l'* ⊑ *g*·*x'* *l'* ⊑ *y'*
using *filter-sum-defl.filter-sum-image*[*OF g gx yy*(*2*) *u*(*2*) *yy*(*3*)] **by** *auto*
show *?thesis* **using** *l' img yy*(*1*) **by** (*intro exI*[*of* - *sinr*·*l'*]) *simp*
qed
qed

lemma *filter-sum-defl-sprod-map*:
assumes *f*: *filter-sum-defl f* **and** *g*: *filter-sum-defl g*
shows *filter-sum-defl* (*sprod-map*·*f*·*g*)
proof (*rule filter-sum-deflI*)
show *deflation* (*sprod-map*·*f*·*g*)
using *f g* **by** (*intro deflation-sprod-map*) (*simp-all add*: *filter-sum-defl-def*)
fix *x y u* **assume** *x*: *sprod-map*·*f*·*g*·*x* ≠ ⊥ **and** *y*: *y* ≠ ⊥
and *xu*: *sprod-map*·*f*·*g*·*x* ⊑ *u* **and** *yu*: *y* ⊑ *u*
obtain *x1 x2* **where** *xx*: *x* = (:*x1*, *x2*:) *x1* ≠ ⊥ *x2* ≠ ⊥ **using** *x* **by** (*cases x*)
auto
with *x* **have** *fx*: *f*·*x1* ≠ ⊥ *g*·*x2* ≠ ⊥ **and** *img*: *sprod-map*·*f*·*g*·*x* = (:*f*·*x1*, *g*·*x2*:)

```
    by simp-all
  obtain u1 u2 where u: u = (:u1, u2:) f·x1 ⊑ u1 g·x2 ⊑ u2
    using xu fx img by (cases u) (auto simp: spair-below)
  obtain y1 y2 where yy: y = (:y1, y2:) y1 ≠ ⊥ y2 ≠ ⊥ y1 ⊑ u1 y2 ⊑ u2
    using yu y u by (cases y) (auto simp: spair-below)
  obtain l1 where l1: l1 ≠ ⊥ l1 ⊑ f·x1 l1 ⊑ y1
    using filter-sum-defl.filter-sum-image[OF f fx(1) yy(2) u(2) yy(4)] by auto
  obtain l2 where l2: l2 ≠ ⊥ l2 ⊑ g·x2 l2 ⊑ y2
    using filter-sum-defl.filter-sum-image[OF g fx(2) yy(3) u(3) yy(5)] by auto
  show ∃ l. l ≠ ⊥ ∧ l ⊑ sprod-map·f·g·x ∧ l ⊑ y
    using l1 l2 img yy(1) by (intro exI[of - (:l1, l2:)]) (simp add: spair-below)
qed

lemma filter-sum-defl-iso:
  fixes abs :: 'r::bifinite → 't::bifinite and rep
  assumes iso-abs: ⋀x. rep·(abs·x) = x and iso-rep: ⋀y. abs·(rep·y) = y
    and f: filter-sum-defl f
  shows filter-sum-defl (abs oo f oo rep)
proof (rule filter-sum-deflI)
  interpret iso abs rep by (rule iso.intro[OF iso-abs iso-rep])
  show deflation (abs oo f oo rep)
   using f by (intro deflation-abs-rep[OF abs-iso rep-iso]) (simp add: filter-sum-defl-def)
  fix x y u assume x: (abs oo f oo rep)·x ≠ ⊥ and y: y ≠ ⊥
    and xu: (abs oo f oo rep)·x ⊑ u and yu: y ⊑ u
  have fx: f·(rep·x) ≠ ⊥ using x by (simp add: abs-bottom-iff)
  have ry: rep·y ≠ ⊥ using y by (simp add: rep-bottom-iff)
  have rep·(abs·(f·(rep·x))) ⊑ rep·u using monofun-cfun-arg[OF xu, of rep] by
simp
  then have fxu: f·(rep·x) ⊑ rep·u by (simp del: rep-iso)
  have ryu: rep·y ⊑ rep·u using yu by (rule monofun-cfun-arg)
  obtain l where l: l ≠ ⊥ l ⊑ f·(rep·x) l ⊑ rep·y
    using filter-sum-defl.filter-sum-image[OF f fx ry fxu ryu] by auto
  have abs·l ⊑ abs·(f·(rep·x)) using l(2) by (rule monofun-cfun-arg)
  moreover have abs·l ⊑ abs·(rep·y) using l(3) by (rule monofun-cfun-arg)
  ultimately show ∃ l. l ≠ ⊥ ∧ l ⊑ (abs oo f oo rep)·x ∧ l ⊑ y
    using l(1) by (intro exI[of - abs·l]) (simp add: abs-bottom-iff)
qed

lemma adm-filter-sum-defl: adm (filter-sum-defl :: ('a::bifinite → 'a) ⇒ bool)
proof (rule admI)
  fix Y :: nat ⇒ 'a → 'a assume Y: chain Y and s: ∀ i. filter-sum-defl (Y i)
  show filter-sum-defl (⨆ i. Y i)
  proof (rule filter-sum-deflI)
    show deflation (⨆ i. Y i)
     by (rule admD[OF adm-deflation Y]) (use s in ‹simp add: filter-sum-defl-def›)
    fix x y u assume x: (⨆ i. Y i)·x ≠ ⊥ and y: y ≠ ⊥
      and xu: (⨆ i. Y i)·x ⊑ u and yu: y ⊑ u
    have chx: chain (λi. Y i·x) by (rule ch2ch-Rep-cfunL[OF Y])
    have lub: (⨆ i. Y i)·x = (⨆ i. Y i·x) by (rule contlub-cfun-fun[OF Y])
```

obtain n **where** n: $Y\ n{\cdot}x \neq \bot$ **using** x *lub-eq-bottom-iff*[*OF chx*] *lub* **by** *auto*
have *below*: $Y\ n{\cdot}x \sqsubseteq (\bigsqcup i.\ Y\ i){\cdot}x$ **by** (*rule monofun-cfun-fun*[*OF is-ub-thelub*[*OF Y*]])
obtain l **where** l: $l \neq \bot\ l \sqsubseteq Y\ n{\cdot}x\ l \sqsubseteq y$
using *filter-sum-defl.filter-sum-image*[*OF s*[*rule-format*, *of n*] *n y below-trans*[*OF below xu*] *yu*]
by *auto*
show $\exists l.\ l \neq \bot \wedge l \sqsubseteq (\bigsqcup i.\ Y\ i){\cdot}x \wedge l \sqsubseteq y$
using l *below-trans*[*OF l*(*2*) *below*] **by** *auto*
qed
qed

Antitone among deflations.

lemma *filter-sum-defl-below*:
assumes *le*: $d \sqsubseteq e$ **and** e: *filter-sum-defl* e **and** d: *deflation* d
shows *filter-sum-defl* d
proof (*rule filter-sum-deflI*[*OF d*])
fix $x\ y\ u$ **assume** x: $d{\cdot}x \neq \bot\ y \neq \bot\ d{\cdot}x \sqsubseteq u\ y \sqsubseteq u$
have $e{\cdot}(d{\cdot}x) = d{\cdot}x$
by (*rule deflation-below-comp2*[*OF d filter-sum-defl.axioms*(*1*)[*OF e*] *le*])
then show $\exists l.\ l \neq \bot \wedge l \sqsubseteq d{\cdot}x \wedge l \sqsubseteq y$
using *filter-sum-defl.filter-sum-image*[*OF e*, *of d*$\cdot x\ y\ u$] x **by** *simp*
qed

12.3 Separated sums

The one-sided separation of the previous subsection with an *atom* of the whole type as witness. Since the atom is global and the second element arbitrary, a witness found at a finite stage of a chain stays a witness in the limit: the predicate is admissible with no condition on the images. (Minimality taken within the image instead would not be: a type whose nonbottom elements form a descending chain without least element has no atoms, yet is the limit of finite deflations whose images do have least nonbottom elements.)

locale *separated-sum-defl* = *deflation d* **for** d :: *'a*::*bifinite* $\to$ *'a* +
assumes *separated-sum-image*:
$d{\cdot}x \neq \bot \Longrightarrow y \neq \bot \Longrightarrow d{\cdot}x \sqsubseteq u \Longrightarrow y \sqsubseteq u \Longrightarrow \exists l.\ \mathit{atom}\ l \wedge l \sqsubseteq d{\cdot}x \wedge l \sqsubseteq y$

lemma (**in** *separated-sum-defl*) *deflation*: *deflation d* **by** (*fact deflation-axioms*)

lemma *separated-sum-deflI*:
assumes *deflation d*
and $\bigwedge x\ y\ u.\ d{\cdot}x \neq \bot \Longrightarrow y \neq \bot \Longrightarrow d{\cdot}x \sqsubseteq u \Longrightarrow y \sqsubseteq u \Longrightarrow \exists l.\ \mathit{atom}\ l \wedge l \sqsubseteq d{\cdot}x \wedge l \sqsubseteq y$
shows *separated-sum-defl d*
using *assms* **by** (*simp add*: *separated-sum-defl-def separated-sum-defl-axioms-def*)

lemma *separated-sum-defl-bottom*: *separated-sum-defl* ($\bot$ :: *'a*::*bifinite* $\to$ *'a*)

by (*rule separated-sum-deflI*[*OF deflation-bottom*]) *simp*

lemma *separated-sum-defl-ID*: *separated-sum-defl* (*ID* :: *'a*::{*bifinite*, *separated-sum*} → *'a*)
by (*rule separated-sum-deflI*[*OF deflation-ID*]) (*simp add*: *separated-sum*)

lemma *separated-sum-class-deflI*:
assumes *d*: *separated-sum-defl* (*ID* :: *'a*::*bifinite* → *'a*)
shows *OFCLASS*(*'a*, *separated-sum-class*)
proof
fix *x y u* :: *'a* **assume** *x* ≠ ⊥ *y* ≠ ⊥ *x* ⊑ *u y* ⊑ *u*
then show ∃ *l*. *atom l* ∧ *l* ⊑ *x* ∧ *l* ⊑ *y*
using *separated-sum-defl.separated-sum-image*[*OF d*, *of x y u*] **by** *simp*
qed

lemma *separated-sum-defl-u-map*:
assumes *f*: *deflation* (*f* :: *'a*::*bifinite* → *'a*) **shows** *separated-sum-defl* (*u-map*·*f*)
proof (*rule separated-sum-deflI*[*OF deflation-u-map*[*OF f*]])
fix *x y u* :: *'a*$_\perp$ **assume** *x*: *u-map*·*f*·*x* ≠ ⊥ **and** *y*: *y* ≠ ⊥
from *x* **obtain** *x'* **where** *x* = *up*·*x'* **by** (*cases x*) *auto*
moreover from *y* **obtain** *y'* **where** *y* = *up*·*y'* **by** (*cases y*) *auto*
ultimately show ∃ *l*. *atom l* ∧ *l* ⊑ *u-map*·*f*·*x* ∧ *l* ⊑ *y*
by (*intro exI*[*of* - *up*·⊥]) (*simp add*: *atom-up-bottom*)
qed

lemma *separated-sum-defl-ssum-map*:
assumes *f*: *separated-sum-defl f* **and** *g*: *separated-sum-defl g*
shows *separated-sum-defl* (*ssum-map*·*f*·*g*)
proof (*rule separated-sum-deflI*)
show *deflation* (*ssum-map*·*f*·*g*)
using *f g* **by** (*intro deflation-ssum-map*) (*simp-all add*: *separated-sum-defl-def*)
fix *x y u* **assume** *x*: *ssum-map*·*f*·*g*·*x* ≠ ⊥ **and** *y*: *y* ≠ ⊥
and *xu*: *ssum-map*·*f*·*g*·*x* ⊑ *u* **and** *yu*: *y* ⊑ *u*
show ∃ *l*. *atom l* ∧ *l* ⊑ *ssum-map*·*f*·*g*·*x* ∧ *l* ⊑ *y*
proof (*cases x*)
case *bottom* **with** *x* **show** *?thesis* **by** *simp*
next
case (*sinl x'*)
with *x* **have** *fx*: *f*·*x'* ≠ ⊥ **and** *img*: *ssum-map*·*f*·*g*·*x* = *sinl*·(*f*·*x'*) **by** *simp-all*
obtain *u'* **where** *u*: *u* = *sinl*·*u'* *f*·*x'* ⊑ *u'*
using *xu fx img* **by** (*cases u*) *auto*
obtain *y'* **where** *yy*: *y* = *sinl*·*y'* *y'* ≠ ⊥ *y'* ⊑ *u'*
using *yu y u* **by** (*cases y*) *auto*
obtain *l'* **where** *l'*: *atom l'* *l'* ⊑ *f*·*x'* *l'* ⊑ *y'*
using *separated-sum-defl.separated-sum-image*[*OF f fx yy*(*2*) *u*(*2*) *yy*(*3*)] **by** *auto*
show *?thesis* **using** *l' img yy*(*1*) **by** (*intro exI*[*of* - *sinl*·*l'*]) (*simp add*: *atom-sinl-iff*)
next
case (*sinr x'*)

```
    with x have gx: g·x' ≠ ⊥ and img: ssum-map·f·g·x = sinr·(g·x') by simp-all
    obtain u' where u: u = sinr·u' g·x' ⊑ u'
      using xu gx img by (cases u) auto
    obtain y' where yy: y = sinr·y' y' ≠ ⊥ y' ⊑ u'
      using yu y u by (cases y) auto
    obtain l' where l': atom l' l' ⊑ g·x' l' ⊑ y'
      using separated-sum-defl.separated-sum-image[OF g gx yy(2) u(2) yy(3)] by
auto
   show ?thesis using l' img yy(1) by (intro exI[of - sinr·l']) (simp add: atom-sinr-iff)
  qed
qed

lemma separated-sum-defl-sprod-map:
  assumes f: separated-sum-defl f and g: separated-sum-defl g
  shows separated-sum-defl (sprod-map·f·g)
proof (rule separated-sum-deflI)
  show deflation (sprod-map·f·g)
    using f g by (intro deflation-sprod-map) (simp-all add: separated-sum-defl-def)
  fix x y u assume x: sprod-map·f·g·x ≠ ⊥ and y: y ≠ ⊥
    and xu: sprod-map·f·g·x ⊑ u and yu: y ⊑ u
  obtain x1 x2 where xx: x = (:x1, x2:) x1 ≠ ⊥ x2 ≠ ⊥ using x by (cases x)
auto
  with x have fx: f·x1 ≠ ⊥ g·x2 ≠ ⊥ and img: sprod-map·f·g·x = (:f·x1, g·x2:)
    by simp-all
  obtain u1 u2 where u: u = (:u1, u2:) f·x1 ⊑ u1 g·x2 ⊑ u2
    using xu fx img by (cases u) (auto simp: spair-below)
  obtain y1 y2 where yy: y = (:y1, y2:) y1 ≠ ⊥ y2 ≠ ⊥ y1 ⊑ u1 y2 ⊑ u2
    using yu y u by (cases y) (auto simp: spair-below)
  obtain l1 where l1: atom l1 l1 ⊑ f·x1 l1 ⊑ y1
   using separated-sum-defl.separated-sum-image[OF f fx(1) yy(2) u(2) yy(4)] by
auto
  obtain l2 where l2: atom l2 l2 ⊑ g·x2 l2 ⊑ y2
     using separated-sum-defl.separated-sum-image[OF g fx(2) yy(3) u(3) yy(5)]
by auto
  show ∃ l. atom l ∧ l ⊑ sprod-map·f·g·x ∧ l ⊑ y
    using l1 l2 img yy(1) atom-nonbottom[OF l1(1)] atom-nonbottom[OF l2(1)]
    by (intro exI[of - (:l1, l2:)]) (simp add: atom-spair spair-below)
qed
```

An isomorphism maps atoms to atoms.

```
lemma iso-atom:
  fixes abs :: 'r::pcpo → 't::pcpo and rep
  assumes iso: iso abs rep and l: atom l shows atom (abs·l)
proof (rule atomI)
  interpret iso abs rep by (fact iso)
  show abs·l ≠ ⊥ using atom-nonbottom[OF l] by (simp add: abs-bottom-iff)
  fix z assume z ⊑ abs·l
  then have rep·z ⊑ l by (simp add: abs-below[symmetric])
  from atomD[OF l this] show z = ⊥ ∨ z = abs·l by (auto simp: rep-bottom-iff)
```

qed

lemma *separated-sum-defl-iso*:
fixes *abs* :: *'r*::*bifinite* → *'t*::*bifinite* **and** *rep*
assumes *iso-abs*: ⋀*x*. *rep*·(*abs*·*x*) = *x* **and** *iso-rep*: ⋀*y*. *abs*·(*rep*·*y*) = *y*
and *f*: *separated-sum-defl f*
shows *separated-sum-defl* (*abs oo f oo rep*)
proof (*rule separated-sum-deflI*)
have *iso*: *iso abs rep* **by** (*rule iso.intro*[*OF iso-abs iso-rep*])
interpret *iso abs rep* **by** (*fact iso*)
show *deflation* (*abs oo f oo rep*)
using *f* **by** (*intro deflation-abs-rep*[*OF abs-iso rep-iso*]) (*simp add*: *separated-sum-defl-def*)
fix *x y u* **assume** *x*: (*abs oo f oo rep*)·*x* ≠ ⊥ **and** *y*: *y* ≠ ⊥
and *xu*: (*abs oo f oo rep*)·*x* ⊑ *u* **and** *yu*: *y* ⊑ *u*
have *fx*: *f*·(*rep*·*x*) ≠ ⊥ **using** *x* **by** (*simp add*: *abs-bottom-iff*)
have *ry*: *rep*·*y* ≠ ⊥ **using** *y* **by** (*simp add*: *rep-bottom-iff*)
have *rep*·(*abs*·(*f*·(*rep*·*x*))) ⊑ *rep*·*u* **using** *monofun-cfun-arg*[*OF xu*, *of rep*] **by** *simp*
then have *fxu*: *f*·(*rep*·*x*) ⊑ *rep*·*u* **by** (*simp del*: *rep-iso*)
have *ryu*: *rep*·*y* ⊑ *rep*·*u* **using** *yu* **by** (*rule monofun-cfun-arg*)
obtain *l* **where** *l*: *atom l l* ⊑ *f*·(*rep*·*x*) *l* ⊑ *rep*·*y*
using *separated-sum-defl.separated-sum-image*[*OF f fx ry fxu ryu*] **by** *auto*
have *abs*·*l* ⊑ *abs*·(*f*·(*rep*·*x*)) **using** *l*(*2*) **by** (*rule monofun-cfun-arg*)
moreover have *abs*·*l* ⊑ *abs*·(*rep*·*y*) **using** *l*(*3*) **by** (*rule monofun-cfun-arg*)
ultimately show ∃ *l*. *atom l* ∧ *l* ⊑ (*abs oo f oo rep*)·*x* ∧ *l* ⊑ *y*
using *iso-atom*[*OF iso l*(*1*)] **by** (*intro exI*[*of - abs*·*l*]) *simp*
qed

lemma *adm-separated-sum-defl*: *adm* (*separated-sum-defl* :: (*'a*::*bifinite* → *'a*) ⇒ *bool*)
proof (*rule admI*)
fix *Y* :: *nat* ⇒ *'a* → *'a* **assume** *Y*: *chain Y* **and** *s*: ∀ *i*. *separated-sum-defl* (*Y i*)
show *separated-sum-defl* (⨆ *i*. *Y i*)
proof (*rule separated-sum-deflI*)
show *deflation* (⨆ *i*. *Y i*)
by (*rule admD*[*OF adm-deflation Y*]) (*use s* **in** ‹*simp add*: *separated-sum-defl-def*›)
fix *x y u* **assume** *x*: (⨆ *i*. *Y i*)·*x* ≠ ⊥ **and** *y*: *y* ≠ ⊥
and *xu*: (⨆ *i*. *Y i*)·*x* ⊑ *u* **and** *yu*: *y* ⊑ *u*
have *chx*: *chain* (λ*i*. *Y i*·*x*) **by** (*rule ch2ch-Rep-cfunL*[*OF Y*])
have *lub*: (⨆ *i*. *Y i*)·*x* = (⨆ *i*. *Y i*·*x*) **by** (*rule contlub-cfun-fun*[*OF Y*])
obtain *n* **where** *n*: *Y n*·*x* ≠ ⊥ **using** *x lub-eq-bottom-iff*[*OF chx*] *lub* **by** *auto*
have *below*: *Y n*·*x* ⊑ (⨆ *i*. *Y i*)·*x* **by** (*rule monofun-cfun-fun*[*OF is-ub-thelub*[*OF Y*]])
obtain *l* **where** *l*: *atom l l* ⊑ *Y n*·*x l* ⊑ *y*
using *separated-sum-defl.separated-sum-image*[*OF s*[*rule-format*, *of n*] *n y* *below-trans*[*OF below xu*] *yu*]
by *auto*
show ∃ *l*. *atom l* ∧ *l* ⊑ (⨆ *i*. *Y i*)·*x* ∧ *l* ⊑ *y*

using *l below-trans*[*OF l*(*2*) *below*] **by** *auto*
qed
qed

12.4 Distributivity

The distributive law with only the meeting element c restricted to the image. The type itself is required to be bounded complete (that class is derived first), so meets and joins are the class operations; other order properties of the components enter the same way, as sort constraints (*filter-sum* for the smash product). Admissibility needs no downward closure, only continuity of meet (*cont-meet-arg*).

locale *distributive-defl* = *deflation d* **for** *d* :: *'a*::{*bifinite*, *bounded-complete*} → *'a* +
assumes *distrib-image*: $a \Uparrow b \Longrightarrow d{\cdot}c \sqcap (a \sqcup b) = d{\cdot}c \sqcap a \sqcup d{\cdot}c \sqcap b$

lemma (**in** *distributive-defl*) *deflation*: *deflation d* **by** (*fact deflation-axioms*)

lemma *distributive-deflI*:
deflation d $\Longrightarrow$ ($\bigwedge a\ b\ c.\ a \Uparrow b \Longrightarrow d{\cdot}c \sqcap (a \sqcup b) = d{\cdot}c \sqcap a \sqcup d{\cdot}c \sqcap b$) $\Longrightarrow$ *distributive-defl d*
by (*simp add*: *distributive-defl-def distributive-defl-axioms-def*)

lemma *distributive-defl-bottom*: *distributive-defl* (⊥ :: *'a*::{*bifinite*, *bounded-complete*} → *'a*)
by (*rule distributive-deflI*[*OF deflation-bottom*]) *simp*

lemma *distributive-defl-ID*: *distributive-defl* (*ID* :: *'a*::{*bifinite*, *distributive-cpo*} → *'a*)
by (*rule distributive-deflI*[*OF deflation-ID*]) (*simp add*: *meet-join-distrib*)

lemma *distributive-cpo-class-deflI*:
assumes *distributive-defl* (*ID* :: *'a*::{*bifinite*, *bounded-complete*} → *'a*)
shows *OFCLASS*(*'a*, *distributive-cpo-class*)
by *intro-classes* (*simp add*: *distributive-defl.distrib-image*[*OF assms*, *simplified*])

lemma *adm-distributive-defl*:
adm (*distributive-defl* :: (*'a*::{*bifinite*, *bounded-complete*} → *'a*) ⇒ *bool*)
proof (*rule admI*)
fix *Y* :: *nat* ⇒ *'a* → *'a*
assume *chain*: *chain Y* **and** *H*: ∀ *i*. *distributive-defl* (*Y i*)
show *distributive-defl* (⨆ *i*. *Y i*)
proof (*rule distributive-deflI*)
show *deflation* (⨆ *i*. *Y i*)
by (*rule admD*[*OF adm-deflation chain*]) (*simp add*: *distributive-defl.deflation H*)
fix *a b c* :: *'a* **assume** *ab*: *a* ⇑ *b*
have *ch*: *chain* (λ*n*. *Y n*·*c*) **by** (*rule ch2ch-Rep-cfunL*[*OF chain*])

have *reach*: $(\bigsqcup i.\ Y\ i)\cdot c = (\bigsqcup n.\ Y\ n\cdot c)$ **by** (*rule contlub-cfun-fun*[*OF chain*])
have *lim*: $(\bigsqcup n.\ Y\ n\cdot c) \sqcap y = (\bigsqcup n.\ Y\ n\cdot c \sqcap y)$ **and** *chy*: *chain* $(\lambda n.\ Y\ n\cdot c \sqcap y)$ **for** *y*
using *cont2contlubE*[*OF cont-meet-arg ch, of y*] *ch2ch-cont*[*OF cont-meet-arg ch, of y*]
by *simp-all*
have *j*: $Y\ n\cdot c \sqcap a \Uparrow Y\ n\cdot c \sqcap b$ **for** *n* **by** (*rule joinableI*[*OF meet-below1 meet-below1*])
have $(\bigsqcup n.\ Y\ n\cdot c) \sqcap (a \sqcup b) = (\bigsqcup n.\ Y\ n\cdot c \sqcap (a \sqcup b))$ **by** (*rule lim*)
also have $\ldots = (\bigsqcup n.\ Y\ n\cdot c \sqcap a \sqcup Y\ n\cdot c \sqcap b)$
by (*simp only*: *distributive-defl.distrib-image*[*OF H*[*rule-format*] *ab*])
also have $\ldots = (\bigsqcup n.\ Y\ n\cdot c \sqcap a) \sqcup (\bigsqcup n.\ Y\ n\cdot c \sqcap b)$ **by** (*rule lub-join-chains*(*3*)[*OF chy chy j*])
also have $\ldots = (\bigsqcup n.\ Y\ n\cdot c) \sqcap a \sqcup (\bigsqcup n.\ Y\ n\cdot c) \sqcap b$ **by** (*simp only*: *lim*[*symmetric*])
finally show $(\bigsqcup i.\ Y\ i)\cdot c \sqcap (a \sqcup b) = (\bigsqcup i.\ Y\ i)\cdot c \sqcap a \sqcup (\bigsqcup i.\ Y\ i)\cdot c \sqcap b$
by (*simp only*: *reach*)
qed
qed

lemma *distributive-defl-iso*:
fixes *abs* :: *'r*::{*bifinite, bounded-complete*} → *'t*::{*bifinite, bounded-complete*}
and *rep*
assumes *iso-abs*: $\bigwedge x.\ rep\cdot(abs\cdot x) = x$ **and** *iso-rep*: $\bigwedge y.\ abs\cdot(rep\cdot y) = y$
and *f*: *distributive-defl f*
shows *distributive-defl* (*abs oo f oo rep*)
proof (*rule distributive-deflI*)
have *iso*: *iso abs rep* **by** (*rule iso.intro*[*OF iso-abs iso-rep*])
interpret *iso abs rep* **by** (*fact iso*)
show *deflation* (*abs oo f oo rep*)
by (*rule deflation-abs-rep*[*OF abs-iso rep-iso distributive-defl.deflation*[*OF f*]])
fix *a b c* :: *'t* **assume** *ab*: $a \Uparrow b$
let *?c* = (*abs oo f oo rep*)·*c*
have *rc*: *rep*·*?c* = *f*·(*rep*·*c*) **by** *simp*
have *cab*: $?c \sqcap a \Uparrow ?c \sqcap b$ **by** (*rule joinableI*[*OF meet-below1 meet-below1*])
have *rep*·(*?c* ⊓ (*a* ⊔ *b*)) = *f*·(*rep*·*c*) ⊓ (*rep*·*a* ⊔ *rep*·*b*)
by (*simp only*: *iso-rep-meet*[*OF iso*] *iso-rep-join*[*OF iso ab*] *rc*)
also have … = *f*·(*rep*·*c*) ⊓ *rep*·*a* ⊔ *f*·(*rep*·*c*) ⊓ *rep*·*b*
using *ab* **by** (*simp add*: *distributive-defl.distrib-image*[*OF f*] *iso-rep-joinable-iff*[*OF iso*])
also have … = *rep*·(*?c* ⊓ *a* ⊔ *?c* ⊓ *b*)
by (*simp only*: *iso-rep-meet*[*OF iso*] *iso-rep-join*[*OF iso cab*] *rc*)
finally show *?c* ⊓ (*a* ⊔ *b*) = *?c* ⊓ *a* ⊔ *?c* ⊓ *b* **by** (*simp only*: *rep-eq*)
qed

lemma *distributive-defl-u-map*:
assumes *f*: *distributive-defl* (*f* :: *'a*::{*bifinite, bounded-complete*} → *'a*)
shows *distributive-defl* (*u-map*·*f*)
proof (*rule distributive-deflI*)

show *deflation* (*u-map*·*f*) **by** (*rule deflation-u-map*[*OF distributive-defl.deflation*[*OF f*]])
fix *a b c* :: '*a u* **assume** *ab*: *a* ⇑ *b*
show *u-map*·*f*·*c* ⊓ (*a* ⊔ *b*) = *u-map*·*f*·*c* ⊓ *a* ⊔ *u-map*·*f*·*c* ⊓ *b*
proof (*cases a*; *cases b*; *cases c*)
fix *a*′ *b*′ *c*′ **assume** ∗: *a* = *up*·*a*′ *b* = *up*·*b*′ *c* = *up*·*c*′
with *ab* **have** *a*′*b*′: *a*′ ⇑ *b*′ **by** *simp*
have *f*·*c*′ ⊓ *a*′ ⇑ *f*·*c*′ ⊓ *b*′ **by** (*rule joinableI*[*OF meet-below1 meet-below1*])
with ∗ *a*′*b*′ **show** *?thesis* **by** (*simp add*: *up-join distributive-defl.distrib-image*[*OF f*])
qed *simp-all*
qed

lemma *distributive-defl-ssum-map*:
assumes *f*: *distributive-defl* (*f* :: '*a*::{*bifinite*, *bounded-complete*} → '*a*)
and *g*: *distributive-defl* (*g* :: '*b*::{*bifinite*, *bounded-complete*} → '*b*)
shows *distributive-defl* (*ssum-map*·*f*·*g*)
proof (*rule distributive-deflI*)
show *deflation* (*ssum-map*·*f*·*g*)
by (*rule deflation-ssum-map*[*OF distributive-defl.deflation*[*OF f*] *distributive-defl.deflation*[*OF g*]])
fix *a b c* :: '*a* ⊕ '*b* **assume** *ab*: *a* ⇑ *b*
have *ml*: *x* ⊓ *x*′ ⇑ *x* ⊓ *x*″ **for** *x x*′ *x*″ :: '*a* **by** (*rule joinableI*[*OF meet-below1 meet-below1*])
have *mr*: *y* ⊓ *y*′ ⇑ *y* ⊓ *y*″ **for** *y y*′ *y*″ :: '*b* **by** (*rule joinableI*[*OF meet-below1 meet-below1*])
show *ssum-map*·*f*·*g*·*c* ⊓ (*a* ⊔ *b*) = *ssum-map*·*f*·*g*·*c* ⊓ *a* ⊔ *ssum-map*·*f*·*g*·*c* ⊓ *b*
using *ab* **by** (*cases a*; *cases b*; *cases c*)
(*simp-all add*: *sinl-join sinr-join distributive-defl.distrib-image*[*OF f*] *distributive-defl.distrib-image*[*OF g*] *ml mr*)
qed

Separation of both components keeps the smash product distributive, as for the class instance: joinable nonbottom pairs meet a third pair to ⊥ together.

lemma *distributive-defl-sprod-map*:
assumes *f*: *distributive-defl* (*f* :: '*a*::{*bifinite*, *bounded-complete*, *filter-sum*} → '*a*)
and *g*: *distributive-defl* (*g* :: '*b*::{*bifinite*, *bounded-complete*, *filter-sum*} → '*b*)
shows *distributive-defl* (*sprod-map*·*f*·*g*)
proof (*rule distributive-deflI*)
show *deflation* (*sprod-map*·*f*·*g*)
by (*rule deflation-sprod-map*[*OF distributive-defl.deflation*[*OF f*] *distributive-defl.deflation*[*OF g*]])
fix *a b c* :: '*a* ⊗ '*b* **assume** *a* ⇑ *b*
let *?x* = *sprod-map*·*f*·*g*·*c*
show *?x* ⊓ (*a* ⊔ *b*) = *?x* ⊓ *a* ⊔ *?x* ⊓ *b*
proof (*cases a* = ⊥ ∨ *b* = ⊥ ∨ *?x* = ⊥)
case *True* **then show** *?thesis* **by** (*elim disjE*) *simp-all*
next

```
    case False
    then obtain a1 a2 b1 b2 c1 c2 where
      ab: a = (:a1, a2:) b = (:b1, b2:) and x: ?x = (:f·c1, g·c2:) and
      nb: a1 ≠ ⊥ a2 ≠ ⊥ b1 ≠ ⊥ b2 ≠ ⊥ f·c1 ≠ ⊥ g·c2 ≠ ⊥
      by (cases a; cases b; cases c) auto
   from ‹a ⇑ b› have j: a1 ⇑ b1 a2 ⇑ b2 by (simp-all add: ab nb spair-joinable-iff)
    have cong: f·c1 ⊓ a1 = ⊥ ⟷ f·c1 ⊓ b1 = ⊥ g·c2 ⊓ a2 = ⊥ ⟷ g·c2 ⊓ b2
= ⊥
     using filter-sum-meet-bottom-cong[OF j(1) nb(1,3)] filter-sum-meet-bottom-cong[OF
j(2) nb(2,4)]
       filter-sum-meet-bottom-cong[OF joinable-sym[THEN iffD1, OF j(1)] nb(3,1)]
       filter-sum-meet-bottom-cong[OF joinable-sym[THEN iffD1, OF j(2)] nb(4,2)]
      by blast+
    have mj: x ⊓ y ⇑ x ⊓ z for x y z :: 'c::bounded-complete
      by (rule joinableI[OF meet-below1 meet-below1])
    have lhs: ?x ⊓ (a ⊔ b) = (:f·c1 ⊓ a1 ⊔ f·c1 ⊓ b1, g·c2 ⊓ a2 ⊔ g·c2 ⊓ b2:)
      by (simp only: x ab spair-join[OF nb(1,2,3,4) j] spair-meet
        distributive-defl.distrib-image[OF f j(1)] distributive-defl.distrib-image[OF g
j(2)])
    have rhs: ?x ⊓ a ⊔ ?x ⊓ b = (:f·c1 ⊓ a1, g·c2 ⊓ a2:) ⊔ (:f·c1 ⊓ b1, g·c2 ⊓
b2:)
      by (simp only: x ab spair-meet)
    show ?thesis
    proof (cases f·c1 ⊓ a1 = ⊥ ∨ g·c2 ⊓ a2 = ⊥)
      case True
      with cong show ?thesis unfolding lhs rhs by auto
    next
      case False
      with cong have f·c1 ⊓ a1 ≠ ⊥ g·c2 ⊓ a2 ≠ ⊥ f·c1 ⊓ b1 ≠ ⊥ g·c2 ⊓ b2 ≠
⊥ by simp-all
      then show ?thesis unfolding lhs rhs by (simp add: spair-join mj)
    qed
  qed
qed

end
theory Domain-Order-Derive
  imports Domain-Order HOL−Eisbach.Eisbach
begin
```

13 Deriving order-class instances of domain-package types

Proof methods packaging the derivation of `Domain_Order.thy`: for a type T defined by the domain package, a map rule $P\ v \Longrightarrow P\ (T\text{-}map{\cdot}v)$ (one premise per type parameter) is proved by fixed-point induction on the map functional, from the package's *T-map-def* and *T.iso-rews*; the instance then

follows at $v = ID$ from *T-map-ID*. The rules for the combinators are collected in named theorem lists, so map rules of earlier types can be registered for nested recursion.

13.1 Fixed-point induction on a map definition

The package defines $T\text{-}map \equiv \Lambda\ v_1\ \ldots\ v_n.\ \mu\ f.\ G\ v_1\ \ldots\ v_n\ f$. One beta step turns an equation $M = (\Lambda\ v.\ B\ v)$ into $M{\cdot}v = B\ v$, again an equation of the same shape with one Λ fewer; its continuity side condition is a single goal for *cont2cont*. After n steps the equation reads $T\text{-}map{\cdot}v_1{\cdot}\ldots{\cdot}v_n = \mu\ f.\ G\ f$, with the parameters free in G, and the induction rule below needs nothing more. The equation is its first premise, for *erule*; the others come easiest first, ending with the step.

lemma *map-beta-step*: $M = (\Lambda\ v.\ B\ v) \Longrightarrow cont\ B \Longrightarrow M{\cdot}v = B\ v$
  by *simp*

lemma *map-fix-induct*:
  fixes $G :: ('a{::}pcpo \to 'a) \Rightarrow 'a \to 'a$
  assumes *eq*: $m = fix{\cdot}(\Lambda\ f.\ G\ f)$ **and** *adm*: *adm P* **and** *bot*: $P\ \bot$
    and *cont*: *cont G*
    and *step*: $\bigwedge f.\ P\ f \Longrightarrow P\ (G\ f)$
  shows $P\ m$
  unfolding *eq* **by** (*rule fix-preserves*[*OF adm bot*], *rule preserves-LAM*[*OF cont*], *erule step*)

For a non-recursive type the functional ignores its argument, and the fixed point is the body itself; this needs neither admissibility nor the $\bot$ case. The rule only unifies when the body does not mention the bound variable.

lemma *map-fix-const*: $m = fix{\cdot}(\Lambda\ f.\ c) \Longrightarrow P\ c \Longrightarrow P\ m$
  by (*simp add*: *fix-const*)

13.2 Registries

The classes handled here (bounded completeness, separation, distributivity) are categories of domains closed under the domain constructions; the registries are prefixed accordingly. Each combinator rule mentions only its own predicate; what a property needs from the others (distributivity of $\otimes$ needs separated, bounded-complete components) is a sort constraint, so the classes are derived in dependency order.

named-theorems *domain-category-adm* ‹*admissibility of the predicates*›
  and *domain-category-bottom* ‹*the predicates at* @{*term* $\bot$}›
  and *domain-category-iso* ‹*transport along an isomorphism, premises in the shape of* ‹*T.iso-rews*››
  and *domain-category-map* ‹*the predicates through the map combinators and registered type maps*›

and *domain-category-ID* ‹*the predicates at* @{*term ID*}, *under class assumptions*›
and *domain-category-deflation* ‹*each predicate implies* @{*const deflation*}›
and *domain-category-class* ‹*class introduction from the predicate at* @{*term ID*}›

lemmas [*domain-category-adm*] = *adm-bounded-complete-defl adm-filter-sum-defl adm-separated-sum-defl adm-distributive-defl*

lemmas [*domain-category-bottom*] = *bounded-complete-defl-bottom filter-sum-defl-bottom separated-sum-defl-bottom distributive-defl-bottom*

lemmas [*domain-category-iso*] = *bounded-complete-defl-iso filter-sum-defl-iso separated-sum-defl-iso distributive-defl-iso*

lemmas [*domain-category-map*] =
bounded-complete-defl-u-map bounded-complete-defl-ssum-map bounded-complete-defl-sprod-map filter-sum-defl-u-map filter-sum-defl-ssum-map filter-sum-defl-sprod-map separated-sum-defl-u-map separated-sum-defl-ssum-map separated-sum-defl-sprod-map distributive-defl-u-map distributive-defl-ssum-map distributive-defl-sprod-map

lemmas [*domain-category-ID*] = *bounded-complete-defl-ID filter-sum-defl-ID separated-sum-defl-ID*
distributive-defl-ID deflation-ID

lemmas [*domain-category-deflation*] = *bounded-complete-defl.deflation filter-sum-defl.deflation separated-sum-defl.deflation distributive-defl.deflation*

lemmas [*domain-category-class*] = *bounded-complete-class-deflI filter-sum-class-deflI separated-sum-class-deflI distributive-cpo-class-deflI*

13.3 Methods

Each step of a method is a single rule application, mostly out of a registry, and every leaf goal is closed under *solves*, so a step can only succeed by doing what it is meant to. For a type T the per-type inputs are all generated by the domain package (*T-map-def*, *T.iso-rews*, *T-map-ID*); no constructor-level lemmas are needed. A typical use:

lemma *distributive-defl-T-map* [*domain-category-map*]:
distributive-defl v ⟹ *distributive-defl w* ⟹ *distributive-defl* (*T-map*·*v*·*w*)
by (*domain-category-map-rule map-def*: *T-map-def iso-rews*: *T.iso-rews*)

instance *T* :: ({*domain*, *distributive-cpo*}, {*domain*, *distributive-cpo*}) *distributive-cpo*
by (*domain-category-instance map-ID*: *T-map-ID*)

where the *bounded-complete* and *filter-sum* instances of T are derived before, if the recursion passes through $\otimes$.

domain-category-map-rule proceeds as follows.

- *T-map-def* is inserted as an equation and beta-reduced one parameter at a time by *map-beta-step*, each continuity side goal closed by *intro cont2cont*. Any arity works, including zero.
- *erule map-fix-induct* matches the reduced equation and the conclusion $P\ m$ at once, which also fixes the parameters left schematic by the beta steps. P comes from higher-order unification; a wrong unifier (e.g. a constant P) cannot pass *solves ‹rule domain-category-adm›*.
- For a non-recursive type, *erule map-fix-const* is tried first: it unifies only when the functional ignores its argument, and then the predicate needs no admissibility or $\bot$ rule. So non-admissible predicates (e.g. finitary levels) can still be pushed through non-recursive types.
- The iso rules take the *abs-iso*/*rep-iso* equations of *T.iso-rews* as premises, so the registries list source theorems as they are, without *OF* or *rotated*.
- *intro domain-category-map domain-category-ID* decomposes the rep type; a combinator rule whose sort constraints the component types do not meet simply does not apply. Side goals are closed by the induction hypothesis or by its deflation part; a non-recursive type without parameters leaves none.
- *declares cont2cont* lets callers add continuity rules.

  domain-category-instance rewrites *ID* to *T-map·ID·…*, finds the map rule *domain-category-map*, and closes the parameter premises from *domain-category-ID*. What the user still decides by hand: the parameter sorts of each instance, and the order of the derivations.

method *domain-category-side* =
  (*assumption* | *erule domain-category-deflation*)

Beta-reduce the inserted map definition, one parameter per step.

method *domain-category-beta* **declares** *cont2cont* =
  ((*drule map-beta-step*, *solves ‹intro cont2cont›*)+)*?*

method *domain-category-map-rule* **uses** *map-def iso-rews* **declares** *cont2cont* =
 (*insert map-def*[*THEN meta-eq-to-obj-eq*], (*domain-category-beta cont2cont*: *cont2cont*),
  (*erule map-fix-const* |
   erule map-fix-induct,
    solves ‹rule domain-category-adm›,
    solves ‹rule domain-category-bottom›,
    solves ‹intro cont2cont›),
  rule domain-category-iso, *solves ‹rule iso-rews›*, *solves ‹rule iso-rews›*,
  intro domain-category-map domain-category-ID, (*domain-category-side*+)*?*)

Instance from the registered map rule: the class property is the predicate at *ID*, and *T-map*·*ID* = *ID*.

method *domain-category-instance* **uses** *map-ID* =
(*rule domain-category-class*, *subst map-ID*[*symmetric*], *rule domain-category-map*,
((*intro domain-category-ID*)+)*?*)

13.4 Future work: a derivation command

A command *derive-order-instances T*, for one type or a mutually recursive group, would automate the remaining manual choices.

- *Package data.* Read the type constructors, *T-map-def*, *iso-rews*, *T-map-ID* and the rep-type functional from the domain package's theory data.
- *Variants.* Walk the functional and collect, per combinator position, the applicable rules: the general ones and the registered map rules of nested types. Sort constraints at closed concrete types are decided by *Sign.of-sort*; those at types mentioning parameters become requirements on the parameter sorts.
- *Order.* Derive the classes in dependency order (bounded completeness and separation before distributivity), registering each instance before the next derivation.
- *Arity.* Each variant at $v = ID$ gives a candidate arity; emit the unique minimal one (least parameter sorts), or fall back to the all-general variant with a warning.
- *Proof.* Instantiate the stored rules directly, building the closed map equation from the known parameters and abstracting P over the map term instead of relying on the unifier. Mutual recursion runs the fixed-point induction on the tuple of maps, with the predicate conjoined componentwise; the registry rules are unchanged.

  Open: there are no rules for *cfun-map* or the powerdomain maps; prior art to mirror is the domain package's own *domain-take-proofs.ML*, AFP *Deriving*, and the relator properties of Transfer/Lifting.

end
theory *MUB-Closure*
imports *Definitions Directed*
begin

14 MUB-closure (in an arbitrary partial order)

The following closure operators are generic over sort *po*, so they apply both to *'a* itself and to *'a compact-basis*.

definition *is-mub* :: *'a::po set* ⇒ *'a* ⇒ *bool*
where *is-mub T a* ≡ *T* <| *a* ∧ (∀ *b*. *T* <| *b* ⟶ *b* ⊑ *a* ⟶ *b* = *a*)

lemma *is-mubI*:
assumes *T* <| *a*
and ⋀*b*. *T* <| *b* ⟹ *b* ⊑ *a* ⟹ *b* = *a*
shows *is-mub T a*
unfolding *is-mub-def* **using** *assms* **by** *simp*

lemma *is-mubD1*: *is-mub T a* ⟹ *T* <| *a*
by (*simp add*: *is-mub-def*)

lemma *is-mubD2*: *is-mub T a* ⟹ *T* <| *b* ⟹ *b* ⊑ *a* ⟹ *b* = *a*
by (*simp add*: *is-mub-def*)

definition *mub-closed* :: *'a::po set* ⇒ *bool*
where *mub-closed T* ⟷ (∀ *S* ⊆ *T*. ∀ *s*. *finite S* ⟶ *is-mub S s* ⟶ *s* ∈ *T*)

definition *mub-closure* :: *'a::po set* ⇒ *'a set*
where *mub-closure S* = ⋂{*T*. *mub-closed T* ∧ *S* ⊆ *T*}

lemma *mub-closed-UNIV*: *mub-closed UNIV*
by (*simp add*: *mub-closed-def*)

lemma *mub-closed-Inter*:
assumes ⋀*T*. *T* ∈ *𝒯* ⟹ *mub-closed T*
shows *mub-closed* (⋂*𝒯*)
using *assms* **unfolding** *mub-closed-def* **by** *blast*

lemma *mub-closed-mub-closure*: *mub-closed* (*mub-closure S*)
unfolding *mub-closure-def* **by** (*rule mub-closed-Inter*) *blast*

lemma *mub-closure-extensive*: *S* ⊆ *mub-closure S*
unfolding *mub-closure-def* **by** *blast*

lemma *mub-closure-least*: *A* ⊆ *B* ⟹ *mub-closed B* ⟹ *mub-closure A* ⊆ *B*
unfolding *mub-closure-def* **by** *blast*

lemma *mub-closure-mono*: *A* ⊆ *B* ⟹ *mub-closure A* ⊆ *mub-closure B*
by (*meson mub-closed-mub-closure mub-closure-extensive mub-closure-least order-trans*)

lemma *mub-closure-idem*: *mub-closure* (*mub-closure S*) = *mub-closure S*
by (*meson mub-closed-mub-closure mub-closure-extensive mub-closure-least subset-antisym*)

lemma *mub-closedI*:
fixes *T*
assumes ⋀*S s*. *finite S* ⟹ *S* ⊆ *T* ⟹ *is-mub S s* ⟹ *s* ∈ *T*

shows *mub-closed T*
unfolding *mub-closed-def*
using *assms* **by** *blast*

lemma *mub-closedD*:
assumes *mub-closed T*
finite S S $\subseteq$ *T is-mub S s*
shows $s \in T$
using *assms* **unfolding** *mub-closed-def* **by** *blast*

15 Fixed sets of deflations are MUB-closed

lemma *is-ub-mono*:
assumes $\bigwedge x\ y.\ x \sqsubseteq y \Longrightarrow f\ x \sqsubseteq f\ y$
shows $A <|\ a \Longrightarrow f\ `\ A <|\ f\ a$
by (*auto intro*: *is-ubI assms dest*: *is-ubD*)

lemma *deflation-ub*:
assumes *d*: *deflation d*
and *fixed*: $A \subseteq \{x.\ d{\cdot}x = x\}$
and *ub*: $A <|\ b$
shows $A <|\ d{\cdot}b$
proof −
from *d* **interpret** *deflation d* **.**
from *monofun-cfun-arg*[*of* - - *d*] *ub* **have** $(\lambda x.\ d{\cdot}x)\ `\ A <|\ d{\cdot}b$
by (*rule is-ub-mono*)
also from *fixed* **have** $(\lambda x.\ d{\cdot}x)\ `\ A = A$ **by** *force*
finally show *?thesis* **.**
qed

lemma *deflation-preserves-mub*:
assumes *d*: *deflation d* **and** *F*: $\bigwedge f.\ f \in F \Longrightarrow d \cdot f = f$
and *mub*: *is-mub F f*
shows $d{\cdot}f = f$
proof −
from *mub* **have** $F <|\ f$ **by** (*rule is-mubD1*)
with *monofun-cfun-arg* **have** $(\lambda x.\ d{\cdot}x)\ `\ F <|\ d{\cdot}f$
by (*rule is-ub-mono*)
also have $(\lambda x.\ d{\cdot}x)\ `\ F = F$ **using** *F* **by** (*simp add*: *image-def*)
finally show *?thesis* **using** *deflation.below*[*OF d*]
by (*rule mub*[*THEN is-mubD2*])
qed

lemma *deflation-fixes-mub-closed*:
assumes *deflation d*
shows *mub-closed* $\{x.\ d{\cdot}x = x\}$
using *deflation-preserves-mub*[*OF assms*]
by (*blast intro*!: *mub-closedI*)

lemma *deflation-mub-closure-fixed*:
assumes *deflation d* **and** $S \subseteq \{x.\ d{\cdot}x = x\}$
shows *mub-closure* $S \subseteq \{x.\ d{\cdot}x = x\}$
by (*rule mub-closure-least*[*OF assms*(*2*) *deflation-fixes-mub-closed*[*OF assms*(*1*)]])

16 Finiteness of MUB-closures in bifinite domains

lemma *finite-compacts-fixed*:
fixes $S :: 'a$::*bifinite set*
assumes *finS*: *finite S* **and** *cpt*: $\bigwedge x.\ x \in S \Longrightarrow$ *compact x*
obtains *d* **where** *finite-deflation d* **and** $S \subseteq \{x.\ d{\cdot}x = x\}$
proof −
from *bifinite* **obtain** *approx* :: *nat* ⇒ $'a \to 'a$ **where** *approx-chain approx* **..**
then interpret *approx-chain approx* **.**
have *idem-mono*: *approx j*·*x* = *x* **if** *ij*: $i \le j$ **and** *fx*: *approx i*·*x* = *x* **for** *i j x*
proof −
have *approx i*·*x* ⊑ *approx j*·*x*
using *ij* **by** (*rule chain-approx*[*THEN chain-mono*, *THEN monofun-cfun-fun*])
with *fx* **have** *x* ⊑ *approx j*·*x* **by** *simp*
with *approx-below* **show** *?thesis* **by** (*metis below-antisym*)
qed
have *ex-i*: ∃ *i*. *approx i*·*x* = *x* **if** $x \in S$ **for** *x*
using *compact-eq-approx*[*OF cpt*[*OF that*]] **.**
obtain *idx* **where** *idx*: $\bigwedge x.\ x \in S \Longrightarrow$ *approx* (*idx x*)·*x* = *x*
using *ex-i* **by** *metis*
define *k* **where** *k* = *Max* (*insert 0* (*idx* ‘ *S*))
have *finI*: *finite* (*insert 0* (*idx* ‘ *S*)) **using** *finS* **by** *simp*
have *k-ge*: *idx x* ≤ *k* **if** $x \in S$ **for** *x*
proof −
have *idx x* ∈ *insert 0* (*idx* ‘ *S*) **using** *that* **by** *blast*
with *finI* **show** *?thesis* **unfolding** *k-def* **by** (*simp add*: *Max-ge*)
qed
have *fixes-k*: *approx k*·*x* = *x* **if** $x \in S$ **for** *x*
using *idem-mono*[*OF k-ge*[*OF that*] *idx*[*OF that*]] **.**
have *finite-deflation* (*approx k*) **by** (*rule finite-deflation-approx*)
moreover have $S \subseteq \{x.\ approx\ k{\cdot}x = x\}$ **using** *fixes-k* **by** *blast*
ultimately show *thesis* **using** *that* **by** *blast*
qed

lemma *finite-mub-closure*:
fixes $S :: 'a$::*bifinite set*
assumes *finS*: *finite S* **and** *cpt*: $\bigwedge x.\ x \in S \Longrightarrow$ *compact x*
shows *finite* (*mub-closure S*) **and** $\bigwedge x.\ x \in$ *mub-closure S* $\Longrightarrow$ *compact x*
proof −
obtain *d* **where** *d*: *finite-deflation d* **and** *Ssub*: $S \subseteq \{x.\ d{\cdot}x = x\}$
by (*rule finite-compacts-fixed*[*OF finS cpt*])
from *d* **have** *defl*: *deflation d* **by** (*rule finite-deflation-imp-deflation*)
have *clo-sub*: *mub-closure S* $\subseteq \{x.\ d{\cdot}x = x\}$
by (*rule deflation-mub-closure-fixed*[*OF defl Ssub*])

```
  have finite {x. d·x = x} using d by (rule finite-deflation.finite-fixes)
  with clo-sub show finite (mub-closure S) by (rule finite-subset)
  show compact x if x ∈ mub-closure S for x
  proof −
    from that clo-sub have d·x = x by blast
    then show compact x using finite-deflation.compact[OF d] by metis
  qed
qed
```

Lifting the bounding results from $'a$ to $'a$ *compact-basis*.

```
lemma compact-mub-bound-below:
  fixes A :: 'a::bifinite set
  assumes fin: finite A and compact: ⋀a. a ∈ A ⟹ compact a
    and ub: A <| b
  obtains a where is-mub A a compact a a ⊑ b
proof −
  from fin compact obtain d where d: finite-deflation d
    and sub: A ⊆ {x. d·x = x}
    using finite-compacts-fixed[of A]
    by force
  from d interpret d: finite-deflation d .
  from d.deflation-axioms sub ub have ub-db: A <| d·b
    by (rule deflation-ub)
  define bounds where bounds = {x. d·x = x ∧ A <| x ∧ x ⊑ b}
  from bounds-def d.finite-fixes have finite bounds by auto
  moreover from ub-db d.idem d.below[of b] have bounds ≠ {}
    unfolding bounds-def by blast
  ultimately have ∃ m ∈ bounds. ∀ b ∈ bounds. b ⊑ m ⟶ m = b
    by (rule order.finite-has-minimal[OF below-order])
  then obtain m where 1: m ∈ bounds
    and 2: ∀ b ∈ bounds. b ⊑ m ⟶ m = b..
  from 1 have fixm: d·m = m and ubm: A <| m and mleb: m ⊑ b
    unfolding bounds-def by auto

  from fixm have p2: compact m using d.compact[of m]
    by simp
  have p1: is-mub A m
  proof (rule is-mubI)
    from 1 show A <| m by (simp add: bounds-def)
  next
    fix c assume A <| c c ⊑ m
    from ‹c ⊑ m› fixm monofun-cfun-arg[of c m d]
    have dclem: d·c ⊑ m by simp
    with mleb have d·c ⊑ b by (rule below-trans[rotated])
    moreover from d.deflation-axioms sub ‹A <| c› have A <| d·c
      by (rule deflation-ub)
    ultimately have d·c ∈ bounds unfolding bounds-def using d.idem
      by auto
    with dclem 2 have m = d·c by fast
```

```
    with d.below[of c] ‹c ⊑ m› show c = m by (metis below-antisym)
  qed
  from p1 p2 mleb show thesis by (rule that)
qed

lemma Rep-compact-basis-ub:
Rep-compact-basis ‘ A <| Rep-compact-basis b ⟷ A <| b
  unfolding is-ub-def compact-le-def by auto

lemma mub-compact-basis-bwd:
  is-mub (Rep-compact-basis ‘ A) (Rep-compact-basis b) ⟹ is-mub A b
  unfolding is-mub-def Rep-compact-basis-ub[symmetric] compact-le-def
  using Rep-compact-basis-inject by blast

lemma compact-mub-basis-bound-below:
  fixes A :: 'a::bifinite compact-basis set
  assumes fin: finite A
    and ub: Rep-compact-basis ‘ A <| b
  obtains a where is-mub A a Rep-compact-basis a ⊑ b
proof −
  from assms compact-mub-bound-below[of Rep-compact-basis ‘ A b]
  obtain c where compact c
      and mub2: is-mub (Rep-compact-basis ‘ A) c
      and cleb: c ⊑ b
    by (metis Rep-compact-basis' finite-imageI imageE)
  let ?c = Abs-compact-basis c
  from ‹compact c› have ∗: Rep-compact-basis ?c = c by simp
  with mub2 mub-compact-basis-bwd have is-mub A ?c by metis
  moreover from ∗ cleb have Rep-compact-basis ?c ⊑ b by simp
  ultimately show ?thesis by (rule that)
qed

lemma mub-compact-basis-fwd:
  fixes A :: 'a::bifinite compact-basis set and a
  assumes fin: finite A and mub: is-mub A a
  shows is-mub (Rep-compact-basis ‘ A) (Rep-compact-basis a)
proof (rule is-mubI)
  show (Rep-compact-basis ‘ A) <| Rep-compact-basis a
    using mub[THEN is-mubD1] is-ub-mono compact-le-def by metis
next
  fix b assume b: Rep-compact-basis ‘ A <| b
  with fin obtain c where
     mub2: is-mub A c
    and cleb: Rep-compact-basis c ⊑ b
    by (rule compact-mub-basis-bound-below)
  assume blea: b ⊑ Rep-compact-basis a
  with cleb compact-le-def below-trans have clea: c ⊑ a by metis
  from mub2[THEN is-mubD1] have A <| c .
  with mub[THEN is-mubD2] clea
```

```
  have c = a by fast
  with cleb blea show b = Rep-compact-basis a by (simp add: po-eq-conv)
qed

lemma finite-mub-closure-cb:
  fixes S :: 'a::bifinite compact-basis set
  assumes finS: finite S
  shows finite (mub-closure S)
proof −
  let ?R = Rep-compact-basis
  have inj: inj ?R
    by (metis compact-basis.Rep-compact-basis-inject injI)
  have cpt: compact x if x ∈ ?R ‘ S for x
    using that using Rep-compact-basis' by auto
  have finRS: finite (?R ‘ S) using finS by simp
  define M where M = mub-closure (?R ‘ S)
  from finite-mub-closure[OF finRS cpt]
  have finM: finite M and compactM: ⋀x. x ∈ M ⟹ compact x
    unfolding M-def by iprover+
  let ?T = {a. ?R a ∈ M}
  have Ssub: S ⊆ ?T
    using mub-closure-extensive[of ?R ‘ S] unfolding M-def by auto
  have Tclosed: mub-closed ?T
  proof (intro mub-closedI)
    fix U u assume U ⊆ ?T
    then have 1: ?R ‘ U ⊆ M by auto
    assume fin: finite U and is-mub U u
    then have 2: is-mub (?R ‘ U) (?R u)
      by (rule mub-compact-basis-fwd)
    from M-def have mub-closed M by (simp add: mub-closed-mub-closure)
    from this fin 1 2 have ?R u ∈ M using mub-closedD by blast
    thus u ∈ ?T by simp
  qed
  with Ssub have mub-closure S ⊆ ?T by (rule mub-closure-least)
  hence ?R ‘ mub-closure S ⊆ M by auto
 with finM have fin-img: finite (?R ‘ mub-closure S) by (rule finite-subset[rotated])
  have inj-on ?R (mub-closure S) using inj by (meson inj-eq inj-onI)
  with fin-img show finite (mub-closure S) by (rule finite-imageD)
qed
```

Proposition 2.2.16 from Abramsky and Jung's Domain Theory - compactness for directed sets

```
lemma directed-compactD:
  fixes x :: 'a::bifinite and D :: 'a set
  assumes compact x directed D D ≠ {}
    x ⊑ lub D
  shows ∃ y ∈ D. x ⊑ y
  using compactD[THEN directed-admD, of x D] assms
  by blast
```

17 The deflation determined by a MUB-closed set of compacts

For a MUB-closed set S of compact basis elements (containing the basis element of $\bot$), the assignment $\lambda a.\ lub\ \{Rep\text{-}compact\text{-}basis\ b\ |b.\ b \in S \wedge b \sqsubseteq a\}$ is monotone, and its *compact-basis.extension* is a deflation whose compact fixed points are exactly S. Working in a locale à la *stable-subdomain*.

locale *mub-closed-basis* =
  fixes S :: *'a*::*bifinite compact-basis set*
  assumes *mub-closed-S*: *mub-closed S*
begin

lemma *bot-in-S*: *compact-bot* $\in$ *S*
proof −
  have $x \sqsubseteq$ *compact-bot* $\Longrightarrow x =$ *compact-bot*
    for x :: *'a compact-basis*
    using *compact-bot-minimal below-antisym* **by** *blast*
  then have *is-mub* ({}::*'a compact-basis set*) *compact-bot*
    unfolding *is-mub-def* **by** *auto*
  with *mub-closedD*[*OF mub-closed-S*] **show** *?thesis* **by** *blast*
qed

definition *below-set* :: *'a* ⇒ *'a compact-basis set*
  where *below-set a* = {$x \in S$. *Rep-compact-basis* $x \sqsubseteq a$}

definition g :: *'a* ⇒ *'a*
  where *g a* = *lub* (*Rep-compact-basis* ‘ *below-set a*)

lemma *below-set-nonempty*: *below-set a* ≠ {}
  using *bot-in-S* **unfolding** *below-set-def* **by** *auto*

lemma *below-set-directed*: *directed* (*below-set a*)
proof (*rule directedI*)
  fix *x y* **assume** *x* ∈ *below-set a y* ∈ *below-set a*
  then have *S*: *x* ∈ *S y* ∈ *S*
    and *a*: *Rep-compact-basis x* ⊑ *a Rep-compact-basis y* ⊑ *a*
    unfolding *below-set-def* **by** *auto*
  from *a* **have** *Rep-compact-basis* ‘ {*x*, *y*} <| *a*
    by (*auto intro*: *is-ubI*)
  with *compact-mub-basis-bound-below*[*of* {*x*, *y*} *a*]
  obtain *z* **where** *mub*: *is-mub* {*x*, *y*} *z* **and** *az*: *Rep-compact-basis z* ⊑ *a*
    by *blast*
  from *S* **have** {*x*, *y*} ⊆ *S* **by** *simp*
  with *mub-closed-S*[*THEN mub-closedD*, *of* {*x*, *y*}] *mub*
  have *z* ∈ *S* **by** *fast*
  with *az* **have** *z* ∈ *below-set a* **unfolding** *below-set-def* **by** *simp*
  moreover from *mub*[*THEN is-mubD1*, *THEN is-ubD*]
  have *x* ⊑ *z* ∧ *y* ⊑ *z* **by** *simp*

ultimately show $\exists z \in$ *below-set a*. $x \sqsubseteq z \wedge y \sqsubseteq z$**..**
qed

corollary *below-set-nonempty′*:
Rep-compact-basis ‘ below-set a $\neq \{\}$
using *below-set-nonempty* **by** *simp*

corollary *below-set-directed′*:
directed (*Rep-compact-basis ‘ below-set a*)
using *compact-le-def below-set-directed*
directed-mono-image[*of Rep-compact-basis below-set a*]
by *metis*

lemma *g-lub*: *Rep-compact-basis ‘ below-set a <<| g a*
unfolding *g-def* **using** *below-set-directed′* **by** (*rule directed-lub*)

lemma *g-approximant*:
$x \in$ *approximants* (*g a*) $\longleftrightarrow$ ($\exists b \in$ *below-set a*. $x \sqsubseteq b$)
unfolding *approximants-def*
proof (*safe*)
fix *x* **assume** *Rep-compact-basis x* $\sqsubseteq$ *g a*
with *directed-compactD*[*OF Rep-compact-basis′ below-set-directed′ below-set-nonempty′*]
have $\exists y \in$ *Rep-compact-basis ‘ below-set a*. *Rep-compact-basis x* $\sqsubseteq$ *y*
unfolding *g-def* **by** *auto*
then show $\exists b \in$*below-set a*. $x \sqsubseteq b$ **by** *fastforce*
next
fix *x b* **assume** $b \in$ *below-set a*
with *directed-lub*[*OF below-set-directed′*, *THEN is-lubD1*, *THEN is-ubD*]
have *Rep-compact-basis b* $\sqsubseteq$ *g a* **unfolding** *g-def* **by** *blast*
moreover assume $x \sqsubseteq b$
ultimately show *Rep-compact-basis x* $\sqsubseteq$ *g a*
unfolding *compact-le-def* **by** (*rule below-trans*[*rotated*])
qed

definition *down-closure* (*A*::′*b*::*po set*) = {*x*. $\exists y \in A$. $x \sqsubseteq y$}

lemma *below-set-alt*: *below-set a* = *S* $\cap$ *approximants a*
unfolding *below-set-def approximants-def* **by** *blast*

corollary *approximants-g*:
approximants (*g a*) = *down-closure* (*below-set a*)
unfolding *down-closure-def* **using** *g-approximant*
by *blast*

lemma *below-set-chain*:
fixes *Y* :: *nat* $\Rightarrow$ ′*a*
assumes *chain Y*
shows *below-set* ($\bigsqcup i$. *Y i*) = ($\bigcup i$. *below-set* (*Y i*))
unfolding *below-set-alt compact-basis.rep-lub*[*OF assms*]

by *simp*

lemma *down-closure-UN*: *down-closure* $(\bigcup i.\ P\ i) = (\bigcup i.\ \textit{down-closure}\ (P\ i))$
unfolding *down-closure-def* **by** *blast*

lemma *below-set-mono*: $a \sqsubseteq b \Longrightarrow \textit{below-set}\ a \subseteq \textit{below-set}\ b$
unfolding *below-set-def* **using** *below-trans* **by** *blast*

lemma *g-mono*: *monofun g*
proof (*rule monofunI*)
fix *a b* :: *'a*
assume *ab*: $a \sqsubseteq b$
show *g a* $\sqsubseteq$ *g b*
using *below-set-mono*[*OF ab*]
by (*fast intro*: *lub-subset-mono*[*OF g-lub g-lub*])
qed

lemma *g-cont*: *cont g*
proof (*rule contI2*, *rule g-mono*)
fix *Y* :: *nat* $\Rightarrow$ *'a*
assume *Y*: *chain Y*
and *gY*: *chain* $(\lambda i.\ g\ (Y\ i))$
show *g* $(\bigsqcup i.\ Y\ i) \sqsubseteq (\bigsqcup i.\ g\ (Y\ i))$
apply (*rule compact-basis.belowI*)
unfolding *approximants-g below-set-chain*[*OF Y*]
compact-basis.rep-lub[*OF gY*] *down-closure-UN* **..**
qed

definition *d* :: *'a* $\rightarrow$ *'a*
where $d \equiv \Lambda\ x.\ g\ x$

lemma *d-lub*: *Rep-compact-basis* ‘ *below-set a* $<<|$ *d*·*a*
unfolding *d-def*
by (*simp add*: *g-lub g-cont*)

lemma *d-eq-lub*: *d*·*a* = *lub* (*Rep-compact-basis* ‘ *below-set a*)
using *d-lub*[*THEN lub-eqI*] **..**

lemma *below-set-ub*: *Rep-compact-basis* ‘ *below-set a* $<|$ *a*
by (*simp add*: *below-set-def is-ub-def*)

corollary *d-below*: *d*·*a* $\sqsubseteq$ *a*
using *d-lub*[*THEN is-lubD2*, *OF below-set-ub*] **.**

lemma *d-fixed*: *d*·(*Rep-compact-basis a*) = *Rep-compact-basis a* **if** $a \in S$
proof (*intro below-antisym d-below*)
let *?a* = *Rep-compact-basis a*
from *that* **have** *a* $\in$ *below-set ?a* **by** (*simp add*: *below-set-def*)
then have *?a* $\in$ *Rep-compact-basis* ‘ *below-set ?a* **by** *simp*

with *d-lub*[*THEN is-lubD1*, *THEN is-ubD*]
show *?a* ⊑ *d*·*?a* **.**
qed

lemma *approximants-closed-down*: *x* ∈ *approximants a* ⟹ *y* ⊑ *x* ⟹ *y* ∈ *approximants a*
unfolding *approximants-def compact-le-def* **by** (*auto dest*: *below-trans*)

lemma *below-set-idem*: *below-set* (*g a*) = *below-set a*
unfolding *below-set-alt approximants-g down-closure-def*
using *approximants-closed-down*
by *auto*

corollary *g-idem*: *g* (*g a*) = *g a*
using *below-set-idem g-def* **by** *auto*

lemma *deflation-d*: *deflation d*
proof (*rule deflation.intro*)
fix *x* **show** *d*·(*d*·*x*) = *d*·*x* **by** (*simp add*: *g-idem g-cont d-def*)
next
fix *x* **show** *d*·*x* ⊑ *x* **by** (*rule d-below*)
qed

lemma *d-fixes*: *d*·(*Rep-compact-basis a*) = *Rep-compact-basis a* ⟷ *a* ∈ *S*
proof
assume *a* ∈ *S*
thus *d*·(*Rep-compact-basis a*) = *Rep-compact-basis a* **by** (*rule d-fixed*)
next
let *?a* = *Rep-compact-basis a*
assume *fx*: *d*·*?a* = *?a*
then have *?a* ⊑ *g ?a* **by** (*simp add*: *d-def g-cont*)
then have *a* ∈ *approximants* (*g ?a*) **by** (*simp add*: *approximants-def*)
then have ∃ *y* ∈ *below-set ?a*. *a* ⊑ *y*
by (*simp add*: *g-approximant*)
then obtain *y* **where** *y*: *y* ∈ *below-set ?a* **and** *le*: *a* ⊑ *y* **by** *blast*
from *y* **have** *y*: *y* ∈ *S* **and** *le2*: *y* ⊑ *a*
unfolding *below-set-def compact-le-def* **by** *auto*
from *le le2* **have** *a* = *y* **by** (*rule below-antisym*)
with *y* **show** *a* ∈ *S* **by** *simp*
qed

When S is finite the extension is a *finite* deflation: its values, and hence its fixed points, all lie in the finite set *Rep-compact-basis ‘ S*.

lemma *d-in-RepS*:
assumes *finite S*
shows *d*·*a* ∈ *Rep-compact-basis ‘ S*
proof −
have *sub*: *Rep-compact-basis ‘ below-set a* ⊆ *Rep-compact-basis ‘ S*
unfolding *below-set-def* **by** *blast*

```
  hence fin: finite (Rep-compact-basis ‘ below-set a) using assms
    by (meson finite-imageI finite-subset)
  have lub (Rep-compact-basis ‘ below-set a) ∈  Rep-compact-basis ‘ below-set a
   by (rule finite-directed-lub-mem[OF below-set-directed′ fin below-set-nonempty′])
  with sub show ?thesis using d-eq-lub by auto
qed

lemma finite-deflation-d:
  assumes finite S
  shows finite-deflation d
proof (rule finite-deflation-intro[OF deflation-d])
  have {x. d·x = x} ⊆ Rep-compact-basis ‘ S
  proof
    fix x assume x ∈ {x. d·x = x}
    hence x = d·x by simp
    also have d·x ∈ Rep-compact-basis ‘ S by (rule d-in-RepS[OF assms])
    finally show x ∈ Rep-compact-basis ‘ S .
  qed
  thus finite {x. d·x = x} using assms by (meson finite-imageI finite-subset)
qed
end

end
theory Deflations
  imports MUB-Closure
begin
```

18 The lattice of finite deflations

A deflation is below another exactly when its fixed-point set is contained in the other's. This is the order-theoretic content behind *cast* being an order-embedding, phrased for raw deflations.

```
lemma deflation-belowI-fixes:
  assumes d: deflation d and e: deflation e
    and fx: ⋀x. e·x = x ⟹ d·x = x
  shows e ⊑ d
  using e fx by (rule deflation.belowI)
```

The compact fixed points of a deflation, as a set of basis elements. This set is MUB-closed (*deflation-fixes-mub-closed*) and contains $\bot$, so it satisfies the assumptions of the *mub-closed-basis* locale.

```
definition dfixes :: ('a::bifinite → 'a) ⇒ 'a compact-basis set
  where dfixes d = {a. d·(Rep-compact-basis a) = Rep-compact-basis a}

lemma mub-closed-dfixes:
  assumes deflation d
  shows mub-closed (dfixes d)
```

```
proof (intro mub-closedI)
  let ?R = Rep-compact-basis
  fix S s assume fin: finite S and S ⊆ dfixes d is-mub S s
  from ‹finite S› ‹is-mub S s› have is-mub (?R ‘ S) (?R s)
    by (rule mub-compact-basis-fwd)
  moreover from ‹S ⊆ dfixes d› have ?R ‘ S ⊆ {x. d·x = x}
    unfolding dfixes-def by auto
  ultimately have ?R s ∈ {x. d·x = x}
    using deflation-fixes-mub-closed[OF assms, THEN mub-closedD]
      fin[THEN finite-imageI] by fast
  then show s ∈ dfixes d by (simp add: dfixes-def)
qed


lemma mub-closed-basis-dfixes:
  assumes deflation d
  shows mub-closed-basis (dfixes d)
  unfolding mub-closed-basis-def
  using mub-closed-dfixes[OF assms] by blast

lemma finite-dfixes:
  assumes finite-deflation d
  shows finite (dfixes d)
proof −
  have inj-on Rep-compact-basis (dfixes d)
    by (meson compact-basis.Rep-compact-basis-inject inj-onI)
  moreover have Rep-compact-basis ‘ dfixes d ⊆ {x. d·x = x}
    unfolding dfixes-def by auto
 moreover have finite {x. d·x = x} using assms by (rule finite-deflation.finite-fixes)
  ultimately show ?thesis by (meson finite-imageD finite-subset)
qed
```

Reconstruction: a finite deflation is recovered from its own compact fixed points by the *mub-closed-basis* construction. Thus *dfixes* and *defl-of-set* are mutually inverse between finite deflations and finite MUB-closed sets containing $\bot$.

```
lemma defl-of-dfixes:
  assumes finite-deflation d
  shows mub-closed-basis.d (dfixes d) = d
proof −
  from assms have defld: deflation d by (rule finite-deflation-imp-deflation)
  interpret L: mub-closed-basis dfixes d
    by (rule mub-closed-basis-dfixes[OF defld])
  have fin: finite (dfixes d) using assms by (rule finite-dfixes)
  have defle: deflation L.d by (rule L.deflation-d)
  have fde: finite-deflation L.d by (rule L.finite-deflation-d[OF fin])
  have eq-fixes: L.d·x = x ⟷ d·x = x for x
  proof
    assume d·x = x
```

moreover have *compact* $(d{\cdot}x)$ **by** (*rule finite-deflation.compact*[*OF assms*])
ultimately have *compact x* **by** *simp*
then obtain *a* **where** *xa*: $x = Rep\text{-}compact\text{-}basis\ a$
using *compact-basis.compact-imp-principal* **by** *blast*
from ‹$d{\cdot}x = x$› *xa* **have** $a \in dfixes\ d$ **unfolding** *dfixes-def* **by** *simp*
thus $L.d{\cdot}x = x$ **using** *xa L.d-fixes* **by** *simp*
next
assume $L.d{\cdot}x = x$
moreover have *compact* $(L.d{\cdot}x)$ **by** (*rule finite-deflation.compact*[*OF fde*])
ultimately have *compact x* **by** *simp*
then obtain *a* **where** *xa*: $x = Rep\text{-}compact\text{-}basis\ a$
using *compact-basis.compact-imp-principal* **by** *blast*
from ‹$L.d{\cdot}x = x$› *xa* **have** $a \in dfixes\ d$ **using** *L.d-fixes* **by** *simp*
thus $d{\cdot}x = x$ **using** *xa* **unfolding** *dfixes-def* **by** *simp*
qed
show $L.d = d$
by (*rule below-antisym*; *rule deflation-belowI-fixes*) (*simp-all add*: *eq-fixes defld defle*)
qed

The MUB-closure of any set containing $\bot$ satisfies the locale assumptions, so it determines a deflation.

lemma *mub-closed-basis-mub-closure*:
shows *mub-closed-basis* (*mub-closure T*)
unfolding *mub-closed-basis-def*
using *mub-closed-mub-closure* **by** *blast*

Pairwise join of finite deflations: the deflation determined by the MUB-closure of the union of their fixed-point sets. By the lattice-embedding (fixed points ordered by inclusion) this is the least upper bound.

definition *join-fin-defl* :: $'a$::*bifinite fin-defl* ⇒ $'a$ *fin-defl* ⇒ $'a$ *fin-defl*
where *join-fin-defl A B* =
Abs-fin-defl (*mub-closed-basis.d*
(*mub-closure* (*dfixes* (*Rep-fin-defl A*) ∪ *dfixes* (*Rep-fin-defl B*))))

lemma *join-fin-defl-aux*:
fixes *A B* :: $'a$::*bifinite fin-defl*
defines *U* ≡ *mub-closure* (*dfixes* (*Rep-fin-defl A*) ∪ *dfixes* (*Rep-fin-defl B*))
shows *mub-closed-basis U* (**is** *?P1*)
and *finite U* (**is** *?P2*)
and *finite-deflation* (*mub-closed-basis.d U*) (**is** *?P3*)
and *Rep-fin-defl* (*join-fin-defl A B*) = *mub-closed-basis.d U* (**is** *?P4*)
proof −
have *finA*: *finite* (*dfixes* (*Rep-fin-defl A*))
by (*rule finite-dfixes*[*OF finite-deflation-Rep-fin-defl*])
have *finB*: *finite* (*dfixes* (*Rep-fin-defl B*))
by (*rule finite-dfixes*[*OF finite-deflation-Rep-fin-defl*])
show *P1*: *?P1* **unfolding** *U-def* **by** (*rule mub-closed-basis-mub-closure*)
show *P2*: *?P2* **unfolding** *U-def*

```
    using finite-mub-closure-cb[of dfixes (Rep-fin-defl A) ∪ dfixes (Rep-fin-defl B)]
      finA finB by simp
  show P3: ?P3 by (rule mub-closed-basis.finite-deflation-d[OF P1 P2])
  show ?P4 unfolding join-fin-defl-def U-def[symmetric]
    by (metis P3 Abs-fin-defl-inverse mem-Collect-eq)
qed

lemma dfixes-mub-closed-basis-d:
  assumes mub-closed-basis S
  shows dfixes (mub-closed-basis.d S) = S
  unfolding dfixes-def using mub-closed-basis.d-fixes[OF assms] by auto
```

The order on finite deflations is inclusion of their fixed-point sets the promised lattice embedding into the MUB-closed sets.

```
lemma fin-defl-below-dfixes:
  fixes A B :: 'a::bifinite fin-defl
  shows A ⊑ B ⟷ dfixes (Rep-fin-defl A) ⊆ dfixes (Rep-fin-defl B)
proof
  assume A ⊑ B
  thus dfixes (Rep-fin-defl A) ⊆ dfixes (Rep-fin-defl B)
    unfolding dfixes-def using fin-defl-belowD by auto
next
  assume sub: dfixes (Rep-fin-defl A) ⊆ dfixes (Rep-fin-defl B)
  show A ⊑ B
  proof (rule fin-defl-belowI)
    fix x assume Rep-fin-defl A·x = x
    moreover have compact (Rep-fin-defl A·x)
      by (rule finite-deflation.compact[OF finite-deflation-Rep-fin-defl])
    ultimately have compact x by simp
    then obtain a where xa: x = Rep-compact-basis a
      using compact-basis.compact-imp-principal by blast
    from ‹Rep-fin-defl A·x = x› xa have a ∈ dfixes (Rep-fin-defl A)
      unfolding dfixes-def by simp
    with sub have a ∈ dfixes (Rep-fin-defl B) by blast
    thus Rep-fin-defl B·x = x using xa unfolding dfixes-def by simp
  qed
qed

lemma dfixes-join-fin-defl:
  fixes A B :: 'a::bifinite fin-defl
  shows dfixes (Rep-fin-defl (join-fin-defl A B)) =
        mub-closure (dfixes (Rep-fin-defl A) ∪ dfixes (Rep-fin-defl B))
  using join-fin-defl-aux(4)[of A B]
    dfixes-mub-closed-basis-d[OF join-fin-defl-aux(1)[of A B]] by simp

lemma join-fin-defl-above1: A ⊑ join-fin-defl A B
  unfolding fin-defl-below-dfixes dfixes-join-fin-defl
  using mub-closure-extensive by blast
```

lemma *join-fin-defl-above2*: $B \sqsubseteq$ *join-fin-defl* A B
unfolding *fin-defl-below-dfixes dfixes-join-fin-defl*
using *mub-closure-extensive* **by** *blast*

lemma *join-fin-defl-least*:
assumes $A \sqsubseteq C$ **and** $B \sqsubseteq C$
shows *join-fin-defl* A $B \sqsubseteq C$
proof −
have *dfixes* (*Rep-fin-defl* A) ∪ *dfixes* (*Rep-fin-defl* B) ⊆ *dfixes* (*Rep-fin-defl* C)
using *assms* **unfolding** *fin-defl-below-dfixes* **by** *blast*
hence *mub-closure* (*dfixes* (*Rep-fin-defl* A) ∪ *dfixes* (*Rep-fin-defl* B))
⊆ *dfixes* (*Rep-fin-defl* C)
by (*rule mub-closure-least*[*OF - mub-closed-dfixes*[*OF deflation-Rep-fin-defl*]])
thus *?thesis* **unfolding** *fin-defl-below-dfixes dfixes-join-fin-defl* **.**
qed

19 Algebraic deflations are determined by their compact fixes

cast is injective but not surjective onto all deflations: only a deflation whose image is bifinite is the lub of the finite deflations below it. Still, an algebraic deflation *cast*·A is the directed lub of the finite deflations in its defining ideal, so it is pinned down by which COMPACT elements it fixes, i.e. by *dfixes* (*cast*·A).

Each finite deflation in the defining ideal of A lies below *cast*·A.

lemma *Rep-fin-defl-below-cast*:
assumes $a \in$ *Rep-defl* A
shows *Rep-fin-defl* $a \sqsubseteq$ *cast*·A
using *assms* **by** (*metis cast-defl-principal cast-below-cast defl.principal-below-iff-mem-rep*)

Larger algebraic deflations fix more compacts (this direction is just monotonicity of *cast*).

lemma *dfixes-cast-mono*:
assumes $A \sqsubseteq B$
shows *dfixes* (*cast*·A) ⊆ *dfixes* (*cast*·B)
proof
fix b **assume** $b \in$ *dfixes* (*cast*·A)
hence *fx*: *cast*·A·(*Rep-compact-basis* b) = *Rep-compact-basis* b
by (*simp add*: *dfixes-def*)
from *assms* **have** *cast*·$A \sqsubseteq$ *cast*·B **by** (*rule monofun-cfun-arg*)
hence *cast*·A·(*Rep-compact-basis* b) $\sqsubseteq$ *cast*·B·(*Rep-compact-basis* b)
by (*rule monofun-cfun-fun*)
with *fx* **have** *le*: *Rep-compact-basis* $b \sqsubseteq$ *cast*·B·(*Rep-compact-basis* b) **by** *simp*
have *cast*·B·(*Rep-compact-basis* b) = *Rep-compact-basis* b
proof (*rule below-antisym*)
show *cast*·B·(*Rep-compact-basis* b) $\sqsubseteq$ *Rep-compact-basis* b **by** (*rule cast.below*)

```
    show Rep-compact-basis b ⊑ cast·B·(Rep-compact-basis b) by (rule le)
  qed
  thus b ∈ dfixes (cast·B) by (simp add: dfixes-def)
qed
```

The converse is the substantive direction: an inclusion of compact fixes forces the deflation order. We test each finite deflation *Rep-fin-defl a* in the ideal of A against $cast{\cdot}B$ using *finite-deflation.compact-belowI*, which only inspects compact fixed points.

```
lemma defl-below-of-dfixes:
  assumes sub: dfixes (cast·A) ⊆ dfixes (cast·B)
  shows A ⊑ B
proof (unfold below-defl-def, rule subsetI)
  fix a assume aA: a ∈ Rep-defl A
  have ra: Rep-fin-defl a ⊑ cast·A using aA by (rule Rep-fin-defl-below-cast)
  have Rep-fin-defl a ⊑ cast·B
  proof (rule Rep-fin-defl.compact-belowI)
    fix x assume cx: compact x and fx: Rep-fin-defl a·x = x
    have caAx: cast·A·x = x
      using fx ra cast.belowD by fastforce
    let ?b = Abs-compact-basis x
    from cx have rb: Rep-compact-basis ?b = x by simp
     with caAx have cast·A·(Rep-compact-basis ?b) = Rep-compact-basis ?b by
simp
    hence ?b ∈ dfixes (cast·A) by (simp add: dfixes-def)
    with sub have ?b ∈ dfixes (cast·B) ..
    hence cast·B·(Rep-compact-basis ?b) = Rep-compact-basis ?b
      by (simp add: dfixes-def)
    with rb show cast·B·x = x by simp
  qed
  hence cast·(defl-principal a) ⊑ cast·B by (simp add: cast-defl-principal)
  hence defl-principal a ⊑ B by (rule cast-below-imp-below)
  thus a ∈ Rep-defl B by (simp add: defl.principal-below-iff-mem-rep)
qed
```

Hence the compact-fixes map is an order embedding of $'a\ defl$ into sets of compact basis elements, and in particular injective: an algebraic deflation is completely determined by the compacts it fixes.

```
theorem defl-below-iff-dfixes:
  A ⊑ B ⟷ dfixes (cast·A) ⊆ dfixes (cast·B)
  using dfixes-cast-mono defl-below-of-dfixes by blast

theorem defl-eq-iff-dfixes:
  A = B ⟷ dfixes (cast·A) = dfixes (cast·B)
  by (metis below-antisym defl-below-iff-dfixes set-eq-subset)
end
theory Koenigs imports Main begin
```

context
  fixes *rel* :: *'a* ⇒ *'a* ⇒ *bool* (**infix** ‹⊑› *50*)
begin

primrec *winnow* :: *nat* ⇒ (*nat* ⇒ *'a set*) ⇒ (*nat* ⇒ *'a set*)
  where *winnow-0*: *winnow 0 S* = *S*
  | *winnow-suc*: *winnow* (*Suc i*) *S j* = {*x* ∈ *winnow i S j*. ∃ *y* ∈ *winnow i S* (*Suc j*). *x* ⊑ *y*}

abbreviation (*input*) *upper-le-set* (**infix** ‹≼♯› *50*)
  where *upper-le-set X Y* ≡ ∀ *y* ∈ *Y*. ∃ *x* ∈ *X*. *x* ⊑ *y*

lemma *winnow-chain*: **assumes** ∀ *i*. *S i* ≼♯ *S* (*Suc i*)
  shows ∀ *i*. *winnow n S i* ≼♯ *winnow n S* (*Suc i*)
proof (*induction n*)
  case *0*
  then show *?case* **using** *assms* **by** *simp*
next
  case (*Suc n*)
  then show *?case* **by** *fastforce*
qed

lemma *winnow-ne-empty*: **assumes** *ne*: ∀ *i*. *S i* ≠ {} **and** *chain*: ∀ *i*. *S i* ≼♯ *S* (*Suc i*)
  shows ∀ *i*. *winnow n S i* ≠ {}
proof (*induction n*)
  case *0*
  then show *?case* **using** *ne* **by** *simp*
next
  case (*Suc n*)
  with *winnow-chain*[*of S n*, *OF chain*]
  show *?case* **by** *fastforce*
qed

lemma *winnow-descending*: *winnow* (*Suc n*) *S i* ⊆ *winnow n S i* **by** *simp*

lemma *winnow-antimono*: *antimono* (λ*n*. *winnow n S i*)
  unfolding *antimono-iff-le-Suc* **by** (*simp add*: *winnow-descending*)

definition *winnow-fix* **where** *winnow-fix S i* = (⋂ *n*. *winnow n S i*)

theorem *descending-finite-chain*:
  fixes *C* :: *nat* ⇒ *'a set*
  assumes ∀ *i*. *C* (*Suc i*) ⊆ *C i*
    and ∀ *i*. *C i* ≠ {}
    and *finite* (*C 0*)
  shows (⋂ *i*. *C i*) ≠ {}
proof
  assume (⋂ *n*. *C n*) = {}

```
  then have ∃ n. x ∉ C n if x ∈ C 0 for x by blast
  note Least = this[THEN LeastI-ex]
  have antimono C unfolding antimono-iff-le-Suc by fact
  note mono = this[THEN antimonoD]
  define I where I ≡ {LEAST n. x ∉ C n | x. x ∈ C 0}
  with ‹finite (C 0)› have finite I by simp
  then have n ≤ Max I if n ∈ I for n by (simp add: that)
  with I-def mono Least have x ∉ C (Max I) if x ∈ C 0 for x
    using that by blast
  with mono ‹∀ i. C i ≠ {}› show False by blast
qed

lemma winnow-fix:
  assumes ne: ∀ i. S i ≠ {}
    and finite: ∀ i. finite (S i)
    and chain: ∀ i. S i ≼♯ S (Suc i)
  shows winnow-fix-ne: winnow-fix S i ≠ {}
    and winnow-fix-strong-chain: ∀ x ∈ winnow-fix S i. ∃ y ∈ winnow-fix S (Suc i).
x ⊑ y
proof −
  show fix-ne: winnow-fix S i ≠ {}
    unfolding winnow-fix-def
  proof (intro allI descending-finite-chain winnow-descending)
    fix i
    show winnow n S i ≠ {} for n using winnow-ne-empty[OF ne chain] by simp
    show finite (winnow 0 S i) using finite by simp
  qed
  show ∀ x ∈ winnow-fix S i. ∃ y ∈ winnow-fix S (Suc i). x ⊑ y
  proof (intro strip)
    fix x assume x ∈ winnow-fix S i
    then have x ∈ winnow n S i for n by (simp add: winnow-fix-def)
    then have x ∈ winnow (Suc n) S i for n .
    then have 1: ∃ y ∈ winnow n S (Suc i). x ⊑ y for n by simp
    have (⋂ n. {y ∈ winnow n S (Suc i). x ⊑ y}) ≠ {}
    proof (intro descending-finite-chain allI, clarsimp)
      show {y ∈ winnow n S (Suc i). x ⊑ y} ≠ {} for n
        using 1 by blast
      show finite {y ∈ winnow 0 S (Suc i). x ⊑ y} by (simp add: finite)
    qed
    then show ∃ y ∈ winnow-fix S (Suc i). x ⊑ y
      unfolding winnow-fix-def by blast
  qed
qed

lemma winnow-fix-subset: winnow-fix S i ⊆ S i
  unfolding winnow-fix-def
  by (intro Inter-lower range-eqI[where x = 0]) simp

lemma obtain-infinite-path:
```

fixes $S :: nat \Rightarrow 'a\ set$
assumes *ne*: $\bigwedge i.\ S\ i \neq \{\}$
and *finite*: $\bigwedge i.\ finite\ (S\ i)$
and *chain*: $\bigwedge i.\ S\ i \preceq\sharp\ S\ (Suc\ i)$
obtains f **where** $\forall\, i.\ f\ i \in S\ i$ **and** $\forall\, i.\ f\ i \sqsubseteq f\ (Suc\ i)$
proof −
from *winnow-fix assms*
have *winnow-fix S i* $\neq \{\}$ **and** $\forall\, x \in$ *winnow-fix S i*. $\exists\, y \in$ *winnow-fix S* (*Suc i*). $x \sqsubseteq y$
for *i*
by *auto*
then have $\exists f.\ \forall\, i.\ f\ i \in$ *winnow-fix S i* $\wedge\ f\ i \sqsubseteq f\ (Suc\ i)$
by (*intro dependent-nat-choice*) *fast+*
with *winnow-fix-subset* **show** *thesis* **using** *that* **by** *blast*
qed

end

end
theory *LowerPD-Representation*
imports *HOLCF HOLCF.LowerPD Distributive Directed Top*

begin

definition *lower-mem* :: $'a$::*bifinite* $\Rightarrow$ ($'a$)$\flat \Rightarrow$ *bool* (**infix** ‹$\in\flat$› *50*)
where $x \in\flat\ X \longleftrightarrow \{x\}\flat \cup\flat\ X = X$

definition *lower-pd-set* :: ($'a$::*bifinite*)$\flat \Rightarrow 'a\ set$
where *lower-pd-set* $X \equiv \{x.\ x \in\flat\ X\}$

lemma *lower-mem-equiv*: $x \in\flat\ X \longleftrightarrow \{x\}\flat \sqsubseteq X$
unfolding *lower-mem-def* **by** (*simp add*: *lower-join-eq lower-pd-lattice.sup.absorb-iff2*)

lemma *lower-mem-principal*: $x \in\flat\ \{y\}\flat \longleftrightarrow x \sqsubseteq y$
unfolding *lower-mem-equiv* **by** *simp*

corollary *lower-principal-rep*: *lower-pd-set* $\{y\}\flat = \{x.\ x \sqsubseteq y\}$
unfolding *lower-mem-principal lower-pd-set-def* **..**

lemma *lower-mem-union*: $x \in\flat\ X \cup\flat\ Y \longleftrightarrow x \in\flat\ X \vee x \in\flat\ Y$
unfolding *lower-mem-equiv*
by (*rule lower-unit-below-plus-iff*)

corollary *lowerset-union*: *lower-pd-set* $(X \cup\flat\ Y)$ = *lower-pd-set* $X \cup$ *lower-pd-set* Y
unfolding *lower-mem-union lower-pd-set-def* **by** *blast*

definition *set-approximants* :: $'a$::*bifinite set* $\Rightarrow$ $'a$ *compact-basis set*
where *set-approximants* $X \equiv \bigcup x \in X.$ *approximants* x

lemma *lower-pd-ideal-PDPlus*:
assumes *lower-le.ideal A*
and $x \in A$
and $y \in A$
shows *PDPlus x y* $\in A$
proof −
from *assms* **obtain** *z* **where** *xy*: $x \leq\flat z$ $y \leq\flat z$ **and** *z*: $z \in A$
by (*blast dest*: *lower-le.idealD2*)
from *xy* **have** *PDPlus x y* $\leq\flat z$ **by** (*simp add*: *lower-le-PDPlus-iff*)
from *assms*(*1*) *this z* **show** *PDPlus x y* $\in A$ **by** (*rule lower-le.idealD3*)
qed

definition *lower-pd-set-ideal* :: *'a*::*bifinite set* ⇒ *'a pd-basis set*
where *lower-pd-set-ideal A* ≡ {*Abs-pd-basis B* | *B*. *B* ⊆ *set-approximants A* ∧ *finite B* ∧ *B* ≠ {}}

lemma *lower-pd-ideal-is-set-ideal*:
assumes *ideal*: *lower-le.ideal A*
defines *KA* ≡ (⋃ *P*∈*A*. *Rep-pd-basis P*)
shows *A* = {*B*. *Rep-pd-basis B* ⊆ *KA*}
proof (*rule subset-antisym*)
show *A* ⊆ {*B*. *Rep-pd-basis B* ⊆ *KA*} **unfolding** *KA-def* **by** *blast*
show {*B*. *Rep-pd-basis B* ⊆ *KA*} ⊆ *A*
proof (*safe*)
fix *B* **assume** *B*: *Rep-pd-basis B* ⊆ *KA*
{
fix *k* **assume** *k* ∈ *Rep-pd-basis B*
with *B* **obtain** *P* **where** *1*: *P* ∈ *A* **and** *2*: *k* ∈ *Rep-pd-basis P*
unfolding *KA-def* **by** *auto*
from *2* **have** *PDUnit k* $\leq\flat$ *P* **unfolding** *lower-le-def*
by (*auto simp add*: *Rep-PDUnit*)
from *ideal this 1* **have** *PDUnit k* ∈ *A* **by** (*rule lower-le.idealD3*)
}
then have *base*: *k* ∈ *Rep-pd-basis B* ⟹ *PDUnit k* ∈ *A* **for** *k* **.**
have *ind*: *Rep-pd-basis C* ⊆ *Rep-pd-basis B* ⟹ *C* ∈ *A* **for** *C*
proof (*induction C rule*: *pd-basis-induct*)
case (*PDUnit a*)
then show *?case* **by** (*simp add*: *Rep-PDUnit base*)
next
case (*PDPlus t u*)
then have *t* ∈ *A* *u* ∈ *A* **by** (*simp-all add*: *Rep-PDPlus*)
with *ideal* **show** *PDPlus t u* ∈ *A* **by** (*rule lower-pd-ideal-PDPlus*)
qed
then show *B* ∈ *A* **by** *simp*
qed
qed

lemma *exists-approx*: ∃ *b*. *b* ∈ *approximants* (*a*::*'a*::*bifinite*)

by (*metis compact-basis.ideal-rep below.ideal-def*)

lemma *set-approximants-ne*: $A \neq \{\} \Longrightarrow$ *set-approximants* $A \neq \{\}$
proof −
assume $A \neq \{\}$
then obtain a **where** a: $a \in A$ **by** *blast*
obtain b **where** $b \in$ *approximants* a **by** (*rule exE*[*OF exists-approx*])
with a **have** $b \in$ *set-approximants* A **unfolding** *set-approximants-def* **by** *blast*
then show *?thesis* **by** *auto*
qed

lemma *lower-pd-set-ideal-mem*: $x \in$ *lower-pd-set-ideal* $A \longleftrightarrow$ *Rep-pd-basis* $x \subseteq$ *set-approximants* A
unfolding *lower-pd-set-ideal-def mem-Collect-eq*
proof
assume $\exists B.\ x =$ *Abs-pd-basis* $B \wedge B \subseteq$ *set-approximants* $A \wedge$ *finite* $B \wedge B \neq \{\}$
then obtain B **where** x: $x =$ *Abs-pd-basis* B **and** B: $B \subseteq$ *set-approximants* A
and *dom*: *finite* B $B \neq \{\}$ **by** *blast*
from *dom* **have** $B \in$ *pd-basis* **unfolding** *pd-basis-def* **by** *simp*
then have *Rep-pd-basis* $x = B$ **unfolding** x **by** (*rule Abs-pd-basis-inverse*)

with B **show** *Rep-pd-basis* $x \subseteq$ *set-approximants* A **by** *simp*
next
assume B: *Rep-pd-basis* $x \subseteq$ *set-approximants* A
show $\exists B.\ x =$ *Abs-pd-basis* $B \wedge$
$B \subseteq$ *set-approximants* $A \wedge$ *finite* $B \wedge B \neq \{\}$
by (*rule exI*[*of* - *Rep-pd-basis* x])
(*simp add*: B *Rep-pd-basis-inverse*[*of* x])
qed

lemma *set-approximants-alt*: *set-approximants* $A = \{x.\ \exists\, y \in A.$ *Rep-compact-basis* $x \sqsubseteq y\}$
unfolding *set-approximants-def approximants-def* **by** *blast*

lemma *set-approximants-downward-closed*: $y \sqsubseteq x \Longrightarrow x \in$ *set-approximants* $A \Longrightarrow$ $y \in$ *set-approximants* A
unfolding *set-approximants-def approximants-def compact-le-def*
using *below-trans* **by** *blast*

lemma *lower-pd-set-ideal*: **assumes** $A \neq \{\}$ **shows** *lower-le.ideal* (*lower-pd-set-ideal* A)
proof (*rule lower-le.idealI*)
obtain b **where** $b \in$ *set-approximants* A **using** *set-approximants-ne*[*OF assms*]
by *blast*
then have *Abs-pd-basis* $\{b\} \in$ *lower-pd-set-ideal* A
unfolding *lower-pd-set-ideal-def* **by** *blast*
then show $\exists x.\ x \in$ *lower-pd-set-ideal* A **by** *auto*
next

fix x y **assume** $x \in$ *lower-pd-set-ideal* A $y \in$ *lower-pd-set-ideal* A
then have *xy*: *Rep-pd-basis* $x \subseteq$ *set-approximants* A *Rep-pd-basis* $y \subseteq$ *set-approximants* A
by (*simp-all add*: *lower-pd-set-ideal-mem*)
then have *Rep-pd-basis* (*PDPlus* x y) $\subseteq$ *set-approximants* A
unfolding *Rep-PDPlus* **by** *simp*
then have $*$: *PDPlus* x $y \in$ *lower-pd-set-ideal* A **by** (*simp add*: *lower-pd-set-ideal-mem*)
show $\exists z \in$ *lower-pd-set-ideal* A. $x \leq\flat z \wedge y \leq\flat z$
using $*$ *PDPlus-lower-le PDPlus-commute*[*of* x y] **by** *metis*
next
show $x \leq\flat y \Longrightarrow y \in$ *lower-pd-set-ideal* $A \Longrightarrow x \in$ *lower-pd-set-ideal* A
for x y
unfolding *lower-pd-set-ideal-mem lower-le-def*
using *set-approximants-downward-closed* **by** *blast*
qed

definition *lower-pd-of-set* :: ('*a*::*bifinite*) *set* ⇒ '*a lower-pd*
where *lower-pd-of-set* A = *Abs-lower-pd* (*lower-pd-set-ideal* A)

lemma *Rep-lower-pd-of-set*: **assumes** $A \neq \{\}$ **shows** *Rep-lower-pd* (*lower-pd-of-set* A) = *lower-pd-set-ideal* A
unfolding *lower-pd-of-set-def*
by (*rule Abs-lower-pd-inverse*) (*simp add*: *lower-pd-set-ideal*[*OF assms*])

definition *lowerset* :: '*a*::*po set* ⇒ '*a set*
where *lowerset* $A = \{y.\ \exists x \in A.\ y \sqsubseteq x\}$

lemma *lowerset-idem*: *lowerset* (*lowerset* A) = *lowerset* A
unfolding *lowerset-def* **using** *below-trans* **by** *blast*

definition *completion* :: '*a*::*cpo set* ⇒ '*a set*
where *completion* $A = \{(\bigsqcup i.\ S\ i) \mid S.\ chain\ S \wedge range\ S \subseteq A\}$

definition *closure* :: '*a*::*cpo set* ⇒ '*a set*
where *closure* A = *completion* (*lowerset* A)

lemma *approximant-lower*:
(x::'*a*::*bifinite compact-basis*) $\in$ *set-approximants* $A \Longrightarrow$ *Rep-compact-basis* $x \in$ *lowerset* A
unfolding *set-approximants-alt lowerset-def* **by** *blast*

lemma *lower-approximants*:
assumes $x \in$ *lowerset* A
shows *approximants* $x \subseteq$ *set-approximants* A
proof −
from *assms*[*unfolded lowerset-def*] **obtain** y **where** *1*: $y \in A$ **and** *2*: $x \sqsubseteq y$
by *blast*
from *2* **have** *approximants* $x \subseteq$ *approximants* y **by** (*rule compact-basis.rep-mono*)
also have *approximants* $y \subseteq$ *set-approximants* A

unfolding *set-approximants-def* **using** *1* **by** *auto*
finally show *?thesis* **.**
qed

lemma *adm-set-approximants*: *adm* ($\lambda x::'a::bifinite.$ *approximants* $x \subseteq$ *set-approximants* A)
by (*rule admI*) (*blast dest*: *compact-basis.rep-lub*)

lemma *adm-not-set-approximants*: *adm* ($\lambda x::'a::bifinite.$ $\neg$(*approximants* $x \subseteq$ *set-approximants* A))
apply (*rule admI*)
unfolding *compact-basis.rep-lub* **by** *fast*

lemma *adm-lower-mem*: *adm* ($\lambda x.$ $x \in\flat$ A)
unfolding *lower-mem-equiv*
by *auto*

lemma *adm-not-lower-mem*: *adm* ($\lambda x.$ $\neg(x \in\flat A)$)
unfolding *lower-mem-equiv*
by *auto*

lemma *closure-approximants*: *approximants* ($x::'a::bifinite$) $\subseteq$ *set-approximants* A
$\longleftrightarrow$
$x \in$ *closure* A
proof
assume $x \in$ *closure* A
from *this*[*unfolded closure-def completion-def*]
obtain S **where** x: $x = (\bigsqcup i.\ S\ i)$ **and** *chain*: *chain* S **and** *ss*: *range* $S \subseteq$ *lowerset* A
by *blast*
from *ss lower-approximants* **have** $\bigwedge i.$ *approximants* ($S\ i$) $\subseteq$ *set-approximants* A
by *auto*
then have *approximants* ($\bigsqcup i.\ S\ i$) $\subseteq$ *set-approximants* A
unfolding *compact-basis.rep-lub*[*OF chain*] **by** *auto*
with x **show** *approximants* $x \subseteq$ *set-approximants* A **by** *simp*
next
assume $*$: *approximants* $x \subseteq$ *set-approximants* A
obtain *approx* :: *nat* $\Rightarrow$ $'a \to 'a$ **where** *approx-chain approx*
by (*rule exE*[*OF bifinite*])
then interpret *approx-chain approx* **.**
define S **where** $S\ i$ = *approx* $i{\cdot}x$ **for** i
have *chain*: *chain* S **unfolding** *S-def* **by** *auto*
have *subset*: *range* $S \subseteq$ *lowerset* A **unfolding** *S-def*
proof (*rule subsetI*, *erule rangeE*, *hypsubst*)
fix i
have *Abs-compact-basis* (*approx* $i{\cdot}x$) $\in$ *approximants* x
by (*simp add*: *approximants-def approx-below*)
then have *Abs-compact-basis* (*approx* $i{\cdot}x$) $\in$ *set-approximants* A
by (*rule subsetD*[*OF* $*$])

```
    then have Rep-compact-basis (Abs-compact-basis (approx i·x)) ∈ lowerset A
      by (rule approximant-lower)
    then show approx i·x ∈ lowerset A by simp
  qed
  have lub: x = (⨆ i. S i) unfolding S-def using lub-approx
    contlub-cfun-fun[OF chain-approx] by simp
  from lub chain subset show x ∈ closure A
    unfolding closure-def completion-def
    by auto
qed

lemma Abs-lower-pd-principal:
  Rep-lower-pd (lower-principal x) = {u. u ≤♭ x}
  unfolding lower-principal-def
  by (intro Abs-lower-pd-inverse CollectI lower-le.ideal-principal)

lemma compact-pd-set-ideal:
  assumes A ≠ {}
  shows {u. u ≤♭ PDUnit b} ⊆ lower-pd-set-ideal A ⟷ approximants (Rep-compact-basis
b) ⊆ set-approximants A
proof −
  have {u. u ≤♭ PDUnit b} ⊆ lower-pd-set-ideal A ⟷ PDUnit b ∈ lower-pd-set-ideal
A
    (is ?ss ⟷ ?mem)
  proof
    show ?ss ⟹ ?mem using lower-le-refl[of PDUnit b] by blast
    show ?mem ⟹ ?ss
      using lower-le.idealD3[OF lower-pd-set-ideal[OF assms], of - PDUnit b]
      by blast
  qed
  also have PDUnit b ∈ lower-pd-set-ideal A ⟷ b ∈ set-approximants A
    unfolding lower-pd-set-ideal-mem Rep-PDUnit by simp
  also have b ∈ set-approximants A ⟷ approximants (Rep-compact-basis b) ⊆
set-approximants A
    unfolding approximants-def
    using set-approximants-downward-closed
    by auto
  finally show ?thesis .
qed

lemma lower-pd-of-set-mem-iff:
  assumes (A::'a::bifinite set) ≠ {}
  shows x ∈♭ lower-pd-of-set A ⟷ x ∈ closure A
  unfolding closure-approximants[symmetric]
proof (induction x rule: compact-basis.principal-induct)
  case 1
  show adm (λa. (a ∈♭ lower-pd-of-set A) =
            (approximants a ⊆ set-approximants A))
    by (simp add: adm-set-approximants adm-not-set-approximants
```

```
      adm-lower-mem adm-not-lower-mem)
next
  case (2 b)
  have Rep-compact-basis b ∈♭ lower-pd-of-set A ⟷
    lower-principal (PDUnit b) ⊑ lower-pd-of-set A
    unfolding lower-mem-equiv by simp
  also have ... ⟷ {u. u ≤♭ PDUnit b} ⊆ lower-pd-set-ideal A
    unfolding below-lower-pd-def
      Rep-lower-pd-of-set[OF assms]
      Abs-lower-pd-principal
    ..
  also note compact-pd-set-ideal[OF assms]
  finally show ?case .
qed

lemma lower-ext-subset:
  assumes ⋀p. {p}♭ ⊑ x ⟹ {p}♭ ⊑ y
  shows x ⊑ y
proof −
  have lower-principal h ⊑ x ⟹ lower-principal h ⊑ y for h
  proof (induct rule: pd-basis-induct)
    case (PDUnit a)
    then show ?case using assms[of Rep-compact-basis a] by simp
  next
    case (PDPlus t u)
    from PDPlus.prems have lower-principal t ⊑ x lower-principal u ⊑ x
      by (metis lower-plus-below-iff lower-plus-principal)+
    with PDPlus.hyps show lower-principal (PDPlus t u) ⊑ y
      by (metis lower-plus-below-iff lower-plus-principal)
  qed

  then have h ⊑ x ⟹ h ⊑ y for h
    by (induct h rule: lower-pd.principal-induct, simp)
  then show x ⊑ y by simp
qed

lemma lower-pd-set-iff: x ⊑ y ⟷ lower-pd-set x ⊆ lower-pd-set y
  unfolding lower-pd-set-def lower-mem-equiv
  by (auto intro: lower-ext-subset below-trans)

corollary lower-pd-set-inj: lower-pd-set x = lower-pd-set y ⟹ x = y
  by (auto intro!: below-antisym simp add: lower-pd-set-iff)

lemma lower-pd-set-meet[simp]: lower-pd-set (x ⊓ y) = lower-pd-set x ∩ lower-pd-set
y
  unfolding lower-pd-set-def lower-mem-equiv
  by auto

lemma lower-pd-set-join[simp]: lower-pd-set (x ⊔ y) = lower-pd-set x ∪ lower-pd-set
```

y
unfolding *lower-join-eq lower-pd-set-def lower-mem-equiv lower-unit-below-plus-iff*
by *auto*

instance *lower-pd* :: (*bifinite*) *distributive-cpo*
proof
fix *a b c* :: *'a lower-pd*
show $c \sqcap (a \sqcup b) = (c \sqcap a) \sqcup (c \sqcap b)$
by (*auto intro*!: *lower-pd-set-inj*)
qed

20 Lower powerdomain elements are generated by their maximal members

The members of a lower powerdomain element form a Scott-closed set: closed downwards and under directed lubs. By Zorn's lemma every member lies below a maximal one, so the set is the down-closure of its maximal members.

lemma *lower-mem-below*: $x \in\flat\ X \Longrightarrow y \sqsubseteq x \Longrightarrow y \in\flat\ X$
unfolding *lower-mem-equiv* **by** (*blast intro*: *below-trans monofun-cfun-arg*)

lemma *lower-mem-directed-lub*:
assumes *D*: *directed D* $D \neq \{\}$ **and** *mem*: $\bigwedge x.\ x \in D \Longrightarrow x \in\flat\ X$
shows *lub D* $\in\flat\ X$
proof −
obtain *C* **where** *C*: *chain C* $\forall\, i.\ C\ i \in D$ $(\bigsqcup i.\ C\ i) =$ *lub D*
by (*rule obtain-directed-chain*[*OF D*])
have $(\bigsqcup i.\ C\ i) \in\flat\ X$ **using** *admD*[*OF adm-lower-mem C*(*1*)] *C*(*2*) *mem* **by** *blast*
then show *?thesis* **by** (*simp only*: *C*(*3*))
qed

lemma *lower-mem-lub*: *chain* $Y \Longrightarrow x \in\flat\ Y\ i \Longrightarrow x \in\flat\ (\bigsqcup i.\ Y\ i)$
unfolding *lower-mem-equiv* **by** (*blast intro*: *below-trans is-ub-thelub*)

lemma *lower-mem-chain*: *chain* $Y \Longrightarrow x \in\flat\ Y\ i \Longrightarrow i \leq j \Longrightarrow x \in\flat\ Y\ j$
unfolding *lower-mem-equiv* **by** (*blast intro*: *below-trans chain-mono*)

lemma *lower-mem-compact-lub*:
assumes *Y*: *chain Y* **and** *x*: *compact x* **and** *mem*: $x \in\flat\ (\bigsqcup i.\ Y\ i)$
obtains *i* **where** $x \in\flat\ Y\ i$
using *mem compact-below-lub-iff*[*OF compact-lower-unit*[*OF x*] *Y*] **by** (*auto simp add*: *lower-mem-equiv*)

definition *lower-maximal* :: *'a*::*bifinite lower-pd* ⇒ *'a set* **where**
lower-maximal X = $\{m.\ m \in\flat\ X \wedge (\forall\, y.\ y \in\flat\ X \longrightarrow m \sqsubseteq y \longrightarrow y = m)\}$

theorem *lower-mem-maximal-above*:

assumes x: $x \in\flat\ X$
obtains m **where** $m \in$ *lower-maximal* $X\ x \sqsubseteq m$
proof −
have *ub*: $\exists\, a \in$ *lower-pd-set* $X.\ T <|\ a$ **if** T: $T \subseteq$ *lower-pd-set* X *directed* T **for** T
proof (*cases* $T = \{\}$)
case *True*
then show *?thesis* **using** x **by** (*auto simp add*: *lower-pd-set-def*)
next
case *False*
have *lub* $T \in\flat\ X$ **using** *lower-mem-directed-lub*[*OF* $T(2)$ *False*] $T(1)$ **by** (*auto simp add*: *lower-pd-set-def*)
then show *?thesis* **using** *is-lubD1*[*OF directed-lub*[*OF* $T(2)$]] **by** (*auto simp add*: *lower-pd-set-def*)
qed
obtain m **where** $m \in$ *lower-pd-set* $X\ x \sqsubseteq m\ \forall\, z \in$ *lower-pd-set* $X.\ m \sqsubseteq z \longrightarrow m = z$
using *obtain-total-above*[*of x lower-pd-set X*] x *ub* **by** (*auto simp add*: *lower-pd-set-def*)
then show *?thesis* **using** *that* **by** (*auto simp add*: *lower-maximal-def lower-pd-set-def*)
qed

corollary *lower-pd-set-maximal*: *lower-pd-set* $X =$ *lowerset* (*lower-maximal* X)
proof (*rule set-eqI*, *rule iffI*)
fix x **assume** $x \in$ *lower-pd-set* X
then obtain m **where** $m \in$ *lower-maximal* $X\ x \sqsubseteq m$
by (*auto elim*: *lower-mem-maximal-above simp add*: *lower-pd-set-def*)
then show $x \in$ *lowerset* (*lower-maximal* X) **unfolding** *lowerset-def* **by** *blast*
next
fix x **assume** $x \in$ *lowerset* (*lower-maximal* X)
then show $x \in$ *lower-pd-set* X
unfolding *lowerset-def lower-maximal-def lower-pd-set-def* **by** (*blast intro*: *lower-mem-below*)
qed

21 Binds and maps through members

A bind lies below Z as soon as the image of every member does.

lemma *lower-bind-below*:
assumes g: $\bigwedge y.\ y \in\flat\ Y \Longrightarrow g{\cdot}y \sqsubseteq Z$
shows *lower-bind*$\cdot Y{\cdot}g \sqsubseteq Z$
proof −
have $W \sqsubseteq Y \longrightarrow$ *lower-bind*$\cdot W{\cdot}g \sqsubseteq Z$ **for** W
proof (*induct* W *rule*: *lower-pd-induct*)
case *adm*
then show *?case* **by** (*rule adm-imp*) *simp-all*
next
case (*lower-unit* a)
then show *?case* **using** g **by** (*simp add*: *lower-mem-equiv*)

```
  next
    case (lower-plus xs ys)
    then show ?case by simp
  qed
  then show ?thesis by simp
qed

lemma lower-bind-mem:
  y ∈♭ Y ⟹ g·y ⊑ lower-bind·Y·g
  using monofun-cfun-fun[of lower-bind·{y}♭ lower-bind·Y g]
  by (simp add: lower-mem-equiv monofun-cfun-arg)

lemma lower-map-mem: x ∈♭ X ⟹ f·x ∈♭ lower-map·f·X
  using monofun-cfun-arg[of {x}♭ X lower-map·f] by (simp add: lower-mem-equiv)
```

A compact member of an image lies below the image of a member.

```
lemma lower-map-mem-compact:
  assumes v: compact v and mem: v ∈♭ lower-map·f·Z
  obtains w where w ∈♭ Z v ⊑ f·w
proof −
  have ∀ v. compact v ⟶ {v}♭ ⊑ lower-map·f·W ⟶ (∃ w. w ∈♭ W ∧ v ⊑ f·w)
for W
  proof (induct W rule: lower-pd-induct)
    case adm
    show ?case
    proof (rule admI, intro allI impI)
      fix Ws v assume Ws: chain Ws
        and all: ∀ i. ∀ v. compact v ⟶ {v}♭ ⊑ lower-map·f·(Ws i) ⟶ (∃ w. w ∈♭
Ws i ∧ v ⊑ f·w)
        and v: compact v and le: {v}♭ ⊑ lower-map·f·(⨆ i. Ws i)
    have {v}♭ ⊑ (⨆ i. lower-map·f·(Ws i)) using le by (simp add: contlub-cfun-arg[OF
Ws])
      then obtain i where {v}♭ ⊑ lower-map·f·(Ws i)
        using compact-below-lub-iff[OF compact-lower-unit[OF v]] Ws by auto
      then obtain w where w ∈♭ Ws i v ⊑ f·w using all v by blast
      then show ∃ w. w ∈♭ (⨆ i. Ws i) ∧ v ⊑ f·w using lower-mem-lub[OF Ws]
by blast
    qed
  next
    case (lower-unit a)
    then show ?case by (auto simp add: lower-mem-principal)
  next
    case (lower-plus xs ys)
    then show ?case by (auto simp add: lower-mem-union)
  qed
  then show ?thesis using v mem that by (auto simp add: lower-mem-equiv)
qed
```

22 Rows with a top adjoined

Mapping *down* over a lower powerdomain element identifies it with a Top-free element of the lower powerdomain over rows with a top: its members are exactly the *down*s of the members.

lemma *lower-mem-map-down*: $v \in\flat$ *lower-map·down·Z* $\longleftrightarrow$ ($\exists\, w.\ w \in\flat Z \wedge v =$ *down·w*)
proof
assume *mem*: $v \in\flat$ *lower-map·down·Z*
obtain *ap* :: *nat* $\Rightarrow$ $'a$ *top* $\rightarrow$ $'a$ *top* **where** *ap*: *approx-chain ap* **using** *bifinite* **by** *blast*
have $\exists\, u.$ *ap i·v* = *down·u* $\wedge$ $u \in\flat Z$ **for** *i*
proof −
have *ap i·v* $\in\flat$ *lower-map·down·Z*
using *mem approx-chain.approx-below*[*OF ap*] **by** (*rule lower-mem-below*)
then obtain *w* **where** *w*: $w \in\flat Z$ **and** *le*: *ap i·v* $\sqsubseteq$ *down·w*
using *approx-chain.compact-approx*[*OF ap*] **by** (*blast elim*: *lower-map-mem-compact*)
show *?thesis*
proof (*cases ap i·v*)
case (*down u*)
then show *?thesis* **using** *le w* **by** (*auto intro*: *lower-mem-below*)
next
case *Top*
then show *?thesis* **using** *le* **by** *simp*
qed
qed
then obtain *us* **where** *us*: $\bigwedge i.$ *ap i·v* = *down·*(*us i*) **and** *mem-us*: $\bigwedge i.$ *us i* $\in\flat$ *Z* **by** *metis*
have *ch*: *chain us*
proof (*rule chainI*)
fix *i*
have *ap i·v* $\sqsubseteq$ *ap* (*Suc i*)*·v*
using *approx-chain.chain-approx*[*OF ap*] **by** (*simp add*: *monofun-cfun-fun chainE*)
then show *us i* $\sqsubseteq$ *us* (*Suc i*) **by** (*simp add*: *us*)
qed
have *v* = ($\bigsqcup i.$ *ap i·v*)
using *approx-chain.chain-approx*[*OF ap*] *approx-chain.lub-approx*[*OF ap*] **by** (*simp flip*: *contlub-cfun-fun*)
also have ... = *down·*($\bigsqcup i.$ *us i*) **by** (*simp add*: *us contlub-cfun-arg*[*OF ch*])
finally show $\exists\, w.\ w \in\flat Z \wedge v =$ *down·w* **using** *admD*[*OF adm-lower-mem ch*] *mem-us* **by** *blast*
next
assume $\exists\, w.\ w \in\flat Z \wedge v =$ *down·w*
then show *v* $\in\flat$ *lower-map·down·Z* **by** (*auto intro*: *lower-map-mem*)
qed

lemma *Top-not-mem-map-down*: $\neg$ *Top* $\in\flat$ *lower-map·down·Z*

by (*simp add*: *lower-mem-map-down*)

lemma *lower-map-down-below-iff*: *lower-map·down·X* $\sqsubseteq$ *lower-map·down·Z* $\longleftrightarrow$ *X* $\sqsubseteq$ *Z*
proof
assume *le*: *lower-map·down·X* $\sqsubseteq$ *lower-map·down·Z*
have *x* $\in\flat$ *Z* **if** *x* $\in\flat$ *X* **for** *x*
proof −
have *down·x* $\in\flat$ *lower-map·down·Z*
using *lower-map-mem*[*OF that*] *le* **by** (*auto simp add*: *lower-mem-equiv intro*: *below-trans*)
then show *?thesis* **by** (*auto simp add*: *lower-mem-map-down dest*: *down-inject*)
qed
then show *X* $\sqsubseteq$ *Z* **by** (*simp add*: *lower-pd-set-iff lower-pd-set-def subset-iff*)
qed (*rule monofun-cfun-arg*)

Conversely every Top-free element comes from the rows.

theorem *lower-map-down-onto*:
assumes *nT*: ¬ *Top* $\in\flat$ *Y*
obtains *Z* **where** *Y* = *lower-map·down·Z*
proof −
define *A* **where** *A* = {*w*. *down·w* $\in\flat$ *Y*}
have ⊥ ∈ *A* **unfolding** *A-def lower-mem-equiv* **by** (*simp add*: *down-strict*)
then have *ne*: *A* ≠ {} **by** *blast*
have *down-closed*: *lowerset A* = *A*
unfolding *lowerset-def A-def* **by** (*auto intro*: *lower-mem-below monofun-cfun-arg*)
have *closed*: *closure A* = *A*
proof (*rule set-eqI*, *rule iffI*)
fix *x* **assume** *x* ∈ *closure A*
then obtain *S* **where** *x*: *x* = (⨆ *i*. *S i*) **and** *S*: *chain S range S* ⊆ *A*
unfolding *closure-def completion-def down-closed* **by** *blast*
have *down·*(⨆ *i*. *S i*) = (⨆ *i*. *down·*(*S i*)) **by** (*rule contlub-cfun-arg*[*OF S*(*1*)])
moreover have (⨆ *i*. *down·*(*S i*)) $\in\flat$ *Y*
using *admD*[*OF adm-lower-mem ch2ch-Rep-cfunR*[*OF S*(*1*)]] *S*(*2*) **unfolding** *A-def* **by** *blast*
ultimately show *x* ∈ *A* **by** (*simp add*: *A-def x*)
next
fix *x* **assume** *x* ∈ *A*
then show *x* ∈ *closure A*
unfolding *closure-def completion-def down-closed*
by (*auto intro*!: *exI*[*of* - λ-. *x*])
qed
have *Y* = *lower-map·down·*(*lower-pd-of-set A*)
proof (*rule lower-pd-set-inj*, *rule set-eqI*)
fix *v*
have *v* $\in\flat$ *Y* $\longleftrightarrow$ (∃ *w*. *w* ∈ *A* ∧ *v* = *down·w*)
using *nT* **by** (*cases v*) (*auto simp add*: *A-def dest*: *down-inject*)
then show *v* ∈ *lower-pd-set Y* $\longleftrightarrow$ *v* ∈ *lower-pd-set* (*lower-map·down·*(*lower-pd-of-set A*))

by (*simp add*: *lower-pd-set-def lower-mem-map-down lower-pd-of-set-mem-iff*[*OF ne*] *closed*)
qed
then show *?thesis* **by** (*rule that*)
qed

end
theory *UpperPD-Representation* **imports** *HOLCF Koenigs Distributive Directed*
begin

lemma *koenig-construction-aux*:
fixes x Y f
assumes *chain*: $\forall i.\ Y\ i \leq\sharp\ Y\ (Suc\ i)$
and *sup*: $x = (\bigsqcup i.\ upper\text{-}principal\ (Y\ i))$
and *f-mem*: $\forall i.\ f\ i \in Rep\text{-}pd\text{-}basis\ (Y\ i)$
and *f-chain*: $\forall i.\ f\ i \sqsubseteq f\ (Suc\ i)$
shows *chain* $(\lambda i.\ Rep\text{-}compact\text{-}basis\ (f\ i))$
and $x \sqsubseteq \{\bigsqcup i.\ Rep\text{-}compact\text{-}basis\ (f\ i)\}\sharp$
proof −
from *f-mem* **have** *le-unit*: $\bigwedge n.\ Y\ n \leq\sharp\ PDUnit\ (f\ n)$ **unfolding** *upper-le-def Rep-PDUnit*
using *below-refl* **by** *blast*
then have *principal-below*: $\bigwedge n.\ upper\text{-}principal\ (Y\ n) \sqsubseteq upper\text{-}principal\ (PDUnit\ (f\ n))$
by *simp*
from *f-chain* **show** *fchain*: *chain* $(\lambda i.\ Rep\text{-}compact\text{-}basis\ (f\ i))$ **by** *simp*
then have *chain* $(\lambda i.\ \{Rep\text{-}compact\text{-}basis\ (f\ i)\}\sharp)$ **by** (*rule chain-monofun*)
with *upper-pd.ch2ch-principal*[*OF chain*] *principal-below*
have $x \sqsubseteq (\bigsqcup n.\ \{Rep\text{-}compact\text{-}basis\ (f\ n)\}\sharp)$
unfolding *sup*
by (*auto intro*: *lub-mono*)
then show $x \sqsubseteq \{\bigsqcup i.\ Rep\text{-}compact\text{-}basis\ (f\ i)\}\sharp$
unfolding *contlub-cfun-arg*[*OF fchain*] **by** *simp*
qed

lemma *upper-pd-has-point*: $\exists x.\ s \sqsubseteq \{x\}\sharp$
proof −
from *upper-pd.obtain-principal-chain*
obtain Y **where** *chain*: $\forall i.\ Y\ i \leq\sharp\ Y\ (Suc\ i)$
and *sup*: $s = (\bigsqcup i.\ upper\text{-}principal\ (Y\ i))$**.**

have *1*: $Rep\text{-}pd\text{-}basis\ (Y\ i) \neq \{\}$ **for** i **by** *simp*
have *2*: *finite* $(Rep\text{-}pd\text{-}basis\ (Y\ i))$ **for** i **by** *simp*
have *3*: $\forall y \in Rep\text{-}pd\text{-}basis\ (Y\ (Suc\ i)).\ \exists x \in Rep\text{-}pd\text{-}basis\ (Y\ i).\ x \sqsubseteq y$ **for** i
using *chain*[*unfolded upper-le-def*] **..**

obtain f **where** f: $\forall i.\ f\ i \in Rep\text{-}pd\text{-}basis\ (Y\ i)\ \forall i.\ f\ i \sqsubseteq f\ (Suc\ i)$
by (*rule obtain-infinite-path*[*of* $\lambda i.\ Rep\text{-}pd\text{-}basis\ (Y\ i)$ $(\sqsubseteq)$, *OF 1 2 3 that*])
from *koenig-construction-aux*(*2*)[*OF chain sup this*]

```
  have s ⊑ {⨆ n. Rep-compact-basis (f n)}♯ .
  then show ?thesis ..
qed

theorem upper-pd-extensionality-lemma:
  fixes x k
  defines y ≡ upper-principal k
  assumes imp: ⋀p. x ⊑ {p}♯ ⟹ y ⊑ {p}♯
  shows y ⊑ x
proof (rule classical)
  assume not-below: ¬(y ⊑ x)

  from upper-pd.obtain-principal-chain
  obtain Y where chain: ∀ i. Y i ≤♯ Y (Suc i)
    and sup: x = (⨆ i. upper-principal (Y i)).
  from chain have chain': chain (λi. upper-principal (Y i))
    by (rule upper-pd.ch2ch-principal)
  from sup not-below chain' have ¬(y ⊑ upper-principal (Y i)) for i
    using below-lub[OF chain', of y] by iprover
  with y-def have not-below-compact: ¬(k ≤♯ (Y i)) for i by simp

  define Bad where Bad i = {q ∈ Rep-pd-basis (Y i). (∀ r ∈ Rep-pd-basis k. ¬(r
⊑ q))}
    for i
  from not-below-compact have 1: Bad i ≠ {} for i
    unfolding upper-le-def Bad-def by fast
  have 2: finite (Bad i) for i unfolding Bad-def by simp
  have 3: ∀ y ∈ Bad (Suc i). ∃ x ∈ Bad i. x ⊑ y for i
  proof
    fix y assume y: y ∈ Bad (Suc i)
    then have y ∈ Rep-pd-basis (Y (Suc i)) unfolding Bad-def by auto
    with chain obtain x where x ∈ Rep-pd-basis (Y i) x ⊑ y
      unfolding upper-le-def by blast
    from y have ∀ r ∈ Rep-pd-basis k. ¬(r ⊑ y) unfolding Bad-def by simp
    with ‹x ⊑ y› below-trans
    have ∀ r ∈ Rep-pd-basis k. ¬(r ⊑ x)
      by blast
    with ‹x ∈ Rep-pd-basis (Y i)› have x ∈ Bad i unfolding Bad-def by auto
    with ‹x ⊑ y› show ∃ x ∈ Bad i. x ⊑ y ..
  qed

  from obtain-infinite-path[of Bad (⊑), OF 1 2 3]
  obtain f where f-mem: ∀ i. f i ∈ Bad i and f-chain: ∀ i. f i ⊑ f (Suc i) .
  from f-mem Bad-def have f-mem': ∀ i. f i ∈ Rep-pd-basis (Y i) by auto
  from koenig-construction-aux[OF chain sup f-mem' f-chain]
  have f-chain: chain (λi. Rep-compact-basis (f i))
    and x-below: x ⊑ {⨆ i. Rep-compact-basis (f i)}♯
    by this+
  from x-below have y ⊑ {⨆ i. Rep-compact-basis (f i)}♯ by (rule imp)
```

```
  with f-chain have y-below: y ⊑ (⨆ i. {Rep-compact-basis (f i)}♯)
    by (simp add: contlub-cfun-arg)
  from y-def have compact y by simp
  with y-below obtain i where y ⊑ {Rep-compact-basis (f i)}♯
    using chain-monofun[OF f-chain] compactD2 by blast
  with y-def have k ≤♯ PDUnit (f i) by simp
  then have ∃ q ∈ Rep-pd-basis k. q ⊑ f i unfolding upper-le-def
    by (simp add: Rep-PDUnit)
  with f-mem Bad-def have False by auto
  then show ?thesis ..
qed

theorem upper-pd-ext-subset:
  assumes ⋀p. x ⊑ {p}♯ ⟹ y ⊑ {p}♯
  shows y ⊑ x
proof −
  have h ⊑ y ⟹ h ⊑ x for h
  proof (induct h rule: upper-pd.principal-induct, simp)
    fix k assume upper-principal k ⊑ y
    from below-trans[OF this assms]
    show upper-principal k ⊑ x
      by (rule upper-pd-extensionality-lemma)
  qed
  then show ?thesis by simp
qed

theorem upper-pd-below-iff-ext-subset:
x ⊑ y ⟷ (∀ p. y ⊑ {p}♯ ⟶ x ⊑ {p}♯)
  by (auto intro: upper-pd-ext-subset below-trans)

definition upper-set where upper-set x = {p. x ⊑ {p}♯}

lemma upper-set-subset-iff: x ⊑ y ⟷ upper-set x ⊇ upper-set y
  unfolding upper-set-def
  by (subst upper-pd-below-iff-ext-subset) auto

lemma upper-set-join[simp]: x ⇑ y ⟹ upper-set (x ⊔ y) = upper-set x ∩ upper-set
y
  unfolding upper-set-def using join-belowI joinableD[of x y] below-trans by blast

lemma upper-set-meet[simp]: upper-set (x ⊓ y) = upper-set x ∪ upper-set y
  unfolding upper-set-def upper-meet-eq upper-plus-below-unit-iff by auto

corollary upper-set-inj: upper-set x = upper-set y ⟹ x = y
  using upper-set-subset-iff by (auto intro!: below-antisym)

instance upper-pd :: (bifinite) distributive-cpo
proof
  fix a b c :: 'a upper-pd
```

assume $a \Uparrow b$
then show $c \sqcap (a \sqcup b) = (c \sqcap a) \sqcup (c \sqcap b)$
by (*auto intro*!: *upper-set-inj*)
qed

lemma *upper-principal-below-unit-iff*:
$\mathit{upper\text{-}principal}\ k \sqsubseteq \{x\}\sharp \longleftrightarrow (\exists\, q \in \mathit{Rep\text{-}pd\text{-}basis}\ k.\ \mathit{Rep\text{-}compact\text{-}basis}\ q \sqsubseteq x)$
proof (*induct k rule*: *pd-basis-induct*)
case (*PDUnit a*)
have $\mathit{upper\text{-}principal}\ (\mathit{PDUnit}\ a) \sqsubseteq \{x\}\sharp \longleftrightarrow \{\mathit{Rep\text{-}compact\text{-}basis}\ a\}\sharp \sqsubseteq \{x\}\sharp$
by (*simp only*: *upper-unit-Rep-compact-basis*)
also have $\ldots \longleftrightarrow \mathit{Rep\text{-}compact\text{-}basis}\ a \sqsubseteq x$
by (*rule upper-unit-below-iff*)
finally show *?case* **by** (*simp add*: *Rep-PDUnit*)
next
case (*PDPlus s t*)
have $\mathit{upper\text{-}principal}\ (\mathit{PDPlus}\ s\ t) \sqsubseteq \{x\}\sharp$
$\longleftrightarrow \mathit{upper\text{-}principal}\ s \sqsubseteq \{x\}\sharp \vee \mathit{upper\text{-}principal}\ t \sqsubseteq \{x\}\sharp$
by (*simp only*: *upper-plus-principal*[*symmetric*] *upper-plus-below-unit-iff*)
also have $\ldots \longleftrightarrow (\exists\, q \in \mathit{Rep\text{-}pd\text{-}basis}\ (\mathit{PDPlus}\ s\ t).\ \mathit{Rep\text{-}compact\text{-}basis}\ q \sqsubseteq x)$
using *PDPlus* **by** (*auto simp*: *Rep-PDPlus*)
finally show *?case* **.**
qed

lemma *chain-finite-nonempty-Inter*:
fixes $\mathcal{F} :: 'a::\mathit{type}\ \mathit{set}\ \mathit{set}$
assumes *ne*: $\mathcal{F} \neq \{\}$
and *chain*: $\bigwedge S\ T.\ [\![S \in \mathcal{F};\ T \in \mathcal{F}]\!] \Longrightarrow S \subseteq T \vee T \subseteq S$
and *fin*: $\bigwedge S.\ S \in \mathcal{F} \Longrightarrow \mathit{finite}\ S$
and *nemp*: $\bigwedge S.\ S \in \mathcal{F} \Longrightarrow S \neq \{\}$
shows $\bigcap \mathcal{F} \neq \{\}$
proof −
from *ne* **obtain** *S0* **where** *S0*: $S0 \in \mathcal{F}$ **by** *blast*
define $\mathcal{G}$ **where** $\mathcal{G} = \{S \in \mathcal{F}.\ S \subseteq S0\}$
have $\mathcal{G} \subseteq \mathit{Pow}\ S0$ **by** (*auto simp*: $\mathcal{G}$-*def*)
with *fin*[*OF S0*] **have** *finite* $\mathcal{G}$ **by** (*blast intro*: *finite-subset*)
moreover have $\mathcal{G} \neq \{\}$ **using** *S0* **by** (*auto simp*: $\mathcal{G}$-*def*)
ultimately obtain *m* **where** *mG*: $m \in \mathcal{G}$
and *m-min*: $\bigwedge T.\ [\![T \in \mathcal{G};\ T \subseteq m]\!] \Longrightarrow m = T$
by (*blast dest*: *finite-has-minimal*)
have *least*: $m \subseteq S$ **if** $S \in \mathcal{F}$ **for** *S*
proof (*cases* $S \subseteq S0$)
case *True*
with *that* **have** $S \in \mathcal{G}$ **by** (*simp add*: $\mathcal{G}$-*def*)
with *chain mG* **have** $S \subseteq m \vee m \subseteq S$ **by** (*auto simp*: $\mathcal{G}$-*def*)
with *m-min* ‹$S \in \mathcal{G}$› **show** $m \subseteq S$ **by** *blast*
next
case *False*
have $m \subseteq S0$ **using** *mG* **by** (*simp add*: $\mathcal{G}$-*def*)

also from *chain*[*OF that S0*] *False* **have** *S0* $\subseteq$ *S* **by** *blast*
finally show $m \subseteq S$ **.**
qed
from *mG* **have** $m \in \mathcal{F}$ **by** (*simp add*: $\mathcal{G}$*-def*)
with *nemp* **obtain** *z* **where** $z \in m$ **by** *blast*
with *least* **have** $z \in \bigcap \mathcal{F}$ **by** *blast*
thus *?thesis* **by** *blast*
qed

Chains have greatest lower bounds in bifinite domains

lemma *bifinite-Chain-glb*:
fixes *A* :: *'a*::*bifinite set*
assumes *Chain A* **and** $A \neq \{\}$
shows $\exists$ *a*. *is-glb A a*
proof −
from *bifinite* **obtain** *approx* :: *nat* $\Rightarrow$ *'a* $\rightarrow$ *'a* **where** *approx-chain approx* **..**
then interpret *approx-chain approx* **.**
have *lub-approx-app*[*simp*]: $(\bigsqcup i.\ approx\ i{\cdot}z) = z$ **for** *z*
by (*simp add*: *contlub-cfun-fun*[*OF chain-approx*, *symmetric*])
define *C* **where** *C i* = ($\lambda x.$ *approx i*·*x*) ‘ *A* **for** *i*
have *C-Chain*: *Chain* (*C i*) **for** *i*
unfolding *C-def*
using *monofun-Rep-cfun2 assms*(*1*)
by (*rule Chain-mono-image*)
have *C-finite*: *finite* (*C i*) **for** *i* **unfolding** *C-def*
by (*simp add*: *finite-deflation.finite-image*)
from *assms*(*2*) **have** *C-ne*: *C i* $\neq \{\}$ **for** *i* **by** (*simp add*: *C-def*)
have *C-glb*: *is-glb* (*C i*) (*glb* (*C i*)) **and** *C-mem*: *glb* (*C i*) $\in$ *C i* **for** *i*
proof −
from *order.finite-has-minimal*[*OF below-order C-finite C-ne*]
obtain *m* **where** *m*: $m \in C\ i$ **and** *m-min*: $\forall\, b \in C\ i.\ b \sqsubseteq m \longrightarrow m = b$ **..**
then have $\forall\, b \in C\ i.\ m \sqsubseteq b$
using *ChainE*[*OF C-Chain m*] **by** *metis*
with *m* **have** *m-glb*: *is-glb* (*C i*) *m*
by (*blast intro*: *is-glbI*)
then have *glb-m*: *glb* (*C i*) = *m*
by (*rule glb-eqI*)
from *m-glb glb-m m* **show** *is-glb* (*C i*) (*glb* (*C i*)) **and** *glb* (*C i*) $\in$ *C i*
by *simp-all*
qed

define *M* **where** *M i* = *glb* (*C i*) **for** *i*
have *M-mem*: *M i* $\in$ *C i* **for** *i* **unfolding** *M-def* **by** (*rule C-mem*)
have *M-obtains*: $\exists\, a{\in}A.$ *M i* = *approx i*·*a* **for** *i*
using *M-mem*[*of i*] **unfolding** *C-def* **by** (*auto simp*: *image-iff*)
from *C-glb*[*THEN is-glb-below*] **have** *M-below-approx*: *M i* $\sqsubseteq$ *approx i*·*x* **if** $x \in A$ **for** *i x*
unfolding *M-def C-def* **using** *that* **by** *blast*
have *M-chain*: *chain M*

```
    by (rule chainI)
    (metis M-obtains M-below-approx below-trans
      monofun-cfun-fun[OF chainE[OF chain-approx]])
  have is-glb A (⨆ i. M i)
  proof (rule is-glbI)
    fix x assume x: x ∈ A
    then have  M i ⊑ approx i·x for i
      by (rule M-below-approx)
    then have (⨆ i. M i) ⊑ (⨆ i. approx i·x)
      by (rule lub-mono[OF M-chain ch2ch-Rep-cfunL[OF chain-approx]])
    then show (⨆ i. M i) ⊑ x by simp
  next
    fix b assume bnd: ∀ x ∈ A. b ⊑ x
    have approx i·b ⊑ M i for i
    proof −
      from M-obtains[of i] obtain a where a: a ∈ A and Ma: M i = approx i·a
by blast
      from bnd a have b ⊑ a by blast
      then have approx i·b ⊑ approx i·a
        by (rule monofun-cfun-arg)
      with Ma show ?thesis by simp
    qed
    then have (⨆ i. approx i·b) ⊑ (⨆ i. M i)
      by (rule lub-mono[OF ch2ch-Rep-cfunL[OF chain-approx] M-chain])
    then show b ⊑ (⨆ i. M i) by simp
  qed
  then show ?thesis ..
qed

lemma upper-principal-below-chain:
  assumes Chain C C ≠ {} ⋀x. x ∈ C ⟹ upper-principal k ⊑ {x}♯
    is-glb C g
  shows upper-principal k ⊑ {g}♯
proof −
  define ℱ where
    ℱ = (λc. {q ∈ Rep-pd-basis k. Rep-compact-basis q ⊑ c}) ‘ C
  have ℱ ≠ {} using assms(2) by (simp add: ℱ-def)
  moreover have S ⊆ T ∨ T ⊆ S if S ∈ ℱ T ∈ ℱ for S T
  proof −
    from that obtain c d where cd: c ∈ C d ∈ C
      and S: S = {q ∈ Rep-pd-basis k. Rep-compact-basis q ⊑ c}
      and T: T = {q ∈ Rep-pd-basis k. Rep-compact-basis q ⊑ d}
      unfolding ℱ-def by blast
    from ChainE[OF assms(1) cd] show ?thesis
      unfolding S T using below-trans by blast
  qed
  moreover have finite S if S ∈ ℱ for S
    using that unfolding ℱ-def by auto
  moreover have S ≠ {} if S ∈ ℱ for S
```

```
  proof −
    from that obtain c where c: c ∈ C
      and S: S = {q ∈ Rep-pd-basis k. Rep-compact-basis q ⊑ c}
      unfolding F-def by blast
    from c have upper-principal k ⊑ {c}♯ by (rule assms(3))
   then show ?thesis unfolding S by (auto simp add: upper-principal-below-unit-iff)
  qed
  ultimately have ⋂F ≠ {} by (rule chain-finite-nonempty-Inter)
  then obtain q where q: q ∈ ⋂F by blast
  from assms(2) obtain c0 where c0 ∈ C by blast
  with q have qmem: q ∈ Rep-pd-basis k unfolding F-def by auto
  have q-lb: Rep-compact-basis q ⊑ c if c ∈ C for c
    using q that unfolding F-def by auto
  from assms(4) q-lb have Rep-compact-basis q ⊑ g
    by (blast intro: is-glb-greatest)
  with qmem have ∃ q∈Rep-pd-basis k. Rep-compact-basis q ⊑ g by blast
  then show upper-principal k ⊑ {g}♯
    by (simp add: upper-principal-below-unit-iff)
qed

lemma upper-set-glb-closed:
  assumes Chain C C ≠ {} C ⊆ upper-set x is-glb C g
  shows g ∈ upper-set x
proof −
  from upper-pd.obtain-principal-chain
  obtain Y where chain: ∀ i. Y i ≤♯ Y (Suc i)
    and sup: x = (⨆ i. upper-principal (Y i)) .
  from chain have chain': chain (λi. upper-principal (Y i))
    by (rule upper-pd.ch2ch-principal)
  have prin-below-x: upper-principal (Y i) ⊑ x for i
    unfolding sup by (rule is-ub-thelub[OF chain'])
  with assms(3) have c ∈ C ⟹ upper-principal (Y i) ⊑ {c}♯ for i c
    unfolding upper-set-def using below-trans by blast
  with assms(1,2,4) have upper-principal (Y i) ⊑ {g}♯ for i
    by (intro upper-principal-below-chain[of C Y i g])
  with chain' have x ⊑ {g}♯ unfolding sup
    by (rule lub-below)
  then show ?thesis by (simp add: upper-set-def)
qed

lemma po-Zorn-lemma-min:
  fixes A :: 'a::po set
  assumes ⋀C. Chain C ⟹ C ⊆ A ⟹ ∃ l ∈ A. ∀ c ∈ C. l ⊑ c
  shows ∃ m ∈ A. ∀ a ∈ A. a ⊑ m ⟶ a = m
proof (rule predicate-Zorn)
  show partial-order-on A (relation-of (λa b. b ⊑ a) A)
    by (rule partial-order-on-relation-ofI) (auto intro: below-trans below-antisym)
next
  fix C assume C ∈ Zorn.Chains (relation-of (λa b. b ⊑ a) A)
```

```
  then have Chain C C ⊆ A
    by (auto simp: Zorn.Chains-def Chain-def relation-of-def)
  then have ∃ l ∈ A. ∀ c ∈ C. l ⊑ c by (rule assms)
  then show ∃ u ∈ A. ∀ a ∈ C. (λa b. b ⊑ a) a u by simp
qed

theorem upper-set-above-minimal:
  assumes p ∈ upper-set x
  shows ∃ y. y ∈ upper-set x ∧ y ⊑ p ∧ (∀ z ∈ upper-set x. z ⊑ y ⟶ z = y)
proof −
  define P where P = {q ∈ upper-set x. q ⊑ p}
  have ∃ m ∈ P. ∀ a ∈ P. a ⊑ m ⟶ a = m
  proof (rule po-Zorn-lemma-min)
    fix C assume C: Chain C C ⊆ P
    show ∃ l ∈ P. ∀ c ∈ C. l ⊑ c
    proof (cases C = {})
      case True
      from assms have p ∈ P by (simp add: P-def)
      with True show ?thesis by blast
    next
      case False
      have C-sub: C ⊆ upper-set x using C(2) by (auto simp: P-def)
      from bifinite-Chain-glb[OF C(1) False] obtain g where g: is-glb C g ..
    from C(1) False C-sub g have gA: g ∈ upper-set x by (rule upper-set-glb-closed)
      from False obtain c0 where c0: c0 ∈ C by blast
      with C(2) have c0 ⊑ p by (auto simp: P-def)
      with g c0 have g ⊑ p using is-glb-below below-trans by blast
      with gA have g ∈ P by (simp add: P-def)
      moreover have ∀ c ∈ C. g ⊑ c using g by (blast intro: is-glb-below)
      ultimately show ?thesis by blast
    qed
  qed
  then obtain m where m: m ∈ P and m-min: ∀ a ∈ P. a ⊑ m ⟶ a = m by
blast
  from m have mA: m ∈ upper-set x and mp: m ⊑ p by (auto simp: P-def)
  have z = m if z: z ∈ upper-set x z ⊑ m for z
  proof −
    from z(2) mp have z ⊑ p by (rule below-trans)
    with z(1) have z ∈ P by (simp add: P-def)
    with m-min z(2) show z = m by blast
  qed
  with mA mp show ?thesis by blast
qed

end
theory Cpo-Topology imports HOLCF HOL.Topological-Spaces begin

definition cpopen (A :: 'a::cpo set) ≡ (∀ x ∈ A. ∀ y. x ⊑ y ⟶ y ∈ A) ∧ (adm
(λx. x ∉ A))
```

lemma *cpopen-upper*: *cpopen A* $\Longrightarrow$ $x \in A \Longrightarrow x \sqsubseteq y \Longrightarrow y \in A$
by (*simp add*: *cpopen-def*)

lemma *cpopen-nadm*: *cpopen A* $\Longrightarrow$ *adm* $(\lambda x.\ x \notin A)$
by (*simp add*: *cpopen-def*)

lemma *cpopen-adm*: *cpopen A* $\Longrightarrow$ *adm* $(\lambda x.\ x \in A)$
by (*fast dest*!: *cpopen-upper intro*!: *adm-upward*)

lemma *adm-Inf*: **assumes** $\forall P \in U.$ *adm P* **shows** *adm* (*Inf U*)
proof (*rule admI*)
show $\forall i.$ *Inf U* (*Y i*) $\Longrightarrow$ *Inf U* ($\bigsqcup i.$ *Y i*) **if** *chain Y* **for** *Y*
proof
fix *P* **assume** *P*: $P \in U$
with *assms* **have** *adm P* **by** *simp*
assume $\forall i.$ *Inf U* (*Y i*)
with *P* **have** $\forall i.$ *P* (*Y i*) **by** *auto*
with *admD*[*OF* ‹*adm P*› *that*] **show** *P* ($\bigsqcup i.$ *Y i*) **by** *simp*
qed
qed

theorem *cpopen-Int*: *cpopen A* $\Longrightarrow$ *cpopen B* $\Longrightarrow$ *cpopen* $(A \cap B)$
unfolding *cpopen-def*
by *auto*

theorem *cpopen-Union*: $\forall P \in U.$ *cpopen P* $\Longrightarrow$ *cpopen* $(\bigcup U)$
unfolding *cpopen-def* **by** *fastforce*

interpretation *cpo-topology*: *topological-space cpopen*
proof
show *cpopen UNIV* **by** (*simp add*: *cpopen-def*)
qed (*fact cpopen-Int cpopen-Union*)+

hide-const *Topological-Spaces.compact*

lemma *cpopen-chain-cases*[*consumes 2*]:
assumes *cpopen*: *cpopen P* **and** *Y*: *chain Y*
obtains ($\bigsqcup i.$ *Y i*) $\notin P$ $\forall i.$ *Y i* $\notin P$ |
j **where** ($\bigsqcup i.$ *Y i*) $\in P$ $\forall i.$ *Y* $(i + j) \in P$
proof (*cases* ($\bigsqcup i.$ *Y i*) $\in P$)
case *1*: *False*
then have *2*: $\forall i.$ *Y i* $\notin P$
by (*auto intro*: *is-ub-thelub*[*OF Y*] *cpopen-upper*[*OF cpopen*])
from *1 2* **show** *?thesis* **by** (*rule that*(*1*))
next
case *1*: *True*
have $\neg\neg(\exists j.$ *Y j* $\in P)$

```
  proof
    assume ∄j. Y j ∈ P
    then have ∀j. Y j ∉ P
      by simp
    with cpopen-nadm[OF cpopen, THEN admD, OF Y]
    have (⨆ i. Y i) ∉ P by simp
    with 1 show False by contradiction
  qed
  then have 2: ∃j. Y j ∈ P by simp
  then obtain j where Y j ∈ P ..
  with cpopen-upper[OF cpopen] chain-mono[OF Y]
  have 2: Y (i + j) ∈ P for i
    by auto
  from 1 2 show ?thesis by (fast intro: that(2))
qed

end
theory BagPD
  imports HOLCF HOL−Library.Extended-Nat
begin
```

23 The bag powerdomain

Finite bags of compact elements with multiplicities in *enat*, where ∞ stands for an open-ended (countable) number of copies. A bag u is above a bag t when every copy of t goes to exactly one copy of u above it and every copy of u is hit: copies may merge but never split. Formally the order is witnessed by an *enat*-labelled plan between the supports (a transport plan with exact row sums and covered column sums, cf. Jones's splitting lemma with the column inequality reversed). The powerdomain is the ideal completion of this preorder, built like HOLCF's own powerdomains.

23.1 Splitting plans

The splitting lemmas are plain HOL, over arbitrary types.

```
default-sort type
```

A plan from a on A to b on B: row sums exactly a, column sums at least b, entries only towards nonzero demands.

```
definition split-plan ::
    'i set ⇒ 'j set ⇒ ('i ⇒ 'v::ordered-comm-monoid-add) ⇒ ('j ⇒ 'v) ⇒ ('i ⇒ 'j
⇒ 'v) ⇒ bool
  where split-plan A B a b m ⟷
    (∀x∈A. sum (m x) B = a x) ∧ (∀z∈B. b z ≤ (∑x∈A. m x z)) ∧
    (∀x z. m x z ≠ 0 ⟶ x ∈ A ∧ z ∈ B ∧ b z ≠ 0)
```

lemma *sum-nonzero-ex*:
fixes $f :: 'i \Rightarrow 'v$::*comm-monoid-add*
assumes *sum f A* $\neq$ *0*
shows $\exists x{\in}A.\ f\ x \neq 0$
using *assms* **by** (*blast elim*: *sum.not-neutral-contains-not-neutral*)

lemma *enat-sum-infty*:
fixes $f :: 'i \Rightarrow$ *enat*
shows *finite A* $\Longrightarrow$ $x \in A \Longrightarrow f\ x = \infty \Longrightarrow$ *sum f A* $= \infty$
using *member-le-sum*[*of x A f*] **by** *simp*

lemma *sum-enat-of-nat*: $(\sum i{\in}I.$ *enat* $(f\ i)) =$ *enat* (*sum f I*)
by (*induct I rule*: *infinite-finite-induct*) (*simp-all add*: *zero-enat-def*)

lemma *sum-update-add*:
fixes $f :: 'i \Rightarrow 'v$::*comm-monoid-add*
assumes *finite A* **and** $x \in A$
shows *sum* $(f(x := f\ x + c))\ A =$ *sum f A* $+ c$
using *sum.remove*[*OF assms*, *of* $f(x := f\ x + c)$] *sum.remove*[*OF assms*, *of f*]
by (*simp add*: *add-ac*)

One more unit in row x, sent to column w.

lemma *split-plan-bump*:
fixes $a :: 'i \Rightarrow$ *nat* **and** $b :: 'j \Rightarrow$ *nat*
assumes *plan*: *split-plan A B a' b' m* **and** *fin*: *finite A finite B*
and *xw*: $x \in A\ w \in B\ b\ w \neq 0$
and *a*: $a = a'(x := a'\ x + 1)$
and *bw*: $b\ w \leq b'\ w + 1$ **and** *bz*: $\bigwedge z.\ z \in B \Longrightarrow z \neq w \Longrightarrow b\ z \leq b'\ z$
and *bnz*: $\bigwedge z.\ b'\ z \neq 0 \Longrightarrow b\ z \neq 0$
shows *split-plan A B a b* $(m(x := (m\ x)(w := m\ x\ w + 1)))$
proof −
let $?m = m(x := (m\ x)(w := m\ x\ w + 1))$
have *row*: *sum* $(?m\ y)\ B = a\ y$ **if** $y \in A$ **for** y
using *plan that sum-update-add*[*OF fin*(*2*) *xw*(*2*), *of m x 1*]
by (*cases* $y = x$) (*simp-all add*: *split-plan-def a*)
have *col*: $b\ z \leq (\sum y{\in}A.\ ?m\ y\ z)$ **if** $z \in B$ **for** z
proof −
have $(\lambda y.\ ?m\ y\ z) = (\lambda y.\ m\ y\ z)(x := m\ x\ z + (\textit{if } z = w \textit{ then } 1 \textit{ else } 0))$
by *auto*
then have $(\sum y{\in}A.\ ?m\ y\ z) = (\sum y{\in}A.\ m\ y\ z) + (\textit{if } z = w \textit{ then } 1 \textit{ else } 0)$
by (*simp only*: *sum-update-add*[*OF fin*(*1*) *xw*(*1*), *of* $\lambda y.\ m\ y\ z$])
then show *?thesis*
using *plan that bw bz*[*of z*] **unfolding** *split-plan-def* **by** (*cases* $z = w$) *auto*
qed
show *?thesis*
using *row col plan xw bnz* **unfolding** *split-plan-def* **by** *auto*
qed

lemma *nat-split-plan*:

```
  fixes a :: 'i ⇒ nat and b :: 'j ⇒ nat
  assumes fin: finite A finite B
    and le: sum b B ≤ sum a A and nz: sum a A ≠ 0 ⟹ ∃ z∈B. b z ≠ 0
  shows ∃ m. split-plan A B a b m
  using le nz
proof (induct sum a A arbitrary: a b)
  case 0
  then show ?case
    using fin by (intro exI[of - λ- -. 0]) (simp add: split-plan-def)
next
  case (Suc n)
```

Take one unit off a nonempty row x and send it to a demanded column w; the demand at w drops by one when the total demand is tight.

```
  obtain x where x: x ∈ A a x ≠ 0
    using sum-nonzero-ex[of a A] Suc.hyps(2) by fastforce
  obtain w where w: w ∈ B b w ≠ 0
    using Suc.prems(2) Suc.hyps(2) by fastforce
  define a' where a' = a(x := a x − 1)
  define b' where b' = (if sum b B ≤ n then b else b(w := b w − 1))
  have a: a = a'(x := a' x + 1)
    using x(2) by (auto simp: a'-def)
  have n = sum a' A
    using sum-update-add[OF fin(1) x(1), of a' 1] Suc.hyps(2) a by simp
  moreover have sum b' B ≤ n ∧ (n ≠ 0 ⟶ (∃ z∈B. b' z ≠ 0))
  proof (cases sum b B ≤ n)
    case True
    then show ?thesis
      using w by (auto simp: b'-def)
  next
    case False
    then have b = b'(w := b' w + 1)
      using w(2) by (auto simp: b'-def)
    then have sum b' B = n
    using sum-update-add[OF fin(2) w(1), of b' 1] False Suc.prems(1) Suc.hyps(2)
by simp
    then show ?thesis
      using sum-nonzero-ex[of b' B] by auto
  qed
  ultimately obtain m where split-plan A B a' b' m
    using Suc.hyps(1) by blast
  then have split-plan A B a b (m(x := (m x)(w := m x w + 1)))
    by (rule split-plan-bump[where b = b, OF - fin x(1) w a])
      (auto simp: b'-def split: if-split-asm)
  then show ?case
    by blast
qed

lemma enat-split-plan:
```

```
  fixes a :: 'i ⇒ enat and b :: 'j ⇒ enat
  assumes fin: finite A finite B
    and le: sum b B ≤ sum a A and nz: sum a A ≠ 0 ⟹ ∃z∈B. b z ≠ 0
  shows ∃m. split-plan A B a b m
proof (cases ∃x∈A. a x = ∞)
  case True
```

An infinite row covers every demanded column; the finite rows all go to one of them.

```
  then obtain xi where xi: xi ∈ A a xi = ∞
    by blast
  then obtain z0 where z0: z0 ∈ B b z0 ≠ 0
    using nz enat-sum-infty[OF fin(1)] by fastforce
  define m where m x z = (if x ∈ A ∧ z ∈ B ∧ b z ≠ 0
    then (if a x = ∞ then ∞ else if z = z0 then a x else 0) else 0) for x z
  have r: sum (m x) B = a x if x ∈ A for x
  proof (cases a x = ∞)
    case True
    then show ?thesis
      using that z0 by (simp add: enat-sum-infty[OF fin(2) z0(1)] m-def)
  next
    case False
    have sum (m x) B = sum (λz. if z = z0 then a x else 0) B
      by (rule sum.cong) (auto simp: m-def that False z0)
    then show ?thesis
      using z0(1) fin(2) by simp
  qed
  have c: b z ≤ (∑x∈A. m x z) if z ∈ B for z
    using that xi by (cases b z = 0) (simp-all add: enat-sum-infty[OF fin(1) xi(1)]
m-def)
  have s: m x z ≠ 0 ⟶ x ∈ A ∧ z ∈ B ∧ b z ≠ 0 for x z
    by (simp add: m-def)
  show ?thesis
    unfolding split-plan-def using r c s by blast
next
  case False
```

All rows are finite, hence so are all demands: reduce to *nat*.

```
  define a' where a' x = the-enat (a x) for x
  define b' where b' z = the-enat (b z) for z
  have a: a x = enat (a' x) if x ∈ A for x
    using False that by (cases a x) (auto simp: a'-def)
  then have sa: sum a A = enat (sum a' A)
    by (simp add: sum-enat-of-nat[symmetric])
  have b: b z = enat (b' z) if z ∈ B for z
  proof −
    have b z ≤ sum b B
      by (rule member-le-sum[OF that - fin(2)]) simp
    then have b z ≤ enat (sum a' A)
```

using *le sa* **by** *simp*
then show *?thesis*
by (*cases b z*) (*auto simp*: *b'-def*)
qed
then have *sb*: *sum b B* = *enat* (*sum b' B*)
by (*simp add*: *sum-enat-of-nat*[*symmetric*])
have ∃ *m*. *split-plan A B a' b' m*
by (*rule nat-split-plan*[*OF fin*]) (*use le nz sa sb b* **in** ‹*auto simp*: *zero-enat-def*›)
then obtain *m* **where** *m*: *split-plan A B a' b' m*
by *blast*
have *split-plan A B a b* (λ*x z*. *enat* (*m x z*))
using *m a b* **by** (*auto simp*: *split-plan-def sum-enat-of-nat zero-enat-def*)
then show *?thesis*
by *blast*
qed

default-sort *bifinite*

23.2 The basis

A basis element is a nonempty finite-support multiplicity function on compact elements.

definition *bag-basis* =
{*f* :: *'a compact-basis* ⇒ *enat*. *finite* {*x*. *f x* ≠ *0*} ∧ {*x*. *f x* ≠ *0*} ≠ {}}

typedef *'a bag-basis* = *bag-basis* :: (*'a compact-basis* ⇒ *enat*) *set*
morphisms *bmult Abs-bag-basis*
by (*rule exI*[*of* - λ*x*. *if x* = *compact-bot then 1 else 0*]) (*simp add*: *bag-basis-def*)

definition *bsupp* :: *'a bag-basis* ⇒ *'a compact-basis set*
where *bsupp t* = {*x*. *bmult t x* ≠ *0*}

lemma *bsupp-iff*: *x* ∈ *bsupp t* ⟷ *bmult t x* ≠ *0*
by (*simp add*: *bsupp-def*)

lemma *finite-bsupp* [*simp*]: *finite* (*bsupp t*)
using *bmult* [*of t*] **by** (*simp add*: *bag-basis-def bsupp-def*)

lemma *bsupp-nonempty* [*simp*]: *bsupp t* ≠ {}
using *bmult* [*of t*] **by** (*simp add*: *bag-basis-def bsupp-def*)

lemma *Abs-bag-basis-inverse'*:
finite {*x*. *f x* ≠ *0*} ⟹ {*x*. *f x* ≠ *0*} ≠ {} ⟹ *bmult* (*Abs-bag-basis f*) = *f*
by (*simp add*: *Abs-bag-basis-inverse bag-basis-def*)

lemma *bag-basis-eqI*: (⋀*x*. *bmult t x* = *bmult u x*) ⟹ *t* = *u*
by (*simp add*: *bmult-inject* [*symmetric*] *fun-eq-iff*)

The basis is countable: encode the graph of the multiplicity function on its

support.

lemma *bag-basis-countable*: $\exists f::'a\ bag\text{-}basis \Rightarrow nat.\ inj\ f$
proof −
  obtain $g :: 'a\ compact\text{-}basis \Rightarrow nat$ **where** *g*: *inj g*
    using *compact-basis.countable* **..**
  define *G* **where** $G\ t = (\lambda x.\ to\text{-}nat\ (g\ x,\ bmult\ t\ x))$ ‘ *bsupp t* **for** $t :: 'a\ bag\text{-}basis$
  have *half*: *bmult u x* = *bmult t x* **if** *G t* = *G u* **and** $x \in bsupp\ t$ **for** *t u x*
  proof −
    have *to-nat* (*g x*, *bmult t x*) ∈ *G t*
      using *that*(*2*) **unfolding** *G-def* **by** (*rule imageI*)
    then have *to-nat* (*g x*, *bmult t x*) ∈ *G u*
      using *that*(*1*) **by** *simp*
    then obtain x' **where** $x' \in bsupp\ u\ g\ x' = g\ x\ bmult\ u\ x' = bmult\ t\ x$
      by (*auto simp*: *G-def*)
    then show *?thesis*
      using *g* **by** (*auto dest*: *injD*)
  qed
  have *inj* (λ*t*. *set-encode* (*G t*))
  proof (*rule injI*)
    fix *t u*
    assume *set-encode* (*G t*) = *set-encode* (*G u*)
    then have *G t* = *G u*
      by (*simp add*: *set-encode-eq G-def*)
    then show *t* = *u*
      using *half*[*of t u*] *half*[*of u t*] **by** (*intro bag-basis-eqI*) (*metis bsupp-iff*)
  qed
  then show *?thesis*
    by *blast*
qed

23.3 Basis constructors

definition $BUnit :: 'a\ compact\text{-}basis \Rightarrow 'a\ bag\text{-}basis$
  where *BUnit a* = *Abs-bag-basis* (λ*x*. *if x* = *a then 1 else 0*)

definition $BPlus :: 'a\ bag\text{-}basis \Rightarrow 'a\ bag\text{-}basis \Rightarrow 'a\ bag\text{-}basis$
  where *BPlus t u* = *Abs-bag-basis* (λ*x*. *bmult t x* + *bmult u x*)

definition $BOmega :: 'a\ bag\text{-}basis \Rightarrow 'a\ bag\text{-}basis$
  where *BOmega t* = *Abs-bag-basis* (λ*x*. *if bmult t x* = *0 then 0 else* ∞)

lemma *bmult-BUnit*: *bmult* (*BUnit a*) *x* = (*if x* = *a then 1 else 0*)
  unfolding *BUnit-def* **by** (*subst Abs-bag-basis-inverse*′) *simp-all*

lemma *bmult-BPlus* [*simp*]: *bmult* (*BPlus t u*) *x* = *bmult t x* + *bmult u x*
proof −
  have {*x*. *bmult t x* + *bmult u x* ≠ *0*} = *bsupp t* ∪ *bsupp u*
    by (*auto simp*: *bsupp-def*)
  then show *?thesis*

unfolding *BPlus-def* **by** (*subst Abs-bag-basis-inverse'*) *simp-all*
qed

lemma *bmult-BOmega*: *bmult* (*BOmega t*) *x* = (*if bmult t x* = *0 then 0 else* ∞)
proof −
have {*x*. (*if bmult t x* = *0 then 0 else* ∞) ≠ (*0*::*enat*)} = *bsupp t*
by (*auto simp*: *bsupp-def*)
then show *?thesis*
unfolding *BOmega-def* **by** (*subst Abs-bag-basis-inverse'*) *simp-all*
qed

lemma *bsupp-BUnit* [*simp*]: *bsupp* (*BUnit a*) = {*a*}
by (*auto simp*: *bsupp-def bmult-BUnit*)

lemma *bsupp-BPlus* [*simp*]: *bsupp* (*BPlus t u*) = *bsupp t* ∪ *bsupp u*
by (*auto simp*: *bsupp-def*)

lemma *bsupp-BOmega* [*simp*]: *bsupp* (*BOmega t*) = *bsupp t*
by (*auto simp*: *bsupp-def bmult-BOmega*)

lemma *BPlus-assoc*: *BPlus* (*BPlus t u*) *v* = *BPlus t* (*BPlus u v*)
by (*rule bag-basis-eqI*) (*simp add*: *add.assoc*)

lemma *BPlus-commute*: *BPlus t u* = *BPlus u t*
by (*rule bag-basis-eqI*) (*simp add*: *add.commute*)

lemma *BOmega-BOmega*: *BOmega* (*BOmega t*) = *BOmega t*
by (*rule bag-basis-eqI*) (*simp add*: *bmult-BOmega*)

lemma *BOmega-BPlus*: *BOmega* (*BPlus t u*) = *BPlus* (*BOmega t*) (*BOmega u*)
by (*rule bag-basis-eqI*) (*simp add*: *bmult-BOmega*)

lemma *BPlus-BOmega-absorb*: *BPlus* (*BOmega t*) *t* = *BOmega t*
by (*rule bag-basis-eqI*) (*simp add*: *bmult-BOmega*)

lemma *BPlus-BOmega-idem*: *BPlus* (*BOmega t*) (*BOmega t*) = *BOmega t*
by (*rule bag-basis-eqI*) (*simp add*: *bmult-BOmega*)

23.4 Basis preorder

A plan from t to u: a splitting plan between the supports whose entries only connect comparable elements.

definition *bag-plan* ::
'a bag-basis ⇒ *'a bag-basis* ⇒ (*'a compact-basis* ⇒ *'a compact-basis* ⇒ *enat*) ⇒ *bool*
where *bag-plan t u a* ⟷
(∀ *x y*. *a x y* ≠ *0* ⟶ *x* ⊑ *y*) ∧ *split-plan* (*bsupp t*) (*bsupp u*) (*bmult t*) (*bmult u*) *a*

definition *bag-le* :: *'a bag-basis* ⇒ *'a bag-basis* ⇒ *bool* (**infix** ‹$\leq_b$› *50*)
where $t \leq_b u \longleftrightarrow (\exists\, a.\ \textit{bag-plan}\ t\ u\ a)$

lemma *bag-planI*:
assumes $\bigwedge x\ y.\ a\ x\ y \neq 0 \Longrightarrow x \sqsubseteq y \wedge x \in \textit{bsupp}\ t \wedge y \in \textit{bsupp}\ u$
and $\bigwedge x.\ x \in \textit{bsupp}\ t \Longrightarrow \textit{sum}\ (a\ x)\ (\textit{bsupp}\ u) = \textit{bmult}\ t\ x$
and $\bigwedge y.\ y \in \textit{bsupp}\ u \Longrightarrow \textit{bmult}\ u\ y \leq (\sum x{\in}\textit{bsupp}\ t.\ a\ x\ y)$
shows *bag-plan t u a*
using *assms* **by** (*auto simp*: *bag-plan-def split-plan-def bsupp-iff*)

lemmas *bag-leI* = *bag-planI* [*THEN bag-le-def* [*THEN iffD2*, *OF exI*]]

lemma *bag-planD*:
assumes *bag-plan t u a*
shows *bag-plan-below*: $a\ x\ y \neq 0 \Longrightarrow x \sqsubseteq y$
and *bag-plan-supp*: $a\ x\ y \neq 0 \Longrightarrow x \in \textit{bsupp}\ t \wedge y \in \textit{bsupp}\ u$
and *bag-plan-row*: $\textit{sum}\ (a\ x)\ (\textit{bsupp}\ u) = \textit{bmult}\ t\ x$
and *bag-plan-col*: $\textit{bmult}\ u\ y \leq (\sum x{\in}\textit{bsupp}\ t.\ a\ x\ y)$
proof −
from *assms* **show** $a\ x\ y \neq 0 \Longrightarrow x \sqsubseteq y\ a\ x\ y \neq 0 \Longrightarrow x \in \textit{bsupp}\ t \wedge y \in \textit{bsupp}\ u$
by (*auto simp*: *bag-plan-def split-plan-def*)
from *assms* **show** $\textit{sum}\ (a\ x)\ (\textit{bsupp}\ u) = \textit{bmult}\ t\ x$
by (*cases* $x \in \textit{bsupp}\ t$)
(*auto simp*: *bag-plan-def split-plan-def bsupp-iff intro*: *sum.neutral*)
from *assms* **show** $\textit{bmult}\ u\ y \leq (\sum x{\in}\textit{bsupp}\ t.\ a\ x\ y)$
by (*cases* $y \in \textit{bsupp}\ u$) (*auto simp*: *bag-plan-def split-plan-def bsupp-iff*)
qed

Row and column sums may be taken over any finite superset of the supports.

lemma *bag-plan-row-on*:
assumes *bag-plan t u a* **and** *finite Y* **and** $\textit{bsupp}\ u \subseteq Y$
shows $\textit{sum}\ (a\ x)\ Y = \textit{bmult}\ t\ x$
proof −
have $\textit{sum}\ (a\ x)\ Y = \textit{sum}\ (a\ x)\ (\textit{bsupp}\ u)$
using *assms bag-plan-supp*[*OF assms*(*1*)] **by** (*intro sum.mono-neutral-right*) *auto*
then show *?thesis*
using *bag-plan-row*[*OF assms*(*1*)] **by** *simp*
qed

lemma *bag-plan-col-on*:
assumes *bag-plan t u a* **and** *finite X* **and** $\textit{bsupp}\ t \subseteq X$
shows $\textit{bmult}\ u\ y \leq (\sum x{\in}X.\ a\ x\ y)$
proof −
have $(\sum x{\in}X.\ a\ x\ y) = (\sum x{\in}\textit{bsupp}\ t.\ a\ x\ y)$
using *assms bag-plan-supp*[*OF assms*(*1*)] **by** (*intro sum.mono-neutral-right*) *auto*
then show *?thesis*

using *bag-plan-col*[*OF assms*(*1*)] **by** *simp*
qed

Every element of either support takes part in the plan.

lemma *bag-plan-row-ex*:
assumes *bag-plan t u a* **and** $x \in$ *bsupp t*
obtains *y* **where** $y \in$ *bsupp u* **and** $a\ x\ y \neq 0$
using *sum-nonzero-ex*[*of a x bsupp u*] *bag-plan-row*[*OF assms*(*1*), *of x*] *assms*(*2*)
by (*auto simp*: *bsupp-iff*)

lemma *bag-plan-col-ex*:
assumes *bag-plan t u a* **and** $y \in$ *bsupp u*
obtains *x* **where** $x \in$ *bsupp t* **and** $a\ x\ y \neq 0$
proof −
have $(\sum x{\in}bsupp\ t.\ a\ x\ y) \neq 0$
using *bag-plan-col*[*OF assms*(*1*), *of y*] *assms*(*2*) **by** (*metis bsupp-iff le-zero-eq*)
then show *?thesis*
using *sum-nonzero-ex that* **by** *blast*
qed

lemma *bag-le-refl* [*simp*]: $t \leq_b t$
by (*rule bag-leI*[**where** $a = \lambda x\ y.$ *if* $x = y$ *then bmult t x else 0*])
(*auto simp*: *bsupp-iff split*: *if-split-asm*)

Transitivity: split the copies flowing through each middle element y along its outgoing row, and add up the pieces.

lemma *bag-le-trans*:
assumes $t \leq_b u$ **and** $u \leq_b v$
shows $t \leq_b v$
proof −
from *assms* **obtain** *a b* **where** *a*: *bag-plan t u a* **and** *b*: *bag-plan u v b*
unfolding *bag-le-def* **by** *blast*
have $\exists m.$ *split-plan* (*bsupp t*) (*bsupp v*) $(\lambda x.\ a\ x\ y)$ (*b y*) *m* **if** $y \in$ *bsupp u* **for** *y*
proof (*rule enat-split-plan*)
show *sum* (*b y*) (*bsupp v*) $\leq (\sum x{\in}bsupp\ t.\ a\ x\ y)$
using *bag-plan-row*[*OF b*] *bag-plan-col*[*OF a*] **by** *simp*
show $\exists z{\in}bsupp\ v.\ b\ y\ z \neq 0$
using *bag-plan-row-ex*[*OF b that*] **by** *blast*
qed *simp-all*
then obtain *M* **where** $\bigwedge y.\ y \in bsupp\ u \Longrightarrow$ *split-plan* (*bsupp t*) (*bsupp v*) $(\lambda x.\ a\ x\ y)$ (*b y*) (*M y*)
by *metis*
then have *Mrow*: $\bigwedge x\ y.\ y \in bsupp\ u \Longrightarrow x \in bsupp\ t \Longrightarrow sum\ (M\ y\ x)\ (bsupp\ v) = a\ x\ y$
and *Mcol*: $\bigwedge y\ z.\ y \in bsupp\ u \Longrightarrow z \in bsupp\ v \Longrightarrow b\ y\ z \leq (\sum x{\in}bsupp\ t.\ M\ y\ x\ z)$
and *Msupp*: $\bigwedge x\ y\ z.\ y \in bsupp\ u \Longrightarrow M\ y\ x\ z \neq 0 \Longrightarrow x \in bsupp\ t \wedge z \in bsupp\ v \wedge b\ y\ z \neq 0$
unfolding *split-plan-def* **by** *blast*+

define c **where** $c\ x\ z = (\sum y{\in}bsupp\ u.\ M\ y\ x\ z)$ **for** $x\ z$
show *?thesis*
proof (*rule bag-leI*[*of c*])
fix $x\ z$
assume $c\ x\ z \neq 0$
then obtain y **where** y: $y \in bsupp\ u\ M\ y\ x\ z \neq 0$
using *sum-nonzero-ex* **unfolding** *c-def* **by** *blast*
then have xz: $x \in bsupp\ t\ z \in bsupp\ v\ b\ y\ z \neq 0$
using *Msupp* **by** *blast+*
have $a\ x\ y \neq 0$
using *Mrow*[*OF y*(*1*) *xz*(*1*)] *y*(*2*) *xz*(*2*) **by** *auto*
then have $x \sqsubseteq y$
by (*rule bag-plan-below*[*OF a*])
also have $y \sqsubseteq z$
using *xz*(*3*) **by** (*rule bag-plan-below*[*OF b*])
finally show $x \sqsubseteq z \wedge x \in bsupp\ t \wedge z \in bsupp\ v$
using *xz* **by** *blast*
next
fix x
assume x: $x \in bsupp\ t$
have $sum\ (c\ x)\ (bsupp\ v) = (\sum y{\in}bsupp\ u.\ sum\ (M\ y\ x)\ (bsupp\ v))$
unfolding *c-def* **by** (*rule sum.swap*)
also have $\ldots = bmult\ t\ x$
using *Mrow x bag-plan-row*[*OF a*] **by** *simp*
finally show $sum\ (c\ x)\ (bsupp\ v) = bmult\ t\ x$ **.**
next
fix z
assume z: $z \in bsupp\ v$
have $bmult\ v\ z \leq (\sum y{\in}bsupp\ u.\ b\ y\ z)$
by (*rule bag-plan-col*[*OF b*])
also have $\ldots \leq (\sum y{\in}bsupp\ u.\ \sum x{\in}bsupp\ t.\ M\ y\ x\ z)$
using *Mcol z* **by** (*intro sum-mono*) *simp*
also have $\ldots = (\sum x{\in}bsupp\ t.\ c\ x\ z)$
unfolding *c-def* **by** (*rule sum.swap*)
finally show $bmult\ v\ z \leq (\sum x{\in}bsupp\ t.\ c\ x\ z)$ **.**
qed
qed

interpretation *bag-le*: *preorder bag-le*
by (*rule preorder.intro*, *rule bag-le-refl*, *rule bag-le-trans*)

23.5 Basic plans

Below a bag, the supports are related in the Egli-Milner order.

lemma *bag-le-supp*:
assumes $t \leq_b u$
shows *bag-le-lower*: $x \in bsupp\ t \Longrightarrow \exists\, y{\in}bsupp\ u.\ x \sqsubseteq y$
and *bag-le-upper*: $y \in bsupp\ u \Longrightarrow \exists\, x{\in}bsupp\ t.\ x \sqsubseteq y$
proof −

obtain *a* **where** *a*: *bag-plan t u a*
using *assms* **unfolding** *bag-le-def* **by** *blast*
show $x \in bsupp\ t \Longrightarrow \exists\, y{\in}bsupp\ u.\ x \sqsubseteq y$
by (*metis a bag-plan-row-ex bag-plan-below*)
show $y \in bsupp\ u \Longrightarrow \exists\, x{\in}bsupp\ t.\ x \sqsubseteq y$
by (*metis a bag-plan-col-ex bag-plan-below*)
qed

Conversely, a bag with only infinite multiplicities is below every bag whose support is Egli-Milner above its own: all entries of the plan can be infinite.

lemma *bag-le-omegaI*:
assumes *inf*: $\bigwedge x.\ x \in bsupp\ t \Longrightarrow bmult\ t\ x = \infty$
and *lower*: $\bigwedge x.\ x \in bsupp\ t \Longrightarrow \exists\, y{\in}bsupp\ u.\ x \sqsubseteq y$
and *upper*: $\bigwedge y.\ y \in bsupp\ u \Longrightarrow \exists\, x{\in}bsupp\ t.\ x \sqsubseteq y$
shows $t \leq_b u$
proof −
define *p* **where** $p\ x\ y = (if\ x \in bsupp\ t \wedge y \in bsupp\ u \wedge x \sqsubseteq y\ then\ \infty\ else\ 0 :: enat)$
for *x y*
show *?thesis*
proof (*rule bag-leI*[*of p*])
fix *x*
assume *x*: $x \in bsupp\ t$
then obtain *y* **where** $y \in bsupp\ u\ x \sqsubseteq y$
using *lower* **by** *blast*
then show $sum\ (p\ x)\ (bsupp\ u) = bmult\ t\ x$
using *x inf* **by** (*simp add*: *enat-sum-infty*[**where** $x = y$] *p-def*)
next
fix *y*
assume *y*: $y \in bsupp\ u$
then obtain *x* **where** $x \in bsupp\ t\ x \sqsubseteq y$
using *upper* **by** *blast*
then show $bmult\ u\ y \leq (\sum x{\in}bsupp\ t.\ p\ x\ y)$
using *y* **by** (*simp add*: *enat-sum-infty*[**where** $x = x$] *p-def*)
qed (*simp add*: *p-def split*: *if-split-asm*)
qed

lemma *bag-le-minimal* [*simp*]: $BOmega\ (BUnit\ compact\text{-}bot) \leq_b t$
by (*rule bag-le-omegaI*) (*auto simp*: *bmult-BOmega bmult-BUnit ex-in-conv*)

lemma *BUnit-mono*: $a \sqsubseteq b \Longrightarrow BUnit\ a \leq_b BUnit\ b$
by (*rule bag-leI*[**where** $a = \lambda x\ y.\ if\ x = a \wedge y = b\ then\ 1\ else\ 0$])
(*simp-all add*: *bmult-BUnit split*: *if-split-asm*)

lemma *BUnit-le-iff* [*simp*]: $BUnit\ a \leq_b BUnit\ b \longleftrightarrow a \sqsubseteq b$
using *bag-le-upper*[*of BUnit a BUnit b b*] *BUnit-mono* **by** *auto*

lemma *BPlus-mono*:
assumes $s \leq_b t$ **and** $u \leq_b v$

shows *BPlus s u* $\leq_b$ *BPlus t v*
proof −
from *assms* **obtain** *a b* **where** *a*: *bag-plan s t a* **and** *b*: *bag-plan u v b*
unfolding *bag-le-def* **by** *blast*
show *?thesis*
proof (*rule bag-leI*[**where** $a = \lambda x\ y.\ a\ x\ y + b\ x\ y$])
fix *x y*
assume $a\ x\ y + b\ x\ y \neq 0$
then show $x \sqsubseteq y \wedge x \in bsupp\ (BPlus\ s\ u) \wedge y \in bsupp\ (BPlus\ t\ v)$
using *bag-planD*(*1*,*2*)[*OF a*] *bag-planD*(*1*,*2*)[*OF b*] **by** *auto*
next
fix *x*
show $sum\ (\lambda y.\ a\ x\ y + b\ x\ y)\ (bsupp\ (BPlus\ t\ v)) = bmult\ (BPlus\ s\ u)\ x$
using *bag-plan-row-on*[*OF a*] *bag-plan-row-on*[*OF b*] **by** (*simp add*: *sum.distrib*)
next
fix *y*
show $bmult\ (BPlus\ t\ v)\ y \leq (\sum x{\in}bsupp\ (BPlus\ s\ u).\ a\ x\ y + b\ x\ y)$
using *bag-plan-col-on*[*OF a*] *bag-plan-col-on*[*OF b*] **by** (*simp add*: *sum.distrib add-mono*)
qed
qed

lemma *BOmega-mono*: $t \leq_b u \Longrightarrow BOmega\ t \leq_b BOmega\ u$
by (*rule bag-le-omegaI*) (*auto simp*: *bmult-BOmega bsupp-iff dest*: *bag-le-supp*)

lemma *BPlus-contract*: $BPlus\ t\ t \leq_b t$
by (*rule bag-leI*[**where** $a = \lambda x\ y.\ if\ x = y\ then\ bmult\ t\ x + bmult\ t\ x\ else\ 0$])
(*auto simp*: *bsupp-iff split*: *if-split-asm*)

lemma *BOmega-le*: $BOmega\ t \leq_b t$
by (*rule bag-le-omegaI*) (*auto simp*: *bmult-BOmega bsupp-iff*)

23.6 Labelled families

A finite family of compact elements with multiplicities, collapsed to a bag. A plan between the index sets pushes forward to a plan between the bags, so operations that emit one element per index (maps, joins) can be shown monotone on indices.

definition *BFamily* ::
$'i$::*type set* ⇒ ($'i$ ⇒ $'a$ *compact-basis*) ⇒ ($'i$ ⇒ *enat*) ⇒ $'a$ *bag-basis*
where $BFamily\ I\ f\ m = Abs\text{-}bag\text{-}basis\ (\lambda x.\ \sum i{\in}\{i{\in}I.\ f\ i = x\}.\ m\ i)$

lemma *bmult-BFamily*:
assumes *finite I* **and** $\exists\, i{\in}I.\ m\ i \neq 0$
shows $bmult\ (BFamily\ I\ f\ m)\ x = (\sum i{\in}\{i{\in}I.\ f\ i = x\}.\ m\ i)$
proof −
have $\{x.\ (\sum i{\in}\{i{\in}I.\ f\ i = x\}.\ m\ i) \neq 0\} = f\ `\ \{i{\in}I.\ m\ i \neq 0\}$
using *assms*(*1*) **by** *auto*

then show *?thesis*
unfolding *BFamily-def* **using** *assms* **by** (*subst Abs-bag-basis-inverse'*) *auto*
qed

lemma *bsupp-BFamily*:
assumes *finite I* **and** $\exists\, i{\in}I.\ m\ i \neq 0$
shows *bsupp* (*BFamily I f m*) = *f* ‘ $\{i{\in}I.\ m\ i \neq 0\}$
unfolding *bsupp-def bmult-BFamily*[*OF assms*] **using** *assms*(*1*) **by** *auto*

Indices of multiplicity zero, and the naming of indices, do not matter.

lemma *BFamily-nonzero*: *finite I* $\Longrightarrow$ *BFamily* $\{i{\in}I.\ m\ i \neq 0\}$ *f m* = *BFamily I f m*
unfolding *BFamily-def* **by** (*rule arg-cong*[**where** *f* = *Abs-bag-basis*], *rule ext*, *rule sum.mono-neutral-left*) *auto*

lemma *BFamily-reindex*:
assumes *bij-betw h I J*
shows *BFamily I* ($\lambda i.\ f\ (h\ i)$) ($\lambda i.\ m\ (h\ i)$) = *BFamily J f m*
proof −
have *bij-betw h* $\{i{\in}I.\ f\ (h\ i) = x\}$ $\{j{\in}J.\ f\ j = x\}$ **for** *x*
using *assms* **unfolding** *bij-betw-def inj-on-def* **by** *auto*
then have $(\sum i{\in}\{i{\in}I.\ f\ (h\ i) = x\}.\ m\ (h\ i)) = (\sum j{\in}\{j{\in}J.\ f\ j = x\}.\ m\ j)$ **for** *x*
by (*rule sum.reindex-bij-betw*)
then show *?thesis*
unfolding *BFamily-def* **by** *simp*
qed

Pushing an exact plan between index sets of nonzero multiplicity forward along the labels.

lemma *BFamily-le-exact*:
fixes *f* :: *'i*::*type* ⇒ *'a compact-basis* **and** *g* :: *'j*::*type* ⇒ *'a compact-basis*
assumes *fin*: *finite I finite J* **and** *ne*: $I \neq \{\}$
and *nzI*: $\bigwedge i.\ i \in I \Longrightarrow m\ i \neq 0$ **and** *nzJ*: $\bigwedge j.\ j \in J \Longrightarrow n\ j \neq 0$
and *below*: $\bigwedge i\ j.\ i \in I \Longrightarrow j \in J \Longrightarrow q\ i\ j \neq 0 \Longrightarrow f\ i \sqsubseteq g\ j$
and *row*: $\bigwedge i.\ i \in I \Longrightarrow sum\ (q\ i)\ J = m\ i$
and *col*: $\bigwedge j.\ j \in J \Longrightarrow n\ j \leq (\sum i{\in}I.\ q\ i\ j)$
shows *BFamily I f m* $\leq_b$ *BFamily J g n*
proof −
have *neI*: $\exists\, i{\in}I.\ m\ i \neq 0$
using *ne nzI* **by** *blast*
have *neJ*: $\exists\, j{\in}J.\ n\ j \neq 0$
by (*metis nzJ row neI sum-nonzero-ex*)
define *a* **where** $a\ x\ y = (\sum i{\in}\{i{\in}I.\ f\ i = x\}.\ \sum j{\in}\{j{\in}J.\ g\ j = y\}.\ q\ i\ j)$ **for** *x y*
have *sI*: *bsupp* (*BFamily I f m*) = *f* ‘ *I*
using *bsupp-BFamily*[*OF fin*(*1*) *neI*] *nzI* **by** *auto*
have *sJ*: *bsupp* (*BFamily J g n*) = *g* ‘ *J*
using *bsupp-BFamily*[*OF fin*(*2*) *neJ*] *nzJ* **by** *auto*
show *?thesis*
proof (*rule bag-leI*[*of a*])

```
    fix x y
    assume a x y ≠ 0
    then obtain i j where i ∈ I f i = x j ∈ J g j = y q i j ≠ 0
      unfolding a-def by (blast dest: sum-nonzero-ex)
    then show x ⊑ y ∧ x ∈ bsupp (BFamily I f m) ∧ y ∈ bsupp (BFamily J g n)
      using below sI sJ by blast
  next
    fix x
    have sum (a x) (g ‘ J) = (∑ i∈{i∈I. f i = x}. ∑ y∈g ‘ J. ∑ j∈{j∈J. g j =
y}. q i j)
      unfolding a-def by (rule sum.swap)
    also have ... = (∑ i∈{i∈I. f i = x}. sum (q i) J)
      using fin by (simp add: sum.group)
    also have ... = bmult (BFamily I f m) x
      using row fin neI by (simp add: bmult-BFamily)
    finally show sum (a x) (bsupp (BFamily J g n)) = bmult (BFamily I f m) x
      by (simp add: sJ)
  next
    fix y
    have bmult (BFamily J g n) y ≤ (∑ j∈{j∈J. g j = y}. ∑ i∈I. q i j)
      unfolding bmult-BFamily[OF fin(2) neJ] using col by (intro sum-mono)
blast
    also have ... = (∑ i∈I. ∑ j∈{j∈J. g j = y}. q i j)
      by (rule sum.swap)
    also have ... = (∑ x∈f ‘ I. a x y)
      unfolding a-def using fin by (simp add: sum.group)
    finally show bmult (BFamily J g n) y ≤ (∑ x∈bsupp (BFamily I f m). a x y)
      by (simp add: sI)
  qed
qed

lemma enat-add-diff-le: a ≤ b ⟹ a + (b − a) = (b::enat)
  by (cases a; cases b) simp-all
```

The general form: the plan may send less than a source holds, provided every source has somewhere to go (the Hoare half), and may touch indices of multiplicity zero.

```
lemma BFamily-le:
  fixes f :: 'i::type ⇒ 'a compact-basis and g :: 'j::type ⇒ 'a compact-basis
  assumes fin: finite I finite J and ne: ∃ i∈I. m i ≠ 0
    and below: ⋀i j. i ∈ I ⟹ j ∈ J ⟹ p i j ≠ 0 ⟹ f i ⊑ g j
    and row: ⋀i. i ∈ I ⟹ sum (p i) J ≤ m i
    and col: ⋀j. j ∈ J ⟹ n j ≤ (∑ i∈I. p i j)
    and lower: ⋀i. i ∈ I ⟹ m i ≠ 0 ⟹ ∃ j∈J. n j ≠ 0 ∧ f i ⊑ g j
  shows BFamily I f m ≤b BFamily J g n
proof −
  define I' where I' = {i∈I. m i ≠ 0}
  define J' where J' = {j∈J. n j ≠ 0}
  obtain s where s: ⋀i. i ∈ I' ⟹ s i ∈ J' ∧ f i ⊑ g (s i)
```

```
  using lower unfolding I'-def J'-def by (metis (mono-tags, lifting) mem-Collect-eq)
 define q where q i j = p i j + (if j = s i then m i − sum (p i) J' else 0) for i j
 have fin': finite I' finite J'
  using fin by (simp-all add: I'-def J'-def)
 have pJ': sum (p i) J' ≤ m i if i ∈ I for i
  using row[OF that] sum-mono2[OF fin(2), of J' p i] by (auto simp: J'-def)
 have p0: p i j = 0 if i ∈ I j ∈ J m i = 0 for i j
  using row[OF that(1)] member-le-sum[OF that(2), of p i] fin(2) that(3) by
simp
 have BFamily I' f m ≤b BFamily J' g n
 proof (rule BFamily-le-exact[where q = q])
  fix i
  assume i: i ∈ I'
  have sum (q i) J' = sum (p i) J' + (m i − sum (p i) J')
   using s[OF i] fin' by (simp add: q-def sum.distrib)
  also have ... = m i
   using pJ' i by (simp add: I'-def enat-add-diff-le)
  finally show sum (q i) J' = m i .
 next
  fix i j
  assume i ∈ I' j ∈ J' q i j ≠ 0
  then show f i ⊑ g j
   using below s by (auto simp: q-def I'-def J'-def split: if-split-asm)
 next
  fix j
  assume j: j ∈ J'
  have n j ≤ (∑ i∈I. p i j)
   using col j by (simp add: J'-def)
  also have ... = (∑ i∈I'. p i j)
   using fin j p0 by (intro sum.mono-neutral-right) (auto simp: I'-def J'-def)
  also have ... ≤ (∑ i∈I'. q i j)
   by (intro sum-mono) (simp add: q-def)
  finally show n j ≤ (∑ i∈I'. q i j) .
 qed (use fin' ne in ‹auto simp: I'-def J'-def›)
 then show ?thesis
  using fin by (simp add: I'-def J'-def BFamily-nonzero)
qed
```

The mass of a bag on a predicate. Refinement moves mass upwards, so the mass on an upward region can only shrink, measured against the mass on any region that contains everything below it.

```
definition bag-mass :: 'a bag-basis ⇒ ('a compact-basis ⇒ bool) ⇒ enat
 where bag-mass t P = (∑ x∈{x∈bsupp t. P x}. bmult t x)

lemma bag-mass-nonzero-iff: bag-mass t P ≠ 0 ⟷ (∃ x∈bsupp t. P x)
proof −
 have bag-mass t P = 0 ⟷ (∀ x∈{x∈bsupp t. P x}. bmult t x = 0)
  unfolding bag-mass-def by (rule sum-eq-0-iff) simp
 then show ?thesis
```

by (*auto simp*: *bsupp-iff*)
qed

lemma *bag-mass-BUnit*: *bag-mass* (*BUnit a*) *P* = (*if P a then 1 else 0*)
proof −
have {*x*∈*bsupp* (*BUnit a*). *P x*} = (*if P a then* {*a*} *else* {})
by *auto*
then show *?thesis*
by (*simp add*: *bag-mass-def bmult-BUnit*)
qed

lemma *bmult-BFamily-sum*:
finite I ⟹ ∃ *i*∈*I*. *m i* ≠ *0* ⟹ *bmult* (*BFamily I f m*) *x* = (∑ *i*∈*I*. *if f i* = *x then m i else 0*)
by (*simp add*: *bmult-BFamily sum.inter-filter*)

lemma *bag-mass-cong*: (⋀*x*. *x* ∈ *bsupp t* ⟹ *P x* = *Q x*) ⟹ *bag-mass t P* = *bag-mass t Q*
unfolding *bag-mass-def* **by** (*rule sum.cong*) *auto*

lemma *bag-mass-disj*:
(⋀*x*. *P x* ⟹ *Q x* ⟹ *False*) ⟹ *bag-mass t* (λ*x*. *P x* ∨ *Q x*) = *bag-mass t P* + *bag-mass t Q*
unfolding *bag-mass-def* **by** (*subst sum.union-disjoint*[*symmetric*]) (*auto intro*: *sum.cong*)

lemma *bag-le-mass*:
assumes *t* ≤$_b$ *t′* **and** *down*: ⋀*x x′*. *x* ⊑ *x′* ⟹ *Q′ x′* ⟹ *Q x*
shows *bag-mass t′ Q′* ≤ *bag-mass t Q*
proof −
obtain *a* **where** *a*: *bag-plan t t′ a*
using *assms*(*1*) **unfolding** *bag-le-def* **by** *blast*
let *?S′* = {*x′*∈*bsupp t′*. *Q′ x′*}
have *bag-mass t′ Q′* ≤ (∑ *x′*∈*?S′*. ∑ *x*∈*bsupp t*. *a x x′*)
unfolding *bag-mass-def* **by** (*intro sum-mono bag-plan-col*[*OF a*])
also have ... = (∑ *x*∈*bsupp t*. ∑ *x′*∈*?S′*. *a x x′*)
by (*rule sum.swap*)
also have ... = (∑ *x*∈{*x*∈*bsupp t*. *Q x*}. ∑ *x′*∈*?S′*. *a x x′*)
by (*rule sum.mono-neutral-right*) (*auto intro*!: *sum.neutral dest*: *bag-plan-below*[*OF a*] *down*)
also have ... ≤ (∑ *x*∈{*x*∈*bsupp t*. *Q x*}. ∑ *x′*∈*bsupp t′*. *a x x′*)
by (*intro sum-mono sum-mono2*) *auto*
also have ... = *bag-mass t Q*
unfolding *bag-mass-def* **using** *bag-plan-row*[*OF a*] **by** *simp*
finally show *?thesis* **.**
qed

A plain congruence for families.

lemma *BFamily-cong*:

$(\bigwedge i.\ i \in I \Longrightarrow f\ i = f'\ i) \Longrightarrow (\bigwedge i.\ i \in I \Longrightarrow m\ i = m'\ i) \Longrightarrow BFamily\ I\ f\ m = BFamily\ I\ f'\ m'$
unfolding *BFamily-def* **by** (*rule arg-cong*[**where** $f = Abs\text{-}bag\text{-}basis$], *rule ext*, *rule sum.cong*) *auto*

23.7 Type definition

typedef $'a\ bag\text{-}pd = \{S::'a\ bag\text{-}basis\ set.\ bag\text{-}le.ideal\ S\}$
by (*rule bag-le.ex-ideal*)

instantiation *bag-pd* :: (*bifinite*) *below*
begin

definition
$x \sqsubseteq y \longleftrightarrow Rep\text{-}bag\text{-}pd\ x \subseteq Rep\text{-}bag\text{-}pd\ y$

instance ..
end

instance *bag-pd* :: (*bifinite*) *po*
using *type-definition-bag-pd below-bag-pd-def*
by (*rule bag-le.typedef-ideal-po*)

instance *bag-pd* :: (*bifinite*) *cpo*
using *type-definition-bag-pd below-bag-pd-def*
by (*rule bag-le.typedef-ideal-cpo*)

definition *bag-principal* :: $'a\ bag\text{-}basis \Rightarrow 'a\ bag\text{-}pd$
where $bag\text{-}principal\ t = Abs\text{-}bag\text{-}pd\ \{u.\ u \leq_b t\}$

interpretation *bag-pd*: *ideal-completion bag-le bag-principal Rep-bag-pd*
using *type-definition-bag-pd below-bag-pd-def*
using *bag-principal-def bag-basis-countable*
by (*rule bag-le.typedef-ideal-completion*)

The bag powerdomain is pointed: the bottom is an open-ended number of unknown rows.

lemma *bag-pd-minimal*: $bag\text{-}principal\ (BOmega\ (BUnit\ compact\text{-}bot)) \sqsubseteq ys$
by (*induct ys rule*: *bag-pd.principal-induct*) *simp-all*

instance *bag-pd* :: (*bifinite*) *pcpo*
by *intro-classes* (*fast intro*: *bag-pd-minimal*)

lemma *inst-bag-pd-pcpo*: $\bot = bag\text{-}principal\ (BOmega\ (BUnit\ compact\text{-}bot))$
by (*rule bag-pd-minimal* [*THEN bottomI*, *symmetric*])

23.8 Unit, plus and omega

definition *bag-unit* :: $'a \to 'a\ bag\text{-}pd$

where *bag-unit* = *compact-basis.extension* (λ*a*. *bag-principal* (*BUnit a*))

definition *bag-plus* :: *'a bag-pd* → *'a bag-pd* → *'a bag-pd*
where *bag-plus* = *bag-pd.extension* (λ*t*. *bag-pd.extension* (λ*u*. *bag-principal* (*BPlus t u*)))

definition *bag-omega* :: *'a bag-pd* → *'a bag-pd*
where *bag-omega* = *bag-pd.extension* (λ*t*. *bag-principal* (*BOmega t*))

abbreviation *bag-add* :: *'a bag-pd* ⇒ *'a bag-pd* ⇒ *'a bag-pd* (**infixl** ‹∪$_b$› *65*)
where *xs* ∪$_b$ *ys* == *bag-plus*·*xs*·*ys*

lemma *bag-unit-Rep-compact-basis* [*simp*]: *bag-unit*·(*Rep-compact-basis a*) = *bag-principal* (*BUnit a*)
unfolding *bag-unit-def* **by** (*simp add*: *compact-basis.extension-principal BUnit-mono*)

lemma *bag-plus-principal* [*simp*]:
bag-principal t ∪$_b$ *bag-principal u* = *bag-principal* (*BPlus t u*)
unfolding *bag-plus-def*
by (*simp add*: *bag-pd.extension-principal bag-pd.extension-mono BPlus-mono*)

lemma *bag-omega-principal* [*simp*]: *bag-omega*·(*bag-principal t*) = *bag-principal* (*BOmega t*)
unfolding *bag-omega-def* **by** (*simp add*: *bag-pd.extension-principal BOmega-mono*)

interpretation *bag-add*: *abel-semigroup bag-add*
proof
fix *xs ys zs* :: *'a bag-pd*
show (*xs* ∪$_b$ *ys*) ∪$_b$ *zs* = *xs* ∪$_b$ (*ys* ∪$_b$ *zs*)
apply (*induct xs rule*: *bag-pd.principal-induct*, *simp*)
apply (*induct ys rule*: *bag-pd.principal-induct*, *simp*)
apply (*induct zs rule*: *bag-pd.principal-induct*, *simp*)
apply (*simp add*: *BPlus-assoc*)
done
show *xs* ∪$_b$ *ys* = *ys* ∪$_b$ *xs*
apply (*induct xs rule*: *bag-pd.principal-induct*, *simp*)
apply (*induct ys rule*: *bag-pd.principal-induct*, *simp*)
apply (*simp add*: *BPlus-commute*)
done
qed

lemmas *bag-plus-assoc* = *bag-add.assoc*
lemmas *bag-plus-commute* = *bag-add.commute*
lemmas *bag-plus-left-commute* = *bag-add.left-commute*

Useful for *simp add*: *bag-plus-ac*

lemmas *bag-plus-ac* = *bag-plus-assoc bag-plus-commute bag-plus-left-commute*

lemma *bag-plus-contract*: *xs* ∪$_b$ *xs* ⊑ *xs*

by (*induct xs rule*: *bag-pd.principal-induct*) (*simp-all add*: *BPlus-contract*)

lemma *bag-omega-below*: *bag-omega·xs* $\sqsubseteq$ *xs*
by (*induct xs rule*: *bag-pd.principal-induct*) (*simp-all add*: *BOmega-le*)

lemma *bag-omega-omega* [*simp*]: *bag-omega·*(*bag-omega·xs*) = *bag-omega·xs*
by (*induct xs rule*: *bag-pd.principal-induct*) (*simp-all add*: *BOmega-BOmega*)

lemma *bag-omega-plus* [*simp*]: *bag-omega·*(*xs* $\cup_b$ *ys*) = *bag-omega·xs* $\cup_b$ *bag-omega·ys*
apply (*induct xs rule*: *bag-pd.principal-induct*, *simp*)
apply (*induct ys rule*: *bag-pd.principal-induct*, *simp*)
apply (*simp add*: *BOmega-BPlus*)
done

lemma *bag-plus-omega-absorb* [*simp*]: *bag-omega·xs* $\cup_b$ *xs* = *bag-omega·xs*
by (*induct xs rule*: *bag-pd.principal-induct*) (*simp-all add*: *BPlus-BOmega-absorb*)

lemma *bag-plus-omega-idem* [*simp*]: *bag-omega·xs* $\cup_b$ *bag-omega·xs* = *bag-omega·xs*
by (*induct xs rule*: *bag-pd.principal-induct*) (*simp-all add*: *BPlus-BOmega-idem*)

lemma *bag-unit-below-iff* [*simp*]: *bag-unit·x* $\sqsubseteq$ *bag-unit·y* $\longleftrightarrow$ *x* $\sqsubseteq$ *y*
apply (*induct x rule*: *compact-basis.principal-induct*, *simp*)
apply (*induct y rule*: *compact-basis.principal-induct*, *simp*)
apply *simp*
done

lemma *bag-unit-eq-iff* [*simp*]: *bag-unit·x* = *bag-unit·y* $\longleftrightarrow$ *x* = *y*
unfolding *po-eq-conv* **by** *simp*

lemma *bag-omega-unit-bottom*: *bag-omega·*(*bag-unit·*($\bot$::'*a*)) = $\bot$
proof −
have *bag-unit·*($\bot$::'*a*) = *bag-principal* (*BUnit compact-bot*)
using *bag-unit-Rep-compact-basis* [*of compact-bot*::'*a compact-basis*] **by** *simp*
then show *?thesis*
by (*simp add*: *inst-bag-pd-pcpo*)
qed

Extended-Nat brings the topological *compact* into scope; the domain-theoretic one is *cpo-class.compact*.

lemma *compact-bag-unit*: *Cpo.compact x* $\Longrightarrow$ *Cpo.compact* (*bag-unit·x*)
by (*auto dest*!: *compact-basis.compact-imp-principal*)

lemma *compact-bag-plus* [*simp*]: ⟦*Cpo.compact xs*; *Cpo.compact ys*⟧ $\Longrightarrow$ *Cpo.compact* (*xs* $\cup_b$ *ys*)
by (*auto dest*!: *bag-pd.compact-imp-principal*)

lemma *compact-bag-omega* [*simp*]: *Cpo.compact xs* $\Longrightarrow$ *Cpo.compact* (*bag-omega·xs*)
by (*auto dest*!: *bag-pd.compact-imp-principal*)

23.9 Embedding-projection pair with the convex powerdomain

Forgetting multiplicities projects a bag onto its support in the convex powerdomain; giving every element multiplicity ω embeds a convex set back. The composite is *bag-omega*, so the multiplicity-free part of the bag powerdomain is a copy of the convex powerdomain.

definition *BSet* :: *'a pd-basis* ⇒ *'a bag-basis*
  where *BSet S* = *Abs-bag-basis* (λ*x*. *if x* ∈ *Rep-pd-basis S then* ∞ *else 0*)

lemma *bmult-BSet*: *bmult* (*BSet S*) *x* = (*if x* ∈ *Rep-pd-basis S then* ∞ *else 0*)
proof −
  have {*x*. (*if x* ∈ *Rep-pd-basis S then* ∞ *else 0*) ≠ (*0*::*enat*)} = *Rep-pd-basis S*
    by *auto*
  then show *?thesis*
    unfolding *BSet-def* **by** (*subst Abs-bag-basis-inverse'*) *simp-all*
qed

lemma *bsupp-BSet* [*simp*]: *bsupp* (*BSet S*) = *Rep-pd-basis S*
  by (*auto simp*: *bsupp-def bmult-BSet*)

definition *bag-pdb* :: *'a bag-basis* ⇒ *'a pd-basis*
  where *bag-pdb t* = *Abs-pd-basis* (*bsupp t*)

lemma *Rep-bag-pdb* [*simp*]: *Rep-pd-basis* (*bag-pdb t*) = *bsupp t*
  unfolding *bag-pdb-def* **by** (*rule Abs-pd-basis-inverse*) (*simp add*: *pd-basis-def*)

lemma *bag-pdb-mono*: *t* ≤$_b$ *u* ⟹ *bag-pdb t* ≤$\natural$ *bag-pdb u*
  unfolding *convex-le-def upper-le-def lower-le-def* **by** (*simp add*: *bag-le-supp*)

lemma *BSet-mono*: *S* ≤$\natural$ *T* ⟹ *BSet S* ≤$_b$ *BSet T*
  unfolding *convex-le-def upper-le-def lower-le-def*
  by (*rule bag-le-omegaI*) (*auto simp*: *bmult-BSet*)

lemma *bag-pdb-BSet* [*simp*]: *bag-pdb* (*BSet S*) = *S*
  by (*simp add*: *bag-pdb-def Rep-pd-basis-inverse*)

lemma *BSet-bag-pdb*: *BSet* (*bag-pdb t*) = *BOmega t*
  by (*rule bag-basis-eqI*) (*simp add*: *bmult-BSet bmult-BOmega bsupp-iff*)

lemma *bag-pdb-BUnit* [*simp*]: *bag-pdb* (*BUnit a*) = *PDUnit a*
  by (*simp add*: *Rep-pd-basis-inject* [*symmetric*] *Rep-PDUnit*)

lemma *bag-pdb-BPlus* [*simp*]: *bag-pdb* (*BPlus t u*) = *PDPlus* (*bag-pdb t*) (*bag-pdb u*)
  by (*simp add*: *Rep-pd-basis-inject* [*symmetric*] *Rep-PDPlus*)

lemma *bag-pdb-BOmega* [*simp*]: *bag-pdb* (*BOmega t*) = *bag-pdb t*

by (*simp add*: *Rep-pd-basis-inject* [*symmetric*])

lemma *BSet-PDUnit*: *BSet* (*PDUnit a*) = *BOmega* (*BUnit a*)
by (*rule bag-basis-eqI*) (*simp add*: *bmult-BSet bmult-BOmega bmult-BUnit Rep-PDUnit*)

lemma *BSet-PDPlus*: *BSet* (*PDPlus S T*) = *BPlus* (*BSet S*) (*BSet T*)
by (*rule bag-basis-eqI*) (*simp add*: *bmult-BSet Rep-PDPlus*)

definition *bag-to-convex* :: *'a bag-pd* → *'a convex-pd*
where *bag-to-convex* = *bag-pd.extension* (λ*t*. *convex-principal* (*bag-pdb t*))

definition *convex-to-bag* :: *'a convex-pd* → *'a bag-pd*
where *convex-to-bag* = *convex-pd.extension* (λ*S*. *bag-principal* (*BSet S*))

lemma *bag-to-convex-principal* [*simp*]:
bag-to-convex·(*bag-principal t*) = *convex-principal* (*bag-pdb t*)
unfolding *bag-to-convex-def* **by** (*simp add*: *bag-pd.extension-principal bag-pdb-mono*)

lemma *convex-to-bag-principal* [*simp*]:
convex-to-bag·(*convex-principal S*) = *bag-principal* (*BSet S*)
unfolding *convex-to-bag-def* **by** (*simp add*: *convex-pd.extension-principal BSet-mono*)

lemma *ep-pair-convex-to-bag*: *ep-pair convex-to-bag bag-to-convex*
proof
fix *x* :: *'a convex-pd*
show *bag-to-convex*·(*convex-to-bag*·*x*) = *x*
by (*induct x rule*: *convex-pd.principal-induct*) *simp-all*
next
fix *y* :: *'a bag-pd*
show *convex-to-bag*·(*bag-to-convex*·*y*) ⊑ *y*
by (*induct y rule*: *bag-pd.principal-induct*) (*simp-all add*: *BSet-bag-pdb BOmega-le*)
qed

lemma *bag-omega-eq*: *bag-omega*·*xs* = *convex-to-bag*·(*bag-to-convex*·*xs*)
by (*induct xs rule*: *bag-pd.principal-induct*) (*simp-all add*: *BSet-bag-pdb*)

lemma *bag-to-convex-unit* [*simp*]: *bag-to-convex*·(*bag-unit*·*x*) = *convex-unit*·*x*
by (*induct x rule*: *compact-basis.principal-induct*) *simp-all*

lemma *bag-to-convex-plus* [*simp*]:
bag-to-convex·(*xs* ∪$_b$ *ys*) = *bag-to-convex*·*xs* ∪♮ *bag-to-convex*·*ys*
apply (*induct xs rule*: *bag-pd.principal-induct*, *simp*)
apply (*induct ys rule*: *bag-pd.principal-induct*, *simp*)
apply *simp*
done

lemma *bag-to-convex-omega* [*simp*]: *bag-to-convex*·(*bag-omega*·*xs*) = *bag-to-convex*·*xs*
by (*induct xs rule*: *bag-pd.principal-induct*) *simp-all*

lemma *convex-to-bag-unit* [*simp*]: *convex-to-bag*·(*convex-unit*·*x*) = *bag-omega*·(*bag-unit*·*x*)
by (*induct x rule*: *compact-basis.principal-induct*) (*simp-all add*: *BSet-PDUnit*)

lemma *convex-to-bag-plus* [*simp*]:
convex-to-bag·(*xs* ∪♮ *ys*) = *convex-to-bag*·*xs* ∪$_b$ *convex-to-bag*·*ys*
apply (*induct xs rule*: *convex-pd.principal-induct*, *simp*)
apply (*induct ys rule*: *convex-pd.principal-induct*, *simp*)
apply (*simp add*: *BSet-PDPlus*)
done

Below ω multiplicities are invisible: the ω-images are ordered exactly as in the convex (Egli-Milner) order.

lemma *bag-omega-below-iff*:
bag-omega·*xs* ⊑ *bag-omega*·*ys* ⟷ *bag-to-convex*·*xs* ⊑ *bag-to-convex*·*ys*
by (*simp add*: *bag-omega-eq ep-pair.e-below-iff* [*OF ep-pair-convex-to-bag*])

23.10 Mapping

A continuous map on elements acts on bags, continuously in the map as well. Its values on compact elements need not be compact, so the basis map reads them through a fixed approximation chain, and the bag map is the limit along the chain.

definition *chosen-approx* :: *nat* ⇒ *'a* → *'a*
where *chosen-approx* = (*SOME a*. *approx-chain a*)

lemma *approx-chain-chosen-approx*: *approx-chain chosen-approx*
unfolding *chosen-approx-def* **using** *bifinite* **by** (*rule someI-ex*)

interpretation *chosen-approx*: *approx-chain chosen-approx*
by (*rule approx-chain-chosen-approx*)

lemma *chain-chosen-approx-app* [*simp*]: *chain* (λ*i*. *chosen-approx i*·*x*)
by (*rule ch2ch-Rep-cfunL*[*OF chosen-approx.chain-approx*])

lemma *lub-chosen-approx-app*: (⨆ *i*. *chosen-approx i*·*x*) = *x*
using *contlub-cfun-fun*[*OF chosen-approx.chain-approx*, *of x*] **by** *simp*

lemma *compact-below-iff*: *x* ⊑ *y* ⟷ *Rep-compact-basis x* ⊑ *Rep-compact-basis y*
unfolding *compact-le-def* **..**

definition *approx-basis* :: *nat* ⇒ *'a* ⇒ *'a compact-basis*
where *approx-basis i x* = *Abs-compact-basis* (*chosen-approx i*·*x*)

lemma *Rep-approx-basis* [*simp*]: *Rep-compact-basis* (*approx-basis i x*) = *chosen-approx i*·*x*
by (*simp add*: *approx-basis-def*)

lemma *approx-basis-mono*: $i \leq j \Longrightarrow x \sqsubseteq y \Longrightarrow$ *approx-basis i x* $\sqsubseteq$ *approx-basis j y*
unfolding *compact-le-def Rep-approx-basis*
by (*rule below-trans*[*OF monofun-cfun-fun*[*OF chain-mono*[*OF chosen-approx.chain-approx*]] *monofun-cfun-arg*])

Finitely many eventual properties hold together eventually.

lemma *finite-eventually*:
fixes $P :: 'x::type \Rightarrow nat \Rightarrow bool$
assumes *finite S* **and** $\bigwedge x.\ x \in S \Longrightarrow \exists\, n.\ \forall\, m{\geq}n.\ P\ x\ m$
shows $\exists\, N.\ \forall\, x{\in}S.\ P\ x\ N$
proof −
obtain *nf* **where** *nf*: $\bigwedge x.\ x \in S \Longrightarrow \forall\, m{\geq}nf\ x.\ P\ x\ m$
using *assms*(*2*) **by** *metis*
have *P x* (*Max* (*insert 0* (*nf* ‘ *S*))) **if** $x \in S$ **for** *x*
using *nf*[*OF that*] *Max-ge*[*of insert 0* (*nf* ‘ *S*) *nf x*] *assms*(*1*) *that* **by** *simp*
then show *?thesis*
by *blast*
qed

Families over the same indices with pointwise larger labels are larger.

lemma *BFamily-le-labels*:
assumes *finite I* **and** $\exists\, i{\in}I.\ m\ i \neq 0$ **and** $\bigwedge i.\ i \in I \Longrightarrow f\ i \sqsubseteq g\ i$
shows *BFamily I f m* $\leq_b$ *BFamily I g m*
by (*rule BFamily-le*[**where** $p = \lambda i\ j.$ *if* $i = j$ *then m i else 0*]) (*use assms* **in** ‹*auto split*: *if-split-asm*›)

definition *BMap* :: $('a \to 'b) \Rightarrow nat \Rightarrow$ *'a bag-basis* $\Rightarrow$ *'b bag-basis*
where *BMap f i t* = *BFamily* (*bsupp t*) ($\lambda x.$ *approx-basis i* (*f*·(*Rep-compact-basis x*))) (*bmult t*)

lemma *bsupp-ex-nonzero*: $\exists\, x{\in}$*bsupp t.* *bmult t x* $\neq 0$
using *bsupp-nonempty*[*of t*] **unfolding** *bsupp-iff* [*symmetric*] **by** *blast*

lemma *BMap-mono*:
assumes *fg*: $f \sqsubseteq g$ **and** *ij*: $i \leq j$ **and** *tu*: $t \leq_b u$
shows *BMap f i t* $\leq_b$ *BMap g j u*
proof −
obtain *P* **where** *P*: *bag-plan t u P*
using *tu* **unfolding** *bag-le-def* **by** *blast*
have *lab*: *approx-basis i* (*f*·(*Rep-compact-basis x*)) $\sqsubseteq$ *approx-basis j* (*g*·(*Rep-compact-basis y*))
if $x \sqsubseteq y$ **for** *x y*
by (*rule approx-basis-mono*[*OF ij monofun-cfun*[*OF fg compact-below-iff* [*THEN iffD1*, *OF that*]]])
show *?thesis*
unfolding *BMap-def*
proof (*rule BFamily-le*[**where** $p = P$])
fix *x*

assume $x \in bsupp\ t$ **and** $bmult\ t\ x \neq 0$
then obtain y **where** y: $y \in bsupp\ u\ P\ x\ y \neq 0$
using *bag-plan-row-ex*[*OF P*] **by** *blast*
then show $\exists\, y{\in}bsupp\ u.\ bmult\ u\ y \neq 0 \wedge$
approx-basis i (*f*·(*Rep-compact-basis x*)) ⊑ *approx-basis j* (*g*·(*Rep-compact-basis y*))
using *lab*[*OF bag-plan-below*[*OF P y*(*2*)]] *bsupp-iff* **by** *blast*
qed (*auto simp*: *bsupp-ex-nonzero bag-plan-row*[*OF P*] *bag-plan-col*[*OF P*] *intro*: *lab bag-plan-below*[*OF P*])
qed

Along a chain of maps, the approximation at a fixed level is eventually reached on the finitely many elements of a basis bag.

lemma *BMap-lub*:
assumes *Y*: *chain Y*
shows $\exists\, N.\ BMap\ (\bigsqcup n.\ Y\ n)\ i\ t \leq_b BMap\ (Y\ N)\ i\ t$
proof −
have $\exists\, N.\ \forall\, x{\in}bsupp\ t.$
approx-basis i ((⨆ *n*. *Y n*)·(*Rep-compact-basis x*)) ⊑ *approx-basis i* (*Y N*·(*Rep-compact-basis x*))
proof (*rule finite-eventually*[*OF finite-bsupp*])
fix *x*
let *?a* = λ*n*. *chosen-approx i*·(*Y n*·(*Rep-compact-basis x*))
have *ch*: *chain ?a*
using *Y* **by** (*intro ch2ch-Rep-cfunR ch2ch-Rep-cfunL*)
have *chosen-approx i*·((⨆ *n*. *Y n*)·(*Rep-compact-basis x*)) = (⨆ *n*. *?a n*)
using *Y* **by** (*simp add*: *contlub-cfun-fun contlub-cfun-arg ch2ch-Rep-cfunL*)
then have *chosen-approx i*·((⨆ *n*. *Y n*)·(*Rep-compact-basis x*)) ⊑ (⨆ *n*. *?a n*)
by *simp*
then obtain *n* **where** *n*: *chosen-approx i*·((⨆ *n*. *Y n*)·(*Rep-compact-basis x*)) ⊑ *?a n*
using *compact-below-lub-iff*[*OF chosen-approx.compact-approx*[*of i* (⨆ *n*. *Y n*)·(*Rep-compact-basis x*)] *ch*]
by *blast*
have ∀ *m*≥*n*. *approx-basis i* ((⨆ *n*. *Y n*)·(*Rep-compact-basis x*)) ⊑ *approx-basis i* (*Y m*·(*Rep-compact-basis x*))
using *n chain-mono*[*OF ch*] **unfolding** *compact-below-iff Rep-approx-basis* **by** (*blast intro*: *below-trans*)
then show $\exists\, n.\ \forall\, m{\geq}n.$
approx-basis i ((⨆ *n*. *Y n*)·(*Rep-compact-basis x*)) ⊑ *approx-basis i* (*Y m*·(*Rep-compact-basis x*))
by *blast*
qed
then show *?thesis*
unfolding *BMap-def* **by** (*blast intro*: *BFamily-le-labels finite-bsupp bsupp-ex-nonzero*)
qed

lemma *chain-BMap*: *chain* (λ*i*. *bag-principal* (*BMap f i t*))
by (*rule chainI*, *rule bag-pd.principal-mono*, *rule BMap-mono*) *simp-all*

lemma *lub-BMap-mono*:
f ⊑ *g* ⟹ *t* ≤$_b$ *u* ⟹ (⨆ *i*. *bag-principal* (*BMap f i t*)) ⊑ (⨆ *i*. *bag-principal* (*BMap g i u*))
by (*rule lub-mono*[*OF chain-BMap chain-BMap*], *rule bag-pd.principal-mono*, *erule* (*1*) *BMap-mono*[*OF - order-refl*])

lemma *cont-bag-map*: *cont* (λ*f*. *bag-pd.extension* (λ*t*. ⨆ *i*. *bag-principal* (*BMap f i t*)))
proof (*rule contI2*)
let *?F* = λ*f t*. ⨆ *i*. *bag-principal* (*BMap f i t*)
show *mono*: *monofun* (λ*f*. *bag-pd.extension* (*?F f*))
by (*rule monofunI*, *rule bag-pd.extension-mono*) (*simp-all add*: *lub-BMap-mono*)
fix *Y* :: *nat* ⇒ *'a* → *'b*
assume *Y*: *chain Y* **and** *chY*: *chain* (λ*n*. *bag-pd.extension* (*?F* (*Y n*)))
have *chF*: *chain* (λ*n*. *?F* (*Y n*) *t*) **for** *t*
using *Y* **by** (*intro chainI lub-BMap-mono*) (*simp-all add*: *chainE*)
show *bag-pd.extension* (*?F* (⨆ *n*. *Y n*)) ⊑ (⨆ *n*. *bag-pd.extension* (*?F* (*Y n*)))
proof (*rule cfun-belowI*)
fix *xs*
show *bag-pd.extension* (*?F* (⨆ *n*. *Y n*))·*xs* ⊑ (⨆ *n*. *bag-pd.extension* (*?F* (*Y n*)))·*xs*
proof (*induct xs rule*: *bag-pd.principal-induct*)
fix *t*
have *?F* (⨆ *n*. *Y n*) *t* ⊑ (⨆ *n*. *?F* (*Y n*) *t*)
proof (*rule lub-below*[*OF chain-BMap*])
fix *i*
obtain *N* **where** *BMap* (⨆ *n*. *Y n*) *i t* ≤$_b$ *BMap* (*Y N*) *i t*
using *BMap-lub*[*OF Y*] **by** *blast*
then have *bag-principal* (*BMap* (⨆ *n*. *Y n*) *i t*) ⊑ *?F* (*Y N*) *t*
by (*blast intro*: *below-trans bag-pd.principal-mono is-ub-thelub*[*OF chain-BMap*])
also have ... ⊑ (⨆ *n*. *?F* (*Y n*) *t*)
by (*rule is-ub-thelub*[*OF chF*])
finally show *bag-principal* (*BMap* (⨆ *n*. *Y n*) *i t*) ⊑ (⨆ *n*. *?F* (*Y n*) *t*) **.**
qed
then show *bag-pd.extension* (*?F* (⨆ *n*. *Y n*))·(*bag-principal t*) ⊑
(⨆ *n*. *bag-pd.extension* (*?F* (*Y n*)))·(*bag-principal t*)
by (*simp add*: *contlub-cfun-fun*[*OF chY*] *bag-pd.extension-principal lub-BMap-mono*)
qed *simp*
qed
qed

definition *bag-map* :: (*'a* → *'b*) → *'a bag-pd* → *'b bag-pd*
where *bag-map* = (Λ *f*. *bag-pd.extension* (λ*t*. ⨆ *i*. *bag-principal* (*BMap f i t*)))

lemma *bag-map-principal*: *bag-map*·*f*·(*bag-principal t*) = (⨆ *i*. *bag-principal* (*BMap f i t*))
unfolding *bag-map-def* **by** (*simp add*: *cont-bag-map bag-pd.extension-principal lub-BMap-mono*)

lemma *BMap-BUnit*: *BMap f i* (*BUnit a*) = *BUnit* (*approx-basis i* (*f*·(*Rep-compact-basis a*)))
by (*rule bag-basis-eqI*) (*auto simp*: *BMap-def bmult-BFamily-sum bmult-BUnit*)

lemma *bag-map-unit* [*simp*]:
fixes *f* :: *'a* → *'b*
shows *bag-map*·*f*·(*bag-unit*·*x*) = *bag-unit*·(*f*·*x*)
proof (*induct x rule*: *compact-basis.principal-induct*)
fix *a*
have *u*: *bag-principal* (*BUnit* (*approx-basis i y*)) = *bag-unit*·(*chosen-approx i*·*y*)
for *i* **and** *y* :: *'b*
using *bag-unit-Rep-compact-basis*[*of approx-basis i y*] **by** *simp*
show *bag-map*·*f*·(*bag-unit*·(*Rep-compact-basis a*)) = *bag-unit*·(*f*·(*Rep-compact-basis a*))
by (*simp add*: *bag-map-principal BMap-BUnit u contlub-cfun-arg*[*symmetric*] *lub-chosen-approx-app*)
qed *simp*

The identity map: the approximations are eventually exact on the compact elements of a basis bag.

lemma *BFamily-bsupp*: *BFamily* (*bsupp t*) (λ*x*. *x*) (*bmult t*) = *t*
proof (*rule bag-basis-eqI*)
fix *z*
have {*x*∈*bsupp t*. *x* = *z*} = (*if z* ∈ *bsupp t then* {*z*} *else* {})
by *auto*
then show *bmult* (*BFamily* (*bsupp t*) (λ*x*. *x*) (*bmult t*)) *z* = *bmult t z*
by (*simp add*: *bmult-BFamily*[*OF finite-bsupp bsupp-ex-nonzero*] *bsupp-iff*)
qed

lemma *bag-map-ID* [*simp*]: *bag-map*·*ID* = (*ID* :: *'a bag-pd* → *'a bag-pd*)
proof (*rule cfun-eqI*)
fix *xs* :: *'a bag-pd*
show *bag-map*·*ID*·*xs* = *ID*·*xs*
proof (*induct xs rule*: *bag-pd.principal-induct*)
fix *t* :: *'a bag-basis*
have ∃ *N*. ∀ *x*∈*bsupp t*. *approx-basis N* (*Rep-compact-basis x*) = *x*
proof (*rule finite-eventually*[*OF finite-bsupp*, **where** *P* = λ*x N*. *approx-basis N* (*Rep-compact-basis x*) = *x*])
fix *x* :: *'a compact-basis*
obtain *n* **where** *n*: *chosen-approx n*·(*Rep-compact-basis x*) = *Rep-compact-basis x*
using *chosen-approx.compact-eq-approx*[*OF Rep-compact-basis'*] **by** *blast*
have *chosen-approx m*·(*Rep-compact-basis x*) = *Rep-compact-basis x* **if** *n* ≤ *m* **for** *m*
by (*rule below-antisym*[*OF chosen-approx.approx-below*])
(*metis n monofun-cfun-fun*[*OF chain-mono*[*OF chosen-approx.chain-approx that*]])
then show ∃ *n*. ∀ *m*≥*n*. *approx-basis m* (*Rep-compact-basis x*) = *x*

```
      by (auto simp: Rep-compact-basis-inject [symmetric])
    qed
  then obtain N where N: ⋀x. x ∈ bsupp t ⟹ approx-basis N (Rep-compact-basis
x) = x
      by blast
    have below: bag-principal (BMap ID i t) ⊑ bag-principal t for i
    proof (rule bag-pd.principal-mono)
      have BMap ID i t ≤b BFamily (bsupp t) (λx. x) (bmult t)
        unfolding BMap-def
       by (rule BFamily-le-labels) (simp-all add: bsupp-ex-nonzero compact-below-iff
chosen-approx.approx-below)
      then show BMap ID i t ≤b t
        by (simp add: BFamily-bsupp)
    qed
    have BMap ID N t = t
      unfolding BMap-def by (subst BFamily-cong[where f' = λx. x and m' =
bmult t]) (simp-all add: N BFamily-bsupp)
    then have bag-principal t ⊑ (⨆ i. bag-principal (BMap ID i t))
      using is-ub-thelub[OF chain-BMap, of ID N t] by simp
    then show bag-map·ID·(bag-principal t) = ID·(bag-principal t)
      by (simp add: bag-map-principal below-antisym lub-below[OF chain-BMap
below])
  qed simp
qed

lemma bmult-BMap:
  bmult (BMap f i t) z = (∑ x∈bsupp t. if approx-basis i (f·(Rep-compact-basis x))
= z then bmult t x else 0)
  by (simp only: BMap-def bmult-BFamily-sum[OF finite-bsupp bsupp-ex-nonzero])

lemma BMap-BPlus: BMap f i (BPlus t u) = BPlus (BMap f i t) (BMap f i u)
proof (rule bag-basis-eqI)
  fix z
  let ?l = λx. approx-basis i (f·(Rep-compact-basis x))
  let ?f = λv x. if ?l x = z then bmult v x else 0
  have ext: sum (?f v) (bsupp t ∪ bsupp u) = sum (?f v) (bsupp v) if bsupp v ⊆
bsupp t ∪ bsupp u for v
    using that by (intro sum.mono-neutral-right) (auto simp: bsupp-iff)
  have bmult (BMap f i (BPlus t u)) z = (∑ x∈bsupp t ∪ bsupp u. ?f t x + ?f u
x)
    unfolding bmult-BMap by (rule sum.cong) simp-all
  also have ... = bmult (BMap f i t) z + bmult (BMap f i u) z
    unfolding sum.distrib bmult-BMap by (simp add: ext)
  finally show bmult (BMap f i (BPlus t u)) z = bmult (BPlus (BMap f i t) (BMap
f i u)) z
    by simp
qed

lemma BMap-BOmega: BMap f i (BOmega t) = BOmega (BMap f i t)
```

proof (*rule bag-basis-eqI*)
fix *z*
let *?l* = λ*x*. *approx-basis i* (*f*·(*Rep-compact-basis x*))
show *bmult* (*BMap f i* (*BOmega t*)) *z* = *bmult* (*BOmega* (*BMap f i t*)) *z*
proof (*cases* ∃ *x*∈*bsupp t*. *?l x* = *z*)
case *True*
then obtain *x* **where** *x*: *x* ∈ *bsupp t ?l x* = *z*
by *blast*
have *bmult* (*BMap f i* (*BOmega t*)) *z* = ∞
unfolding *bmult-BMap* **using** *x* **by** (*intro enat-sum-infty*[**where** *x* = *x*])
(*simp-all add*: *bmult-BOmega bsupp-iff*)
moreover have *bmult* (*BMap f i t*) *z* ≠ *0*
unfolding *bmult-BMap* **using** *x* **by** (*subst sum-eq-0-iff*) (*auto simp*: *bsupp-iff*)
ultimately show *?thesis*
by (*simp add*: *bmult-BOmega*)
next
case *False*
then show *?thesis*
by (*simp add*: *bmult-BMap bmult-BOmega*)
qed
qed

lemma *bag-map-plus* [*simp*]:
fixes *f* :: *'a* → *'b*
shows *bag-map*·*f*·(*xs* ∪*b ys*) = *bag-map*·*f*·*xs* ∪*b bag-map*·*f*·*ys*
proof (*induct xs rule*: *bag-pd.principal-induct*)
fix *t*
show *bag-map*·*f*·(*bag-principal t* ∪*b ys*) = *bag-map*·*f*·(*bag-principal t*) ∪*b bag-map*·*f*·*ys*
proof (*induct ys rule*: *bag-pd.principal-induct*)
fix *u*
have *lubplus*: (⨆ *i*. *P i* ∪*b Q i*) = (⨆ *i*. *P i*) ∪*b* (⨆ *i*. *Q i*)
if *chain P chain Q* **for** *P Q* :: *nat* ⇒ *'b bag-pd*
using *lub-APP*[*OF ch2ch-Rep-cfunR*[*OF that*(*1*)] *that*(*2*)] *contlub-cfun-arg*[*OF*
that(*1*), *of bag-plus*]
by *simp*
have *bag-map*·*f*·(*bag-principal t* ∪*b bag-principal u*) =
(⨆ *i*. *bag-principal* (*BMap f i t*) ∪*b bag-principal* (*BMap f i u*))
by (*simp add*: *bag-map-principal BMap-BPlus*)
then show *bag-map*·*f*·(*bag-principal t* ∪*b bag-principal u*) =
bag-map·*f*·(*bag-principal t*) ∪*b bag-map*·*f*·(*bag-principal u*)
by (*simp add*: *lubplus chain-BMap bag-map-principal del*: *bag-plus-principal*)
qed *simp*
qed *simp*

lemma *bag-map-omega* [*simp*]: *bag-map*·*f*·(*bag-omega*·*xs*) = *bag-omega*·(*bag-map*·*f*·*xs*)
by (*induct xs rule*: *bag-pd.principal-induct*)
(*simp-all add*: *bag-map-principal BMap-BOmega contlub-cfun-arg*[*OF chain-BMap*])

end